Learn Being Forward

COURSES · CAMPUS PARTNERSHIPS · PLACEMENTS

COURSE NOTES · BASIC TO ADVANCED

TypeScript Programming

Complete notes for the Learn Being Forward TypeScript Programming syllabus: every module, topic and subtopic explained step by step, with diagrams, verified examples and a practice checkpoint for each module.

19	3	43	376
modules	levels	minimum days	TypeScript programs

How to use these notes

These notes follow the **Learn Being Forward TypeScript Programming syllabus** exactly: the same 19 modules, in the same order, with every topic and subtopic explained step by step. Each subtopic number (for example **3.7.2**) matches the syllabus, so you can always find where you are.

Level	Modules	Minimum	Outcome
1. Beginner	1 to 7	11 days	Write strictly typed code with basic types, functions, interfaces, unions, literal types and enums, and use typed JavaScript libraries
2. Intermediate	8 to 13	14 days	Use narrowing, typed classes, generics, modules and declaration files, utility types, and type DOM and async code
3. Advanced	14 to 19	18 days	Build advanced and type-level types, use decorators, configure the compiler and tooling, apply TypeScript in React and Node.js, and migrate real projects
Total	1 to 19	43 days	About 8 weeks at 6 study days per week

Read
Read the concept for each subtopic and note the key points in your own words.

Type & run
Type out and run every example. Do not copy and paste code. Compare with the OUTPUT box.

Checkpoint
Finish the practice checkpoint at the end of each module before moving on.

Revise
At the end of each level, revise its modules and build one small project.

Every TypeScript program in these notes was compiled with the **TypeScript 6 compiler in strict mode** (`tsc --strict`) and then run with **Node.js 22** to produce the output shown. Boxes labelled **COMPILER ERRORS** show the real messages tsc printed for code that is deliberately wrong: those programs never ran, which is exactly the point of a type checker. Examples that need a framework, a database, a network or a package that could not be installed (React, Express, Prisma, Zod, Vitest) show the output those programs produce, and say so. Values that depend on the moment or the machine, such as dates, timings and random numbers, will differ when you run the programs.

HOW TO READ THE BOXES

Dark boxes with a gold top line are **TypeScript code**. Dashed cream boxes labelled **OUTPUT** show what the program prints. Gold-bordered boxes are notes and tips, red boxes warn about common mistakes, and navy boxes hold the key idea to remember.

Course plan at a glance

Module	Topic	Level	Min days
01	Introduction to TypeScript	Beginner	1
02	Setup and the TypeScript compiler	Beginner	1
03	Basic types and type inference	Beginner	2
04	Functions	Beginner	2
05	Object types, type aliases and interfaces	Beginner	2

Module	Topic	Level	Min days
06	Union, intersection and literal types	Beginner	2
07	Working with JavaScript and the type system basics	Beginner	1
08	Type narrowing and type guards	Intermediate	2
09	Classes and object-oriented TypeScript	Intermediate	3
10	Generics	Intermediate	3
11	Modules, namespaces and declaration files	Intermediate	2
12	Built-in utility types	Intermediate	2
13	TypeScript in the browser and with asynchronous code	Intermediate	2
14	Advanced types	Advanced	4
15	Type-level patterns and advanced techniques	Advanced	3
16	Decorators	Advanced	2
17	Compiler configuration and tooling	Advanced	2
18	TypeScript with frameworks and back ends	Advanced	4
19	Migration, projects and interview preparation	Advanced	3

Table of contents

LEVEL 1 · BEGINNER LEVEL · MODULES 1 TO 7

Module 01 · Introduction to TypeScript 7

1.1 What is TypeScript? 7
1.2 History of TypeScript 11
1.3 Why do we need TypeScript? 12
1.4 Where is TypeScript used? 14
1.5 TypeScript vs JavaScript 15
1.6 Prerequisites recap 16

Module 02 · Setup and the TypeScript compiler 19

2.1 Installing TypeScript 19
2.2 Editor setup 21
2.3 First program 22
2.4 Running TypeScript directly 23
2.5 tsconfig.json basics 25
2.6 Compile errors 31

Module 03 · Basic types and type inference 36

3.1 Type annotations 36
3.2 Type inference 38
3.3 Primitive types 40
3.4 Arrays 42
3.5 Tuples 44
3.6 Special types 47
3.7 Type assertions 51

Module 04 · Functions 55

4.1 Typing functions 55
4.2 Parameters 58
4.3 Function types 60
4.4 Arrow functions and typing this 63
4.5 Function overloads 64
4.6 Async functions 66

Module 05 · Object types, type aliases and interfaces 71

5.1 Object types 71
5.2 Type aliases 73
5.3 Interfaces 75
5.4 Interface vs type alias 78
5.5 Index signatures 79
5.6 Nested and composed object types 81
5.7 Structural typing 82

Module 06 · Union, intersection and literal types 86

6.1 Union types 86
6.2 Intersection types 89
6.3 Literal types 91
6.4 Enums 93
6.5 null and undefined handling 97

Module 07 · Working with JavaScript and the type system basics 101

7.1 Type compatibility 101
7.2 Using JavaScript libraries 103
7.3 Gradual adoption 105
7.4 Reading type errors 108

LEVEL 2 · INTERMEDIATE LEVEL · MODULES 8 TO 13

Module 08 · Type narrowing and type guards 114

8.1 Narrowing basics 114
8.2 Discriminated unions 119
8.3 User-defined type guards 123
8.4 Narrowing pitfalls 125

Module 09 · Classes and object-oriented TypeScript 130

9.1 Class basics 130
9.2 Access modifiers 133
9.3 Getters, setters and static members 136
9.4 Inheritance 137
9.5 Abstract classes 139
9.6 Interfaces and classes 141
9.7 Classes as types 142
9.8 OOP design in TypeScript 144

Module 10 · Generics 149

10.1 Why generics 149
10.2 Generic functions 150
10.3 Generic constraints 153
10.4 Generic interfaces and type aliases 156
10.5 Generic classes 157
10.6 Built-in generics 159
10.7 Generics best practices 161

Module 11 · Modules, namespaces and declaration files 164

11.1 ES modules in TypeScript 164
11.2 Module resolution 168
11.3 Namespaces 171
11.4 Declaration files 172
11.5 Triple-slash directives (overview) 177

Module 12 · Built-in utility types 180

12.1 Property modifiers 180
12.2 Picking and omitting 182
12.3 Union utilities 185
12.4 Function utilities 186
12.5 Async and string utilities 188
12.6 Other utilities 190
12.7 Real-world usage 190

Module 13 · TypeScript in the browser and with asynchronous code 196

13.1 DOM types 196
13.2 Event types 199
13.3 Async TypeScript 202
13.4 Browser storage with types 206

LEVEL 3 · ADVANCED LEVEL · MODULES 14 TO 19

Module 14 · Advanced types 212

- 14.1 Type operators 212
- 14.2 Conditional types 215
- 14.3 Mapped types 218
- 14.4 Template literal types 221
- 14.5 Recursive types 224
- 14.6 Variadic tuple types 227

Module 15 · Type-level patterns and advanced techniques 230

- 15.1 Literal and inference control 230
- 15.2 Modelling data safely 233
- 15.3 Type safety at boundaries 237
- 15.4 Variance 241
- 15.5 Declaration merging and augmentation in depth 244
- 15.6 Type performance 245

Module 16 · Decorators 249

- 16.1 Introduction 249
- 16.2 Types of decorators 251
- 16.3 Decorator factories 255
- 16.4 Practical decorators 258
- 16.5 Decorators in frameworks 259

Module 17 · Compiler configuration and tooling 265

- 17.1 tsconfig in depth 265
- 17.2 Project references and monorepos 269
- 17.3 Build tools 271
- 17.4 Code quality 272
- 17.5 Running TypeScript on the server 274
- 17.6 Compiler internals (overview) 276

Module 18 · TypeScript with frameworks and back ends 280

- 18.1 TypeScript with React 280
- 18.2 TypeScript with Node.js and Express 285
- 18.3 TypeScript with databases 291
- 18.4 End-to-end type safety 293
- 18.5 Testing TypeScript 295

Module 19 · Migration, projects and interview preparation 304

- 19.1 Migrating JavaScript to TypeScript 304
- 19.2 Best practices 308
- 19.3 Type challenges 311
- 19.4 Capstone projects 313
- 19.5 Interview preparation 316

LEVEL 1

Beginner level

Outcome: write strictly typed code with basic types, functions, interfaces, unions, literal types and enums, and use typed JavaScript libraries.

7 MODULES · 11 DAYS MINIMUM

Module 01 · Introduction to TypeScript	**1 day**
Module 02 · Setup and the TypeScript compiler	**1 day**
Module 03 · Basic types and type inference	**2 days**
Module 04 · Functions	**2 days**
Module 05 · Object types, type aliases and interfaces	**2 days**
Module 06 · Union, intersection and literal types	**2 days**
Module 07 · Working with JavaScript and the type system basics	**1 day**

MODULE 01 · BEGINNER

Introduction to TypeScript

Minimum duration **1 day** | Level **Beginner**

This module explains what TypeScript is, why it was created on top of JavaScript, and where it is used, before any serious code is written. It covers the definition of TypeScript and what "a superset of JavaScript" and "types are erased" mean, the history of the language from 2012 to the native Go compiler, the problems it solves, where it is used in industry, how it compares with JavaScript, and the JavaScript you need to know first.

IN THIS MODULE

1.1 What is TypeScript?
1.2 History of TypeScript
1.3 Why do we need TypeScript?
1.4 Where is TypeScript used?
1.5 TypeScript vs JavaScript
1.6 Prerequisites recap

1.1 What is TypeScript?

JavaScript runs everywhere: in every browser, on servers with Node.js, in mobile and desktop apps. But JavaScript is **dynamically typed**: a variable can hold any kind of value, and mistakes such as calling a method that does not exist, passing a string where a number is expected or forgetting that a value can be `undefined` are only discovered when that line actually runs, often in front of a user. **TypeScript** was created to find those mistakes earlier, while you are still writing the code.

1.1.1 Definition: a statically typed superset of JavaScript that compiles to plain JavaScript

Statically typed

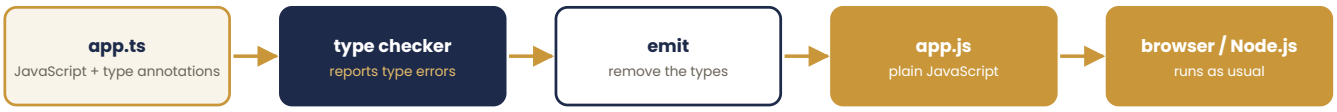
Types are checked **before** the program runs, by a tool (the type checker) that reads your source code. Errors appear in the editor and in the build.

Superset of JavaScript

TypeScript contains **all** of JavaScript and adds syntax for types on top. Anything you know about JavaScript still applies.

Compiles to JavaScript

Browsers and Node.js run JavaScript, so the TypeScript compiler (`tsc`) checks the code and outputs plain `.js` files.



TYPESCRIPT IS CHECKED AT COMPILE TIME AND TURNED INTO ORDINARY JAVASCRIPT

A FIRST TYPESCRIPT PROGRAM: THE ANNOTATIONS SAY WHAT EACH VALUE MUST BE

main.ts

```ts
function totalPrice(price: number, quantity: number): number {
  return price * quantity;
}

const items: string[] = ["notebook", "pen", "bag"];
console.log(totalPrice(120, 3));
console.log(items.map((item) => item.toUpperCase()).join(", "));
```

OUTPUT

```
360
NOTEBOOK, PEN, BAG
```

The only differences from JavaScript are the **type annotations** after a colon: `price: number`, `: number` for the return value, and `string[]` for an array of strings. If you call `totalPrice("120", 3)`, TypeScript refuses to compile it instead of silently producing a wrong answer at run time.

THREE BUGS CAUGHT BEFORE THE PROGRAM RUNS

main.ts

```ts
function totalPrice(price: number, quantity: number): number {
  return price * quantity;
}

totalPrice("120", 3);
const items: string[] = ["notebook", "pen"];
items.push(42);
console.log(items[0].toUppercase());
```

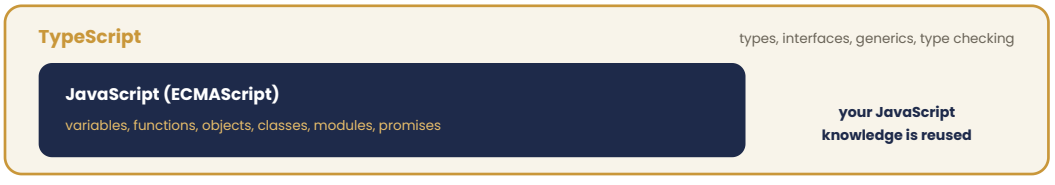
COMPILER ERRORS (tsc --strict)

```
main.ts(5,12): error TS2345: Argument of type 'string' is not assignable to parameter of type
'number'.
main.ts(7,12): error TS2345: Argument of type 'number' is not assignable to parameter of type
'string'.
main.ts(8,22): error TS2551: Property 'toUppercase' does not exist on type 'string'. Did you mean
'toUpperCase'?
```

Each error names the file, the line and column, an error code (TS2345, TS2551) and an explanation. Notice the last one: TypeScript even suggests the correct method name. In JavaScript all three lines would run, and the third would crash with *TypeError: items[0].toUppercase is not a function*.

1.1.2 "JavaScript with syntax for types"

This is the official tagline of TypeScript, and it is the best way to think about the language. TypeScript is not a different language with its own runtime, standard library or way of doing things. You write normal modern JavaScript (`const`, arrow functions, classes, modules, `async`/`await`, array methods) and add **types** where they are useful: on function parameters, on object shapes, on return values. The vast majority of the code in a TypeScript file is plain JavaScript.



TYPESCRIPT IS JAVASCRIPT PLUS A LAYER OF TYPES

1.1.3 Every valid JavaScript program is valid TypeScript

Because TypeScript is a superset, you can rename a `.js` file to `.ts` and it is still a TypeScript program; the compiler simply infers as many types as it can. This is what makes gradual adoption possible (Module 7): teams convert one file at a time. The only catch is that TypeScript may now **report errors** in that code, because it finds real or possible bugs that JavaScript ignored. The program is still syntactically valid; the type checker is just stricter.

PLAIN JAVASCRIPT IN A .TS FILE: ONE STRICT-MODE ERROR IS REPORTED, BUT THE JAVASCRIPT IS STILL EMITTED AND RUNS main.ts

```
// This is plain JavaScript, saved as main.ts: no annotations at all.
const student = { name: "Asha", marks: [78, 91, 85] };

function average(nums) {
  return nums.reduce((a, b) => a + b, 0) / nums.length;
}

console.log(`${student.name}: ${average(student.marks).toFixed(1)}`);
```

TERMINAL

```
$ tsc --strict main.ts
main.ts(4,18): error TS7006: Parameter 'nums' implicitly has an 'any' type.
main.ts(5,23): error TS7006: Parameter 'a' implicitly has an 'any' type.
main.ts(5,26): error TS7006: Parameter 'b' implicitly has an 'any' type.
$ node main.js
Asha: 84.7
```

NOTE

In strict mode the parameter `nums` is reported as implicitly having type `any` (error TS7006), yet tsc still writes `main.js`, which runs normally. Adding `nums: number[]` removes the error. Errors do not stop the output unless you set `noEmitOnError` (2.6.2).

1.1.4 Types are erased at compile time (no runtime type checking)

When TypeScript compiles a file, it **removes** every type annotation, interface and type alias. The output contains no trace of the types, so there is **no type checking while the program runs**. This has two important consequences: TypeScript adds no runtime cost (the JavaScript is as fast as hand-written JavaScript), and TypeScript cannot protect you from data that arrives at run time, such as JSON from an API, user input or a file. That data must be validated with real code (13.3.3, 15.3).

ORDER.TS TypeScript

```
interface Order {
  id: number;
  product: string;
  price: number;
}

const order: Order = { id: 101, product: "Keyboard", price: 1499 };

function withGst(o: Order): number {
  return Math.round(o.price * 1.18);
}

console.log(`${o_name(order)} costs Rs ${withGst(order)} with GST`);

function o_name(o: Order): string {
  return `#${o.id} ${o.product}`;
}
```

THE TYPES DISAPPEAR FROM THE COMPILED JAVASCRIPT terminal

```
tsc --strict --target es2022 order.ts
cat order.js
node order.js
```

TERMINAL

```
$ tsc --strict --target es2022 order.ts
$ cat order.js
"use strict";
const order = { id: 101, product: "Keyboard", price: 1499 };
function withGst(o) {
    return Math.round(o.price * 1.18);
}
console.log(`${o_name(order)} costs Rs ${withGst(order)} with GST`);
function o_name(o) {
    return `#${o.id} ${o.product}`;
}
$ node order.js
#101 Keyboard costs Rs 1769 with GST
```

The `interface Order` and all the `: Order`, `: number` and `: string` annotations are gone; what remains is exactly the JavaScript you would have written by hand. Because the types do not exist at run time, code such as `if (x instanceof Order)` is impossible for an interface: you check values with JavaScript (`typeof`, `in`, `Array.isArray`), which Module 8 covers.

TYPES ARE ERASED: DATA FROM OUTSIDE IS NOT CHECKED AUTOMATICALLY main.ts

```
interface User { name: string; age: number; }

// JSON from the outside world: TypeScript must trust what we say about it
const raw = '{"name": "Ravi", "age": "twenty"}';
const user = JSON.parse(raw) as User;     // a promise to the compiler, not a check

console.log(typeof user.age);             // still a string at run time
console.log(user.age + 1);                // string concatenation, not addition
```

OUTPUT

```
string
twenty1
```

WATCH OUT

A type annotation is a **claim** that the compiler checks against your code, not against real data. Values that come from JSON, forms, files or other programs must be validated at run time.

1.2 History of TypeScript

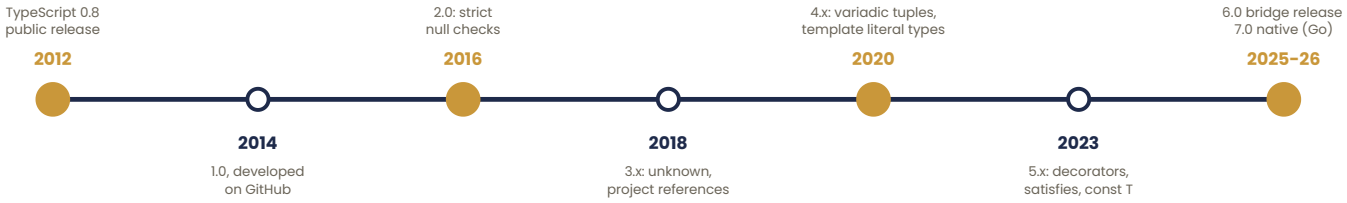
1.2.1 Created at Microsoft by Anders Hejlsberg (also the creator of C#), released in 2012

By 2010 teams at Microsoft and elsewhere were writing very large JavaScript applications (Outlook on the web, Office web apps, Bing Maps) and found them hard to maintain: renaming a function or changing an object's shape could break code anywhere, and editors could not offer reliable autocompletion. A team led by **Anders Hejlsberg**, the designer of Turbo Pascal, Delphi and C#, built a typed superset that would compile to standard JavaScript and stay close to the ECMAScript standard. TypeScript 0.8 was released publicly in **October 2012**, and version 1.0 followed in 2014.

1.2.2 Open source on GitHub

TypeScript has been open source (Apache 2.0 licence) from the start, and since 2014 it has been developed in the open on GitHub (`microsoft/TypeScript`), with public issues, design notes and roadmaps. New versions are released roughly every three months, with a beta, a release candidate and a final release, and they are installed from npm.

1.2.3 Major milestones



MILESTONES IN THE HISTORY OF TYPESCRIPT

Version	Year	Highlights
2.0	2016	**strict null checks**: `null` and `undefined` become separate types; never; readonly; control-flow analysis
2.1 to 2.8	2016–18	`keyof`, mapped types, conditional types and `infer`, the `strict` flag
3.x	2018–20	the `unknown` type, project references and `tsc --build`, tuples in rest parameters, optional chaining and nullish coalescing, `import type`
4.x	2020–22	variadic tuple types, labelled tuples, **template literal types**, key remapping in mapped types, recursive conditional types, `Awaited`
5.x	2023–25	**standard ECMAScript decorators**, the `satisfies` operator (4.9), **const type parameters**, `NoInfer`, inferred type predicates, `erasableSyntaxOnly`, faster builds
6.0	2026	a bridge release that modernises defaults and deprecates old options before the move to the native compiler
7.0	2026	the compiler ported to Go ("Project Corsa"): the same language, about 10 times faster

1.2.4 The native TypeScript compiler written in Go (TypeScript 7, overview)

Until version 6, the TypeScript compiler was itself written in TypeScript and ran on Node.js. In 2025 the team announced a **native port to Go** (code-named Corsa, available first as the `@typescript/native-preview` package with the `tsgo` command). Go compiles to machine code and makes it easy to run work in parallel across CPU cores, so type checking large projects becomes about **10 times faster**, editor responsiveness improves and memory use drops. The language itself does not change: the same code and the same `tsconfig.json` work with both compilers. TypeScript 6.0 is the last version of the JavaScript-based compiler and prepares projects for 7.0 by deprecating rarely used options.

1.3 Why do we need TypeScript?

1.3.1 Catching bugs at compile time instead of in production

Studies of open-source projects have estimated that a static type system would have caught around 15% of the bugs that reached public repositories, and teams that adopt TypeScript consistently report fewer "undefined is not a function" style errors. The earlier a bug is found, the cheaper it is to fix: a red underline in the editor costs seconds, a failing build costs minutes, a production incident costs hours and reputation.



THE RELATIVE COST OF FIXING A BUG GROWS THE LATER IT IS FOUND (ILLUSTRATIVE)

TYPICAL JAVASCRIPT BUGS THAT TYPESCRIPT REPORTS IMMEDIATELY

main.ts

```ts
type Student = { name: string; marks: number[]; email?: string };

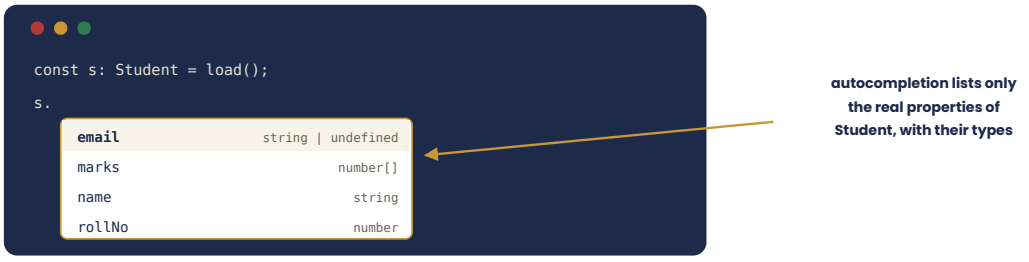
function report(s: Student) {
  const best = Math.max(...s.marks);
  console.log(s.nmae + " scored " + best);          // typo in a property name
  console.log(s.email.toLowerCase());               // email may be missing
  return best > "90";                               // comparing a number with a string
}
```

COMPILER ERRORS (tsc --strict)

```
main.ts(5,17): error TS2339: Property 'nmae' does not exist on type 'Student'.
main.ts(6,15): error TS18048: 's.email' is possibly 'undefined'.
main.ts(7,10): error TS2365: Operator '>' cannot be applied to types 'number' and 'string'.
```

1.3.2 Better editor support: autocompletion, refactoring, go-to-definition

The type checker also powers the editor. Because it knows the type of every expression, VS Code and other editors can offer **accurate autocompletion** (type `student.` and see exactly its properties), show **documentation and types on hover**, **go to the definition** of any function or type, **find all references**, and **rename** a symbol everywhere it is used. This works even in plain JavaScript files, because VS Code runs the TypeScript language service behind the scenes, but it is far more precise with real type annotations.



TYPES LET THE EDITOR COMPLETE CODE ACCURATELY

1.3.3 Self-documenting code

A function signature such as `function sendInvoice(order: Order, email: string, options?: { cc?: string[] }): Promise<boolean>` tells every reader what the function expects and returns, without opening its body or a separate document. Unlike comments, these "docs" cannot go out of date: if the code changes and the types do not match, the build fails.

1.3.4 Safer refactoring in large codebases

Changing the shape of data is one of the riskiest operations in a large JavaScript codebase: you must find every place that reads or writes the old property, and a search for a common word like `name` finds hundreds of unrelated matches. With TypeScript you change the type definition, and the compiler lists **every** place that no longer fits. Renaming with the editor's "Rename Symbol" command updates only the real references.

AFTER RENAMING A PROPERTY, THE COMPILER LISTS EVERY PLACE THAT MUST CHANGE main.ts

```
interface Product {
  id: number;
  title: string;          // renamed from "name" to "title"
  price: number;
}

function label(p: Product) { return `${p.name}: Rs ${p.price}`; }
function search(list: Product[], q: string) { return list.filter((p) => p.name.includes(q)); }
const pen: Product = { id: 1, name: "Pen", price: 20 };
```

COMPILER ERRORS (tsc --strict)

```
main.ts(7,42): error TS2339: Property 'name' does not exist on type 'Product'.
main.ts(8,75): error TS2339: Property 'name' does not exist on type 'Product'.
main.ts(9,31): error TS2353: Object literal may only specify known properties, and 'name' does not
exist in type 'Product'.
```

1.3.5 Team productivity and maintainability

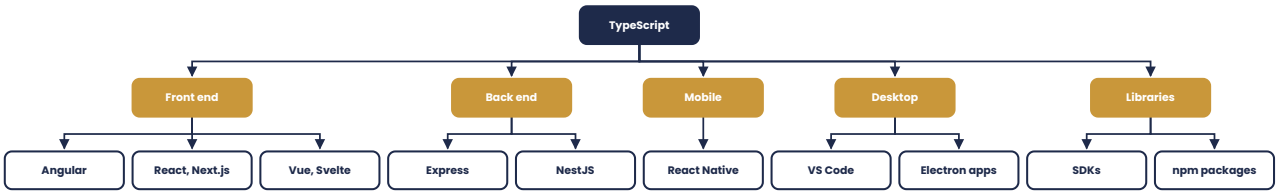
In a team, types act as a **contract** between parts of the system written by different people: the front-end developer knows exactly what shape the API returns, the author of a utility knows how it may be called, and new team members can explore an unfamiliar codebase with the editor. Code reviews focus on logic rather than on spotting typos, and onboarding is faster.

1.3.6 Industry standard for modern front-end and Node.js projects

TypeScript is now the default choice for professional JavaScript development. It is consistently among the most used and most loved languages in developer surveys, and by 2025 it had become the most used language on GitHub by number of contributors. Most new front-end projects (React, Angular, Vue,

Svelte) and Node.js back ends start in TypeScript, the major frameworks are written in it, and job postings for front-end and full-stack roles usually list it as a requirement.

1.4 Where is TypeScript used?



WHERE TYPESCRIPT IS USED

1.4.1 Angular (built with TypeScript)

Google's **Angular** framework is written in TypeScript and requires it: components, services and modules are TypeScript classes, and Angular's decorators (`@Component`, `@Injectable`, `@Input`, Module 16) and dependency injection rely on its type information. Angular was the first major framework to adopt TypeScript (2016) and helped make it mainstream.

1.4.2 React, Next.js, Vue and Svelte projects

React components are typed with TypeScript props (Module 18); `create-vite` and **Next.js** generate TypeScript projects by default (`.tsx` files contain JSX plus types). **Vue 3** was rewritten in TypeScript and supports it in single-file components with `<script setup lang="ts">`, and **Svelte** and SvelteKit support it out of the box.

1.4.3 Node.js back ends (Express, NestJS)

On the server, TypeScript types request bodies, database rows and configuration. **Express** apps use the `@types/express` declarations, and **NestJS**, a structured framework inspired by Angular, is written in TypeScript and uses decorators for controllers and dependency injection. ORMs such as **Prisma** and **Drizzle** generate TypeScript types from the database schema, so a query result has an exact type.

1.4.4 Mobile apps with React Native

React Native builds native Android and iOS apps from React components, and its templates (including Expo) use TypeScript by default. The same types can be shared with the web front end and the back end.

1.4.5 Desktop apps (VS Code itself is written in TypeScript)

Visual Studio Code, one of the most popular code editors in the world, is written in TypeScript and runs on **Electron** (Chromium plus Node.js). Many other desktop apps built with Electron or Tauri, such as Slack, Discord, Figma's desktop app and Microsoft Teams, use TypeScript for much of their code.

1.4.6 Libraries and SDKs

Library authors publish type declarations (`.d.ts` files, Module 11) with their packages, or write the whole library in TypeScript, so that users get autocompletion and type checking. Popular examples include RxJS, Zod, Prisma, tRPC, Axios, date-fns, the AWS SDK, Stripe, and the official SDKs of AI providers such as Anthropic and OpenAI. Even libraries written in JavaScript usually have community-maintained types on DefinitelyTyped (7.2.2).

1.5 TypeScript vs JavaScript

Aspect	JavaScript	TypeScript
Typing	dynamic: types checked while the code runs	static: types checked before the code runs (and erased)
Errors found	at run time, when the faulty line executes	in the editor and during the build
Build step	none needed (run .js directly)	compile with tsc, or strip types (Node.js, tsx, bundlers)
Runtime behaviour	the reference	identical: the output is JavaScript
Editor support	good (inferred)	excellent: precise completion, refactoring, navigation
Learning curve	lower	JavaScript plus the type system
File extensions	.js, .mjs, .cjs, .jsx	.ts, .mts, .cts, .tsx (and .d.ts declarations)
Best for	small scripts, quick prototypes, learning	medium and large apps, teams, libraries, long-lived code

1.5.1 Static vs dynamic typing

In a **dynamically typed** language such as JavaScript or Python, types belong to **values**: a variable can hold a number now and a string later, and an operation is checked only when it runs. In a **statically typed** language such as Java, C# or TypeScript, types also belong to **variables and expressions**, and a checker verifies before the program runs that every operation is used with compatible types. TypeScript's type system is also **structural** (5.7): two types are compatible if they have the same shape, regardless of their names.

JAVASCRIPT (DYNAMIC): PROBLEMS APPEAR ONLY AS WRONG RESULTS AT RUN TIME

main.js

```
let total = 100;
total = "one hundred";          // allowed: the variable just holds a different value
console.log(total * 2);         // NaN, discovered only when this line runs

function area(width, height) { return width * height; }
console.log(area(5));           // NaN: height is undefined, no warning
```

OUTPUT

```
NaN
NaN
```

THE SAME CODE AS TYPESCRIPT (STATIC): BOTH PROBLEMS ARE REPORTED BEFORE RUNNING

main.ts

```
let total = 100;
total = "one hundred";

function area(width: number, height: number) { return width * height; }
console.log(area(5));
```

COMPILER ERRORS (tsc --strict)

```
main.ts(2,1): error TS2322: Type 'string' is not assignable to type 'number'.
main.ts(5,13): error TS2554: Expected 2 arguments, but got 1.
```

1.5.2 Compile step

JavaScript files run directly. TypeScript needs a step that turns `.ts` into `.js`: the official compiler `tsc` (which also type-checks), a bundler such as Vite or esbuild that only strips the types very quickly, or a runtime that strips types itself (Node.js 22.18+ and 23.6+, Deno, Bun). In practice the step is automatic: your editor checks types as you type, and `npm run dev` or `npm run build` handles compilation (Module 2 and Module 17).

1.5.3 Same runtime behaviour

TypeScript never changes what your code does at run time: the emitted JavaScript behaves exactly like the source without its types. That means everything about JavaScript's behaviour (the event loop, closures, `this`, prototypes, coercion rules) still applies, and TypeScript code can call any JavaScript library and be called from JavaScript. A few older TypeScript features do generate code (enums, namespaces, parameter properties and legacy decorators); modern guidance, and the `erasableSyntaxOnly` option, prefers syntax that can simply be erased (6.4.5).

1.5.4 When plain JavaScript is still fine

- small scripts and one-off automation that fit on a screen or two;
- quick prototypes and experiments where speed matters more than safety;
- learning programming and JavaScript itself (learn JavaScript first, then add types);
- projects with no build step where you do not want one (you can still get type checking with JSDoc comments and `// @ts-check`, 7.3).

For anything that will grow, live for months, be shared with a team or be published as a library, TypeScript usually pays for itself quickly.

1.6 Prerequisites recap

1.6.1 JavaScript concepts needed: variables, functions, objects, arrays, classes, modules, promises

TypeScript adds types to JavaScript, so you need solid JavaScript first. The program below uses every concept this book assumes you already know. If any line is unclear, revise it before Module 3 (the LBF JavaScript notes cover all of them).

THE JAVASCRIPT YOU SHOULD BE COMFORTABLE WITH BEFORE LEARNING TYPESCRIPT

main.js

```
// variables and template literals
const course = "TypeScript";
let seats = 30;

// functions: declarations, arrow functions, default and rest parameters
function enrol(name, ...extra) { seats -= 1 + extra.length; return `${name} enrolled in ${course}`; }
const discount = (fee, pct = 10) => fee - (fee * pct) / 100;

// objects, arrays, destructuring and array methods
const students = [{ name: "Asha", marks: 91 }, { name: "Ravi", marks: 76 }, { name: "Meera", marks: 84
}];
const toppers = students.filter((s) => s.marks > 80).map(({ name }) => name);

// classes
class Batch {
  constructor(title, size) { this.title = title; this.size = size; }
  describe() { return `${this.title} (${this.size} learners)`; }
}

// promises and async/await
const fetchFee = () => new Promise((resolve) => setTimeout(() => resolve(14999), 50));

async function main() {
  console.log(enrol("Kiran", "Zoya"), "| seats left:", seats);
  console.log("toppers:", toppers, "| fee after discount:", discount(await fetchFee()));
  console.log(new Batch("Weekend batch", 24).describe());
}
main();
```

OUTPUT

```
Kiran enrolled in TypeScript | seats left: 28
toppers: [ 'Asha', 'Meera' ] | fee after discount: 13499.1
Weekend batch (24 learners)
```

Concept	Why it matters in TypeScript
variables (let, const)	TypeScript infers different types for let and const (3.2.3)
functions, arrows, default/rest parameters	parameters and return values get types (Module 4)
objects, arrays, destructuring	object types, interfaces, tuples and array types (Modules 3 and 5)
classes and this	access modifiers, abstract classes, implements (Module 9)
modules (import/export)	type-only imports, declaration files, module resolution (Module 11)
promises, async/await, fetch	Promise<T>, typing API responses (Modules 4 and 13)

TypeScript is a statically typed superset of JavaScript: JavaScript with syntax for types. Every JavaScript program is valid TypeScript; the compiler checks types before the code runs and then erases them, so the output is plain JavaScript with identical behaviour and there is no runtime type checking. It was created at Microsoft by Anders Hejlsberg (2012), is open source, gained strict null checks (2.0), unknown (3.x), template literal and variadic tuple types (4.x), standard decorators, satisfies and const type parameters (5.x), and is moving to a native Go compiler (7.0). It catches bugs early, powers editor features, documents code, makes refactoring safe and is the industry standard for front ends (Angular, React, Vue, Svelte), Node.js back ends, mobile, desktop apps and libraries.

PRACTICE CHECKPOINT

Explain in your own words what **"superset of JavaScript"** and **"types are erased"** mean, and list **three bugs** TypeScript would catch that JavaScript would not.

Sample answer. "Superset" means that TypeScript contains all of JavaScript and only adds to it: any JavaScript code is also TypeScript, and the extra syntax is for describing types. "Types are erased" means that the compiler uses the types only to check the code and then removes them, producing plain JavaScript; at run time no types exist, so they cost nothing but also cannot check data that arrives while the program runs. The program below demonstrates three bugs that TypeScript catches (a misspelled property, a missing argument and a possibly undefined value), each of which JavaScript would only reveal at run time, if at all.

SAMPLE SOLUTION: THREE BUGS CAUGHT AT COMPILE TIME main.ts

```
interface Student { name: string; email?: string; marks: number[] }
const s: Student = { name: "Asha", marks: [91, 84] };

console.log(s.nmae);                                    // 1. misspelled property
function average(marks: number[], round: boolean) { return 0; }
average(s.marks);                                       // 2. missing argument
console.log(s.email.toUpperCase());                     // 3. email may be undefined
```

COMPILER ERRORS (tsc --strict)

```
main.ts(4,15): error TS2339: Property 'nmae' does not exist on type 'Student'.
main.ts(6,1): error TS2554: Expected 2 arguments, but got 1.
main.ts(7,13): error TS18048: 's.email' is possibly 'undefined'.
```

MODULE 02 · BEGINNER

Setup and the TypeScript compiler

Minimum duration **1 day** | Level **Beginner**

By the end of this module you can install TypeScript, compile and run TypeScript programs, and understand the basic compiler configuration. It covers Node.js and npm, global and local installs, editor support in VS Code and the online Playground, a first program compiled with tsc and run with Node.js, running TypeScript directly with tsx, Node.js type stripping, Deno and Bun, the essential tsconfig.json options (target, module, rootDir, outDir, strict, include and exclude, watch mode), and how to read compile errors.

IN THIS MODULE

2.1 Installing TypeScript
2.2 Editor setup
2.3 First program
2.4 Running TypeScript directly
2.5 tsconfig.json basics
2.6 Compile errors

2.1 Installing TypeScript

2.1.1 Node.js and npm

The TypeScript compiler is itself a program distributed on **npm**, the package registry of the JavaScript world, and it runs on **Node.js**. Install the current **LTS** (long-term support) version of Node.js from nodejs.org (or with a version manager such as `nvm`, `fnm` or `volta`); it includes npm and `npx`. Check both in a terminal:

CHECKING NODE.JS AND NPM — terminal

```
node -v
npm -v
```

```
TERMINAL
$ node -v
v22.22.2
$ npm -v
10.9.7
```

Node.js 22 or later is recommended: it runs modern JavaScript, supports ES modules and can even run simple TypeScript files directly (2.4.2).

2.1.2 Global vs local install: npm install -D typescript

You can install TypeScript in two ways. A **global** install (`npm install -g typescript`) puts a `tsc` command on your PATH for every folder: convenient for experiments, but every project then shares one version. The recommended way for real projects is a **local** install as a **development dependency**: `npm install -D typescript` (short for `--save-dev`). The exact version is recorded in `package.json` and `package-lock.json`, so every developer and the CI server use the same compiler, and different projects can use different versions.

CREATING A PROJECT WITH A LOCAL TYPESCRIPT INSTALL

terminal

```
mkdir marks-app && cd marks-app
npm init -y
npm install -D typescript @types/node
```

TERMINAL

```
Wrote to /home/ravi/marks-app/package.json
added 3 packages, and audited 4 packages in 2s
found 0 vulnerabilities
```

`@types/node` adds type declarations for Node.js APIs such as `process`, `fs` and `path` (7.2.2).

PACKAGE.JSON AFTER THE INSTALL, WITH USEFUL SCRIPTS ADDED

JavaScript

```
{
  "name": "marks-app",
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "build": "tsc",
    "dev": "tsc --watch",
    "start": "node dist/index.js",
    "typecheck": "tsc --noEmit"
  },
  "devDependencies": {
    "@types/node": "^24.0.0",
    "typescript": "^6.0.0"
  }
}
```

A locally installed compiler lives in `node_modules/.bin`. Run it through `npx tsc` or an npm script (`npm run build`), never by assuming a global `tsc`. Devdependencies are not needed in production because only the compiled JavaScript is deployed.

	Global install	Local install (recommended)
command	`npm install -g typescript`	`npm install -D typescript`
run with	`tsc`	`npx tsc` or `npm run build`
version	one for the whole machine	pinned per project in package.json
best for	quick experiments, learning	every real project, teams, CI

2.1.3 Checking the version: npx tsc -v

CHECKING THE COMPILER VERSION AND HELP

terminal

```
npx tsc -v
npx tsc --help | head -n 12
```

TERMINAL

```
$ npx tsc -v
Version 6.0.3
$ npx tsc --help | head -n 12
tsc: The TypeScript Compiler - Version 6.0.3

COMMON COMMANDS

  tsc
  Compiles the current project (tsconfig.json in the working directory.)

  tsc app.ts util.ts
  Ignoring tsconfig.json, compiles the specified files with default compiler options.

  tsc -b
  Build a composite project in the working directory.
```

This book uses TypeScript 6. Everything taught here also works in TypeScript 5.x unless a section says otherwise, and the code is ready for the native TypeScript 7 compiler (1.2.4).

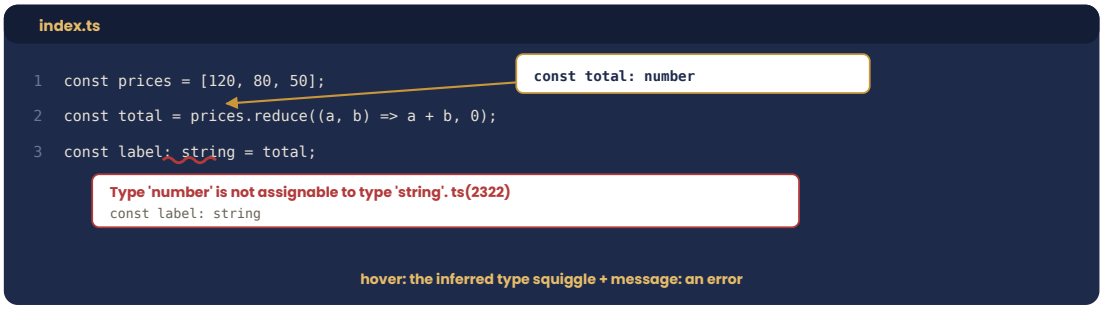
2.2 Editor setup

2.2.1 VS Code's built-in TypeScript support

Visual Studio Code supports TypeScript out of the box: no extension is needed. It runs the **TypeScript language service** (tsserver) in the background, which type-checks open files as you type and powers completion, navigation and refactoring. By default VS Code uses its own bundled TypeScript version; in a project it is better to use the workspace version so the editor and the build agree (command palette: *TypeScript: Select TypeScript Version → Use Workspace Version*). Useful commands: **F12** go to definition, **Shift+F12** find references, **F2** rename symbol, **Ctrl+.** quick fixes (add missing import, fix spelling, implement interface), *Organize Imports*, and *TypeScript: Restart TS Server* if the editor gets confused. Recommended extensions: ESLint, Prettier and Error Lens (shows errors inline).

2.2.2 Hover information and error squiggles

Hover over any variable, function or property to see its **inferred type** and documentation; this is the fastest way to learn what TypeScript thinks a value is. Errors appear as **red squiggles** under the faulty code, warnings and unused code are dimmed, and the **Problems** panel (Ctrl+Shift+M) lists every error in open files with the same messages and codes that `tsc` prints.



HOVER SHOWS INFERRED TYPES; RED SQUIGGLES SHOW ERRORS WITH THEIR TS CODES

2.2.3 The TypeScript Playground (online)

The **TypeScript Playground** at `typescriptlang.org/play` runs the compiler in your browser. Type code on the left and immediately see errors, the generated JavaScript (the *.JS* tab), the generated declarations (*.D.TS*), and the output of `console.log` (*Logs*). You can switch the TypeScript version (including nightly builds), change compiler options in the *TS Config* menu, and share a link to your code. It is ideal for experimenting with types, asking questions and comparing how options change the output.

2.3 First program

2.3.1 Writing hello.ts

Create a folder, open it in VS Code and add a file named `hello.ts`. TypeScript source files use the `.ts` extension (`.tsx` when they contain JSX).

2.3.2 Compiling with tsc hello.ts

2.3.3 Reading the generated hello.js

2.3.4 Running with Node.js

HELLO.TS — TypeScript

```typescript
function greet(name: string, batch: number): string {
  return `Hello, ${name}! Welcome to batch ${batch} at Learn Being Forward.`;
}

const learners: string[] = ["Asha", "Ravi"];
for (const learner of learners) {
  console.log(greet(learner, 12));
}
```

COMPILE, INSPECT THE JAVASCRIPT, THEN RUN IT

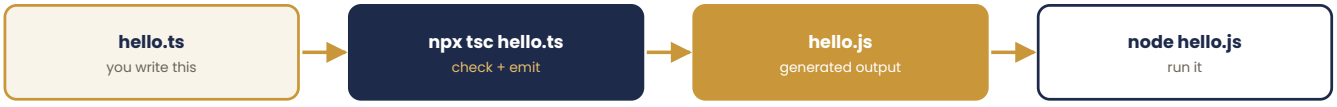
terminal

```
npx tsc hello.ts
ls
cat hello.js
node hello.js
```

TERMINAL

```
$ npx tsc hello.ts
$ ls
hello.js
hello.ts
node_modules
$ cat hello.js
"use strict";
function greet(name, batch) {
    return `Hello, ${name}! Welcome to batch ${batch} at Learn Being Forward.`;
}
const learners = ["Asha", "Ravi"];
for (const learner of learners) {
    console.log(greet(learner, 12));
}
$ node hello.js
Hello, Asha! Welcome to batch 12 at Learn Being Forward.
Hello, Ravi! Welcome to batch 12 at Learn Being Forward.
```

Three things happened. `tsc hello.ts` type-checked the file and wrote `hello.js` next to it. The JavaScript is the same code with the annotations (`: string`, `: number`, `: string[]`) removed; everything else, even the formatting, is preserved as far as possible. Finally **Node.js runs the JavaScript**, not the TypeScript. When you compile a single file like this, tsc uses its default options; real projects put options in `tsconfig.json` (2.5).



THE COMPILE-AND-RUN CYCLE

NOTE

Never edit the generated `.js` files: they are overwritten every time you compile. Edit the `.ts` source, and in projects keep the output in a separate folder such as `dist` that is listed in `.gitignore`.

2.4 Running TypeScript directly

Compiling to a file and then running it is the production workflow. During development it is faster to run a `.ts` file in one step. Note that these tools **do not type-check**: they only remove the types and run the code. Keep the editor (or `tsc --noEmit`) for checking.

2.4.1 tsx and ts-node

tsx ("TypeScript execute", `npm install -D tsx`) runs TypeScript files with esbuild under the hood, is very fast, supports both ES modules and CommonJS, and has a watch mode (`tsx watch src/index.ts`) that restarts the program on every save. **ts-node** is the older tool; it can type-check while running but is slower and harder to configure with ES modules, so most new projects use tsx.

HELLO.TS

TypeScript

```typescript
const courses: Record<string, number> = { TypeScript: 43, Python: 70 };
for (const [name, days] of Object.entries(courses)) {
  console.log(`${name.padEnd(10)} ${days} days`);
}
```

RUNNING A TYPESCRIPT FILE IN ONE STEP WITH TSX

terminal

```
npx tsx hello.ts
```

TERMINAL

```
$ npx tsx hello.ts
TypeScript 43 days
Python     70 days
```

2.4.2 Native type stripping in Node.js

Since **Node.js 22.18** (and 23.6), Node.js can run `.ts` files directly: `node app.ts`. It replaces the type annotations with spaces ("type stripping") and runs the result, with no separate tool. It supports only **erasable** syntax, meaning TypeScript that can be removed without generating code: annotations, interfaces, type aliases, generics, `as` and `satisfies`. Features that generate JavaScript (enums, namespaces, parameter properties) are rejected, and the TypeScript option `erasableSyntaxOnly` (6.4.5, 17.5) makes tsc report them too. Imports must use the real file extension (`import { add } from "./math.ts"`), and like tsx, Node.js performs no type checking.

PRICES.TS

TypeScript

```typescript
interface Item {
  name: string;
  price: number;
}

const cart: Item[] = [
  { name: "Notebook", price: 60 },
  { name: "Geometry box", price: 145 },
];

const total: number = cart.reduce((sum, item) => sum + item.price, 0);
console.log(`Items: ${cart.length}, total: Rs ${total}`);
```

NODE.JS RUNS TYPESCRIPT DIRECTLY BY STRIPPING THE TYPES

terminal

```
node prices.ts
```

TERMINAL

```
$ node prices.ts
Items: 2, total: Rs 205
```

2.4.3 Deno and Bun

Deno (by the original creator of Node.js) and **Bun** are alternative JavaScript runtimes that run TypeScript natively: `deno run app.ts` and `bun app.ts`. Deno also type-checks with `deno check` and includes a formatter, linter and test runner; Bun focuses on speed and includes a package manager, bundler and test runner. Both are compatible with most npm packages.

Way to run	Command	Type-checks?	Typical use
tsc then node	`npx tsc && node dist/index.js`	yes (tsc)	production builds
tsx	`npx tsx watch src/index.ts`	no	development of Node.js apps
Node.js type stripping	`node src/index.ts`	no	scripts, simple apps (erasable syntax only)
Deno / Bun	`deno run app.ts`, `bun app.ts`	Deno: deno check	projects using those runtimes
Vite / bundlers	`npm run dev`	no (run tsc --noEmit)	front-end apps (17.3)

2.5 tsconfig.json basics

A `tsconfig.json` file marks the root of a TypeScript project and holds its compiler options. When you run `tsc` **with no file names**, it finds `tsconfig.json` in the current folder (or a parent), and compiles the files it lists with its options. Editors read the same file, so the editor and the build behave identically.

2.5.1 tsc --init

`npx tsc --init` creates a starter `tsconfig.json` with recommended settings and comments explaining the most common options. The file is JSON with comments (JSONC), so comments and trailing commas are allowed.

GENERATING A STARTER TSCONFIG.JSON (TYPESCRIPT 6)

terminal

```
npx tsc --init
cat tsconfig.json
```

TERMINAL

```
$ npx tsc --init

Created a new tsconfig.json

You can learn more at https://aka.ms/tsconfig
$ cat tsconfig.json
{
  // Visit https://aka.ms/tsconfig to read more about this file
  "compilerOptions": {
    // File Layout
    // "rootDir": "./src",
    // "outDir": "./dist",

    // Environment Settings
    // See also https://aka.ms/tsconfig/module
    "module": "nodenext",
    "target": "esnext",
    "types": [],
    // For nodejs:
    // "lib": ["esnext"],
    // "types": ["node"],
    // and npm install -D @types/node

    // Other Outputs
    "sourceMap": true,
    "declaration": true,
    "declarationMap": true,

    // Stricter Typechecking Options
    "noUncheckedIndexedAccess": true,
    "exactOptionalPropertyTypes": true,

    // Style Options
    // "noImplicitReturns": true,
    // "noImplicitOverride": true,
    // "noUnusedLocals": true,
    // "noUnusedParameters": true,
    // "noFallthroughCasesInSwitch": true,
    // "noPropertyAccessFromIndexSignature": true,

    // Recommended Options
    "strict": true,
    "jsx": "react-jsx",
    "verbatimModuleSyntax": true,
    "isolatedModules": true,
    "noUncheckedSideEffectImports": true,
    "moduleDetection": "force",
    "skipLibCheck": true,
  }
}
```

The generated defaults are modern and strict: `strict` plus two extra checks (`noUncheckedIndexedAccess`, `exactOptionalPropertyTypes`, explained in 5.5.3 and 17.1.1), ES-module output for Node.js (`"module": "nodenext"`), the newest JavaScript as target, and settings that keep code compatible with fast single-file tools (`verbatimModuleSyntax`, `isolatedModules`). Uncomment `rootDir` and `outDir` to separate sources from output.

2.5.2 target and module

`target` chooses the JavaScript version that tsc outputs. Newer syntax that the target does not support is rewritten ("down-levelled"); for example, with `"target": "es5"` arrow functions become `function` expressions and classes become constructor functions. Modern Node.js and browsers support ES2022+, so `es2022`, `es2023` or `esnext` are usual choices, and the output stays close to your source. `module` chooses the module format of the output: `nodenext` (or `node20`) for Node.js, which outputs ES modules or CommonJS according to `package.json`'s `"type"`; `esnext` or `preserve` for code that a bundler such as Vite processes; `commonjs` for older Node.js projects. Module 11 covers module resolution in detail.

ORDER.TS

TypeScript

```
class Order {
  status = "new";
  constructor(public id: number, public coupon?: { code: string; pct?: number }) {}
  discount(): number {
    return this.coupon?.pct ?? 0;
  }
}

const order = new Order(7, { code: "LBF10", pct: 10 });
console.log(order.id, order.status, order.discount(), new Order(8).coupon?.code ?? "no coupon");
```

THE SAME SOURCE COMPILED FOR TWO TARGETS: NEWER SYNTAX IS REWRITTEN FOR OLDER TARGETS

terminal

```
npx tsc --target es2022 --outDir new order.ts
npx tsc --target es2017 --outDir old order.ts
cat new/order.js
cat old/order.js
node old/order.js
```

TERMINAL

```
$ npx tsc --target es2022 --outDir new order.ts
$ npx tsc --target es2017 --outDir old order.ts
$ cat new/order.js
"use strict";
class Order {
    id;
    coupon;
    status = "new";
    constructor(id, coupon) {
        this.id = id;
        this.coupon = coupon;
    }
    discount() {
        return this.coupon?.pct ?? 0;
    }
}
const order = new Order(7, { code: "LBF10", pct: 10 });
console.log(order.id, order.status, order.discount(), new Order(8).coupon?.code ?? "no coupon");
$ cat old/order.js
"use strict";
var _a, _b;
class Order {
    constructor(id, coupon) {
        this.id = id;
        this.coupon = coupon;
        this.status = "new";
    }
    discount() {
        var _a, _b;
        return (_b = (_a = this.coupon) === null || _a === void 0 ? void 0 : _a.pct) !== null && _b
!== void 0 ? _b : 0;
    }
}
const order = new Order(7, { code: "LBF10", pct: 10 });
console.log(order.id, order.status, order.discount(), (_b = (_a = new Order(8).coupon) === null || _a
=== void 0 ? void 0 : _a.code) !== null && _b !== void 0 ? _b : "no coupon");
$ node old/order.js
7 new 10 no coupon
```

With `es2022` the output is the source minus types (the parameter properties `public id` become assignments, 9.2.4). With `es2017`, optional chaining `?.` and nullish coalescing `??` (ES2020) are rewritten as explicit `null`/`undefined` checks with temporary variables, and the class field `status` moves into the constructor, because those features are newer than ES2017. Note that tsc rewrites **syntax** only; it does not add missing built-in methods such as `Array.prototype.at` (that needs a polyfill), and the `lib` option (17.1.2) controls which built-ins the type checker assumes exist.

2.5.3 rootDir and outDir

`rootDir` is the folder that contains your source files (usually `src`) and **`outDir`** is where compiled files go (usually `dist` or `build`). tsc mirrors the folder structure: `src/utils/math.ts` becomes

`dist/utils/math.js`. Keeping output separate means you never confuse generated files with sources and can delete `dist` at any time.

2.5.4 strict mode

`"strict": true` switches on a whole family of checks at once: `noImplicitAny` (a value whose type cannot be inferred must be annotated), `strictNullChecks` (`null` and `undefined` are separate types, 3.3.3), `strictFunctionTypes`, `strictBindCallApply`, `strictPropertyInitialization` (class fields must be initialised, 9.1.2), `noImplicitThis`, `useUnknownInCatchVariables` (13.3.4), `alwaysStrict` and newer checks as they are added. Without strict mode TypeScript misses many of the bugs it exists to catch. **Always enable strict in new projects**; in TypeScript 6 it is on by default. When migrating JavaScript you can start looser and tighten over time (19.1.2).

THREE CHECKS THAT STRICT MODE ADDS

main.ts

```ts
function average(marks) {                 // noImplicitAny
  return marks.reduce((a, b) => a + b) / marks.length;
}

const names = ["Asha", "Ravi"];
const found = names.find((n) => n.startsWith("Z"));
console.log(found.length);                // strictNullChecks: found may be undefined

class Batch {
  title: string;                          // strictPropertyInitialization
  size = 0;
}
```

COMPILER ERRORS (tsc --strict)

```
main.ts(1,18): error TS7006: Parameter 'marks' implicitly has an 'any' type.
main.ts(2,24): error TS7006: Parameter 'a' implicitly has an 'any' type.
main.ts(2,27): error TS7006: Parameter 'b' implicitly has an 'any' type.
main.ts(7,13): error TS18048: 'found' is possibly 'undefined'.
main.ts(10,3): error TS2564: Property 'title' has no initializer and is not definitely assigned in the
constructor.
```

2.5.5 include and exclude

`include` lists the files or glob patterns to compile (for example `["src"]` or `["src/**/*.ts"]`; `**` matches any folder depth). **`exclude`** removes paths from that set, such as test fixtures or generated code; `node_modules` and the `outDir` are excluded automatically. There is also **`files`**, an explicit list of individual files for very small projects. Files imported by included files are always compiled, even if they are outside `include`.

TSCONFIG.JSON JavaScript

```
{
  "compilerOptions": {
    "target": "es2022",
    "module": "nodenext",
    "rootDir": "src",
    "outDir": "dist",
    "strict": true,
    "types": ["node"],
    "skipLibCheck": true
  },
  "include": ["src"],
  "exclude": ["src/**/*.test.ts"]
}
```

SRC/INDEX.TS TypeScript

```
import { grade, average } from "./utils/grades.js";

const marks: number[] = [78, 91, 85, 66];
const avg = average(marks);
console.log(`Average ${avg.toFixed(1)}, grade ${grade(avg)}`);
```

SRC/UTILS/GRADES.TS TypeScript

```
export function average(values: number[]): number {
  return values.reduce((sum, v) => sum + v, 0) / values.length;
}

export function grade(mark: number): "A" | "B" | "C" | "D" {
  if (mark >= 85) return "A";
  if (mark >= 70) return "B";
  if (mark >= 55) return "C";
  return "D";
}
```

A SMALL PROJECT: SOURCES IN SRC, COMPILED OUTPUT IN DIST terminal

```
npx tsc
find dist -type f | sort
node dist/index.js
```

TERMINAL

```
$ npx tsc
$ find dist -type f | sort
dist/index.js
dist/utils/grades.js
$ node dist/index.js
Average 80.0, grade B
```

NOTE

With `"module": "nodenext"`, relative imports must include the extension of the **output** file: write `"./utils/grades.js"` in the `.ts` source, because that is the file Node.js will load. (With the `rewriteRelativeImportExtensions` option or with bundlers you may write `.ts`; see 11.2.)

2.5.6 Watch mode tsc --watch

`tsc --watch` (or `-w`) compiles the project and then keeps running, recompiling incrementally whenever a source file changes; it reports errors each time. Run it in one terminal and your program (for example `node --watch dist/index.js`, which restarts on change) in another. Add `--preserveWatchOutput` to stop it clearing the screen.

SRC/TAX.TS TypeScript

```
const rate: number = 18;
const price = 2500;
console.log(`GST at ${rate}%: Rs ${(price * rate) / 100}`);
```

WATCH MODE RECOMPILES ON EVERY SAVE AND REPORTS NEW ERRORS terminal

```
npx tsc --watch --preserveWatchOutput &
sleep 8
sed -i 's/const rate: number = 18/const rate: number = "18"/' src/tax.ts
sleep 6
sed -i 's/const rate: number = "18"/const rate: number = 18/' src/tax.ts
sleep 6
node dist/tax.js
```

```
TERMINAL
$ npx tsc --watch --preserveWatchOutput &
$ sleep 8
  [server log] 8:11:06 PM - Starting compilation in watch mode...
$ sed -i 's/const rate: number = 18/const rate: number = "18"/' src/tax.ts
$ sleep 6
  [server log] 8:11:15 PM - Found 0 errors. Watching for file changes.
  [server log] 8:11:15 PM - File change detected. Starting incremental compilation...
  [server log] src/tax.ts(1,7): error TS2322: Type 'string' is not assignable to type 'number'.
  [server log] 8:11:15 PM - Found 1 error. Watching for file changes.
$ sed -i 's/const rate: number = "18"/const rate: number = 18/' src/tax.ts
$ sleep 6
  [server log] 8:11:19 PM - File change detected. Starting incremental compilation...
  [server log] 8:11:19 PM - Found 0 errors. Watching for file changes.
$ node dist/tax.js
GST at 18%: Rs 450
```

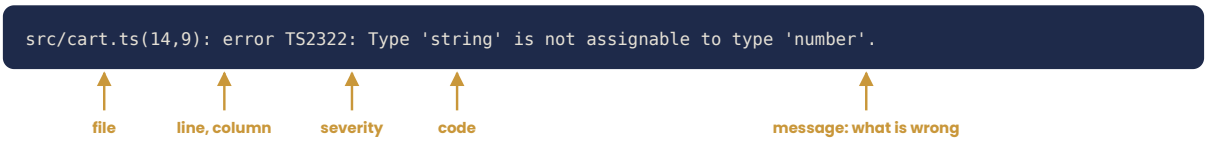
Lines marked [server log] are printed by the background `tsc --watch` process. The commands after it simulate editing the file twice: once introducing an error and once fixing it. Timestamps will differ on your machine.

2.6 Compile errors

2.6.1 Reading TypeScript error messages and codes

Every error has the same structure: **file(line,column)**, the word **error**, a code **TSxxxx** and a message. In a project build the location is printed as `file:line:column` with the offending code underlined, and a summary at the end. The code is useful for searching documentation and for suppressing a specific error knowingly (`// @ts-expect-error`, 7.3.3). Long messages are nested: the first line says what failed, and each indented line explains why, getting more specific (7.4.1). Always read the **first** error first; later errors are often consequences of it.

Anatomy of a TypeScript error



THE PARTS OF AN ERROR MESSAGE

Code	Message (shortened)	Usual cause
TS2322	Type X is not assignable to type Y	assigning or returning a value of the wrong type
TS2345	Argument of type X is not assignable to parameter of type Y	calling a function with the wrong argument type
TS2339	Property P does not exist on type T	typo, or the property is not part of the type
TS2554	Expected N arguments, but got M	missing or extra arguments
TS18048 / TS2531	X is possibly undefined / null	using a value that may be missing without a check
TS7006	Parameter implicitly has an any type	an unannotated parameter in strict mode
TS2304	Cannot find name X	missing import or declaration, or a typo
TS2307	Cannot find module X or its type declarations	package not installed, or missing @types package
TS2564	Property has no initializer and is not definitely assigned	class field not set in the constructor

TSCONFIG.JSON JavaScript

```
{ "compilerOptions": { "strict": true, "noEmit": true, "pretty": true, "types": [] }, "include":
["src"] }
```

SRC/CART.TS TypeScript

```
type CartItem = { name: string; price: number; qty: number };

const items: CartItem[] = [
  { name: "Pen", price: 20, qty: 3 },
  { name: "Ruler", price: 15, qty: 2 },
];

function total(list: CartItem[]): number {
  return list.reduce((sum, i) => sum + i.price * i.qty, 0);
}

const discount = "10";
let payable: number = total(items);
payable = payable - discount;
console.log(totl(items));
```

ERRORS IN A PROJECT BUILD, WITH THE CODE UNDERLINED (TSC --PRETTY) terminal

```
npx tsc --noEmit; echo "exit code: $?"
```

```
TERMINAL
$ npx tsc --noEmit; echo "exit code: $?"
src/cart.ts:14:21 - error TS2363: The right-hand side of an arithmetic operation must be of type
'any', 'number', 'bigint' or an enum type.

14 payable = payable - discount;
                       ~~~~~~~~

src/cart.ts:15:13 - error TS2552: Cannot find name 'totl'. Did you mean 'total'?

15 console.log(totl(items));
               ~~~~

  src/cart.ts:8:10
    8 function total(list: CartItem[]): number {
               ~~~~~
    'total' is declared here.


Found 2 errors in the same file, starting at: src/cart.ts:14

exit code: 2
```

The exit code is non-zero when there are errors, so build scripts and CI pipelines stop. `--noEmit` only type-checks, which is how projects that compile with another tool (Vite, esbuild, tsx) still verify their types.

2.6.2 noEmitOnError

By default tsc writes the JavaScript files **even when there are type errors**, because the errors are about types and the JavaScript can usually still run (this helps gradual migration, 1.1.3). In a real project you usually do not want broken code in `dist`: set `"noEmitOnError": true` and tsc writes nothing if any error is found.

SRC/APP.TS TypeScript

```
const seats: number = "thirty";
console.log(seats);
```

WITH NOEMITONERROR NOTHING IS WRITTEN WHILE ERRORS EXIST

terminal

```
npx tsc
ls dist 2>/dev/null || echo "dist was not created"
sed -i 's/"noEmitOnError": true/"noEmitOnError": false/' tsconfig.json
npx tsc
ls dist
```

TERMINAL

```
$ npx tsc
src/app.ts(1,7): error TS2322: Type 'string' is not assignable to type 'number'.
$ ls dist 2>/dev/null || echo "dist was not created"
dist was not created
$ sed -i 's/"noEmitOnError": true/"noEmitOnError": false/' tsconfig.json
$ npx tsc
src/app.ts(1,7): error TS2322: Type 'string' is not assignable to type 'number'.
$ ls dist
app.js
```

Install Node.js LTS (with npm), then add TypeScript to each project as a dev dependency (npm install -D typescript) and run it with npx tsc or npm scripts. VS Code supports TypeScript natively: hover for inferred types, red squiggles for errors, F12, F2 and quick fixes; the online Playground is great for experiments. tsc file.ts type-checks and writes plain JavaScript, which Node.js runs. tsx, Node.js type stripping (erasable syntax only), Deno and Bun run .ts files directly but do not type-check. tsconfig.json (tsc --init) holds the options: target (output JavaScript version), module (output module format), rootDir and outDir (src to dist), strict (always on), include and exclude; tsc --watch recompiles on change. Errors show file(line,col), a TS code and a message; read the first one first, use --noEmit to only check, and noEmitOnError to avoid writing output when there are errors.

PRACTICE CHECKPOINT

Create a project with **tsc --init**, set **outDir** to **dist** and **strict** to true, write a small program, and run it in **watch mode**.

Sample solution. The commands below create the project, adjust the generated configuration (rootDir and outDir uncommented; the starter file already has `"strict": true`), write a small fee calculator in `src`, build it once and run it. During development, keep `npx tsc --watch` running in one terminal and `node --watch dist/index.js` in another.

SRC/INDEX.TS

TypeScript

```
const fees: Record<string, number> = { TypeScript: 9999, Python: 14999, Java: 12999 };

function withGst(amount: number, rate = 18): number {
  return Math.round(amount * (1 + rate / 100));
}

for (const [course, fee] of Object.entries(fees)) {
  console.log(`${course.padEnd(11)} Rs ${fee} -> Rs ${withGst(fee)} incl. GST`);
}
```

SAMPLE SOLUTION: A STRICT PROJECT COMPILED FROM SRC TO DIST

terminal

```
npx tsc --init > /dev/null
sed -i 's|// "rootDir": "./src"|"rootDir": "./src"|; s|// "outDir": "./dist"|"outDir": "./dist"|'
tsconfig.json
grep -E '"(rootDir|outDir|strict|module|target)"' tsconfig.json
npx tsc
ls dist
node dist/index.js
```

TERMINAL

```
$ npx tsc --init > /dev/null
$ sed -i 's|// "rootDir": "./src"|"rootDir": "./src"|; s|// "outDir": "./dist"|"outDir": "./dist"|'
tsconfig.json
$ grep -E '"(rootDir|outDir|strict|module|target)"' tsconfig.json
    "rootDir": "./src",
    "outDir": "./dist",
    "module": "nodenext",
    "target": "esnext",
    "strict": true,
$ npx tsc
$ ls dist
index.d.ts
index.d.ts.map
index.js
index.js.map
$ node dist/index.js
TypeScript  Rs 9999 -> Rs 11799 incl. GST
Python      Rs 14999 -> Rs 17699 incl. GST
Java        Rs 12999 -> Rs 15339 incl. GST
```

MODULE 03 · BEGINNER

Basic types and type inference

Minimum duration **2 days** | Level **Beginner**

This module covers type annotations and every basic type TypeScript provides. You will learn the annotation syntax for variables, parameters and return values, how type inference works and when to rely on it, literal widening with let and const, the primitive types (string, number, boolean, bigint, symbol, null and undefined), arrays and readonly arrays, tuples with optional, rest and named elements, the special types any, unknown, void, never, object and {}, and type assertions with as and the non-null assertion operator.

IN THIS MODULE

3.1 Type annotations
3.2 Type inference
3.3 Primitive types
3.4 Arrays
3.5 Tuples
3.6 Special types
3.7 Type assertions

3.1 Type annotations

A **type annotation** tells TypeScript what type a variable, parameter or return value must have. It is written after the name, separated by a colon. Once a variable has a type, every assignment and every use of it is checked against that type.

3.1.1 Syntax let name: string = "Ravi";



THE PARTS OF AN ANNOTATED VARIABLE DECLARATION

ANNOTATING VARIABLES — main.ts

```
let studentName: string = "Ravi";
let marks: number = 87.5;
let isPassed: boolean = marks >= 40;
const course: string = "TypeScript";

studentName = "Asha";            // fine: still a string
console.log(studentName, marks, isPassed, course);

let score: number;               // declared now, assigned later
score = 92;
console.log(`score = ${score}`);
```

OUTPUT

```
Asha 87.5 true TypeScript
score = 92
```

ASSIGNING THE WRONG TYPE, AND USING A VARIABLE BEFORE IT IS ASSIGNED main.ts

```
let studentName: string = "Ravi";
studentName = 42;

let marks: number = "87";
let isPassed: boolean = "yes";

let score: number;
console.log(score);
```

COMPILER ERRORS (tsc --strict)

```
main.ts(2,1): error TS2322: Type 'number' is not assignable to type 'string'.
main.ts(4,5): error TS2322: Type 'string' is not assignable to type 'number'.
main.ts(5,5): error TS2322: Type 'string' is not assignable to type 'boolean'.
main.ts(8,13): error TS2454: Variable 'score' is used before being assigned.
```

3.1.2 Annotating variables, parameters and return values

The three places where annotations matter most are **function parameters** (TypeScript cannot guess what callers will pass, so in strict mode unannotated parameters are errors), **return values** (optional, but an explicit return type documents the function and catches mistakes inside it), and variables that are declared without a value.

PARAMETER AND RETURN-VALUE ANNOTATIONS main.ts

```
function percentage(obtained: number, total: number): number {
  return (obtained / total) * 100;
}

function describe(name: string, pct: number): string {
  return `${name} scored ${pct.toFixed(1)}%`;
}

const pct: number = percentage(436, 500);
console.log(describe("Meera", pct));
```

OUTPUT

```
Meera scored 87.2%
```

THE RETURN TYPE IS CHECKED INSIDE THE FUNCTION, THE PARAMETERS AT EVERY CALL main.ts

```
function percentage(obtained: number, total: number): number {
  const pct = (obtained / total) * 100;
  return pct.toFixed(1);                // returns a string, not a number
}

percentage(436);                        // missing argument
percentage("436", 500);                 // wrong argument type
```

COMPILER ERRORS (tsc --strict)

```
main.ts(3,3): error TS2322: Type 'string' is not assignable to type 'number'.
main.ts(6,1): error TS2554: Expected 2 arguments, but got 1.
main.ts(7,12): error TS2345: Argument of type 'string' is not assignable to parameter of type
'number'.
```

3.2 Type inference

3.2.1 How TypeScript infers types

You do not have to annotate everything. When a variable is initialised, TypeScript **infers** its type from the value: `let count = 5` gives `count` the type `number`, exactly as if you had written `let count: number = 5`. Inference also works for return values (from the `return` statements), array literals (from their elements), object literals (from their properties), and callback parameters (from where the callback is passed, called **contextual typing**). Hover in the editor to see what was inferred.

TYPES INFERRED FROM INITIAL VALUES, RETURNS AND CALLBACKS main.ts

```
let count = 5;                                  // number
let city = "Mysuru";                            // string
let scores = [78, 91, 85];                      // number[]
let student = { name: "Asha", age: 20 };        // { name: string; age: number }
let mixed = [1, "two", 3];                      // (string | number)[]

function double(n: number) { return n * 2; }    // return type inferred as number

scores.forEach((s) => console.log(s.toFixed(1)));   // s inferred as number from the array

console.log(typeof count, typeof city, Array.isArray(scores), student.name, mixed.length, double(21));
```

OUTPUT

```
78.0
91.0
85.0
number string true Asha 3 42
```

INFERRED TYPES ARE ENFORCED EXACTLY LIKE ANNOTATED ONES main.ts

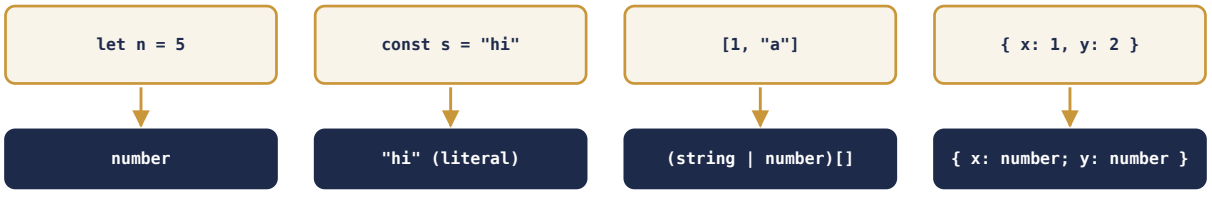
```
let count = 5;
count = "five";

let student = { name: "Asha", age: 20 };
student.email = "asha@lbf.in";

let scores = [78, 91, 85];
scores.push("100");
```

COMPILER ERRORS (tsc --strict)

```
main.ts(2,1): error TS2322: Type 'string' is not assignable to type 'number'.
main.ts(5,9): error TS2339: Property 'email' does not exist on type '{ name: string; age: number; }'.
main.ts(8,13): error TS2345: Argument of type 'string' is not assignable to parameter of type
'number'.
```



WHAT TYPESCRIPT INFERS FROM SOME COMMON INITIALISERS

3.2.2 When to annotate and when to rely on inference

A good rule used by most teams: **let inference work inside your code, and annotate the boundaries**. Annotate function parameters (required), the return types of exported or public functions (they document the API and stop accidental changes), variables declared without a value, and values where the inferred type is wider or narrower than you want (an empty array `[]`, an object you will add properties to). Do not annotate obvious local variables such as `const total = price * qty`: it adds noise without adding safety.

ANNOTATING AT THE BOUNDARIES, INFERRING INSIDE main.ts

```ts
// Annotate: parameters, exported return types, empty collections, delayed initialisation
export function averageOf(values: number[]): number {
  const sum = values.reduce((a, b) => a + b, 0);     // inferred: number (no annotation needed)
  return values.length ? sum / values.length : 0;
}

const toppers: string[] = [];                        // without the annotation this would be never[]
toppers.push("Asha");

let message: string;                                 // assigned later in a branch
if (averageOf([90, 80]) > 75) message = "good batch";
else message = "needs revision";

console.log(averageOf([70, 85, 90]).toFixed(2), toppers, message);
```

OUTPUT

```
81.67 [ 'Asha' ] good batch
```

3.2.3 let vs const inference (widening)

With `const`, the variable can never change, so TypeScript infers the narrowest possible type: the **literal type** of the value (`const size = "large"` has type `"large"`, not `string`). With `let`, the variable may be reassigned, so the literal type is **widened** to its general type (`let size = "large"` has type `string`). Literal types become very useful with unions in Module 6.

CONST KEEPS LITERAL TYPES; LET WIDENS THEM main.ts

```ts
const size = "large";          // type: "large"
let otherSize = "large";       // type: string (widened)
const max = 100;               // type: 100
let limit = 100;               // type: number

type Size = "small" | "medium" | "large";
function orderTea(s: Size) { return `one ${s} tea`; }

console.log(orderTea(size));   // OK: "large" is a Size
otherSize = "extra-large";     // OK: any string
console.log(otherSize, max, limit);
```

OUTPUT

```
one large tea
extra-large 100 100
```

A WIDENED STRING IS NOT ASSIGNABLE TO A UNION OF LITERALS

main.ts

```ts
type Size = "small" | "medium" | "large";
function orderTea(s: Size) { return `one ${s} tea`; }

let otherSize = "large";
orderTea(otherSize);
```

COMPILER ERRORS (tsc --strict)

```
main.ts(5,10): error TS2345: Argument of type 'string' is not assignable to parameter of type 'Size'.
```

Fixes: use `const`, annotate `let otherSize: Size = "large"`, or use `as const` (6.3.3). Objects and arrays are widened too, even when declared with `const`, because their properties and elements can still change: `const p = { size: "large" }` has type `{ size: string }`.

3.3 Primitive types

3.3.1 string, number, boolean

The three everyday primitive types are written in lowercase: `string` (any text, in single, double or backtick quotes), `number` (all numbers: integers and floating-point values share one type, including `NaN` and `Infinity`), and `boolean` (`true` or `false`). Do not use the capitalised `String`, `Number` and `Boolean`: those are the wrapper object types, which you almost never want.

STRING, NUMBER AND BOOLEAN

main.ts

```ts
let firstName: string = "Asha";
let greeting: string = `Hello, ${firstName}!`;
let roll: number = 42;
let fee: number = 14_999.5;           // numeric separators are allowed
let hex: number = 0xff;
let binary: number = 0b1010;
let ratio: number = 1 / 3;
let active: boolean = true;

console.log(greeting, roll, fee, hex, binary, ratio.toFixed(3), active);
console.log(Number.MAX_SAFE_INTEGER, 0.1 + 0.2, 10 / 0);
```

OUTPUT

```
Hello, Asha! 42 14999.5 255 10 0.333 true
9007199254740991 0.30000000000000004 Infinity
```

3.3.2 bigint and symbol

`bigint` holds integers of any size, written with an `n` suffix (`123n`) or created with `BigInt(...)`; it is used when values exceed `Number.MAX_SAFE_INTEGER` ($2^{53} - 1$), for example very large IDs or exact financial integers. bigint and number cannot be mixed in arithmetic without converting. `symbol` is a unique, immutable value created with `Symbol("description")`, used as a property key that cannot clash with others; `unique symbol` is the type of a specific symbol declared with `const`. Both need `target` / `lib` ES2020 or later.

BIGINT AND SYMBOL main.ts

```ts
const big: bigint = 9_007_199_254_740_993n;
const unsafe: number = 9_007_199_254_740_993;
console.log(big, unsafe);                       // the number lost precision
console.log(big * 2n, BigInt(10) ** 20n);

const id: symbol = Symbol("id");
const another = Symbol("id");
console.log(id === another, id.description);    // every symbol is unique

const user = { name: "Ravi", [id]: 101 };
console.log(user[id], Object.keys(user));       // symbol keys are hidden from Object.keys
```

OUTPUT

```
9007199254740993n 9007199254740992
18014398509481986n 10000000000000000000n
false id
101 [ 'name' ]
```

BIGINT AND NUMBER CANNOT BE MIXED main.ts

```ts
const big: bigint = 100n;
const total = big + 5;
```

COMPILER ERRORS (tsc --strict)

```
main.ts(2,15): error TS2365: Operator '+' cannot be applied to types 'bigint' and '5'.
```

3.3.3 null and undefined with strictNullChecks

`undefined` means "no value has been assigned" (a missing property, an omitted argument, a variable without a value) and `null` means "intentionally empty". With `strictNullChecks` (part of `strict`), they are **separate types**: a `string` variable can never hold `null` or `undefined`. When a value may be absent, you say so with a **union** such as `string | null`, and TypeScript then forces you to check before using it. This single feature prevents the most common JavaScript crash: *Cannot read properties of undefined*.

WITH STRICTNULLCHECKS, NULL AND UNDEFINED MUST BE HANDLED EXPLICITLY main.ts

```ts
let email: string = null;

function findStudent(id: number): string | undefined {
  return id === 1 ? "Asha" : undefined;
}
const name = findStudent(2);
console.log(name.toUpperCase());
```

COMPILER ERRORS (tsc --strict)

```
main.ts(1,5): error TS2322: Type 'null' is not assignable to type 'string'.
main.ts(7,13): error TS18048: 'name' is possibly 'undefined'.
```

CHECKING FOR UNDEFINED BEFORE USE (NARROWING, MODULE 8) main.ts

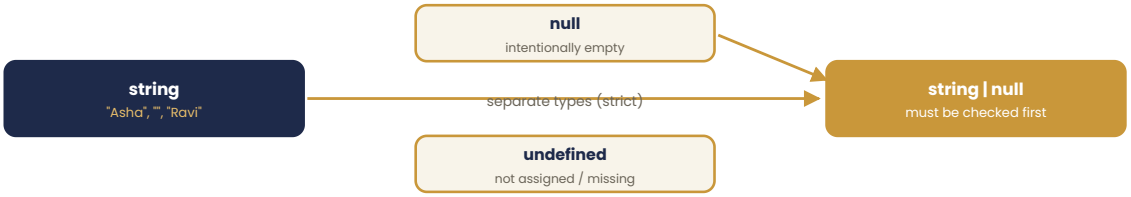
```
function findStudent(id: number): string | undefined {
  return id === 1 ? "Asha" : undefined;
}

const name = findStudent(2);
if (name !== undefined) {
  console.log(name.toUpperCase());          // narrowed to string here
} else {
  console.log("not found");
}
console.log(findStudent(1)?.toUpperCase() ?? "UNKNOWN");   // optional chaining + default

let middleName: string | null = null;       // intentionally empty
middleName = "Kumar";
console.log(middleName.length);             // TypeScript knows it is a string now
```

OUTPUT

```
not found
ASHA
5
```



UNDER STRICTNULLCHECKS, NULL AND UNDEFINED ARE ONLY ALLOWED WHEN THE TYPE SAYS SO

3.4 Arrays

3.4.1 number[] and Array<number>

An array type is written as the element type followed by `[]` (`number[]`, `string[]`) or with the generic form `Array<number>`. The two are identical; most code uses `T[]` for simple types and `Array<...>` when the element type is complicated. Arrays of mixed types use a union: `(string | number)[]` (the parentheses matter: `string | number[]` means "a string or an array of numbers").

TYPED ARRAYS AND THE METHODS THAT KEEP THEIR TYPES main.ts

```
const marks: number[] = [78, 91, 85];
const names: Array<string> = ["Asha", "Ravi"];
const ids: (string | number)[] = [101, "A-202", 303];

marks.push(66);
const passed = marks.filter((m) => m >= 70);          // number[]
const labels = names.map((n) => n.toUpperCase());      // string[]
const total = marks.reduce((sum, m) => sum + m, 0);   // number

console.log(passed, labels, total, ids);
const first = marks[0];          // number (or undefined at run time if the array is empty!)
console.log(first, marks[10]);
```

OUTPUT

```
[ 78, 91, 85 ] [ 'ASHA', 'RAVI' ] 320 [ 101, 'A-202', 303 ]
78 undefined
```

ELEMENTS ARE CHECKED ON CREATION, INSERTION AND USE main.ts

```ts
const marks: number[] = [78, 91, 85];
marks.push("90");
const names: string[] = [1, 2];
const total: string = marks.reduce((a, b) => a + b, 0);
```

COMPILER ERRORS (tsc --strict)

```
main.ts(2,12): error TS2345: Argument of type 'string' is not assignable to parameter of type
'number'.
main.ts(3,26): error TS2322: Type 'number' is not assignable to type 'string'.
main.ts(3,29): error TS2322: Type 'number' is not assignable to type 'string'.
main.ts(4,7): error TS2322: Type 'number' is not assignable to type 'string'.
```

NOTE

`marks[10]` has type `number` even though it is `undefined` at run time. The option `noUncheckedIndexedAccess` (5.5.3) makes index access return `number | undefined` so you are forced to check.

3.4.2 Readonly arrays readonly string[]

`readonly string[]` (or `ReadonlyArray<string>`) is an array type without the mutating methods (`push`, `pop`, `splice`, `sort`, index assignment). Use it for parameters to promise callers that a function will not modify their array, and for constants that must not change. It is a compile-time check only; for run-time immutability use `Object.freeze`.

A READONLY ARRAY HAS NO MUTATING METHODS main.ts

```ts
function average(values: readonly number[]): number {
  values.sort();                              // would change the caller's array
  values[0] = 0;
  return values.reduce((a, b) => a + b, 0) / values.length;
}

const WEEKDAYS: readonly string[] = ["Mon", "Tue", "Wed", "Thu", "Fri"];
WEEKDAYS.push("Sat");
```

COMPILER ERRORS (tsc --strict)

```
main.ts(2,10): error TS2339: Property 'sort' does not exist on type 'readonly number[]'.
main.ts(3,3): error TS2542: Index signature in type 'readonly number[]' only permits reading.
main.ts(8,10): error TS2339: Property 'push' does not exist on type 'readonly string[]'.
```

WORKING WITH READONLY ARRAYS BY COPYING main.ts

```ts
function average(values: readonly number[]): number {
  const sorted = [...values].sort((a, b) => a - b);   // copy first, then sort the copy
  console.log("median:", sorted[Math.floor(sorted.length / 2)]);
  return values.reduce((a, b) => a + b, 0) / values.length;
}

const marks = [91, 64, 78, 85, 70];
console.log(average(marks).toFixed(1), marks);         // the original is unchanged
```

OUTPUT

```
median: 78
77.6 [ 91, 64, 78, 85, 70 ]
```

3.4.3 Multi-dimensional arrays

An array of arrays is written by repeating the brackets: `number[][]` is a 2-D grid of numbers (a matrix, a game board, marks per student per subject), and `string[][][]` has three dimensions.

A 2-D ARRAY OF MARKS AND A TIC-TAC-TOE BOARD main.ts

```ts
// rows = students, columns = subjects
const marks: number[][] = [
  [78, 91, 85],
  [66, 72, 80],
  [95, 88, 92],
];
const subjects: string[] = ["Maths", "Science", "English"];

for (const [i, subject] of subjects.entries()) {
  const column = marks.map((row) => row[i]);
  const avg = column.reduce((a, b) => a + b, 0) / column.length;
  console.log(`${subject.padEnd(8)} average ${avg.toFixed(1)}`);
}

const board: string[][] = Array.from({ length: 3 }, () => Array(3).fill("."));
board[1][1] = "X";
console.log(board.map((row) => row.join(" ")).join("\n"));
```

```
OUTPUT
Maths    average 79.7
Science  average 83.7
English  average 85.7
. . .
. X .
. . .
```

3.5 Tuples

3.5.1 Fixed-length typed arrays [string, number]

A **tuple** is an array with a fixed number of elements where each position has its own type. `[string, number]` means "exactly two elements: a string, then a number". Tuples are useful for small, positional groups of values: a pair of coordinates, a key-value entry, or a function that returns two results (like React's `useState`, which returns `[value, setter]`). At run time a tuple is an ordinary array.



THE TUPLE [STRING, NUMBER, BOOLEAN]: A TYPE FOR EACH POSITION

TUPLES FOR PAIRS AND MULTIPLE RETURN VALUES main.ts

```ts
let student: [string, number] = ["Asha", 91];
const [name, marks] = student;                    // destructuring keeps each type
console.log(name.toUpperCase(), marks.toFixed(1));

function minMax(values: number[]): [number, number] {
  return [Math.min(...values), Math.max(...values)];
}
const [low, high] = minMax([78, 91, 64, 85]);
console.log(`range ${low} to ${high}`);

const entries: [string, number][] = Object.entries({ maths: 91, physics: 84 });
for (const [subject, mark] of entries) console.log(subject, mark);
```

OUTPUT

```
ASHA 91.0
range 64 to 91
maths 91
physics 84
```

TUPLES CHECK THE TYPE AT EACH POSITION AND THE LENGTH main.ts

```ts
let student: [string, number] = ["Asha", 91];
student = [91, "Asha"];                 // wrong order
student = ["Ravi", 85, true];           // too many elements
const third = student[2];
```

COMPILER ERRORS (tsc --strict)

```
main.ts(2,12): error TS2322: Type 'number' is not assignable to type 'string'.
main.ts(2,16): error TS2322: Type 'string' is not assignable to type 'number'.
main.ts(3,1): error TS2322: Type '[string, number, boolean]' is not assignable to type '[string,
number]'.
  Source has 3 element(s) but target allows only 2.
main.ts(4,23): error TS2493: Tuple type '[string, number]' of length '2' has no element at index '2'.
```

3.5.2 Optional and rest elements in tuples

A tuple element can be **optional** with `?` (it must come after the required ones): `[string, number, string?]` allows two or three elements. A **rest element** `...T[]` allows any number of elements of that type, at the end, the start or the middle: `[string, ...number[]]` is "a string followed by any number of numbers".

OPTIONAL AND REST ELEMENTS main.ts

```ts
type Point = [number, number, number?];          // z is optional
const p2: Point = [3, 4];
const p3: Point = [3, 4, 12];

function length(p: Point): number {
  const [x, y, z = 0] = p;                        // default for the optional element
  return Math.hypot(x, y, z);
}
console.log(length(p2), length(p3));

type MarksRow = [string, ...number[]];            // a name, then any number of marks
const rows: MarksRow[] = [["Asha", 91, 84, 77], ["Ravi", 66], ["Meera"]];
for (const [student, ...marks] of rows) {
  console.log(student, marks.length ? Math.max(...marks) : "no marks yet");
}
```

OUTPUT

```
5 13
Asha 91
Ravi 66
Meera no marks yet
```

3.5.3 Named tuple elements

Tuple elements can have **labels** that document what each position means: `[name: string, marks: number]`. Labels appear in editor hints and error messages but do not change the type; positions are still accessed by index or destructuring. All elements must be labelled if any are.

LABELLED TUPLE ELEMENTS main.ts

```ts
type StudentRecord = [name: string, roll: number, grade?: string];
type Range = [start: number, end: number];

function slicePage(items: string[], [start, end]: Range): string[] {
  return items.slice(start, end);
}

const rec: StudentRecord = ["Kiran", 17];
console.log(rec, slicePage(["a", "b", "c", "d", "e"], [1, 4]));
```

OUTPUT

```
[ 'Kiran', 17 ] [ 'b', 'c', 'd' ]
```

3.5.4 Readonly tuples

`readonly [number, number]` is a tuple that cannot be modified. Tuples created with `as const` (6.3.3) are readonly and keep literal types: `const origin = [0, 0] as const` has type `readonly [0, 0]`.

READONLY TUPLES CANNOT BE CHANGED main.ts

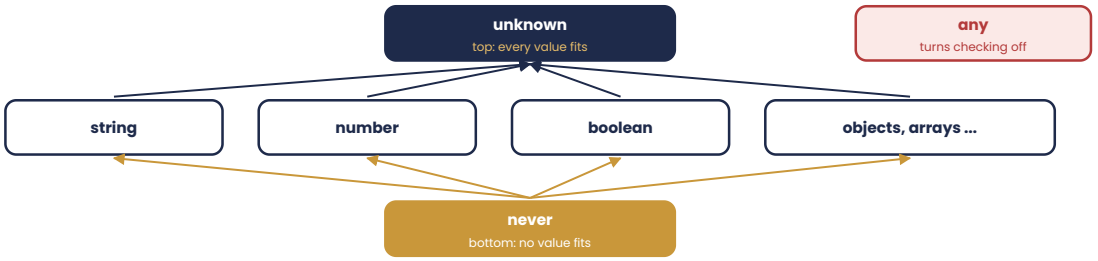
```ts
const origin: readonly [number, number] = [0, 0];
origin[0] = 5;
origin.push(1);

const rgb = [201, 151, 58] as const;      // readonly [201, 151, 58]
rgb[1] = 0;
```

COMPILER ERRORS (tsc --strict)

```
main.ts(2,8): error TS2540: Cannot assign to '0' because it is a read-only property.
main.ts(3,8): error TS2339: Property 'push' does not exist on type 'readonly [number, number]'.
main.ts(6,5): error TS2540: Cannot assign to '1' because it is a read-only property.
```

3.6 Special types



UNKNOWN IS ABOVE EVERY TYPE, NEVER IS BELOW EVERY TYPE, AND ANY ESCAPES THE TYPE SYSTEM

3.6.1 any and why to avoid it

`any` switches off type checking for a value: you can assign anything to it, read any property, call it as a function, and pass it anywhere, and TypeScript reports nothing. Worse, `any` is **contagious**: values derived from it are also `any`, so errors spread silently. It exists as an escape hatch for gradual migration and truly dynamic code. In strict mode, implicit `any` is an error (`noImplicitAny`); an explicit `any` should be rare, commented, and replaced with `unknown` or a proper type whenever possible. Linters can forbid it (`@typescript-eslint/no-explicit-any`).

ANY HIDES MISTAKES UNTIL THE PROGRAM RUNS main.ts

```ts
let data: any = JSON.parse('{"name": "Asha", "marks": [91, 84]}');

console.log(data.nmae);                  // typo: no error, prints undefined
const total: number = data.marks.sum();  // no error at compile time ...
console.log(total);                      // ... crashes at run time
```

TERMINAL

```
$ tsc --strict main.ts
(no errors reported)
$ node main.js
undefined
main.js:5
const total = data.marks.sum(); // no error at compile time ...
                         ^

TypeError: data.marks.sum is not a function
```

3.6.2 unknown as the safe alternative

`unknown` also accepts **any** value, but you cannot **do** anything with it (read properties, call it, pass it to a typed parameter) until you have checked what it is. That makes it the right type for data whose shape you do not know yet: `JSON.parse` results, API responses, caught errors and user input. The checks you write (`typeof`, `Array.isArray`, `in`, `instanceof`) **narrow** the type, as Module 8 explains.

UNKNOWN MUST BE CHECKED BEFORE USE main.ts

```
let data: unknown = JSON.parse('{"name": "Asha"}');
console.log(data.name);
const n: number = data;
data();
```

COMPILER ERRORS (tsc --strict)

```
main.ts(2,13): error TS18046: 'data' is of type 'unknown'.
main.ts(3,7): error TS2322: Type 'unknown' is not assignable to type 'number'.
main.ts(4,1): error TS18046: 'data' is of type 'unknown'.
```

NARROWING UNKNOWN WITH RUN-TIME CHECKS main.ts

```
function describe(value: unknown): string {
  if (typeof value === "string") return `text of length ${value.length}`;
  if (typeof value === "number") return `number ${value.toFixed(2)}`;
  if (Array.isArray(value)) return `array of ${value.length} items`;
  if (value !== null && typeof value === "object" && "name" in value) return `object named
${String(value.name)}`;
  return "something else";
}

for (const v of ["LBF", 3.14159, [1, 2, 3], { name: "Ravi" }, null, true]) {
  console.log(describe(v));
}
```

OUTPUT

```
text of length 3
number 3.14
array of 3 items
object named Ravi
something else
something else
```

	any	unknown
assign any value to it	yes	yes
assign it to a typed variable	yes (unsafe)	only after narrowing
access properties or call it	yes, unchecked	only after narrowing
spreads to other values	yes (contagious)	no
use for	migration, last resort	external data, catch clauses, generic inputs

3.6.3 void

`void` is the return type of a function that does not return a useful value (it returns `undefined` implicitly). It is mainly used for return types: `function log(msg: string): void`. A function **type** with a `void` return (`()`

`=> void`) accepts functions that do return something; the result is simply ignored, which is why `arr.forEach(x => list.push(x))` is allowed even though `push` returns a number.

VOID FUNCTIONS main.ts

```
function logMessage(msg: string): void {
  console.log(`[LOG] ${msg}`);
}

const result = logMessage("server started");
console.log(result);                          // undefined

const list: number[] = [];
[1, 2, 3].forEach((n) => list.push(n * 10));    // forEach expects (n) => void; push's number is
ignored
console.log(list);
```

OUTPUT

```
[LOG] server started
undefined
[ 10, 20, 30 ]
```

A VOID FUNCTION CANNOT RETURN A VALUE main.ts

```
function logMessage(msg: string): void {
  return msg.length;
}
```

COMPILER ERRORS (tsc --strict)

```
main.ts(2,3): error TS2322: Type 'number' is not assignable to type 'void'.
```

3.6.4 never

`never` is the type with **no values**. It is the return type of functions that never finish normally: they always throw or loop forever. It also appears when TypeScript has eliminated every possibility (for example, after checking every member of a union), which makes it the basis of **exhaustiveness checking** (8.2.3): if a new case is added and not handled, a value of that case cannot be assigned to `never`, and the compiler reports it.

NEVER FOR FUNCTIONS THAT THROW AND FOR EXHAUSTIVENESS CHECKS

main.ts

```
function fail(message: string): never {
  throw new Error(message);
}

function parseMarks(input: string): number {
  const n = Number(input);
  if (Number.isNaN(n)) fail(`not a number: ${input}`);
  return n;                                        // TypeScript knows we only get here if n is valid
}

type Light = "red" | "amber" | "green";
function next(light: Light): Light {
  switch (light) {
    case "red": return "green";
    case "green": return "amber";
    case "amber": return "red";
    default: {
      const impossible: never = light;           // every case handled, so light is never here
      return impossible;
    }
  }
}

console.log(parseMarks("87"), next("red"));
try { parseMarks("eighty"); } catch (e) { console.log((e as Error).message); }
```

OUTPUT

```
87 green
not a number: eighty
```

3.6.5 object vs Object vs {}

These three similar-looking types are a common source of confusion. `object` (lowercase) means **any non-primitive value**: objects, arrays, functions, but not strings, numbers, booleans, symbols, bigints, null or undefined. `{}` (an empty object type) means **any value except null and undefined**, because every other value, including primitives, has properties. `Object` (capitalised) behaves almost like `{}` and should not be used. To describe a real object, write its shape (Module 5) or use `Record<string, unknown>`.

OBJECT ACCEPTS NON-PRIMITIVES; {} ACCEPTS ANYTHING EXCEPT NULL AND UNDEFINED

main.ts

```
let a: object = { x: 1 };
a = [1, 2];
a = () => 0;
a = "text";                    // error: string is a primitive

let b: {} = "text";            // OK: any non-null value
b = 42;
b = null;                      // error

let c: object = { name: "Asha" };
console.log(c.name);           // error: object has no known properties
```

COMPILER ERRORS (tsc --strict)

```
main.ts(4,1): error TS2322: Type 'string' is not assignable to type 'object'.
main.ts(8,1): error TS2322: Type 'null' is not assignable to type '{}'.
main.ts(11,15): error TS2339: Property 'name' does not exist on type 'object'.
```

Type	Accepts	Rejects	Use it?
`object`	objects, arrays, functions	primitives, null, undefined	rarely (when only "non-primitive" matters)
`{}`	every value except null and undefined	null, undefined	rarely, in generic constraints (T extends {})
`Object`	almost the same as {}	null, undefined	no
`{ name: string }`	values with that shape	others	yes: describe real shapes
`Record<string, unknown>`	objects with any string keys	primitives, arrays (mostly)	yes: dictionaries of unknown values

3.7 Type assertions

A **type assertion** tells the compiler "trust me, this value has this type". It does not convert or check anything at run time; it only changes what the compiler believes. Assertions are needed when you know more than TypeScript can work out, for example the exact element type returned by `document.getElementById`, or the shape of JSON you have just validated. Prefer narrowing (Module 8) or validation whenever possible.

3.7.1 as syntax

TYPE ASSERTIONS WITH AS — main.ts

```ts
interface Student { name: string; marks: number[] }

const raw: unknown = JSON.parse('{"name": "Asha", "marks": [91, 84, 77]}');
const student = raw as Student;                // assertion: we promise the shape
console.log(student.name, Math.max(...student.marks));

const input = "  42  " as string;
const value = (Number(input.trim()) as number) + 8;
console.log(value);
```

```
OUTPUT
Asha 91
50
```

TypeScript only allows assertions between types that **overlap** (one is assignable to the other). Asserting a `string` as a `number` is rejected because it is almost certainly a mistake. Going through `unknown` (`x as unknown as T`) bypasses this check; if you find yourself writing a double assertion, the design probably needs rethinking.

ASSERTIONS BETWEEN UNRELATED TYPES ARE REJECTED — main.ts

```ts
const roll = "42" as number;
```

```
COMPILER ERRORS (tsc --strict)
main.ts(1,14): error TS2352: Conversion of type 'string' to type 'number' may be a mistake because
neither type sufficiently overlaps with the other. If this was intentional, convert the expression to
'unknown' first.
```

3.7.2 Angle-bracket syntax and JSX conflicts

The original assertion syntax puts the type in angle brackets before the value: `<string>value`. It means exactly the same as `value as string`, but it is **not allowed in .tsx files**, because `<string>` looks like a JSX element. For consistency, modern code always uses `as`.

ANGLE-BRACKET AND AS ASSERTIONS ARE EQUIVALENT main.ts

```
const data: unknown = "Learn Being Forward";
const a = (<string>data).length;        // angle-bracket assertion (not allowed in .tsx)
const b = (data as string).length;      // the same, works everywhere
console.log(a, b);
```

```
OUTPUT
19 19
```

3.7.3 Non-null assertion !

The postfix `!` operator removes `null` and `undefined` from a type: `value!` says "this is definitely not null or undefined here". It is common with DOM lookups (`document.getElementById("app")!`) and `Map.get` after a `has` check. If you are wrong, the program crashes at run time with the very error strict null checks exist to prevent, so use it only when you are certain, and prefer an explicit check.

THE NON-NULL ASSERTION AND THE SAFER ALTERNATIVE main.ts

```
const fees = new Map<string, number>([["TypeScript", 9999], ["Python", 14999]]);

if (fees.has("Python")) {
  const fee = fees.get("Python")!;            // get() returns number | undefined; we just checked
  console.log(fee * 1.18);
}

const names = ["Asha", "Ravi", "Meera"];
const longest = names.find((n) => n.length > 4)!;   // we know one exists
console.log(longest.toUpperCase());

const missing = names.find((n) => n.startsWith("Z"));
console.log(missing?.toUpperCase() ?? "no match");  // the safe alternative
```

```
OUTPUT
17698.82
MEERA
no match
```

3.7.4 Risks of assertions

Every assertion is a place where the compiler stops protecting you. If the claim is false, nothing warns you: the error appears later, somewhere else, at run time. Use assertions sparingly, keep them close to the place where the value is produced, and prefer alternatives: annotate the variable, narrow with a check, validate external data (15.3), or use `satisfies` (15.1.2) to check a value without changing its type.

A WRONG ASSERTION COMPILES FINE AND FAILS AT RUN TIME

main.ts

```ts
interface User { id: number; email: string }

const fromApi: unknown = JSON.parse('{"id": "7"}');      // the API changed: no email, id is a string
const user = fromApi as User;                             // the compiler believes us

console.log(typeof user.id, user.id + 1);                 // "7" + 1 = "71", not 8
try {
  console.log(user.email.toLowerCase());
} catch (e) {
  console.log("crash:", (e as Error).message);
}
```

OUTPUT

```
string 71
crash: Cannot read properties of undefined (reading 'toLowerCase')
```

WATCH OUT

Never use `as` to silence an error you do not understand. Read the error (7.4) and fix the types instead; an assertion that "makes the red line go away" usually hides a real bug.

Annotations (name: type) declare the types of variables, parameters and return values; inference supplies types from initial values, returns and context, so annotate boundaries and let inference work inside. const infers literal types, let widens them. The primitives are string, number, boolean, bigint and symbol; with strictNullChecks, null and undefined are separate types that must be included in a union and checked. Arrays are T[] or Array<T>, readonly T[] blocks mutation, and T[][] is 2-D. Tuples fix length and per-position types and support optional (?), rest (...T[]), named and readonly elements. any disables checking (avoid it); unknown accepts anything but must be narrowed before use; void marks functions with no useful return; never has no values (throwing functions, exhaustiveness); object means non-primitive and {} means non-null. Assertions (as, the old <T> form, and the non-null !) change only what the compiler believes, so use them rarely.

PRACTICE CHECKPOINT

A **student marks program** with fully typed variables, arrays and a **tuple per student [name, marks]**, compiled with zero errors in strict mode.

SAMPLE SOLUTION: A TYPED STUDENT MARKS REPORT

main.ts

```ts
type StudentEntry = [name: string, marks: readonly number[]];

const SUBJECTS: readonly string[] = ["Maths", "Science", "English", "Computer"];
const PASS_MARK: number = 40;

const students: StudentEntry[] = [
  ["Asha", [91, 84, 77, 95]],
  ["Ravi", [66, 38, 72, 80]],
  ["Meera", [88, 92, 85, 90]],
  ["Kiran", [55, 61, 49, 70]],
];

function average(marks: readonly number[]): number {
  return marks.reduce((sum, m) => sum + m, 0) / marks.length;
}
```

```
function grade(avg: number): string {
  if (avg >= 85) return "A";
  if (avg >= 70) return "B";
  if (avg >= 55) return "C";
  return "D";
}

function failedSubjects(marks: readonly number[]): string[] {
  return marks.flatMap((m, i) => (m < PASS_MARK ? [SUBJECTS[i]] : []));
}

let topper: StudentEntry | undefined;
let bestAverage: number = -1;

console.log("Name    Avg    Grade  Failed");
for (const entry of students) {
  const [name, marks] = entry;
  const avg: number = average(marks);
  const failed: string[] = failedSubjects(marks);
  if (avg > bestAverage) {
    bestAverage = avg;
    topper = entry;
  }
  console.log(`${name.padEnd(7)} ${avg.toFixed(1).padStart(5)}  ${grade(avg).padEnd(6)}
${failed.join(", ") || "-"}`);
}

const subjectAverages: [subject: string, avg: number][] = SUBJECTS.map((s, i) => [s,
average(students.map(([, m]) => m[i]))]);
const [hardest] = [...subjectAverages].sort((a, b) => a[1] - b[1]);

console.log(`Topper: ${topper ? topper[0] : "none"} (${bestAverage.toFixed(1)})`);
console.log(`Hardest subject: ${hardest[0]} (average ${hardest[1].toFixed(1)})`);
```

OUTPUT

```
Name    Avg    Grade  Failed
Asha     86.8  A      -
Ravi     64.0  C      Science
Meera    88.8  A      -
Kiran    58.8  C      -
Topper: Meera (88.8)
Hardest subject: Science (average 68.8)
```

MODULE 04 · BEGINNER

Functions

Minimum duration **2 days** | Level **Beginner**

Functions are where most type errors appear, because they are the boundaries between pieces of code. This module covers typing every kind of function: parameter and return types (including inferred, void and never returns), optional, default, rest and destructured parameters, function type expressions, callbacks, call and construct signatures, arrow functions and the this parameter, function overloads, and async functions that return Promise<T>.

IN THIS MODULE

4.1 Typing functions
4.2 Parameters
4.3 Function types
4.4 Arrow functions and typing this
4.5 Function overloads
4.6 Async functions

4.1 Typing functions

4.1.1 Parameter types and return types

Every parameter gets a type annotation, and the return type is written after the parameter list. Together they form the function's **signature**: a contract that is checked in two directions. Inside the function, the body must return the declared type on every path; at every call, the arguments must match the parameters in number and type.



THE PARTS OF A TYPED FUNCTION DECLARATION

PARAMETER AND RETURN TYPES main.ts

```
function fee(base: number, gstPct: number): number {
  return Math.round(base * (1 + gstPct / 100));
}

function initials(fullName: string): string {
  return fullName.split(" ").map((part) => part[0].toUpperCase()).join("");
}

function isEligible(age: number, marks: number): boolean {
  return age >= 17 && marks >= 60;
}

console.log(fee(9999, 18), initials("lohit kumar"), isEligible(19, 72));
```

OUTPUT

```
11799 LK true
```

THE SIGNATURE IS CHECKED INSIDE THE FUNCTION AND AT EVERY CALL main.ts

```
function fee(base: number, gstPct: number): number {
  if (base < 0) {
    return "invalid";
  }
}

fee(9999);
fee(9999, "18");
fee(9999, 18, true);
```

COMPILER ERRORS (tsc --strict)

```
main.ts(1,45): error TS2366: Function lacks ending return statement and return type does not include
'undefined'.
main.ts(3,5): error TS2322: Type 'string' is not assignable to type 'number'.
main.ts(7,1): error TS2554: Expected 2 arguments, but got 1.
main.ts(8,11): error TS2345: Argument of type 'string' is not assignable to parameter of type
'number'.
main.ts(9,15): error TS2554: Expected 2 arguments, but got 3.
```

The first error (TS2366) appears because one path (when `base` is not negative) falls off the end of the function and returns `undefined`, which is not a `number`. Explicit return types catch exactly this kind of forgotten `return`.

4.1.2 Inferred return types

If you omit the return type, TypeScript infers it from the `return` statements: a union if different paths return different types, `void` if nothing is returned. Inference is fine for small local functions and callbacks. For exported functions and public methods, writing the return type is good practice: it documents the API, gives clearer errors (at the wrong `return` rather than at a distant caller) and stops an accidental change in the body from silently changing the function's type.

RETURN TYPES INFERRED FROM THE RETURN STATEMENTS main.ts

```ts
function grade(mark: number) {            // inferred: "A" | "B" | "C"
  if (mark >= 85) return "A";
  if (mark >= 60) return "B";
  return "C";
}

function parse(input: string) {           // inferred: number | null
  const n = Number(input);
  return Number.isNaN(n) ? null : n;
}

function logTotal(values: number[]) {     // inferred: void
  console.log(values.reduce((a, b) => a + b, 0));
}

const g = grade(72);
const p = parse("x");
logTotal([1, 2, 3]);
console.log(g, p);
```

```
OUTPUT
6
B null
```

4.1.3 void and never return types

`void` marks a function that returns nothing useful (it may `return;` with no value). `never` marks a function that never returns at all: it always throws, or runs forever (a server loop). TypeScript uses `never` functions in control-flow analysis: code after a call to one is unreachable, and narrowing continues as if that branch ended (3.6.4).

VOID FOR "NO RESULT", NEVER FOR "DOES NOT RETURN" main.ts

```ts
function printReceipt(item: string, amount: number): void {
  if (amount <= 0) return;                // early return with no value is allowed
  console.log(`${item.padEnd(10)} Rs ${amount}`);
}

function invalid(field: string, value: unknown): never {
  throw new RangeError(`invalid ${field}: ${JSON.stringify(value)}`);
}

function checkedQty(qty: number): number {
  if (!Number.isInteger(qty) || qty < 1) invalid("quantity", qty);
  return qty;                             // qty is still a number here
}

printReceipt("Notebook", 60);
printReceipt("Free pen", 0);
try {
  console.log(checkedQty(3), checkedQty(2.5));
} catch (e) {
  console.log((e as Error).name, (e as Error).message);
}
```

```
OUTPUT
Notebook   Rs 60
RangeError invalid quantity: 2.5
```

4.2 Parameters

4.2.1 Optional parameters ?

A `?` after a parameter name makes it **optional**: callers may leave it out, and inside the function its type includes `undefined`, so you must handle that case. Optional parameters must come after all required ones.

OPTIONAL PARAMETERS INCLUDE UNDEFINED main.ts

```
function fullName(first: string, last?: string): string {
  // last has type string | undefined here
  return last ? `${first} ${last}` : first;
}

function greet(name: string, title?: string) {
  console.log(`Hello, ${title?.toUpperCase() ?? ""}${title ? " " : ""}${name}`);
}

console.log(fullName("Asha"), "|", fullName("Asha", "Rao"));
greet("Kumar");
greet("Kumar", "dr");
```

OUTPUT

```
Asha | Asha Rao
Hello, Kumar
Hello, DR Kumar
```

AN OPTIONAL PARAMETER MUST BE CHECKED, AND CANNOT PRECEDE A REQUIRED ONE main.ts

```
function fullName(first: string, last?: string): string {
  return first + " " + last.toUpperCase();
}

function badOrder(a?: number, b: number) { return b; }
```

COMPILER ERRORS (tsc --strict)

```
main.ts(2,24): error TS18048: 'last' is possibly 'undefined'.
main.ts(5,31): error TS1016: A required parameter cannot follow an optional parameter.
```

4.2.2 Default parameters

A **default value** (`gstPct = 18`) also makes a parameter optional, but inside the function its type does **not** include `undefined`, because the default fills the gap. The type is inferred from the default, so an annotation is usually unnecessary. Defaults apply when the argument is missing or explicitly `undefined` (not when it is `null`).

DEFAULT PARAMETERS main.ts

```
function priceWithTax(amount: number, gstPct = 18, roundTo = 1): number {
  // gstPct: number (never undefined inside)
  const total = amount * (1 + gstPct / 100);
  return Math.round(total / roundTo) * roundTo;
}

console.log(priceWithTax(1000));              // 1180
console.log(priceWithTax(1000, 5));           // 1050
console.log(priceWithTax(1234, undefined, 10)); // skip gstPct, use its default: 1460
```

OUTPUT

```
1180
1050
1460
```

4.2.3 Rest parameters ...nums: number[]

A **rest parameter** collects any number of remaining arguments into an array. Its type is an array type (`...marks: number[]`) or a tuple type for a fixed pattern. Only the last parameter can be a rest parameter. When you call a function that has fixed parameters with a spread array, TypeScript needs to know the array has the right length, so spreading a tuple works but spreading a `number[]` into fixed parameters does not.

REST PARAMETERS AND SPREADING TUPLES main.ts

```
function sum(...nums: number[]): number {
  return nums.reduce((total, n) => total + n, 0);
}

function enrol(course: string, ...students: string[]): string {
  return `${course}: ${students.length} enrolled (${students.join(", ")})`;
}

const marks = [78, 91, 85];
console.log(sum(), sum(5), sum(1, 2, 3), sum(...marks));
console.log(enrol("TypeScript", "Asha", "Ravi", "Meera"));

function area(width: number, height: number) { return width * height; }
const size: [number, number] = [12, 5];          // a tuple has a known length
console.log(area(...size));
```

OUTPUT

```
0 5 6 254
TypeScript: 3 enrolled (Asha, Ravi, Meera)
60
```

SPREADING AN ARRAY OF UNKNOWN LENGTH INTO FIXED PARAMETERS main.ts

```
function area(width: number, height: number) { return width * height; }
const dims = [12, 5];
area(...dims);
```

COMPILER ERRORS (tsc --strict)

```
main.ts(3,6): error TS2556: A spread argument must either have a tuple type or be passed to a rest
parameter.
```

4.2.4 Destructured parameters with types

When a function takes an options object, destructure it in the parameter list and put the type annotation **after the whole pattern**, not inside it. Writing `{ name: string }` inside the pattern would rename a property, because in destructuring syntax a colon means "rename". Defaults can be given in the pattern, and a default `= {}` for the whole object makes the argument optional.

DESTRUCTURED OBJECT PARAMETERS WITH A TYPE AND DEFAULTS main.ts

```
type InvoiceOptions = { customer: string; items: number[]; discountPct?: number; currency?: string };

function invoice({ customer, items, discountPct = 0, currency = "Rs" }: InvoiceOptions): string {
  const gross = items.reduce((a, b) => a + b, 0);
  const net = gross * (1 - discountPct / 100);
  return `${customer}: ${currency} ${net.toFixed(2)}`;
}

function paginate({ page = 1, size = 10 }: { page?: number; size?: number } = {}) {
  return `items ${(page - 1) * size + 1} to ${page * size}`;
}

console.log(invoice({ customer: "Asha", items: [499, 1299] }));
console.log(invoice({ customer: "Ravi", items: [999], discountPct: 10, currency: "INR" }));
console.log(paginate(), "|", paginate({ page: 3 }), "|", paginate({ page: 2, size: 25 }));

const [first, second]: [string, number] = ["batch", 12];   // array destructuring with a tuple type
console.log(first, second);
```

OUTPUT

```
Asha: Rs 1798.00
Ravi: INR 899.10
items 1 to 10 | items 21 to 30 | items 26 to 50
batch 12
```

WATCH OUT

`function f({ name: string })` does **not** type `name` as a string: it destructures `name` into a new variable called `string` (of type any). Always write `function f({ name }: { name: string })`.

4.3 Function types

4.3.1 Function type expressions (a: number, b: number) => number

Functions are values in JavaScript, so they have types. A **function type expression** looks like an arrow function: `(a: number, b: number) => number` describes any function that takes two numbers and returns a number. The parameter names are only documentation; compatibility depends on positions and types. Give function types a name with a type alias to reuse them.

A NAMED FUNCTION TYPE USED FOR SEVERAL FUNCTIONS main.ts

```ts
type Operation = (a: number, b: number) => number;

const add: Operation = (a, b) => a + b;          // a and b are inferred from the type
const subtract: Operation = (x, y) => x - y;     // names do not have to match
const power: Operation = Math.pow;               // any compatible function works

const operations: Record<string, Operation> = { "+": add, "-": subtract, "^": power };
for (const [symbol, op] of Object.entries(operations)) {
  console.log(`8 ${symbol} 2 = ${op(8, 2)}`);
}
```

OUTPUT

```
8 + 2 = 10
8 - 2 = 6
8 ^ 2 = 64
```

FUNCTION TYPES ARE CHECKED LIKE ANY OTHER TYPE main.ts

```ts
type Operation = (a: number, b: number) => number;

const concat: Operation = (a: string, b: string) => a + b;
const half: Operation = (a) => a / 2;            // fewer parameters is fine ...
half(10);                                          // ... but callers must pass both
```

COMPILER ERRORS (tsc --strict)

```
main.ts(3,7): error TS2322: Type '(a: string, b: string) => string' is not assignable to type
'Operation'.
  Types of parameters 'a' and 'a' are incompatible.
    Type 'number' is not assignable to type 'string'.
main.ts(5,1): error TS2554: Expected 2 arguments, but got 1.
```

Notice that `half`, which ignores its second parameter, is accepted: a function with **fewer** parameters can be used where more are provided, just as JavaScript callbacks often ignore extra arguments (that is why `arr.forEach(x => ...)` works even though forEach passes three arguments).

4.3.2 Typing callbacks

A **callback** is a function passed to another function. Type the callback parameter with a function type, and TypeScript will infer the types of the callback's own parameters at every call site (contextual typing), so the person writing the callback rarely needs annotations. Use `void` as the callback return type when the result is ignored.

TYPED CALLBACKS WITH INFERRED PARAMETER TYPES main.ts

```
type Student = { name: string; marks: number };

function processAll(students: Student[], callback: (s: Student, index: number) => void): void {
  students.forEach((s, i) => callback(s, i));
}

function retry(task: () => boolean, attempts: number, onFail?: (attempt: number) => void): boolean {
  for (let i = 1; i <= attempts; i++) {
    if (task()) return true;
    onFail?.(i);
  }
  return false;
}

const batch: Student[] = [{ name: "Asha", marks: 91 }, { name: "Ravi", marks: 66 }];
processAll(batch, (s, i) => console.log(`${i + 1}. ${s.name} ${s.marks >= 70 ? "distinction" :
"pass"}`));

let calls = 0;
const ok = retry(() => ++calls === 3, 5, (n) => console.log(`attempt ${n} failed`));
console.log("succeeded:", ok, "after", calls, "calls");
```

OUTPUT

```
1. Asha distinction
2. Ravi pass
attempt 1 failed
attempt 2 failed
succeeded: true after 3 calls
```

4.3.3 Call signatures in object types

A function type expression cannot describe a function that **also has properties** (JavaScript functions are objects). For that, write a **call signature** inside an object type: `{ (input: string): number; description: string }`. Note the colon instead of the arrow.

A CALL SIGNATURE FOR A FUNCTION WITH PROPERTIES main.ts

```
type Validator = {
  (value: string): boolean;          // call signature: it can be called
  rule: string;                      // and it has properties
};

function makeMinLength(min: number): Validator {
  const check = (value: string) => value.trim().length >= min;
  return Object.assign(check, { rule: `at least ${min} characters` });
}

const passwordCheck = makeMinLength(8);
for (const pw of ["secret", "learnbeing2026"]) {
  console.log(`"${pw}" ${passwordCheck(pw) ? "passes" : "fails"}: ${passwordCheck.rule}`);
}
```

OUTPUT

```
"secret" fails: at least 8 characters
"learnbeing2026" passes: at least 8 characters
```

4.3.4 Construct signatures (overview)

A **construct signature**, written with `new`, describes something that can be called with `new`, such as a class: `{ new (name: string): Student }` or `new (name: string) => Student`. It lets you write factories that accept a class as an argument. Classes and `typeof ClassName` are covered in 9.7.

PASSING CLASSES TO A FACTORY THROUGH A CONSTRUCT SIGNATURE main.ts

```
class Circle { constructor(public r: number) {} area() { return Math.PI * this.r ** 2; } }
class Square { constructor(public side: number) {} area() { return this.side ** 2; } }

type ShapeConstructor = new (size: number) => { area(): number };

function createAll(ctor: ShapeConstructor, sizes: number[]) {
  return sizes.map((s) => new ctor(s));
}

for (const shape of [...createAll(Circle, [1, 2]), ...createAll(Square, [3])]) {
  console.log(shape.constructor.name, shape.area().toFixed(2));
}
```

```
OUTPUT
Circle 3.14
Circle 12.57
Square 9.00
```

4.4 Arrow functions and typing this

4.4.1 Typed arrow functions

Arrow functions are typed exactly like function declarations: annotate the parameters in the parentheses and the return type after them, before the arrow: `(price: number): number => price * 2`. When an arrow is assigned to a variable with a function type, or passed as a callback, its parameter types are inferred. Arrow functions do not have their own `this`; they use the `this` of the surrounding code, which is why they are popular for callbacks inside classes.

TYPED ARROW FUNCTIONS AND A FUNCTION THAT RETURNS A FUNCTION main.ts

```
const square = (n: number): number => n * n;
const toPercent = (part: number, whole: number) => `${((part / whole) * 100).toFixed(1)}%`;
const isEven = (n: number): boolean => n % 2 === 0;
const makeCounter = (start = 0) => {
  let count = start;
  return (): number => ++count;          // returns a function of type () => number
};

console.log(square(9), toPercent(436, 500), [1, 2, 3, 4].filter(isEven));
const next = makeCounter(10);
console.log(next(), next(), next());
```

```
OUTPUT
81 87.2% [ 2, 4 ]
11 12 13
```

4.4.2 Declaring this parameter

In a regular function, `this` depends on how the function is called, which is a classic source of JavaScript bugs. TypeScript lets you declare the expected type of `this` as a fake **first parameter named this**: `function (this: HTMLButtonElement, ev: MouseEvent)`. It is erased from the output and is not a real parameter (callers do not pass it), but the compiler checks the body against it and reports calls where `this` would be wrong. With `noImplicitThis` (part of strict), using `this` in a function without a declared type is an error.

A DECLARED THIS PARAMETER main.ts

```
interface Account { owner: string; balance: number }

function deposit(this: Account, amount: number): string {
  this.balance += amount;
  return `${this.owner}: Rs ${this.balance}`;
}

const acc: Account = { owner: "Asha", balance: 1000 };
console.log(deposit.call(acc, 500));              // this = acc

const wallet = { owner: "Ravi", balance: 200, deposit };
console.log(wallet.deposit(300));                 // this = wallet (it has owner and balance)
```

OUTPUT

```
Asha: Rs 1500
Ravi: Rs 500
```

WRONG OR MISSING THIS IS REPORTED main.ts

```
interface Account { owner: string; balance: number }
function deposit(this: Account, amount: number) { this.balance += amount; }

deposit(500);                                     // called with no this
const loose = { name: "X", deposit };
loose.deposit(100);                               // this has the wrong shape

function show() { return this.owner; }            // this implicitly any
```

COMPILER ERRORS (tsc --strict)

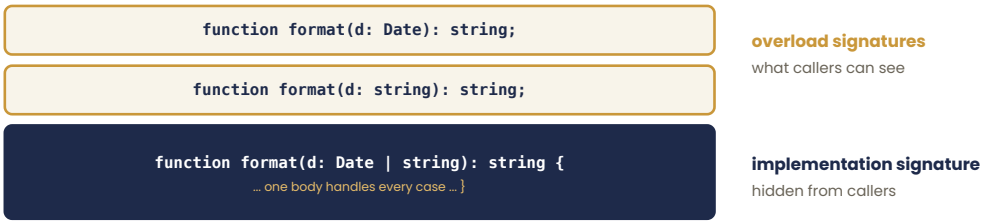
```
main.ts(4,1): error TS2684: The 'this' context of type 'void' is not assignable to method's 'this' of
type 'Account'.
main.ts(6,1): error TS2684: The 'this' context of type '{ name: string; deposit: (this: Account,
amount: number) => void; }' is not assignable to method's 'this' of type 'Account'.
  Type '{ name: string; deposit: (this: Account, amount: number) => void; }' is missing the following
properties from type 'Account': owner, balance
main.ts(8,26): error TS2683: 'this' implicitly has type 'any' because it does not have a type
annotation.
```

4.5 Function overloads

4.5.1 Overload signatures and the implementation signature

Sometimes a function returns different types depending on the types of its arguments: given a `string` it returns a `Date`, given a `number` it returns a `Date` too, given two numbers something else. **Overloads** describe each allowed way of calling the function. You write several **overload signatures** (no body),

followed by one **implementation signature** with a body that is general enough to handle all of them. Callers see only the overload signatures; the implementation signature is hidden.



OVERLOAD SIGNATURES FOLLOWED BY A SINGLE IMPLEMENTATION

AN OVERLOADED FORMAT FUNCTION main.ts

```ts
function formatDate(date: Date): string;
function formatDate(isoText: string): string;
function formatDate(day: number, month: number, year: number): string;
function formatDate(a: Date | string | number, month?: number, year?: number): string {
  let d: Date;
  if (a instanceof Date) d = a;
  else if (typeof a === "string") d = new Date(a);
  else d = new Date(year!, month! - 1, a);
  return d.toLocaleDateString("en-IN", { day: "2-digit", month: "short", year: "numeric" });
}

console.log(formatDate(new Date(2026, 9, 7)));
console.log(formatDate("2026-01-26"));
console.log(formatDate(15, 8, 1947));
```

OUTPUT

```
07 Oct 2026
26 Jan 2026
15 Aug 1947
```

CALLS MUST MATCH ONE OF THE OVERLOAD SIGNATURES main.ts

```ts
function formatDate(date: Date): string;
function formatDate(isoText: string): string;
function formatDate(day: number, month: number, year: number): string;
function formatDate(a: Date | string | number, month?: number, year?: number): string {
  return String(a);
}

formatDate(15, 8);                // matches no overload
formatDate(true);
```

COMPILER ERRORS (tsc --strict)

```
main.ts(8,1): error TS2575: No overload expects 2 arguments, but overloads do exist that expect either
1 or 3 arguments.
main.ts(9,12): error TS2769: No overload matches this call.
  Overload 1 of 3, '(date: Date): string', gave the following error.
    Argument of type 'boolean' is not assignable to parameter of type 'Date'.
  Overload 2 of 3, '(isoText: string): string', gave the following error.
    Argument of type 'boolean' is not assignable to parameter of type 'string'.
```

OVERLOADS THAT RELATE THE INPUT TYPE TO THE OUTPUT TYPE

main.ts

```ts
// Overloads can also give a more precise return type for each input
function parseValue(input: string): number;
function parseValue(input: string[]): number[];
function parseValue(input: string | string[]): number | number[] {
  return Array.isArray(input) ? input.map(Number) : Number(input);
}

const one = parseValue("42");            // type: number
const many = parseValue(["1", "2"]);     // type: number[]
console.log(one + 1, many.map((n) => n * 10));
```

OUTPUT

```
43 [ 10, 20 ]
```

4.5.2 When to use overloads vs union types

Overloads are more complex to write and read, so prefer simpler tools first. If every input produces the **same** return type, a **union parameter** is enough (`function len(x: string | unknown[]): number`). If the return type **depends on** the input type in a simple way, a **generic** often works better (Module 10). Use overloads when the **number** of parameters differs between forms, or when specific input types map to specific output types and a generic cannot express it. Also note that you cannot call an overloaded function with a value of the union type unless an overload accepts the union.

Situation	Best tool	Example
same return type for several input types	union parameter	`len(x: string | any[]): number`
return type mirrors the input type	generic	`first<T>(arr: T[]): T | undefined`
different numbers of parameters	overloads	`formatDate(date)` or `formatDate(d, m, y)`
specific input types map to specific outputs	overloads (or conditional types)	`parseValue(string): number`, `parseValue(string[]): number[]`

4.6 Async functions

4.6.1 Promise<T> return types

An `async` function always returns a **Promise**. Its return type is written `Promise<T>`, where `T` is the type of the value the function `return`s when it resolves: `async function getMarks(): Promise<number[]>`. An async function that returns nothing has type `Promise<void>`. TypeScript reports an error if you annotate an async function with a non-Promise type.

ASYNC FUNCTIONS RETURN PROMISE<T>

main.ts

```
type Course = { code: string; title: string; fee: number };

const delay = (ms: number) => new Promise<void>((resolve) => setTimeout(resolve, ms));

async function fetchCourse(code: string): Promise<Course> {
  await delay(20);                                          // simulate a network call
  if (code !== "TS101") throw new Error(`course ${code} not found`);
  return { code, title: "TypeScript Programming", fee: 9999 };
}

async function announce(message: string): Promise<void> {
  await delay(10);
  console.log("announcement:", message);
}

fetchCourse("TS101").then((c) => console.log(c.title, c.fee));
announce("new batch starts Monday");
```

OUTPUT

```
announcement: new batch starts Monday
TypeScript Programming 9999
```

THE RETURN TYPE OF AN ASYNC FUNCTION MUST BE A PROMISE OF THE RETURNED VALUE

main.ts

```
async function count(): number {
  return 5;
}

async function getName(): Promise<string> {
  return 42;
}
```

COMPILER ERRORS (tsc --strict)

```
main.ts(1,25): error TS1064: The return type of an async function or method must be the global
Promise<T> type. Did you mean to write 'Promise<number>'?
main.ts(6,3): error TS2322: Type 'number' is not assignable to type 'string'.
```

4.6.2 Typing await results

`await` unwraps a promise: if `p` has type `Promise<Course>`, then `await p` has type `Course`, so in most code you never write the types of awaited values. `Promise.all` with a tuple of promises returns a tuple of results with each position typed correctly, and `Promise.allSettled` returns results you must check for `status`. Remember that a promise can reject: wrap awaits in `try`/`catch`, where the error has type `unknown` (13.3.4). The utility type `Awaited<T>` computes the unwrapped type (12.5.1).

AWAIT, PROMISE.ALL WITH TYPED TUPLES, AND ALLSETTLED

main.ts

```ts
type Course = { code: string; fee: number };
type Seat = { course: string; left: number };

const wait = <T>(value: T, ms: number) => new Promise<T>((resolve) => setTimeout(() => resolve(value),
ms));
const getCourse = (code: string) => wait<Course>({ code, fee: 9999 }, 30);
const getSeats = (code: string) => wait<Seat>({ course: code, left: 4 }, 10);
const getRating = (code: string): Promise<number> =>
  code === "TS101" ? wait(4.8, 5) : Promise.reject(new Error("no ratings yet"));

async function summary(code: string): Promise<string> {
  const course = await getCourse(code);                    // Course
  const [seats, rating] = await Promise.all([getSeats(code), getRating(code)]);   // [Seat, number]
  return `${course.code}: Rs ${course.fee}, ${seats.left} seats, rated ${rating.toFixed(1)}`;
}

async function main() {
  console.log(await summary("TS101"));
  try {
    console.log(await summary("PY201"));
  } catch (e: unknown) {
    console.log("failed:", e instanceof Error ? e.message : String(e));
  }
  const results = await Promise.allSettled([getRating("TS101"), getRating("JV301")]);
  for (const r of results) console.log(r.status === "fulfilled" ? `ok ${r.value}` : `error ${(r.reason
as Error).message}`);
}
main();
```

OUTPUT

```
TS101: Rs 9999, 4 seats, rated 4.8
failed: no ratings yet
ok 4.8
error no ratings yet
```

A function signature gives every parameter a type and (optionally) the return type; the compiler checks the body returns that type on every path and that every call matches. Return types can be inferred, but annotate exported functions; void means no useful result and never means the function never returns. Optional parameters (?) include undefined and come last; default values make a parameter optional without undefined; rest parameters (...xs: T[]) collect extra arguments; type destructured parameters after the whole pattern. Function type expressions ((a: number) => number) and named aliases type callbacks, whose parameters are then inferred; call signatures describe callable objects with properties and construct signatures (new (...) => T) describe classes. Arrow functions take the surrounding this; a this: Type pseudo-parameter types this in regular functions. Overloads list several call forms followed by one implementation, but prefer unions or generics when they suffice. Async functions return Promise<T>; await unwraps it, Promise.all keeps tuple types, and rejected promises are caught with catch (e: unknown).

PRACTICE CHECKPOINT

A **typed calculator** using a **function type for operations**, a function with **optional and default parameters**, and an **overloaded format function** that accepts a **string or a Date**.

SAMPLE SOLUTION: A TYPED CALCULATOR WITH A FUNCTION TYPE, OPTIONAL AND DEFAULT PARAMETERS, AND OVERLOADS main.ts

```
type Operation = (a: number, b: number) => number;
type OpSymbol = "+" | "-" | "*" | "/" | "%" | "^";

const operations: Record<OpSymbol, Operation> = {
  "+": (a, b) => a + b,
  "-": (a, b) => a - b,
  "*": (a, b) => a * b,
  "/": (a, b) => {
    if (b === 0) throw new RangeError("division by zero");
    return a / b;
  },
  "%": (a, b) => a % b,
  "^": Math.pow,
};

// optional (label) and default (decimals) parameters
function calculate(a: number, op: OpSymbol, b: number, decimals = 2, label?: string): string {
  const result = operations[op](a, b);
  const text = `${a} ${op} ${b} = ${Number(result.toFixed(decimals))}`;
  return label ? `[${label}] ${text}` : text;
}

// overloaded format: a Date, or an ISO date string, plus an optional style
function formatStamp(date: Date, style?: "short" | "long"): string;
function formatStamp(iso: string, style?: "short" | "long"): string;
function formatStamp(value: Date | string, style: "short" | "long" = "short"): string {
  const d = typeof value === "string" ? new Date(value) : value;
  if (Number.isNaN(d.getTime())) return "invalid date";
  return d.toLocaleString("en-IN", style === "short"
    ? { day: "2-digit", month: "2-digit", year: "numeric", timeZone: "UTC" }
    : { dateStyle: "full", timeZone: "UTC" });
}

const history: string[] = [];
const run = (a: number, op: OpSymbol, b: number, decimals?: number, label?: string) => {
  try {
    history.push(calculate(a, op, b, decimals, label));
  } catch (e: unknown) {
    history.push(`${a} ${op} ${b} failed: ${e instanceof Error ? e.message : e}`);
  }
};

run(12, "+", 30);
run(10, "/", 3);
run(10, "/", 3, 5, "precise");
run(2, "^", 10, undefined, "power");
run(7, "/", 0);

console.log(`Calculator log (${formatStamp("2026-10-07T09:30:00Z", "long")})`);
history.forEach((line, i) => console.log(`${i + 1}. ${line}`));
console.log("short date:", formatStamp(new Date(Date.UTC(2026, 0, 26))), "| bad:", formatStamp("not a
date"));
```

OUTPUT

```
Calculator log (Wednesday, 7 October 2026)
1. 12 + 30 = 42
2. 10 / 3 = 3.33
3. [precise] 10 / 3 = 3.33333
4. [power] 2 ^ 10 = 1024
5. 7 / 0 failed: division by zero
short date: 26/01/2026 | bad: invalid date
```

MODULE 05 · BEGINNER

Object types, type aliases and interfaces

Minimum duration **2 days** | Level **Beginner**

Object types describe the shape of data, and most of the types you write in real projects are object types. This module covers inline object types with optional and readonly properties and excess property checks, naming types with type aliases and interfaces (methods, extending, declaration merging), how to choose between interface and type, index signatures and noUncheckedIndexedAccess, nested and composed object types, and the structural ("duck") typing that makes TypeScript different from Java and C#.

IN THIS MODULE

5.1 Object types
5.2 Type aliases
5.3 Interfaces
5.4 Interface vs type alias
5.5 Index signatures
5.6 Nested and composed object types
5.7 Structural typing

5.1 Object types

5.1.1 Inline object type annotations

An **object type** lists the properties an object must have and the type of each: `{ name: string; marks: number }`. Properties are separated by semicolons or commas (semicolons are conventional). You can write an object type directly in an annotation ("inline"), which is fine for one-off use; for anything reused, give it a name (5.2, 5.3).

INLINE OBJECT TYPES — main.ts

```
let student: { name: string; roll: number; marks: number[] } = {
  name: "Asha",
  roll: 17,
  marks: [91, 84, 77],
};

function printCard(s: { name: string; roll: number }): void {
  console.log(`#${s.roll} ${s.name}`);
}

printCard(student);                       // extra property marks is fine here (5.1.4)
student.marks.push(95);
console.log(student.marks.length);
```

OUTPUT

```
#17 Asha
4
```

MISSING, WRONGLY TYPED AND UNKNOWN PROPERTIES main.ts

```
let student: { name: string; roll: number } = { name: "Asha" };
student.roll = "17";
console.log(student.age);
```

COMPILER ERRORS (tsc --strict)

```
main.ts(1,5): error TS2741: Property 'roll' is missing in type '{ name: string; }' but required in
type '{ name: string; roll: number; }'.
main.ts(2,1): error TS2322: Type 'string' is not assignable to type 'number'.
main.ts(3,21): error TS2339: Property 'age' does not exist on type '{ name: string; roll: number; }'.
```

5.1.2 Optional properties ?

A `?` after a property name makes it optional: the object may leave it out, and reading it gives a value whose type includes `undefined`. Use optional chaining (`?.`), nullish coalescing (`??`) or an explicit check before using it. With `exactOptionalPropertyTypes` (17.1.1) you also cannot explicitly assign `undefined` to an optional property unless its type includes it.

OPTIONAL PROPERTIES WITH ?. AND ?? main.ts

```
type Contact = { name: string; email?: string; phone?: string; address?: { city: string; pin?: string
} };

const people: Contact[] = [
  { name: "Asha", email: "asha@lbf.in" },
  { name: "Ravi", phone: "98450 12345", address: { city: "Mysuru", pin: "570001" } },
  { name: "Meera", address: { city: "Pune" } },
];

for (const p of people) {
  const reach = p.email ?? p.phone ?? "no contact";
  const pin = p.address?.pin ?? "-";
  console.log(`${p.name.padEnd(6)} ${reach.padEnd(14)} ${p.address?.city ?? "?"} ${pin}`);
}
```

OUTPUT

```
Asha   asha@lbf.in    ? -
Ravi   98450 12345    Mysuru 570001
Meera  no contact     Pune -
```

5.1.3 Readonly properties

`readonly` before a property name means it can be set when the object is created but never reassigned afterwards. It is ideal for identifiers, creation dates and configuration. Like all TypeScript checks it disappears at run time, and it is **shallow**: a readonly property that holds an array or object can still have its contents changed unless those are readonly too.

READONLY BLOCKS REASSIGNMENT OF THE PROPERTY ITSELF main.ts

```ts
type Order = { readonly id: number; readonly items: string[]; status: string };

const order: Order = { id: 101, items: ["pen"], status: "new" };
order.status = "shipped";          // fine
order.id = 102;                    // error
order.items = [];                  // error
order.items.push("book");          // allowed: readonly is shallow
```

COMPILER ERRORS (tsc --strict)

```
main.ts(5,7): error TS2540: Cannot assign to 'id' because it is a read-only property.
main.ts(6,7): error TS2540: Cannot assign to 'items' because it is a read-only property.
```

5.1.4 Excess property checks

Because of structural typing (5.7), an object with **extra** properties is normally assignable to a type that needs fewer. But when you write an **object literal directly** where a type is expected, TypeScript performs an **excess property check**: any property the type does not know about is an error, because it is usually a typo (`colour` for `color`) or a misunderstanding. The check does not apply when the object comes from a variable.

EXCESS PROPERTY CHECKS APPLY TO FRESH OBJECT LITERALS main.ts

```ts
type Options = { color?: string; size?: number };
function draw(o: Options) { return o; }

draw({ colour: "red", size: 3 });          // typo caught

const opts = { colour: "red", size: 3 };
draw(opts);                                 // no error: not a fresh literal

const bad: Options = { size: 3, weight: 10 };
```

COMPILER ERRORS (tsc --strict)

```
main.ts(4,8): error TS2561: Object literal may only specify known properties, but 'colour' does not
exist in type 'Options'. Did you mean to write 'color'?
main.ts(9,33): error TS2353: Object literal may only specify known properties, and 'weight' does not
exist in type 'Options'.
```

Notice that the second call is accepted: `opts` has a `size` property that fits, and the extra `colour` is ignored. This is one reason to annotate object variables at their declaration (`const opts: Options = {...}`), so the check happens where the literal is written.

5.2 Type aliases

5.2.1 type User = { ... }

A **type alias** gives a name to any type with the `type` keyword. Names are written in PascalCase by convention. An alias does not create a new type; it is simply another name for the same type, so values are compatible with anything of the same shape.

A TYPE ALIAS FOR AN OBJECT TYPE main.ts

```ts
type User = {
  readonly id: number;
  name: string;
  email: string;
  isAdmin?: boolean;
};

function displayName(user: User): string {
  return user.isAdmin ? `${user.name} (admin)` : user.name;
}

const users: User[] = [
  { id: 1, name: "Lohit", email: "lohit@lbf.in", isAdmin: true },
  { id: 2, name: "Asha", email: "asha@lbf.in" },
];

users.forEach((u) => console.log(u.id, displayName(u), u.email));
```

OUTPUT

```
1 Lohit (admin) lohit@lbf.in
2 Asha asha@lbf.in
```

5.2.2 Aliasing primitives, unions and functions

Unlike interfaces, aliases can name **any** type, not only object shapes: primitives (to document meaning), unions, tuples, function types and the results of type operators. This makes `type` the more general tool.

ALIASES FOR PRIMITIVES, UNIONS, TUPLES AND FUNCTIONS main.ts

```ts
type Rupees = number;                                  // documents meaning (not a separate type!)
type Id = string | number;                             // union
type Point = [x: number, y: number];                   // tuple
type Status = "draft" | "published" | "archived";      // union of literals
type Formatter = (value: number) => string;            // function type
type Product = { id: Id; title: string; price: Rupees; status: Status };

const rupee: Formatter = (v) => `Rs ${v.toLocaleString("en-IN")}`;

const catalogue: Product[] = [
  { id: 1, title: "TypeScript course", price: 9999, status: "published" },
  { id: "B-7", title: "Interview pack", price: 1499, status: "draft" },
];

const origin: Point = [0, 0];
for (const p of catalogue) console.log(String(p.id).padEnd(4), p.title.padEnd(18), rupee(p.price),
p.status);
console.log("origin", origin);
```

OUTPUT

```
1    TypeScript course  Rs 9,999 published
B-7  Interview pack     Rs 1,499 draft
origin [ 0, 0 ]
```

NOTE

`type Rupees = number` is only a label: a `Rupees` value and any other `number` are interchangeable, so the compiler will not stop you adding rupees to kilograms. To make such types incompatible, use **branded types** (15.2.1).

5.3 Interfaces

5.3.1 Declaring interfaces

An **interface** is the other way to name an object type: `interface Book { ... }`, with no equals sign. For plain object shapes, an interface and a type alias are almost interchangeable. Interfaces come from the object-oriented tradition (Java, C#), read naturally for "things" in your domain, and have two abilities aliases lack: `extends` with clear error messages and **declaration merging**.

DECLARING AND USING AN INTERFACE — main.ts

```ts
interface Book {
  isbn: string;
  title: string;
  author: string;
  year: number;
}

const books: Book[] = [
  { isbn: "978-1", title: "Effective TypeScript", author: "Dan Vanderkam", year: 2024 },
  { isbn: "978-2", title: "Programming TypeScript", author: "Boris Cherny", year: 2019 },
];

function newest(list: Book[]): Book {
  return list.reduce((a, b) => (b.year > a.year ? b : a));
}

console.log(newest(books).title);
```

OUTPUT

```
Effective TypeScript
```

5.3.2 Optional and readonly members

OPTIONAL AND READONLY MEMBERS IN AN INTERFACE — main.ts

```ts
interface Member {
  readonly memberId: number;
  name: string;
  phone?: string;
  readonly joined: Date;
}

const m: Member = { memberId: 501, name: "Kiran", joined: new Date(2026, 0, 5) };
m.phone = "99729 34418";                // optional member added later
console.log(m.memberId, m.name, m.phone, m.joined.getFullYear());
```

OUTPUT

```
501 Kiran 99729 34418 2026
```

5.3.3 Method signatures

Interfaces (and object type aliases) can declare **methods** in two ways: method syntax `describe(): string` or property syntax with a function type `describe: () => string`. They behave the same in most code; property syntax is checked slightly more strictly for parameter types under `strictFunctionTypes` (15.4.3), which is why some style guides prefer it.

METHOD SIGNATURES IN AN INTERFACE main.ts

```
interface Shape {
  name: string;
  area(): number;                              // method syntax
  scale: (factor: number) => Shape;            // property syntax
  describe?(): string;                         // optional method
}

function rectangle(w: number, h: number): Shape {
  return {
    name: "rectangle",
    area: () => w * h,
    scale: (f) => rectangle(w * f, h * f),
    describe() { return `${w} x ${h}`; },
  };
}

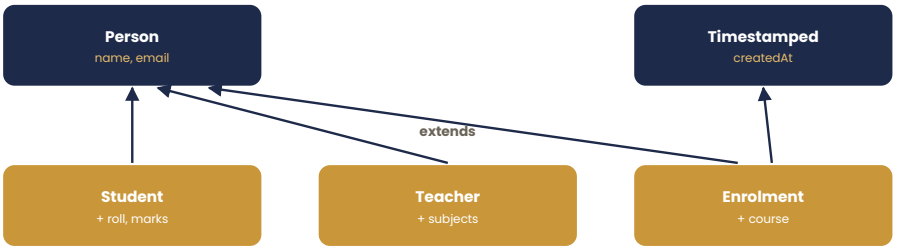
const r = rectangle(3, 4);
const big = r.scale(2);
console.log(r.area(), big.area(), big.describe?.() ?? "no description");
```

OUTPUT

```
12 48 6 x 8
```

5.3.4 Extending interfaces (single and multiple)

`interface B extends A` creates a type with all of A's members plus new ones. An interface can extend **several** interfaces at once (`extends A, B`), and it can extend a type alias of an object type too. Extending is the natural way to model "is a kind of" relationships and to build specific types from shared bases. If an extended member is redeclared, its type must be compatible with the original.



SINGLE AND MULTIPLE INTERFACE INHERITANCE

EXTENDING ONE OR MORE INTERFACES main.ts

```
interface Person { name: string; email: string }
interface Timestamped { createdAt: Date }

interface Student extends Person { roll: number; marks: number[] }
interface Teacher extends Person { subjects: string[] }
interface Enrolment extends Person, Timestamped { course: string }   // multiple

const s: Student = { name: "Asha", email: "asha@lbf.in", roll: 7, marks: [91, 84] };
const t: Teacher = { name: "Lohit", email: "lohit@lbf.in", subjects: ["TypeScript", "Java"] };
const e: Enrolment = { name: "Ravi", email: "ravi@lbf.in", course: "TypeScript", createdAt: new
Date(2026, 8, 1) };

function contact(p: Person): string { return `${p.name} <${p.email}>`; }   // accepts all three
[s, t, e].forEach((p) => console.log(contact(p)));
console.log(e.createdAt.toDateString(), t.subjects.join(" & "));
```

OUTPUT

```
Asha <asha@lbf.in>
Lohit <lohit@lbf.in>
Ravi <ravi@lbf.in>
Tue Sep 01 2026 TypeScript & Java
```

AN EXTENDING INTERFACE CANNOT CHANGE A MEMBER TO AN INCOMPATIBLE TYPE main.ts

```
interface Person { name: string; age: number }
interface Employee extends Person { age: string }
```

COMPILER ERRORS (tsc --strict)

```
main.ts(2,11): error TS2430: Interface 'Employee' incorrectly extends interface 'Person'.
  Types of property 'age' are incompatible.
    Type 'string' is not assignable to type 'number'.
```

5.3.5 Declaration merging

If you declare an interface with the same name twice in the same scope, TypeScript **merges** them into one interface with the members of both. Type aliases cannot do this (a duplicate alias is an error). Merging is mostly used to **extend existing types** that you do not own: adding a property to the global `Window`, adding a field to Express's `Request`, or adding methods to a library's types (module augmentation, 11.4.5 and 15.5). In your own code, avoid splitting one interface across declarations: it makes the type hard to find.

THREE DECLARATIONS OF APPCONFIG MERGE INTO ONE main.ts

```
interface AppConfig { apiUrl: string }
interface AppConfig { timeoutMs: number }          // merged with the first declaration
interface AppConfig { retries?: number }

const config: AppConfig = { apiUrl: "https://api.lbf.in", timeoutMs: 5000 };
console.log(config);
```

OUTPUT

```
{ apiUrl: 'https://api.lbf.in', timeoutMs: 5000 }
```

TYPE ALIASES CANNOT BE MERGED main.ts

```
type AppConfig = { apiUrl: string };
type AppConfig = { timeoutMs: number };
```

COMPILER ERRORS (tsc --strict)

```
main.ts(1,6): error TS2300: Duplicate identifier 'AppConfig'.
main.ts(2,6): error TS2300: Duplicate identifier 'AppConfig'.
```

5.4 Interface vs type alias

5.4.1 Differences and similarities

Feature	interface	type alias
describe an object shape	yes	yes
optional, readonly, methods, index signatures	yes	yes
name a union, tuple, primitive or function type	no	yes
extend / combine	`extends` (one or many)	intersection `&` (6.2)
declaration merging	yes	no (duplicate name error)
implemented by a class (`implements`)	yes	yes, if it is an object type
mapped and conditional types	no	yes (Module 14)
error messages and performance	shows the name; extends is cached	may expand the full type; deep intersections can be slower

INTERFACES AND TYPE ALIASES WITH THE SAME SHAPE ARE COMPATIBLE main.ts

```
// The same shape written both ways is fully interchangeable
interface PointI { x: number; y: number }
type PointT = { x: number; y: number };

const a: PointI = { x: 1, y: 2 };
const b: PointT = a;                    // compatible: same structure
const c: PointI = b;

// extends vs intersection
interface Point3I extends PointI { z: number }
type Point3T = PointT & { z: number };
const p: Point3I = { x: 1, y: 2, z: 3 };
const q: Point3T = p;
console.log(c, q);
```

OUTPUT

```
{ x: 1, y: 2 } { x: 1, y: 2, z: 3 }
```

5.4.2 Team conventions for choosing one

Both are fine; consistency matters more than the choice. Two common conventions: (1) the TypeScript handbook's advice, **use interface for object shapes until you need a feature of type**, which many teams follow because of better error messages and the extends performance benefit; (2) **use type for everything**, because it is one tool that works everywhere (unions, mapped types), reserving interface for when merging is needed. Whichever you choose, enforce it with the ESLint rule `@typescript-eslint/consistent-type-definitions`. This book generally uses interfaces for domain objects and aliases for unions and computed types.

5.5 Index signatures

5.5.1 [key: string]: number

Sometimes you do not know the property names in advance: a dictionary of marks by subject, word counts, HTTP headers. An **index signature** describes such objects: `{ [subject: string]: number }` means "any string key, each with a number value". The key type can be `string`, `number`, `symbol`, template literal types or a union of these. Known properties can sit alongside the signature, but their types must be compatible with it.

INDEX SIGNATURES FOR DICTIONARIES — main.ts

```ts
interface MarksBySubject {
  [subject: string]: number;
}

const marks: MarksBySubject = { maths: 91, physics: 84 };
marks.chemistry = 77;                          // any string key is allowed
marks["computer science"] = 95;

for (const [subject, mark] of Object.entries(marks)) console.log(subject.padEnd(17), mark);

function wordCounts(text: string): { [word: string]: number } {
  const counts: { [word: string]: number } = {};
  for (const w of text.toLowerCase().match(/[a-z]+/g) ?? []) counts[w] = (counts[w] ?? 0) + 1;
  return counts;
}
console.log(wordCounts("Learn TypeScript, practise TypeScript, master TypeScript"));
```

OUTPUT

```
maths             91
physics           84
chemistry         77
computer science  95
{ learn: 1, typescript: 3, practise: 1, master: 1 }
```

VALUES AND KNOWN PROPERTIES MUST MATCH THE INDEX SIGNATURE main.ts

```ts
interface Headers {
  [name: string]: string;
  contentLength: number;        // must be compatible with the index signature
}

const marks: { [subject: string]: number } = { maths: 91 };
marks.physics = "eighty";
```

COMPILER ERRORS (tsc --strict)

```
main.ts(3,3): error TS2411: Property 'contentLength' of type 'number' is not assignable to 'string'
index type 'string'.
main.ts(7,1): error TS2322: Type 'string' is not assignable to type 'number'.
```

5.5.2 Record<string, number> (preview)

The built-in utility type `Record<K, V>` is a shorter way to write an object type with keys `K` and values `V`: `Record<string, number>` is equivalent to `{ [key: string]: number }`. Its real power appears when `K` is a **union of literals**: `Record<"maths" | "physics", number>` requires **exactly** those keys, so forgetting one is an error. Module 12 covers Record and the other utility types.

RECORD WITH STRING KEYS AND WITH A UNION OF KEYS main.ts

```ts
type Subject = "maths" | "physics" | "chemistry";

const passMarks: Record<Subject, number> = { maths: 35, physics: 33, chemistry: 33 };
const prices: Record<string, number> = { pen: 20, notebook: 60 };

function passed(scores: Record<Subject, number>): Subject[] {
  return (Object.keys(scores) as Subject[]).filter((s) => scores[s] >= passMarks[s]);
}

console.log(passed({ maths: 72, physics: 30, chemistry: 41 }), prices.pen);
```

OUTPUT

```
[ 'maths', 'chemistry' ] 20
```

A RECORD OVER A UNION REQUIRES EVERY KEY main.ts

```ts
type Subject = "maths" | "physics" | "chemistry";
const passMarks: Record<Subject, number> = { maths: 35, physics: 33 };
```

COMPILER ERRORS (tsc --strict)

```
main.ts(2,7): error TS2741: Property 'chemistry' is missing in type '{ maths: number; physics: number;
}' but required in type 'Record<Subject, number>'.
```

5.5.3 noUncheckedIndexedAccess

With a plain index signature, TypeScript assumes every key exists: `marks["biology"]` has type `number`, even though it is `undefined` at run time. The compiler option `noUncheckedIndexedAccess` fixes this hole by adding `| undefined` to every index access on index signatures and arrays, forcing a check. It is not part of `strict` but is turned on by `tsc --init` in TypeScript 6 and recommended for new projects. Iterating with `for...of` or `Object.entries` is unaffected, because those values definitely exist.

TSCONFIG.JSON JavaScript

```
{ "compilerOptions": { "strict": true, "noUncheckedIndexedAccess": true, "noEmit": true, "types": []
}, "files": ["marks.ts"] }
```

MARKS.TS TypeScript

```
const marks: Record<string, number> = { maths: 91, physics: 84 };
const scores: number[] = [78, 91];

const bio = marks["biology"];          // number | undefined
console.log(bio.toFixed(1));           // error: possibly undefined

const first = scores[0];               // number | undefined
console.log(first + 1);                // error

if (bio !== undefined) console.log(bio.toFixed(1));   // OK after a check
for (const s of scores) console.log(s.toFixed(1));     // OK: for...of values exist
```

WITH NOUNCHECKEDINDEXEDACCESS, INDEX ACCESS MAY BE UNDEFINED terminal

```
npx tsc --noEmit -p .
```

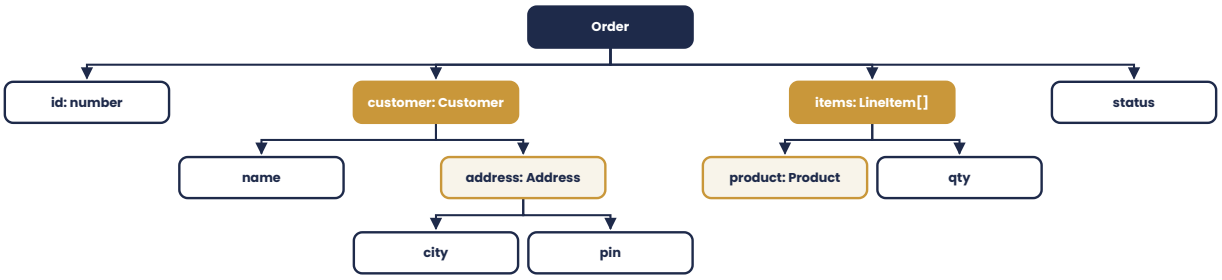
TERMINAL

```
$ npx tsc --noEmit -p .
marks.ts(5,13): error TS18048: 'bio' is possibly 'undefined'.
marks.ts(8,13): error TS18048: 'first' is possibly 'undefined'.
```

5.6 Nested and composed object types

5.6.1 Objects inside objects and arrays of objects

Real data is nested: an order has a customer, a list of line items and an address; each line item has a product. You can write nested types inline, but it is clearer to give each level its own named type and **compose** them. Named parts can be reused (the same `Address` for customers and branches) and produce readable error messages.



AN ORDER TYPE COMPOSED FROM SMALLER NAMED TYPES

A NESTED ORDER MODEL BUILT FROM SMALL INTERFACES main.ts

```
interface Address { line1: string; city: string; pin: string }
interface Customer { name: string; phone: string; address: Address }
interface Product { sku: string; title: string; price: number }
interface LineItem { product: Product; qty: number }
interface Order {
  readonly id: number;
  customer: Customer;
  items: LineItem[];
  status: "placed" | "shipped" | "delivered";
  deliveryAddress?: Address;              // falls back to the customer's address
}

const order: Order = {
  id: 9001,
  customer: { name: "Asha", phone: "98450 12345", address: { line1: "12 Temple Rd", city: "Mysuru",
pin: "570001" } },
  items: [
    { product: { sku: "NB-01", title: "Notebook", price: 60 }, qty: 4 },
    { product: { sku: "PN-02", title: "Gel pen", price: 25 }, qty: 10 },
  ],
  status: "placed",
};

const total = order.items.reduce((sum, { product, qty }) => sum + product.price * qty, 0);
const ship = order.deliveryAddress ?? order.customer.address;
console.log(`Order ${order.id} for ${order.customer.name}: ${order.items.length} lines, Rs ${total}`);
console.log(`Ship to ${ship.line1}, ${ship.city} ${ship.pin}`);
console.log(order.items.map((i) => `${i.qty} x ${i.product.title}`).join(", "));
```

OUTPUT

```
Order 9001 for Asha: 2 lines, Rs 490
Ship to 12 Temple Rd, Mysuru 570001
4 x Notebook, 10 x Gel pen
```

ERRORS IN NESTED OBJECTS POINT TO THE EXACT PROPERTY main.ts

```
interface Address { city: string; pin: string }
interface Customer { name: string; address: Address }

const c: Customer = { name: "Ravi", address: { city: "Pune", pin: 411001 } };
```

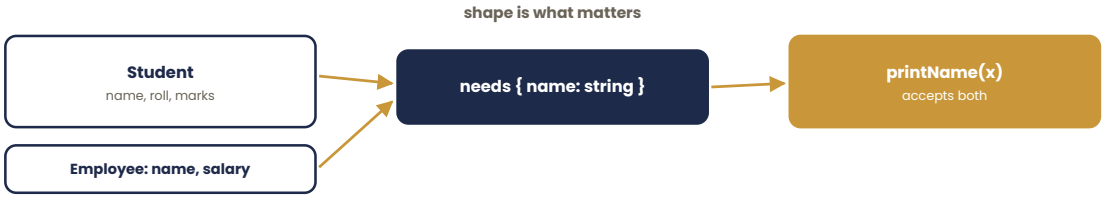
COMPILER ERRORS (tsc --strict)

```
main.ts(4,62): error TS2322: Type 'number' is not assignable to type 'string'.
```

5.7 Structural typing

5.7.1 "If it looks like a duck" compatibility

TypeScript's type system is **structural**: a value is compatible with a type if it has **at least the required properties with compatible types**, regardless of what the value's type is called or where it was declared. This is often called "duck typing" ("if it walks like a duck and quacks like a duck, it is a duck"), checked at compile time. It matches how JavaScript code actually works: functions care about the properties they use, not about class names.



ANY VALUE WITH THE REQUIRED PROPERTIES FITS, WHATEVER ITS TYPE IS CALLED

STRUCTURAL COMPATIBILITY ACROSS CLASSES, INTERFACES AND PLAIN OBJECTS main.ts

```
interface Named { name: string }

class Student { constructor(public name: string, public roll: number) {} }
interface Employee { name: string; salary: number }

function printName(x: Named): void { console.log("Name:", x.name); }

const emp: Employee = { name: "Lohit", salary: 90000 };
const city = { name: "Mysuru", population: 1_200_000 };

printName(new Student("Asha", 7));    // a class instance
printName(emp);                        // a different interface
printName(city);                       // a plain object: it has a name, so it fits

interface Point { x: number; y: number }
interface Vector { x: number; y: number }
const p: Point = { x: 3, y: 4 };
const v: Vector = p;                   // identical structures are interchangeable
console.log(Math.hypot(v.x, v.y));
```

OUTPUT

```
Name: Asha
Name: Lohit
Name: Mysuru
5
```

5.7.2 Nominal vs structural type systems

Java, C# and Swift use **nominal** typing: two types are compatible only if they are the same declared type or one explicitly inherits from or implements the other, so a `Point` is never a `Vector` even with identical fields. TypeScript chose structural typing because JavaScript objects are created everywhere without classes (object literals, JSON, library results), and a nominal system would make ordinary JavaScript patterns painful. The trade-off: values with the same shape but different meanings (a `UserId` and an `OrderId`, both `number`; rupees and dollars) are not distinguished. When that matters, you can simulate nominal types with **brands** (15.2.1) or use class private fields, which make classes nominal.

	Nominal (Java, C#)	Structural (TypeScript)
compatible when	same declared type or explicit inheritance	required properties match
Point vs Vector with x, y	incompatible	compatible
plain object as an interface	impossible: must be an instance	fine if the shape fits
strength	distinguishes same-shaped concepts	fits JavaScript, flexible, less boilerplate
simulate the other	interfaces for flexibility	branded types, private fields (15.2.1)

Object types list properties and their types; ? makes a property optional (its type includes undefined) and readonly blocks reassignment (shallowly). Fresh object literals get excess property checks that catch typos. type aliases name any type (objects, primitives, unions, tuples, functions); interfaces name object shapes, support methods, extend one or more interfaces, and merge when declared twice. They are interchangeable for plain shapes; pick one convention and enforce it. Index signatures ([key: string]: T) and Record<K, V> type dictionaries; noUncheckedIndexedAccess makes index access include undefined. Compose nested data from small named types. TypeScript is structural: a value fits a type if it has the required properties, whatever its declared name, unlike nominal Java and C#.

PRACTICE CHECKPOINT

Model a **Library system** with **Book**, **Member** and **Loan** interfaces (including **optional, readonly and nested** properties) and functions that use them.

SAMPLE SOLUTION: A LIBRARY MODEL WITH NESTED, OPTIONAL AND READONLY MEMBERS

main.ts

```
interface Address { city: string; pin: string }

interface Book {
  readonly isbn: string;
  title: string;
  authors: readonly string[];
  year: number;
  copies: number;
  tags?: string[];
}

interface Member {
  readonly id: number;
  name: string;
  email: string;
  phone?: string;
  address: Address;
  plan: "basic" | "premium";
}

interface Loan {
  readonly loanId: number;
  book: Pick<Book, "isbn" | "title">;
  memberId: Member["id"];
  readonly issued: Date;
  due: Date;
  returned?: Date;
}

interface Library {
  books: Book[];
  members: Member[];
  loans: Loan[];
}

const DAY = 24 * 60 * 60 * 1000;
const lib: Library = {
  books: [
    { isbn: "978-1", title: "Effective TypeScript", authors: ["Dan Vanderkam"], year: 2024, copies: 2,
tags: ["typescript"] },
    { isbn: "978-2", title: "Clean Code", authors: ["Robert C. Martin"], year: 2008, copies: 1 },
  ],
  members: [
```

```
    { id: 1, name: "Asha", email: "asha@lbf.in", address: { city: "Mysuru", pin: "570001" }, plan:
"premium" },
    { id: 2, name: "Ravi", email: "ravi@lbf.in", phone: "98450 12345", address: { city: "Pune", pin:
"411001" }, plan: "basic" },
  ],
  loans: [],
};

function available(library: Library, isbn: string): number {
  const book = library.books.find((b) => b.isbn === isbn);
  if (!book) return 0;
  const out = library.loans.filter((l) => l.book.isbn === isbn && !l.returned).length;
  return book.copies - out;
}

function issue(library: Library, isbn: string, memberId: number, on: Date): Loan | string {
  const book = library.books.find((b) => b.isbn === isbn);
  const member = library.members.find((m) => m.id === memberId);
  if (!book || !member) return "unknown book or member";
  if (available(library, isbn) < 1) return `no copies of "${book.title}" left`;
  const days = member.plan === "premium" ? 21 : 14;
  const loan: Loan = { loanId: library.loans.length + 1, book: { isbn, title: book.title }, memberId,
issued: on, due: new Date(on.getTime() + days * DAY) };
  library.loans.push(loan);
  return loan;
}

function overdue(library: Library, today: Date): string[] {
  return library.loans
    .filter((l) => !l.returned && l.due < today)
    .map((l) => {
      const m = library.members.find((x) => x.id === l.memberId);
      const late = Math.ceil((today.getTime() - l.due.getTime()) / DAY);
      return `${m?.name ?? "?"} (${m?.address.city}): "${l.book.title}", ${late} days late, contact
${m?.phone ?? m?.email}`;
    });
}

const start = new Date(2026, 8, 1);
for (const [isbn, member] of [["978-2", 2], ["978-2", 1], ["978-1", 1]] as const) {
  const r = issue(lib, isbn, member, start);
  console.log(typeof r === "string" ? `refused: ${r}` : `loan ${r.loanId}: "${r.book.title}" due
${r.due.toDateString()}`);
}
console.log("Effective TypeScript copies left:", available(lib, "978-1"));
console.log(overdue(lib, new Date(2026, 8, 20)));
```

OUTPUT

```
loan 1: "Clean Code" due Tue Sep 15 2026
refused: no copies of "Clean Code" left
loan 2: "Effective TypeScript" due Tue Sep 22 2026
Effective TypeScript copies left: 1
[ 'Ravi (Pune): "Clean Code", 5 days late, contact 98450 12345' ]
```

MODULE 06 · BEGINNER

Union, intersection and literal types

Minimum duration **2 days** | Level **Beginner**

Combining types is how TypeScript models real-world data that can take several forms. This module covers union types (A | B) and how to work with them safely, unions of object types, intersection types (A & B) and how they differ from extending interfaces, literal types and unions of literals, literal inference and as const, enums (numeric, string and const) and why many teams prefer literal unions, erasableSyntaxOnly, and handling null and undefined with optional chaining and nullish coalescing.

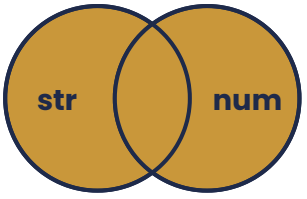
IN THIS MODULE

6.1 Union types
6.2 Intersection types
6.3 Literal types
6.4 Enums
6.5 null and undefined handling

6.1 Union types

6.1.1 string | number

A **union type** `A | B` means "a value that is either an A or a B". Read the `|` as "or". Unions describe values that can legitimately take several forms: an ID that may be a number or a string, a function argument that accepts a single item or an array, a result that may be a value or `null`. A union can have any number of members: `string | number | boolean`.



STRING | NUMBER: A VALUE FROM EITHER SET

A VARIABLE AND A PARAMETER WITH UNION TYPES main.ts

```ts
let rollNo: string | number;
rollNo = 17;
rollNo = "LBF-017";

type Id = string | number;
function formatId(id: Id): string {
  return `ID[${id}]`;
}

const ids: Id[] = [101, "A-7", 202];
console.log(rollNo, ids.map(formatId).join(" "));
```

OUTPUT

```
LBF-017 ID[101] ID[A-7] ID[202]
```

ONLY MEMBERS OF THE UNION ARE ALLOWED main.ts

```ts
let rollNo: string | number = 17;
rollNo = true;
rollNo = null;
```

COMPILER ERRORS (tsc --strict)

```
main.ts(2,1): error TS2322: Type 'boolean' is not assignable to type 'string | number'.
main.ts(3,1): error TS2322: Type 'null' is not assignable to type 'string | number'.
```

6.1.2 Working with unions safely (introduction to narrowing)

With a union you can only use operations that are valid for **every** member. `toUpperCase()` exists on strings but not on numbers, so it is not allowed on `string | number` until you check which one you have. A check such as `typeof id === "string"` **narrows** the type inside the branch: TypeScript's control-flow analysis knows that in the `if` block `id` is a `string`, and in the `else` block a `number`. Module 8 covers all the narrowing techniques; `typeof`, `Array.isArray` and equality checks are enough for now.

A METHOD THAT EXISTS ONLY ON ONE MEMBER CANNOT BE CALLED DIRECTLY main.ts

```ts
function shout(id: string | number) {
  return id.toUpperCase();
}
```

COMPILER ERRORS (tsc --strict)

```
main.ts(2,13): error TS2339: Property 'toUpperCase' does not exist on type 'string | number'.
  Property 'toUpperCase' does not exist on type 'number'.
```

NARROWING A UNION WITH TYPEOF AND ARRAY.ISARRAY

main.ts

```ts
function shout(id: string | number): string {
  if (typeof id === "string") {
    return id.toUpperCase();                // id: string here
  }
  return id.toFixed(0).padStart(4, "0");     // id: number here
}

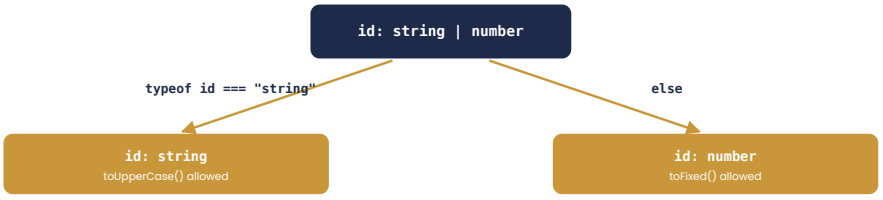
function total(input: number | number[]): number {
  return Array.isArray(input) ? input.reduce((a, b) => a + b, 0) : input;
}

function len(x: string | unknown[]): number {
  return x.length;                           // OK: both members have length
}

console.log(shout("lbf"), shout(7), total(5), total([1, 2, 3]), len("hello"), len([1, 2]));
```

OUTPUT

```
LBF 0007 5 6 5 2
```



CONTROL-FLOW NARROWING SPLITS A UNION INTO ITS MEMBERS

6.1.3 Unions of object types

Unions of object types model data that comes in several shapes. When the shapes share a property, you can read that property directly. To use a property that only some members have, check for it, for example with the `in` operator. The most robust pattern gives every member a common **literal** property such as `kind` or `type`; checking it narrows to exactly one member. This is a **discriminated union**, one of the most important patterns in TypeScript (8.2).

A DISCRIMINATED UNION AND NARROWING WITH THE IN OPERATOR main.ts

```
type Circle = { kind: "circle"; radius: number };
type Rectangle = { kind: "rectangle"; width: number; height: number };
type Shape = Circle | Rectangle;

function area(s: Shape): number {
  if (s.kind === "circle") {
    return Math.PI * s.radius ** 2;          // s: Circle
  }
  return s.width * s.height;                 // s: Rectangle
}

const shapes: Shape[] = [{ kind: "circle", radius: 2 }, { kind: "rectangle", width: 3, height: 4 }];
for (const s of shapes) console.log(s.kind.padEnd(9), area(s).toFixed(2));

type Teacher = { name: string; subjects: string[] };
type Learner = { name: string; batch: number };
function describe(p: Teacher | Learner): string {
  return "subjects" in p ? `${p.name} teaches ${p.subjects.join(", ")}` : `${p.name} is in batch
${p.batch}`;
}
console.log(describe({ name: "Lohit", subjects: ["TypeScript"] }), "|", describe({ name: "Asha",
batch: 12 }));
```

OUTPUT

```
circle    12.57
rectangle 12.00
Lohit teaches TypeScript | Asha is in batch 12
```

A PROPERTY THAT EXISTS ON ONLY ONE MEMBER NEEDS A CHECK FIRST main.ts

```
type Circle = { kind: "circle"; radius: number };
type Rectangle = { kind: "rectangle"; width: number; height: number };

function area(s: Circle | Rectangle) {
  return s.width * s.height;
}
```

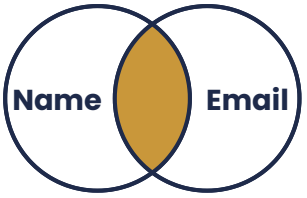
COMPILER ERRORS (tsc --strict)

```
main.ts(5,12): error TS2339: Property 'width' does not exist on type 'Circle | Rectangle'.
  Property 'width' does not exist on type 'Circle'.
main.ts(5,22): error TS2339: Property 'height' does not exist on type 'Circle | Rectangle'.
  Property 'height' does not exist on type 'Circle'.
```

6.2 Intersection types

6.2.1 A & B

An **intersection type** `A & B` means "a value that is **both** an A and a B": it must have every property of A and every property of B. Read `&` as "and". For object types, an intersection combines their properties. The name comes from set theory: the values that belong to both types. For primitive types the intersection is usually empty, so `string & number` is `never`.



NAMED & EMAILABLE: A VALUE THAT SATISFIES BOTH TYPES (IT HAS BOTH SETS OF PROPERTIES)

6.2.2 Combining object types

BUILDING A TYPE FROM REUSABLE PIECES WITH &

main.ts

```ts
type HasId = { id: number };
type HasTimestamps = { createdAt: Date; updatedAt?: Date };
type Person = { name: string; email: string };

type User = HasId & HasTimestamps & Person;                    // all properties of all three

const u: User = { id: 1, name: "Asha", email: "asha@lbf.in", createdAt: new Date(2026, 0, 5) };

function withAudit<T extends object>(entity: T, by: string): T & { auditedBy: string } {
  return { ...entity, auditedBy: by };
}

const audited = withAudit(u, "admin");
console.log(audited.name, audited.id, audited.auditedBy, audited.createdAt.getFullYear());
```

OUTPUT

```
Asha 1 admin 2026
```

AN INTERSECTION REQUIRES EVERY PROPERTY; CONFLICTING PRIMITIVES GIVE NEVER

main.ts

```ts
type HasId = { id: number };
type Person = { name: string };
const p: HasId & Person = { id: 1 };

type Impossible = string & number;           // never
const x: Impossible = "a";
```

COMPILER ERRORS (tsc --strict)

```
main.ts(3,7): error TS2322: Type '{ id: number; }' is not assignable to type 'HasId & Person'.
  Property 'name' is missing in type '{ id: number; }' but required in type 'Person'.
main.ts(6,7): error TS2322: Type '"a"' is not assignable to type 'never'.
```

6.2.3 Intersection vs interface extension

`interface B extends A` and `type B = A & {...}` usually produce the same shape, but they treat **conflicts** differently. `extends` checks that redeclared members are compatible and reports an error at the declaration. An intersection never reports an error: conflicting property types are intersected too, often producing `never` for that property, and the problem only shows up when you try to create a value. Interfaces also give shorter error messages and are cached by the compiler, so for building object types from a base, `extends` is generally preferable; intersections are the tool for combining types on the fly, mixins and generic helpers.

EXTENDS REPORTS CONFLICTS AT ONCE; INTERSECTIONS SILENTLY PRODUCE NEVER main.ts

```
interface Base { status: string }
interface Child extends Base { status: number }        // error here, at the declaration

type ChildT = Base & { status: number };               // no error here ...
const c: ChildT = { status: 1 };                        // ... but status is now string & number =
never
```

COMPILER ERRORS (tsc --strict)

```
main.ts(2,11): error TS2430: Interface 'Child' incorrectly extends interface 'Base'.
  Types of property 'status' are incompatible.
    Type 'number' is not assignable to type 'string'.
main.ts(5,21): error TS2322: Type 'number' is not assignable to type 'never'.
```

6.3 Literal types

6.3.1 String, number and boolean literal types

A **literal type** is a type that contains exactly one value: `"GET"` is the type whose only value is the string `"GET"`; `42` is a type; `true` is a type. On its own a literal type is not very useful, but combined in unions it describes a fixed set of allowed values precisely, something JavaScript developers previously did with comments and runtime checks.

SINGLE-VALUE LITERAL TYPES main.ts

```
let method: "GET" = "GET";
const answer: 42 = 42;
let yes: true = true;
console.log(method, answer, yes);
```

OUTPUT

```
GET 42 true
```

ONLY THE LITERAL VALUE IS ALLOWED main.ts

```
let method: "GET" = "GET";
method = "POST";
const dice: 1 | 2 | 3 | 4 | 5 | 6 = 7;
```

COMPILER ERRORS (tsc --strict)

```
main.ts(2,1): error TS2322: Type '"POST"' is not assignable to type '"GET"'.
main.ts(3,7): error TS2322: Type '7' is not assignable to type '1 | 2 | 3 | 4 | 5 | 6'.
```

6.3.2 Unions of literals "small" | "medium" | "large"

A union of literal types works like a lightweight **enumeration**: `type Size = "small" | "medium" | "large"`. The editor autocompletes the allowed values, typos are compile errors, and in a `switch` TypeScript can check that every value is handled (8.2.3). This is the idiomatic way to model statuses, modes, directions, HTTP methods and similar fixed sets in TypeScript.

UNIONS OF LITERALS FOR SIZES, HTTP METHODS, DICE AND ORDER STATES

main.ts

```
type Size = "small" | "medium" | "large";
type HttpMethod = "GET" | "POST" | "PUT" | "DELETE";
type DiceRoll = 1 | 2 | 3 | 4 | 5 | 6;
type OrderStatus = "pending" | "paid" | "shipped" | "delivered" | "cancelled";

const PRICES: Record<Size, number> = { small: 30, medium: 45, large: 60 };

function teaPrice(size: Size, extraShot = false): number {
  return PRICES[size] + (extraShot ? 10 : 0);
}

function request(method: HttpMethod, url: string): string {
  return `${method} ${url}`;
}

function roll(): DiceRoll {
  return (Math.floor(Math.random() * 6) + 1) as DiceRoll;
}

const next: Record<OrderStatus, OrderStatus | null> = {
  pending: "paid", paid: "shipped", shipped: "delivered", delivered: null, cancelled: null,
};

console.log(teaPrice("medium"), teaPrice("large", true), request("POST", "/api/orders"));
console.log("rolled a valid die:", [1, 2, 3, 4, 5, 6].includes(roll()));
let status: OrderStatus = "pending";
while (status) {
  process.stdout.write(status + " -> ");
  const n: OrderStatus | null = next[status];
  if (!n) break;
  status = n;
}
console.log("end");
```

OUTPUT

```
45 70 POST /api/orders
rolled a valid die: true
pending -> paid -> shipped -> delivered -> end
```

TYPOS AND UNKNOWN VALUES ARE COMPILE ERRORS

main.ts

```
type Size = "small" | "medium" | "large";
function teaPrice(size: Size) { return size; }

teaPrice("Medium");
teaPrice("xl");
```

COMPILER ERRORS (tsc --strict)

```
main.ts(4,10): error TS2345: Argument of type '"Medium"' is not assignable to parameter of type
'Size'.
main.ts(5,10): error TS2345: Argument of type '"xl"' is not assignable to parameter of type 'Size'.
```

6.3.3 Literal inference and as const

As 3.2.3 showed, `const` variables keep literal types but object properties and array elements are widened (`{ method: "GET" }` has type `{ method: string }`). This causes errors when an object is later passed where a literal union is required. The `as const` assertion fixes it: it makes TypeScript infer the narrowest type for the whole expression, with every property `readonly` and every value a literal, and arrays become

readonly tuples. It is also the standard way to derive a union type from a list of values: `typeof SIZES[number]`.

OBJECT PROPERTIES ARE WIDENED TO STRING main.ts

```
type HttpMethod = "GET" | "POST";
function request(method: HttpMethod, url: string) { return `${method} ${url}`; }

const req = { method: "GET", url: "/api/courses" };     // method: string (widened)
request(req.method, req.url);
```

COMPILER ERRORS (tsc --strict)

```
main.ts(5,9): error TS2345: Argument of type 'string' is not assignable to parameter of type
'HttpMethod'.
```

AS CONST KEEPS LITERAL TYPES AND DERIVES UNION TYPES FROM VALUES main.ts

```
type HttpMethod = "GET" | "POST";
function request(method: HttpMethod, url: string) { return `${method} ${url}`; }

const req = { method: "GET", url: "/api/courses" } as const;   // method: "GET", readonly
console.log(request(req.method, req.url));

const SIZES = ["small", "medium", "large"] as const;            // readonly ["small", "medium",
"large"]
type Size = (typeof SIZES)[number];                              // "small" | "medium" | "large"

function isSize(value: string): value is Size {                  // a type guard (8.3)
  return (SIZES as readonly string[]).includes(value);
}

for (const input of ["medium", "huge"]) console.log(input, isSize(input) ? "is a valid size" : "is not
a size");

const ROLES = { admin: "ADMIN", editor: "EDITOR", viewer: "VIEWER" } as const;
type Role = (typeof ROLES)[keyof typeof ROLES];                  // "ADMIN" | "EDITOR" | "VIEWER"
const r: Role = ROLES.editor;
console.log(r, SIZES.length);
```

OUTPUT

```
GET /api/courses
medium is a valid size
huge is not a size
EDITOR 3
```

The last pattern, a **constant object with `as const`** plus a derived union type, gives you named constants (`ROLES.editor`) and a type, with no extra JavaScript. It is the modern replacement for many enums (6.4.4). The type operators `typeof`, `keyof` and indexed access `T[K]` used here are explained fully in 14.1.

6.4 Enums

An **enum** ("enumeration") is a TypeScript feature that defines a set of named constants: `enum Direction { Up, Down, Left, Right }`. Unlike most TypeScript syntax, an enum is not just a type: it **generates a JavaScript object** at run time. Enums are familiar to developers coming from Java and C#, and you will meet them in existing codebases and frameworks.

6.4.1 Numeric enums

By default enum members are numbered from 0. You can set the first value (later ones continue counting) or every value explicitly. Numeric enums get a **reverse mapping**: `Direction[0]` gives `"Up"`. A weakness: any number is assignable to a numeric enum type in some situations, and the values are opaque when logged (you see `2`, not a name).

NUMERIC ENUMS AND REVERSE MAPPINGS main.ts

```
enum Direction { Up, Down, Left, Right }                    // 0, 1, 2, 3
enum HttpStatus { OK = 200, Created = 201, BadRequest = 400, NotFound = 404 }
enum Priority { Low = 1, Medium, High }                       // 1, 2, 3

function move(d: Direction): string {
  switch (d) {
    case Direction.Up: return "y - 1";
    case Direction.Down: return "y + 1";
    case Direction.Left: return "x - 1";
    case Direction.Right: return "x + 1";
  }
}

console.log(Direction.Left, Direction[2], move(Direction.Up));     // value, reverse mapping
console.log(HttpStatus.NotFound, Priority.High, HttpStatus[201]);
console.log(Object.keys(Direction));                                  // includes the reverse keys
```

OUTPUT

```
2 Left y - 1
404 3 Created
[
  '0',    '1',
  '2',    '3',
  'Up',   'Down',
  'Left', 'Right'
]
```

STATUS.TS TypeScript

```
enum Priority { Low = 1, Medium, High }
const p: Priority = Priority.High;
```

AN ENUM COMPILES TO A REAL JAVASCRIPT OBJECT terminal

```
npx tsc --target es2022 status.ts
cat status.js
```

TERMINAL

```
$ npx tsc --target es2022 status.ts
$ cat status.js
"use strict";
var Priority;
(function (Priority) {
    Priority[Priority["Low"] = 1] = "Low";
    Priority[Priority["Medium"] = 2] = "Medium";
    Priority[Priority["High"] = 3] = "High";
})(Priority || (Priority = {}));
const p = Priority.High;
```

6.4.2 String enums

In a **string enum** every member must be initialised with a string. The values are readable in logs, JSON and databases, and only enum members (not arbitrary strings) are assignable to the enum type, which makes string enums nominal-ish. There is no reverse mapping.

A STRING ENUM main.ts

```
enum OrderStatus {
  Pending = "PENDING",
  Paid = "PAID",
  Shipped = "SHIPPED",
}

function label(s: OrderStatus): string {
  return s === OrderStatus.Shipped ? "on its way" : s.toLowerCase();
}

const status = OrderStatus.Paid;
console.log(status, label(status), label(OrderStatus.Shipped));
console.log(JSON.stringify({ id: 7, status }));
```

OUTPUT

```
PAID paid on its way
{"id":7,"status":"PAID"}
```

A PLAIN STRING IS NOT ASSIGNABLE TO A STRING ENUM main.ts

```
enum OrderStatus { Pending = "PENDING", Paid = "PAID" }
function label(s: OrderStatus) { return s; }

label("PAID");
```

COMPILER ERRORS (tsc --strict)

```
main.ts(4,7): error TS2345: Argument of type '"PAID"' is not assignable to parameter of type
'OrderStatus'.
```

6.4.3 Const enums

A `const enum` is removed completely at compile time: every use is **inlined** as its value (`Priority.High` becomes `3 /* Priority.High */`) and no object is generated. It saves a little code, but it causes problems across module boundaries and with single-file compilers (Babel, esbuild, `isolatedModules`), and it has no run-time object to iterate over. Modern guidance is to avoid const enums in libraries and in most applications.

LEVELS.TS TypeScript

```
const enum Level { Beginner = 1, Intermediate, Advanced }
const mine = Level.Advanced;
console.log(mine === Level.Advanced);
```

A CONST ENUM IS INLINED AND LEAVES NO OBJECT BEHIND

terminal

```
npx tsc --target es2022 levels.ts
cat levels.js
```

TERMINAL

```
$ npx tsc --target es2022 levels.ts
$ cat levels.js
"use strict";
const mine = 3 /* Level.Advanced */;
console.log(mine === 3 /* Level.Advanced */);
```

6.4.4 Enums vs literal unions (and why many teams prefer unions)

A union of string literals does almost everything an enum does, with zero runtime code: values are plain strings (so JSON from an API fits directly), autocompletion and exhaustiveness checks work, and there is nothing to import. When you also need the list of values at run time, use an `as const` array or object (6.3.3). Teams that prefer enums value the namespace (`Status.Paid`), the ability to rename values in one place, and familiarity from other languages.

	enum	union of literals / as const object
runtime code	generates an object	none (or a plain const object)
values in JSON / APIs	must convert or use string enums	plain strings fit directly
assign a matching string	not allowed (string enum)	allowed
iterate over all values	Object.values (numeric enums include reverse keys)	from the as const array/object
works with type stripping / erasableSyntaxOnly	no	yes
autocompletion, exhaustiveness	yes	yes

A CONST OBJECT WITH A DERIVED UNION INSTEAD OF AN ENUM

main.ts

```
// The modern alternative: a const object + a derived union type
const OrderStatus = { Pending: "PENDING", Paid: "PAID", Shipped: "SHIPPED" } as const;
type OrderStatus = (typeof OrderStatus)[keyof typeof OrderStatus];   // "PENDING" | "PAID" | "SHIPPED"

function canCancel(s: OrderStatus): boolean {
  return s === OrderStatus.Pending || s === OrderStatus.Paid;
}

const fromApi: OrderStatus = "SHIPPED";             // plain strings from JSON fit
console.log(canCancel(OrderStatus.Paid), canCancel(fromApi), Object.values(OrderStatus));
```

OUTPUT

```
true false [ 'PENDING', 'PAID', 'SHIPPED' ]
```

6.4.5 erasableSyntaxOnly and enums (overview)

Tools that run TypeScript by simply deleting the types (Node.js type stripping, 2.4.2) cannot handle syntax that **generates** code: enums, namespaces with values, parameter properties (`constructor(private x:`

`number)`) and `import x = require()`. The compiler option `"erasableSyntaxOnly": true` (TypeScript 5.8+) makes tsc report these constructs, so your code is guaranteed to run under type stripping. Projects that adopt it replace enums with `as const` objects and parameter properties with ordinary fields (17.5.1).

APP.TS TypeScript

```
enum Color { Red, Green }

class Point {
  constructor(public x: number, public y: number) {}
}

type Size = "S" | "M";                    // fine: types are erasable
const size: Size = "M";
console.log(Color.Red, new Point(1, 2), size);
```

ERASABLESYNTAXONLY REJECTS SYNTAX THAT GENERATES JAVASCRIPT terminal

```
npx tsc --noEmit --erasableSyntaxOnly --target es2022 app.ts
```

TERMINAL

```
$ npx tsc --noEmit --erasableSyntaxOnly --target es2022 app.ts
app.ts(1,6): error TS1294: This syntax is not allowed when 'erasableSyntaxOnly' is enabled.
app.ts(4,15): error TS1294: This syntax is not allowed when 'erasableSyntaxOnly' is enabled.
app.ts(4,33): error TS1294: This syntax is not allowed when 'erasableSyntaxOnly' is enabled.
```

6.5 null and undefined handling

6.5.1 Optional chaining and nullish coalescing with types

With strict null checks, values that may be missing have types like `string | undefined` or `User | null`. Two JavaScript operators work hand in hand with these types. **Optional chaining** `a?.b` stops and returns `undefined` if `a` is `null` or `undefined`, so the result type gains `| undefined`; it works for property access, calls (`fn?.()`) and indexing (`arr?.[0]`). **Nullish coalescing** `a ?? b` returns `b` only when `a` is `null` or `undefined`, and TypeScript removes `null | undefined` from the result type. Unlike `||`, it keeps valid falsy values such as `0` and `""`.

OPTIONAL CHAINING, NULLISH COALESCING AND ??= WITH TYPED OPTIONAL VALUES

main.ts

```ts
interface Profile { bio?: string; social?: { github?: string; followers?: number } }
interface User { name: string; profile?: Profile; lastLogin: Date | null; onLogin?: (name: string) =>
void }

const users: User[] = [
  { name: "Asha", profile: { bio: "", social: { github: "asha-dev", followers: 0 } }, lastLogin: new
Date(2026, 9, 1) },
  { name: "Ravi", lastLogin: null, onLogin: (n) => console.log(`  welcome back, ${n}`) },
];

for (const u of users) {
  const github = u.profile?.social?.github ?? "not linked";          // string
  const followers = u.profile?.social?.followers ?? "unknown";       // number | "unknown"
  const bioOr = u.profile?.bio || "(no bio)";                          // || replaces "" too
  const bioNullish = u.profile?.bio ?? "(no bio)";                     // ?? keeps ""
  const login = u.lastLogin?.toDateString() ?? "never";
  console.log(`${u.name}: github=${github}, followers=${followers}, bio||=${bioOr}, bio??
="${bioNullish}", last=${login}`);
  u.onLogin?.(u.name);                                                 // call only if defined
}

let retries: number | undefined;
retries ??= 3;                                                         // assign only if nullish
console.log("retries:", retries);
```

OUTPUT

```
Asha: github=asha-dev, followers=0, bio||=(no bio), bio??="", last=Thu Oct 01 2026
Ravi: github=not linked, followers=unknown, bio||=(no bio), bio??="(no bio)", last=never
  welcome back, Ravi
retries: 3
```

Expression	Type when a: T | undefined	Result when a is missing	Notes
`a?.b`	type of b | undefined	undefined	stops the chain safely
`a ?? fallback`	T | type of fallback	fallback	keeps 0, "", false
`a || fallback`	T | type of fallback	fallback	also replaces 0, "", false
`a!`	T	crash later (no check)	only if you are certain (3.7.3)
`a ??= value`	T	assigns value	logical nullish assignment

A union A | B accepts a value of either type; only operations valid for every member are allowed until you narrow with checks such as typeof, Array.isArray, in or a shared literal property (a discriminated union). An intersection A & B requires both types at once, combining object properties; conflicting members become never, whereas interface extends reports conflicts. Literal types contain one value, and unions of literals model fixed sets of values with autocompletion and exhaustiveness checks; as const keeps literal types (readonly) and lets you derive unions with typeof X[number] or typeof O[keyof typeof O]. Enums (numeric with reverse mapping, string, const) generate runtime code; many teams prefer literal unions or as const objects, which also satisfy erasableSyntaxOnly. Optional chaining (?.) and nullish coalescing (??) work with optional types, and ?? keeps valid falsy values.

PRACTICE CHECKPOINT

A **payment function** that accepts a union of **CardPayment**, **UpiPayment** and **CashPayment**, and an **order status type** using a literal union.

SAMPLE SOLUTION: A PAYMENT FUNCTION OVER A UNION OF PAYMENT TYPES AND A LITERAL UNION FOR ORDER STATUS

main.ts

```
type OrderStatus = "created" | "payment_pending" | "paid" | "failed" | "refunded";

interface CardPayment { method: "card"; amount: number; cardLast4: string; network: "visa" |
"mastercard" | "rupay"; emiMonths?: 3 | 6 | 12 }
interface UpiPayment { method: "upi"; amount: number; vpa: string }
interface CashPayment { method: "cash"; amount: number; tendered: number }
type Payment = CardPayment | UpiPayment | CashPayment;

interface Order { id: number; total: number; status: OrderStatus; note?: string }

const CARD_FEE: Record<CardPayment["network"], number> = { visa: 1.8, mastercard: 1.8, rupay: 0 };

function pay(order: Order, payment: Payment): Order {
  if (payment.amount !== order.total) {
    return { ...order, status: "failed", note: `amount ${payment.amount} does not match total
${order.total}` };
  }
  switch (payment.method) {
    case "card": {
      const fee = (payment.amount * CARD_FEE[payment.network]) / 100;
      const emi = payment.emiMonths ? `, EMI ${payment.emiMonths} x Rs ${(payment.amount /
payment.emiMonths).toFixed(2)}` : "";
      return { ...order, status: "paid", note: `${payment.network} **** ${payment.cardLast4}, fee Rs
${fee.toFixed(2)}${emi}` };
    }
    case "upi":
      if (!/^[\w.-]+@[a-z]+$/.test(payment.vpa)) return { ...order, status: "failed", note: `invalid
UPI id ${payment.vpa}` };
      return { ...order, status: "paid", note: `UPI ${payment.vpa}` };
    case "cash": {
      const change = payment.tendered - payment.amount;
      if (change < 0) return { ...order, status: "payment_pending", note: `Rs ${-change} more needed`
};
      return { ...order, status: "paid", note: `cash, change Rs ${change}` };
    }
  }
}

const statusLabel: Record<OrderStatus, string> = {
  created: "Created", payment_pending: "Awaiting payment", paid: "Paid", failed: "Failed", refunded:
"Refunded",
};

const attempts: [Order, Payment][] = [
  [{ id: 1, total: 12000, status: "created" }, { method: "card", amount: 12000, cardLast4: "4242",
network: "visa", emiMonths: 6 }],
  [{ id: 2, total: 499, status: "created" }, { method: "upi", amount: 499, vpa: "asha@okbank" }],
  [{ id: 3, total: 499, status: "created" }, { method: "upi", amount: 499, vpa: "not a vpa" }],
  [{ id: 4, total: 750, status: "created" }, { method: "cash", amount: 750, tendered: 1000 }],
  [{ id: 5, total: 750, status: "created" }, { method: "cash", amount: 750, tendered: 500 }],
  [{ id: 6, total: 300, status: "created" }, { method: "card", amount: 250, cardLast4: "1111",
network: "rupay" }],
];

for (const [order, payment] of attempts) {
```

```
  const result = pay(order, payment);
  console.log(`#${result.id} ${payment.method.padEnd(4)} ${statusLabel[result.status].padEnd(16)}
${result.note ?? ""}`);
}
```

OUTPUT

```
#1 card Paid             visa **** 4242, fee Rs 216.00, EMI 6 x Rs 2000.00
#2 upi  Paid             UPI asha@okbank
#3 upi  Failed           invalid UPI id not a vpa
#4 cash Paid             cash, change Rs 250
#5 cash Awaiting payment Rs 250 more needed
#6 card Failed           amount 250 does not match total 300
```

MODULE 07 · BEGINNER

Working with JavaScript and the type system basics

Minimum duration **1 day** | Level **Beginner**

This module connects TypeScript to existing JavaScript code and builds intuition for how the type checker thinks. It covers assignability, subtypes and supertypes, the top type unknown and the bottom type never, using JavaScript libraries that ship their own types or have @types packages on DefinitelyTyped (and what to do when no types exist), adopting TypeScript gradually with allowJs, checkJs, JSDoc annotations and the @ts-check, @ts-ignore and @ts-expect-error comments, and how to read long error messages and fix the most common errors.

IN THIS MODULE

7.1 Type compatibility
7.2 Using JavaScript libraries
7.3 Gradual adoption
7.4 Reading type errors

7.1 Type compatibility

Most TypeScript errors are about one question: **is this value assignable to that type?** Assignments, function arguments, return values and property values are all checked the same way. Understanding the rules behind the answer makes error messages much easier to read.

7.1.1 Assignability rules

A type **S is assignable to T** if every value of S is also a valid T. For object types this is structural (5.7): S must have every required property of T, with assignable property types; extra properties are fine (except in fresh object literals, 5.1.4). For unions, S is assignable to `A | B` if it is assignable to A or to B, and a union `A | B` is assignable to T only if **both** A and B are. Literal types are assignable to their base types (`"GET"` to `string`), and readonly arrays are not assignable to mutable ones. For functions, the parameters and return types are compared (15.4).

ASSIGNMENTS THAT ARE ALLOWED

main.ts

```
interface Point2D { x: number; y: number }
interface Point3D { x: number; y: number; z: number }

const p3: Point3D = { x: 1, y: 2, z: 3 };
const p2: Point2D = p3;                          // OK: Point3D has everything Point2D needs

let id: string | number = "A-7";                 // "A-7" (a string) fits the union
const method: "GET" = "GET";
const s: string = method;                         // literal type -> its base type

const mutable: number[] = [1, 2];
const ro: readonly number[] = mutable;            // mutable -> readonly is fine

console.log(p2, id, s, ro);
```

OUTPUT

```
{ x: 1, y: 2, z: 3 } A-7 GET [ 1, 2 ]
```

ASSIGNMENTS THAT ARE NOT ALLOWED, AND WHY

main.ts

```
interface Point2D { x: number; y: number }
interface Point3D { x: number; y: number; z: number }

const p2: Point2D = { x: 1, y: 2 };
const p3: Point3D = p2;                           // missing z

let n: number = Math.random() > 0.5 ? 1 : "one";   // string | number is not a number

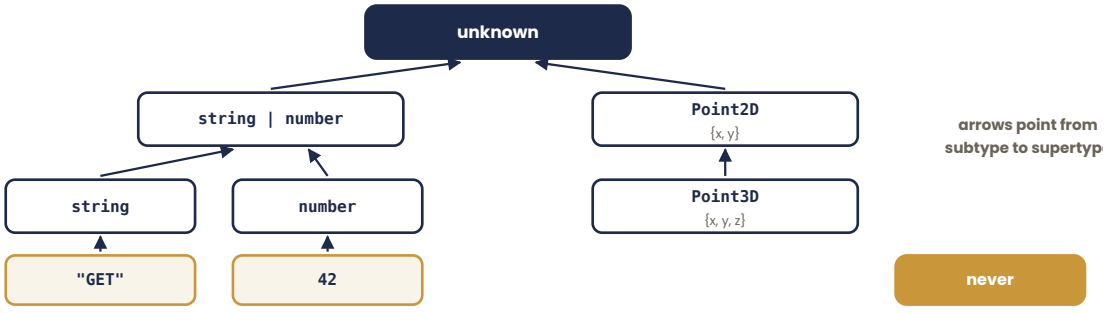
const ro: readonly number[] = [1, 2];
const mutable: number[] = ro;                     // would allow push on a readonly array
```

COMPILER ERRORS (tsc --strict)

```
main.ts(5,7): error TS2741: Property 'z' is missing in type 'Point2D' but required in type 'Point3D'.
main.ts(7,5): error TS2322: Type 'string | number' is not assignable to type 'number'.
  Type 'string' is not assignable to type 'number'.
main.ts(10,7): error TS4104: The type 'readonly number[]' is 'readonly' and cannot be assigned to the
mutable type 'number[]'.
```

7.1.2 Subtypes and supertypes

If S is assignable to T, S is a **subtype** of T and T is a **supertype** of S. A subtype is more specific: it has more properties (`Point3D` is a subtype of `Point2D`) or fewer possible values (`"GET"` is a subtype of `string`, `string` of `string | number`). Values can always flow from a subtype to a supertype ("upcasting"), which is safe; flowing the other way requires a check (narrowing) or an assertion.



A SMALL PART OF THE TYPE HIERARCHY: UNKNOWN AT THE TOP, NEVER AT THE BOTTOM

UPCASTING IS AUTOMATIC; DOWNCASTING NEEDS A CHECK main.ts

```ts
interface Animal { name: string }
interface Dog extends Animal { breed: string }

const dog: Dog = { name: "Tommy", breed: "Indie" };
const animal: Animal = dog;                    // subtype -> supertype: always OK

function greet(a: Animal) { return `Hello ${a.name}`; }
console.log(greet(dog));                       // a Dog can go where an Animal is expected

function isDog(a: Animal): a is Dog { return "breed" in a; }
if (isDog(animal)) console.log(animal.breed);  // supertype -> subtype needs a check
```

```
OUTPUT
Hello Tommy
Indie
```

7.1.3 Top type (unknown) and bottom type (never)

`unknown` is the **top type**: every type is a subtype of it, so any value can be assigned to an `unknown` variable, but an `unknown` value cannot be assigned to anything else (except `unknown` and `any`) without narrowing. `never` is the **bottom type**: it is a subtype of every type, so a `never` value could be assigned anywhere, but no value can be assigned to `never`. In a union, `never` disappears (`string | never` is `string`); in an intersection, `unknown` disappears (`string & unknown` is `string`). `any` is special: it acts as both top and bottom, which is exactly why it breaks type safety.

UNKNOWN ACCEPTS EVERYTHING; NEVER FITS EVERYWHERE BUT ACCEPTS NOTHING main.ts

```ts
let u: unknown = 42;          // anything goes in
let n: number = u;            // nothing comes out without a check

function fail(): never { throw new Error("x"); }
const s: string = fail();     // fine: never fits everywhere
let nothing: never = 1;       // nothing fits into never

let a: any = "text";
let num: number = a;          // any fits everywhere (unsafely), so no error here
```

```
COMPILER ERRORS (tsc --strict)
main.ts(2,5): error TS2322: Type 'unknown' is not assignable to type 'number'.
main.ts(6,5): error TS2322: Type '1' is not assignable to type 'never'.
```

7.2 Using JavaScript libraries

npm packages are JavaScript; their types come from **declaration files** (`.d.ts`), which contain only type information (11.4). When you import a package, TypeScript looks for declarations in three places: the package itself, a matching `@types/` package, or declarations you write.

7.2.1 Libraries that ship their own types

Many modern packages are written in TypeScript or include hand-written `.d.ts` files, referenced by the `"types"` (or `"exports"`) field of their `package.json`. Examples: Axios, Zod, date-fns, Prisma, Vue, Vite, Express 5, most SDKs. You install them as usual and get full types immediately; on npmjs.com such packages show a blue **TS** badge.

NODE_MODULES/RUPEE-FORMAT/INDEX.D.TS

TypeScript

```
export interface FormatOptions { symbol?: boolean }
export declare function formatRupees(amount: number, options?: FormatOptions): string;
```

APP.TS

TypeScript

```
import { formatRupees } from "rupee-format";
console.log(formatRupees(1234567));
```

A PACKAGE THAT SHIPS ITS OWN .D.TS FILE

terminal

```
cat node_modules/rupee-format/package.json
npx tsc --noEmit -p . && echo "types found: no errors"
```

TERMINAL

```
$ cat node_modules/rupee-format/package.json
{ "name": "rupee-format", "version": "1.2.0", "main": "index.js", "types": "index.d.ts" }
$ npx tsc --noEmit -p . && echo "types found: no errors"
types found: no errors
```

7.2.2 DefinitelyTyped and @types/* packages

For popular packages written in plain JavaScript, the community maintains declarations in the **DefinitelyTyped** repository on GitHub, published to npm under the `@types` scope. The package `lodash` has types in `@types/lodash`, Express 4 in `@types/express`, Node.js's built-in modules in `@types/node`. Install them as dev dependencies; TypeScript picks them up automatically from `node_modules/@types` (the `types` compiler option can restrict which global ones are loaded). Keep the major version of the `@types` package in line with the library's.

INSTALLING A JAVASCRIPT LIBRARY AND ITS COMMUNITY TYPES

terminal

```
npm install lodash
npm install -D @types/lodash
```

TERMINAL

```
added 1 package in 1s
added 1 package in 1s
```

LODASH WITH TYPES FROM @TYPES/LODASH

main.ts

```
import _ from "lodash";

const marks = [{ name: "Asha", score: 91 }, { name: "Ravi", score: 66 }, { name: "Meera", score: 84
}];
const best = _.maxBy(marks, "score");          // type: { name: string; score: number } | undefined
const byGrade = _.groupBy(marks, (m) => (m.score >= 80 ? "A" : "B"));
console.log(best?.name, Object.keys(byGrade), _.chunk([1, 2, 3, 4, 5], 2));
```

OUTPUT

```
Asha [ 'A', 'B' ] [ [ 1, 2 ], [ 3, 4 ], [ 5 ] ]
```

Needs the lodash and @types/lodash packages. Hover over maxBy to see the generic types that @types/lodash provides.

7.2.3 What to do when no types exist

If a package has neither built-in nor `@types` declarations, importing it in strict mode gives error **TS7016** ("Could not find a declaration file for module"). Options, from quickest to best: (1) a one-line **shorthand ambient declaration**, `declare module "pkg";`, which types everything from the package as `any`; (2) write a small declaration file describing just the functions you use (11.4.3), which gives real safety; (3) contribute the declarations to DefinitelyTyped or to the package itself. Put local declarations in a folder such as `types/` included by tsconfig.

APP.TS TypeScript

```
import slugify from "old-slugify";
console.log(slugify("Learn Being Forward TypeScript"));
```

FIXING TS7016 WITH A SHORTHAND DECLARATION, THEN A PROPER ONE terminal

```
npx tsc --noEmit -p .
echo 'declare module "old-slugify";' > types/old-slugify.d.ts
npx tsc --noEmit -p . && echo "compiles (everything from old-slugify is any)"
printf 'declare module "old-slugify" {\n  export default function slugify(text: string, sep?: string):
string;\n}\n' > types/old-slugify.d.ts
npx tsc --noEmit -p . && echo "compiles with real types"
```

TERMINAL

```
$ npx tsc --noEmit -p .
app.ts(1,21): error TS7016: Could not find a declaration file for module 'old-slugify'.
'node_modules/old-slugify/index.js' implicitly has an 'any' type.
$ echo 'declare module "old-slugify";' > types/old-slugify.d.ts
$ npx tsc --noEmit -p . && echo "compiles (everything from old-slugify is any)"
compiles (everything from old-slugify is any)
$ printf 'declare module "old-slugify" {\n  export default function slugify(text: string, sep?:
string): string;\n}\n' > types/old-slugify.d.ts
$ npx tsc --noEmit -p . && echo "compiles with real types"
compiles with real types
```

7.3 Gradual adoption

You do not have to convert a JavaScript project in one go. TypeScript can check plain `.js` files, read types from JSDoc comments, and live side by side with `.ts` files, so a team can migrate file by file while the application keeps running (19.1 gives a full migration strategy).

7.3.1 allowJs and checkJs

`"allowJs": true` lets the compiler include `.js` files in the project: they are compiled to the output folder and can be imported from `.ts` files (TypeScript infers types from them). `"checkJs": true` goes further and **reports type errors in JavaScript files**, using inference and JSDoc comments. Start with `allowJs`, then turn on `checkJs` and fix the reported problems, then rename files to `.ts` one at a time.

TSCONFIG.JSON JavaScript

```
{ "compilerOptions": { "allowJs": true, "checkJs": true, "noEmit": true, "strict": false, "target":
"es2022", "types": [] }, "include": ["src"] }
```

SRC/CART.JS

JavaScript

```
const cart = [
  { name: "Pen", price: 20, qty: 3 },
  { name: "Ruler", price: 15, qty: 2 },
];

function total(items) {
  return items.reduce((sum, i) => sum + i.price * i.qty, 0);
}

console.log(total(cart).toFixed(2));
console.log(totl(cart));                  // typo in a function name
const discount = Math.round("10%");       // string passed to Math.round
cart.push({ name: "Bag", price: "499", qty: 1 });   // price as a string
```

CHECKJS REPORTS BUGS IN PLAIN JAVASCRIPT FILES

terminal

```
npx tsc -p .
```

TERMINAL

```
$ npx tsc -p .
src/cart.js(11,13): error TS2552: Cannot find name 'totl'. Did you mean 'total'?
src/cart.js(12,29): error TS2345: Argument of type 'string' is not assignable to parameter of type
'number'.
src/cart.js(13,26): error TS2322: Type 'string' is not assignable to type 'number'.
```

7.3.2 JSDoc type annotations in JavaScript files

In `.js` files you can add types through **JSDoc** comments, which TypeScript understands: `@param {number} price`, `@returns {string}`, `@type {string[]}`, `@typedef` to define an object type, and even `@template` for generics and `@import` to import types. The code stays plain JavaScript (no build step), yet the editor and `tsc` type-check it. Some well-known projects, such as Svelte, use JSDoc types instead of `.ts` files for this reason.

TSCONFIG.JSON

JavaScript

```
{ "compilerOptions": { "allowJs": true, "checkJs": true, "noEmit": true, "strict": true, "target":
"es2022", "types": [] }, "include": ["src"] }
```

SRC/FEES.JS

JavaScript

```
/**
 * @typedef {Object} Course
 * @property {string} title
 * @property {number} fee
 * @property {"online" | "classroom"} mode
 */

/**
 * Fee including GST.
 * @param {Course} course
 * @param {number} [gstPct=18]   optional, with a default
 * @returns {number}
 */
function totalFee(course, gstPct = 18) {
  return Math.round(course.fee * (1 + gstPct / 100));
}

/** @type {Course[]} */
const courses = [
  { title: "TypeScript", fee: 9999, mode: "online" },
  { title: "Python", fee: 14999, mode: "classroom" },
];

for (const c of courses) console.log(`${c.title}: Rs ${totalFee(c)}`);
totalFee({ title: "Java", fee: "12999", mode: "hybrid" });   // two type errors
```

JSDOC TYPES IN A JAVASCRIPT FILE, CHECKED BY TSC

terminal

```
npx tsc -p .
node src/fees.js
```

```
TERMINAL
$ npx tsc -p .
src/fees.js(25,27): error TS2322: Type 'string' is not assignable to type 'number'.
src/fees.js(25,41): error TS2322: Type '"hybrid"' is not assignable to type '"online" | "classroom"'.
$ node src/fees.js
TypeScript: Rs 11799
Python: Rs 17699
```

7.3.3 // @ts-check, // @ts-ignore, // @ts-expect-error

Three special comments control checking line by line or file by file. `// @ts-check` at the top of a `.js` file turns on type checking for that file only, even without `checkJs` (VS Code honours it with no configuration at all), and `// @ts-nocheck` turns it off for a file. `// @ts-ignore` suppresses **any** error on the next line, silently and forever, even after the problem is fixed or changes. `// @ts-expect-error` also suppresses the error on the next line, but **reports an error if there is no error to suppress**, so stale suppressions are found. Always prefer `@ts-expect-error` with a comment explaining why, and use both sparingly.

DEMO.TS

TypeScript

```ts
// @ts-ignore  -- silences whatever is on the next line
const a: number = "not a number";

// @ts-expect-error  -- legacy API returns a string here, fixed in v2 (ticket LBF-142)
const b: number = "also not a number";

// @ts-expect-error  -- this line is actually fine, so tsc reports the unused directive
const c: number = 42;

console.log(a, b, c);
```

@TS-IGNORE HIDES ERRORS; @TS-EXPECT-ERROR INSISTS THAT AN ERROR EXISTS

terminal

```
npx tsc --noEmit --strict --target es2022 demo.ts
```

TERMINAL

```
$ npx tsc --noEmit --strict --target es2022 demo.ts
demo.ts(7,1): error TS2578: Unused '@ts-expect-error' directive.
```

7.4 Reading type errors

7.4.1 Breaking down long error messages

Errors about complex types can run to many lines, but they follow a pattern. The **first line** states the overall failure: "Type X is not assignable to type Y". Each **indented line** below it zooms in one level: which property, which element, which overload failed, ending with the precise mismatch at the bottom. Read the first line to know *where*, then jump to the **last** line to know *what*. Hover in the editor to see the full types involved, and if the message mentions huge inline types, give them names (interfaces or aliases) so errors become shorter.

NESTED ERRORS: THE LAST LINE NAMES THE EXACT MISMATCH

main.ts

```ts
interface Address { city: string; pin: string }
interface Customer { name: string; address: Address; tags: string[] }
interface Order { id: number; customer: Customer }

const order: Order = {
  id: 1,
  customer: { name: "Asha", address: { city: "Mysuru", pin: 570001 }, tags: ["vip"] },
};

const orders: Order[] = [{ id: 2, customer: { name: "Ravi", address: { city: "Pune", pin: "411001" },
tags: [1] } }];
```

COMPILER ERRORS (tsc --strict)

```
main.ts(7,56): error TS2322: Type 'number' is not assignable to type 'string'.
main.ts(10,110): error TS2322: Type 'number' is not assignable to type 'string'.
```



HOW TO READ A NESTED ERROR MESSAGE

7.4.2 Common errors and their fixes

Error	What it means	Typical fix
TS2322 / TS2345 not assignable	the value's type does not fit the target	fix the value, widen the target type, or narrow first
TS2339 property does not exist	typo, or the property is not on every union member	fix the name, add it to the type, or narrow (in, kind check)
TS18048 / TS2532 possibly undefined	the value may be missing	check with if, use ?. and ??, or provide a default
TS7006 implicit any	a parameter has no type in strict mode	annotate the parameter
TS2554 expected N arguments	wrong number of arguments	pass the missing arguments or make the parameter optional
TS2304 cannot find name	missing import, declaration or global type	import it, declare it, or add the lib/@types package
TS2307 / TS7016 module not found / no declaration	package or its types are missing	install the package and its @types, or declare the module
TS2564 not definitely assigned	a class field is not set in the constructor	initialise it, set it in the constructor, or mark it optional
TS2353 object literal may only specify known properties	excess property in a fresh literal	fix the typo or add the property to the type
TS2769 no overload matches this call	arguments do not fit any overload	read each overload's reason; check argument types and count

A FILE WITH SEVERAL COMMON ERRORS

main.ts

```
interface Student { name: string; marks: number[]; email?: string }

function average(s) {
  return s.marks.reduce((a, b) => a + b) / s.marks.length;
}

const list: Student[] = [{ name: "Asha", marks: [91, 84] }];
const first = list.find((s) => s.name === "Asha");
console.log(first.email.toLowerCase());
console.log(average(list[0], true));
console.log(list[0].mark);
```

COMPILER ERRORS (tsc --strict)

```
main.ts(3,18): error TS7006: Parameter 's' implicitly has an 'any' type.
main.ts(4,26): error TS7006: Parameter 'a' implicitly has an 'any' type.
main.ts(4,29): error TS7006: Parameter 'b' implicitly has an 'any' type.
main.ts(9,13): error TS18048: 'first' is possibly 'undefined'.
main.ts(9,13): error TS18048: 'first.email' is possibly 'undefined'.
main.ts(10,30): error TS2554: Expected 1 arguments, but got 2.
main.ts(11,21): error TS2551: Property 'mark' does not exist on type 'Student'. Did you mean 'marks'?
```

THE SAME FILE WITH EACH ERROR FIXED

main.ts

```
interface Student { name: string; marks: number[]; email?: string }

function average(s: Student): number {                          // TS7006: annotate
  return s.marks.reduce((a, b) => a + b, 0) / s.marks.length;
}

const list: Student[] = [{ name: "Asha", marks: [91, 84] }];
const first = list.find((s) => s.name === "Asha");
console.log(first?.email?.toLowerCase() ?? "no email on file");  // TS18048: handle undefined
console.log(average(list[0]));                                  // TS2554: remove extra
argument
console.log(list[0].marks);                                     // TS2339: fix the typo
```

OUTPUT

```
no email on file
87.5
[ 91, 84 ]
```

Assignability decides most errors: S fits T if every S value is a valid T (structurally for objects; literal types fit their base types; mutable arrays fit readonly ones but not the reverse). Subtypes are more specific and flow freely to supertypes; going down needs a check. unknown is the top type (accepts everything, gives nothing without narrowing), never is the bottom type, and any behaves as both. Library types come from the package itself, from @types packages on DefinitelyTyped, or from your own declarations (declare module "pkg" as a quick fix, error TS7016 otherwise). Adopt gradually with allowJs and checkJs, JSDoc type comments and // @ts-check; prefer @ts-expect-error (which fails when unused) over @ts-ignore. Read nested errors from the first line for where and the last line for what, and fix common errors with annotations, narrowing, optional chaining and correct names.

PRACTICE CHECKPOINT

Take a small **JavaScript project**, enable **checkJs**, fix the reported issues, then **convert two files fully to TypeScript**.

Sample solution. The starting point is a two-file JavaScript project (a student report). Step 1 enables `allowJs` and `checkJs` and lets tsc report problems. Step 2 shows the converted `.ts` files, with types for the data, the bugs fixed, and a strict build that compiles and runs.

SRC/REPORT.JS JavaScript

```
import { average, grade } from "./grades.js";

const students = [
  { name: "Asha", marks: [91, 84, 77] },
  { name: "Ravi", marks: [66, 72, 58] },
];

for (const s of students) {
  const avg = average(s.marks);
  console.log(s.name.padEnd(6), avg.toFixed(1), grade(avg));
}
console.log("Top:", students.sort((a, b) => average(b.marks) - average(a.marks))[0].nmae);
console.log(grade("90"));
```

STEP 1: CHECKJS REPORTS THE PROBLEMS IN THE JAVASCRIPT FILES terminal

```
npx tsc -p .
```

TERMINAL

```
$ npx tsc -p .
src/grades.js(1,25): error TS7006: Parameter 'marks' implicitly has an 'any' type.
src/grades.js(2,24): error TS7006: Parameter 'a' implicitly has an 'any' type.
src/grades.js(2,27): error TS7006: Parameter 'b' implicitly has an 'any' type.
src/grades.js(5,23): error TS7006: Parameter 'avg' implicitly has an 'any' type.
src/report.js(12,85): error TS2339: Property 'nmae' does not exist on type '{ name: string; marks:
number[]; }'.
```

TSCONFIG.JSON JavaScript

```
{ "compilerOptions": { "strict": true, "rootDir": "src", "outDir": "dist", "target": "es2022",
"module": "nodenext", "types": [] }, "include": ["src"] }
```

PACKAGE.JSON JavaScript

```
{ "type": "module" }
```

SRC/GRADES.TS TypeScript

```ts
export type Grade = "A" | "B" | "C";

export function average(marks: readonly number[]): number {
  return marks.length ? marks.reduce((a, b) => a + b, 0) / marks.length : 0;
}

export function grade(avg: number): Grade {
  if (avg >= 85) return "A";
  if (avg >= 70) return "B";
  return "C";
}
```

SRC/REPORT.TS TypeScript

```ts
import { average, grade, type Grade } from "./grades.js";

interface Student { name: string; marks: number[] }

const students: Student[] = [
  { name: "Asha", marks: [91, 84, 77] },
  { name: "Ravi", marks: [66, 72, 58] },
];

const rows: { name: string; avg: number; grade: Grade }[] = students.map((s) => {
  const avg = average(s.marks);
  return { name: s.name, avg, grade: grade(avg) };
});

for (const r of rows) console.log(r.name.padEnd(6), r.avg.toFixed(1), r.grade);
const top = [...rows].sort((a, b) => b.avg - a.avg)[0];
console.log("Top:", top?.name ?? "none");
console.log(grade(Number("90")));
```

STEP 2: BOTH FILES CONVERTED TO STRICT TYPESCRIPT terminal

```
npx tsc -p .
node dist/report.js
```

TERMINAL

```
$ npx tsc -p .
$ node dist/report.js
Asha   84.0 B
Ravi   65.3 C
Top: Asha
A
```

LEVEL 2

Intermediate level

Outcome: use narrowing, typed classes, generics, modules and declaration files, utility types, and type DOM and async code.

6 MODULES · 14 DAYS MINIMUM

Module 08 · Type narrowing and type guards	**2 days**
Module 09 · Classes and object-oriented TypeScript	**3 days**
Module 10 · Generics	**3 days**
Module 11 · Modules, namespaces and declaration files	**2 days**
Module 12 · Built-in utility types	**2 days**
Module 13 · TypeScript in the browser and with asynchronous code	**2 days**

MODULE 08 · INTERMEDIATE

Type narrowing and type guards

Minimum duration **2 days** | Level **Intermediate**

Narrowing is how TypeScript works out the exact type of a value inside a branch of code, and it is the key to using unions safely. This module covers control-flow analysis, narrowing with typeof, truthiness, equality, in and instanceof, discriminated (tagged) unions with switch and exhaustiveness checking using never, user-defined type guards with type predicates, inferred type predicates and assertion functions, and the common pitfalls: narrowing lost in callbacks and narrowing unknown data from APIs.

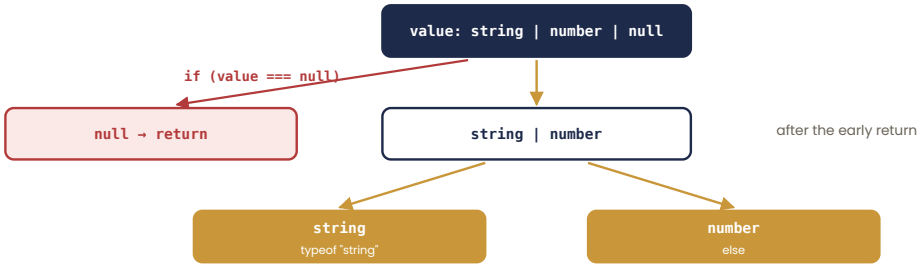
IN THIS MODULE

8.1 Narrowing basics
8.2 Discriminated unions
8.3 User-defined type guards
8.4 Narrowing pitfalls

8.1 Narrowing basics

8.1.1 Control flow analysis

TypeScript does not give a variable one fixed type for its whole life. It follows every path through your code (`if` / `else`, `switch`, `return`, `throw`, loops, `&&`, `||`, `?:`) and tracks the **narrowed type** of each variable at each point. This is called **control-flow analysis**. A check that removes possibilities from a union is a **type guard**. After an early `return` or `throw`, the rest of the function knows the eliminated cases cannot occur.



EACH CHECK REMOVES MEMBERS FROM THE UNION ALONG ITS BRANCH

CONTROL-FLOW ANALYSIS NARROWS A UNION STEP BY STEP main.ts

```
function describe(value: string | number | null): string {
  // value: string | number | null
  if (value === null) {
    return "nothing";                    // value: null
  }
  // value: string | number   (null was handled above)
  if (typeof value === "string") {
    return `text "${value.trim()}"`;     // value: string
  }
  return `number ${value.toFixed(2)}`;   // value: number
}

console.log(describe(null), "|", describe("  LBF  "), "|", describe(3.14159));

let x: string | number = 10;          // declared type: string | number, narrowed to number by the assignment
console.log(typeof x, x + 1);
```

OUTPUT

```
nothing | text "LBF" | number 3.14
number 11
```

8.1.2 typeof guards

The JavaScript `typeof` operator returns one of `"string"`, `"number"`, `"bigint"`, `"boolean"`, `"symbol"`, `"undefined"`, `"object"` or `"function"`, and TypeScript narrows on comparisons with these strings. Watch out for the classic JavaScript quirk: `typeof null === "object"`, so a `typeof x === "object"` check leaves `null` in the type, and arrays are objects too.

TYPEOF NARROWING, AND THE TYPEOF NULL TRAP main.ts

```
function format(input: string | number | boolean | bigint | (() => string)): string {
  if (typeof input === "string") return input.toUpperCase();
  if (typeof input === "number") return input.toFixed(1);
  if (typeof input === "bigint") return `${input}n`;
  if (typeof input === "function") return input();        // narrowed to () => string
  return input ? "yes" : "no";                            // only boolean is left
}

console.log([format("lbf"), format(2.345), format(10n), format(() => "called"), format(false)].join(" | "));

function size(x: string[] | Record<string, number> | null): number {
  if (typeof x === "object") {
    // x: string[] | Record<string, number> | null   <- null is still possible!
    if (x === null) return 0;
    return Array.isArray(x) ? x.length : Object.keys(x).length;
  }
  return -1;
}
console.log(size(["a", "b"]), size({ a: 1 }), size(null));
```

OUTPUT

```
LBF | 2.3 | 10n | called | no
2 1 0
```

8.1.3 Truthiness narrowing

In a condition, JavaScript converts values to booleans: `0`, `NaN`, `""`, `0n`, `null`, `undefined` and `false` are falsy; everything else is truthy. TypeScript uses truthiness to narrow: inside `if (name)`, `null` and `undefined` are removed. The danger is that truthiness also removes legitimate values such as `0` and the empty string, so prefer explicit checks (`!== undefined`, `!= null`) when those values are valid.

TRUTHINESS NARROWING AND ITS TRAP WITH 0 AND "" main.ts

```
function greet(name?: string | null): string {
  if (name) return `Hello, ${name}`;          // name: string (non-empty)
  return "Hello, guest";                       // "", null or undefined
}

function showStock(qty: number | undefined): string {
  if (!qty) return "unknown";                  // BUG: 0 is treated like undefined
  return `${qty} in stock`;
}

function showStockFixed(qty: number | undefined): string {
  if (qty === undefined) return "unknown";
  return qty === 0 ? "out of stock" : `${qty} in stock`;
}

console.log(greet("Asha"), "|", greet(""), "|", greet(null));
console.log(showStock(0), "|", showStockFixed(0), "|", showStockFixed(undefined), "|",
showStockFixed(12));

const names = ["Asha", "", "Ravi", null, undefined, "Meera"];
const real = names.filter(Boolean);          // (string | null | undefined)[]: Boolean does not narrow
const realTyped = names.filter((n): n is string => !!n);   // string[]
console.log(real.length, realTyped.map((n) => n.toUpperCase()));
```

```
OUTPUT
Hello, Asha | Hello, guest | Hello, guest
unknown | out of stock | unknown | 12 in stock
3 [ 'ASHA', 'RAVI', 'MEERA' ]
```

8.1.4 Equality narrowing

Comparisons with `===`, `!==`, `==` and `!=` narrow too. Comparing with a literal narrows to that literal; comparing two variables of different union types narrows both to their common types; and the loose comparison `x != null` removes **both** `null` and `undefined` at once (the one widely accepted use of `!=`). `switch` statements narrow in each `case` in the same way.

NARROWING WITH ===, != NULL AND LITERAL COMPARISONS

main.ts

```
function compare(a: string | number, b: string | boolean): string {
  if (a === b) {
    return `both are the string "${a.toUpperCase()}"`;   // the only common type is string
  }
  return "different";
}

function lengthOf(text: string | null | undefined): number {
  if (text != null) return text.length;     // removes null AND undefined
  return 0;
}

type Unit = "cm" | "m" | "km";
function toMetres(value: number, unit: Unit): number {
  if (unit === "cm") return value / 100;
  if (unit === "km") return value * 1000;
  return value;                               // unit: "m"
}

console.log(compare("lbf", "lbf"), "|", compare(1, true), "|", lengthOf(null), lengthOf("abc"),
toMetres(3, "km"));
```

OUTPUT

```
both are the string "LBF" | different | 0 3 3000
```

8.1.5 in operator narrowing

`"prop" in obj` checks at run time whether an object has a property (own or inherited). TypeScript narrows a union of object types to the members that have (or may have) that property in the true branch, and to the others in the false branch. It is the natural guard when union members have different properties but no shared tag. In TypeScript 4.9+, `in` also narrows `unknown` objects by adding the property to the type.

NARROWING WITH THE IN OPERATOR main.ts

```
interface Fish { name: string; swim(): string }
interface Bird { name: string; fly(): string }
interface Robot { name: string; fly?(): string; charge(): string }

function move(animal: Fish | Bird | Robot): string {
  if ("swim" in animal) return animal.swim();             // Fish
  if ("charge" in animal) return animal.charge();         // Robot
  return animal.fly();                                     // Bird
}

const pets: (Fish | Bird | Robot)[] = [
  { name: "Nemo", swim: () => "Nemo swims" },
  { name: "Tweety", fly: () => "Tweety flies" },
  { name: "R2", charge: () => "R2 charges" },
];
pets.forEach((p) => console.log(move(p)));

function hasId(x: unknown): string {
  if (typeof x === "object" && x !== null && "id" in x) return `id = ${String(x.id)}`;   // x: object
& Record<"id", unknown>
  return "no id";
}
console.log(hasId({ id: 7 }), "|", hasId("seven"));
```

OUTPUT

```
Nemo swims
Tweety flies
R2 charges
id = 7 | no id
```

8.1.6 instanceof narrowing

`x instanceof C` checks whether `C.prototype` is in the prototype chain of `x`. TypeScript narrows `x` to the instance type of `C`. It works for classes and built-in constructors such as `Date`, `Error` (and its subclasses), `Map` and `RegExp`, but not for interfaces or type aliases, which do not exist at run time. It is the standard way to handle different error classes in a `catch` block.

INSTANCEOF WITH CUSTOM ERRORS AND BUILT-IN CLASSES main.ts

```
class ValidationError extends Error { constructor(public field: string, msg: string) { super(msg); } }
class NetworkError extends Error { constructor(public status: number) { super(`HTTP ${status}`); } }

function handle(err: unknown): string {
  if (err instanceof ValidationError) return `fix the ${err.field} field: ${err.message}`;
  if (err instanceof NetworkError) return err.status >= 500 ? "server down, retry later" : `request
failed (${err.status})`;
  if (err instanceof Error) return `unexpected: ${err.message}`;
  return `unknown error: ${String(err)}`;
}

function toDate(value: Date | string | number): string {
  const d = value instanceof Date ? value : new Date(value);
  return d.toISOString().slice(0, 10);
}

const failures: unknown[] = [new ValidationError("email", "missing @"), new NetworkError(503), new
RangeError("too big"), "oops"];
failures.forEach((f) => console.log(handle(f)));
console.log(toDate(new Date(Date.UTC(2026, 0, 26))), toDate("2026-08-15"), toDate(0));
```

```
OUTPUT
fix the email field: missing @
server down, retry later
unexpected: too big
unknown error: oops
2026-01-26 2026-08-15 1970-01-01
```

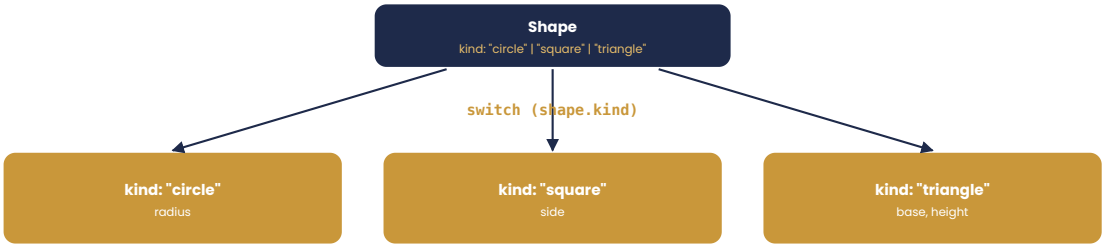
Guard	Narrows	Works on	Watch out for
`typeof x === "string"`	primitives, functions	any value	typeof null is "object"
truthiness `if (x)`	removes null, undefined (and falsy)	any value	also removes 0, "", NaN, false
`x === value`, `x != null`	literals, null and undefined	any value	== null catches both null and undefined
`"p" in x`	object unions by property	objects	x must be an object; optional properties
`x instanceof C`	class instances	classes, built-ins	not for interfaces; fails across realms (iframes)
`Array.isArray(x)`	arrays	any value	returns any[] for unknown input
`x.kind === "..."`	discriminated unions	tagged objects	the tag must be a literal type (8.2)

8.2 Discriminated unions

8.2.1 Tagged unions with a shared kind or type property

A **discriminated union** (tagged union) is a union of object types that all share one property, the **discriminant** (commonly `kind`, `type` or `status`), whose type in each member is a different **literal**. Checking the discriminant narrows the value to exactly one member, with all of its specific properties

available. It is the most important modelling pattern in TypeScript: Redux actions, API results, events, shapes, payment methods and state machines are all naturally discriminated unions.



A DISCRIMINATED UNION: THE LITERAL KIND PROPERTY SELECTS ONE MEMBER

8.2.2 Narrowing with switch

A DISCRIMINATED UNION NARROWED WITH SWITCH main.ts

```
interface Circle { kind: "circle"; radius: number }
interface Square { kind: "square"; side: number }
interface Triangle { kind: "triangle"; base: number; height: number }
type Shape = Circle | Square | Triangle;

function area(shape: Shape): number {
  switch (shape.kind) {
    case "circle":
      return Math.PI * shape.radius ** 2;          // shape: Circle
    case "square":
      return shape.side ** 2;                      // shape: Square
    case "triangle":
      return (shape.base * shape.height) / 2;      // shape: Triangle
  }
}

const shapes: Shape[] = [{ kind: "circle", radius: 1 }, { kind: "square", side: 3 }, { kind:
"triangle", base: 4, height: 5 }];
for (const s of shapes) console.log(s.kind.padEnd(8), area(s).toFixed(2));
```

OUTPUT

```
circle   3.14
square   9.00
triangle 10.00
```

LOADING, SUCCESS AND ERROR STATES AS A DISCRIMINATED UNION main.ts

```
// API results are a classic discriminated union: the tag is "status"
type ApiResult<T> =
  | { status: "loading" }
  | { status: "success"; data: T; fetchedAt: Date }
  | { status: "error"; error: string; retryable: boolean };

function render(r: ApiResult<string[]>): string {
  if (r.status === "loading") return "Loading...";
  if (r.status === "error") return `Error: ${r.error}${r.retryable ? " (retrying)" : ""}`;
  return `${r.data.length} courses: ${r.data.join(", ")}`;     // only success is left
}

const states: ApiResult<string[]>[] = [
  { status: "loading" },
  { status: "success", data: ["TypeScript", "Python"], fetchedAt: new Date() },
  { status: "error", error: "timeout", retryable: true },
];
states.forEach((s) => console.log(render(s)));
```

OUTPUT

```
Loading...
2 courses: TypeScript, Python
Error: timeout (retrying)
```

MEMBER-SPECIFIC PROPERTIES ARE ONLY AVAILABLE AFTER CHECKING THE TAG main.ts

```
type ApiResult = { status: "success"; data: string[] } | { status: "error"; error: string };

function render(r: ApiResult) {
  console.log(r.data);                  // not on every member
  if (r.status === "error") console.log(r.data);   // not on the error member
}
```

COMPILER ERRORS (tsc --strict)

```
main.ts(4,17): error TS2339: Property 'data' does not exist on type 'ApiResult'.
  Property 'data' does not exist on type '{ status: "error"; error: string; }'.
main.ts(5,43): error TS2339: Property 'data' does not exist on type '{ status: "error"; error: string;
}'.
```

8.2.3 Exhaustiveness checking with never

When every member of a union has been handled, the narrowed type in the remaining branch is `never`. Assigning the value to a variable of type `never` in the `default` case (or calling a helper such as `assertNever(x: never)`) turns this into a compile-time guarantee: if someone later adds a new member to the union and forgets to handle it, the value is no longer `never` and the compiler reports an error **at every switch that needs updating**. The helper also throws at run time in case unexpected data arrives anyway.

AN EXHAUSTIVE SWITCH WITH AN ASSERTNEVER HELPER

main.ts

```
type Shape =
  | { kind: "circle"; radius: number }
  | { kind: "square"; side: number }
  | { kind: "triangle"; base: number; height: number };

function assertNever(value: never): never {
  throw new Error(`unhandled case: ${JSON.stringify(value)}`);
}

function perimeterHint(shape: Shape): string {
  switch (shape.kind) {
    case "circle": return `2πr with r = ${shape.radius}`;
    case "square": return `4 × ${shape.side}`;
    case "triangle": return "sum of the three sides";
    default: return assertNever(shape);        // shape: never here
  }
}

console.log(perimeterHint({ kind: "square", side: 5 }));
try {
  perimeterHint(JSON.parse('{"kind": "hexagon"}'));     // bad data at run time
} catch (e) {
  console.log((e as Error).message);
}
```

OUTPUT

```
4 × 5
unhandled case: {"kind":"hexagon"}
```

AFTER ADDING A MEMBER, THE COMPILER POINTS TO EVERY SWITCH THAT DOES NOT HANDLE IT

main.ts

```
type Shape =
  | { kind: "circle"; radius: number }
  | { kind: "square"; side: number }
  | { kind: "rectangle"; width: number; height: number };     // newly added member

function assertNever(value: never): never { throw new Error("unhandled"); }

function area(shape: Shape): number {
  switch (shape.kind) {
    case "circle": return Math.PI * shape.radius ** 2;
    case "square": return shape.side ** 2;
    default: return assertNever(shape);
  }
}
```

COMPILER ERRORS (tsc --strict)

```
main.ts(12,33): error TS2345: Argument of type '{ kind: "rectangle"; width: number; height: number; }'
is not assignable to parameter of type 'never'.
```

TIP

Without a `default` case, TypeScript can still detect a missing case if the function has an explicit return type: the end of the function becomes reachable, and you get "Function lacks ending return statement" (TS2366). The `assertNever` pattern gives a clearer message and also works for `if` / `else` chains and functions returning `void`.

8.3 User-defined type guards

8.3.1 Type predicates value is Fish

When the check you need is more complex than `typeof` or `in`, put it in a function and give the function a **type predicate** as its return type: `function isFish(pet: Fish | Bird): pet is Fish`. The function returns a boolean at run time; at compile time, calling it in a condition narrows the argument to `Fish` when it returns `true` (and to the other members when it returns `false`). Type guards are the standard way to validate `unknown` data and to filter arrays with correct types. The compiler trusts your predicate, so the check inside must really be correct.

TYPE PREDICATES FOR UNIONS, ARRAY FILTERS AND UNKNOWN DATA main.ts

```
interface Fish { name: string; swim(): void }
interface Bird { name: string; fly(): void }

function isFish(pet: Fish | Bird): pet is Fish {
  return typeof (pet as Fish).swim === "function";
}

const pets: (Fish | Bird)[] = [
  { name: "Nemo", swim() { console.log("  Nemo swims"); } },
  { name: "Polly", fly() { console.log("  Polly flies"); } },
  { name: "Dory", swim() { console.log("  Dory swims"); } },
];

for (const pet of pets) {
  if (isFish(pet)) pet.swim();          // pet: Fish
  else pet.fly();                       // pet: Bird
}

const fish: Fish[] = pets.filter(isFish);              // filter keeps the narrowed type
console.log(fish.map((f) => f.name));

interface User { id: number; name: string; email?: string }
function isUser(value: unknown): value is User {
  if (typeof value !== "object" || value === null) return false;
  const v = value as Record<string, unknown>;
  return typeof v.id === "number" && typeof v.name === "string" && (v.email === undefined || typeof
v.email === "string");
}

const parsed: unknown[] = [JSON.parse('{"id": 1, "name": "Asha"}'), JSON.parse('{"id": "2", "name":
"Ravi"}'), null];
parsed.forEach((p) => console.log(isUser(p) ? `valid user ${p.name}` : `rejected
${JSON.stringify(p)}`));
```

OUTPUT

```
  Nemo swims
  Polly flies
  Dory swims
[ 'Nemo', 'Dory' ]
valid user Asha
rejected {"id":"2","name":"Ravi"}
rejected null
```

8.3.2 Inferred type predicates

Since TypeScript 5.5, the compiler **infers** a type predicate for simple functions whose body returns a narrowing expression, so you often do not need to write `x is T` yourself. The most visible benefit: `arr.filter(x => x !== undefined)` now produces an array without `undefined`. The inference applies only when the function returns a boolean expression that narrows the parameter and does not mutate it; for truthiness checks (`x => !!x`) it is not inferred, because a `false` result would not prove the value is null (it could be `0` or `""`).

TYPE PREDICATES INFERRED AUTOMATICALLY (TYPESCRIPT 5.5+) main.ts

```
const raw: (number | undefined | null)[] = [91, undefined, 84, null, 0, 77];

const marks = raw.filter((m) => m != null);                // number[] (predicate inferred)
console.log(marks.reduce((a, b) => a + b, 0), marks);

const isString = (x: unknown) => typeof x === "string";    // inferred: (x: unknown) => x is string
const items: unknown[] = ["a", 1, "b", true];
const strings = items.filter(isString);                    // string[]
console.log(strings.map((s) => s.toUpperCase()));

const truthy = raw.filter((m) => !!m);                     // still (number | null | undefined)[]
console.log(truthy.length);
```

```
OUTPUT
252 [ 91, 84, 0, 77 ]
[ 'A', 'B' ]
3
```

8.3.3 Assertion functions asserts value is string

An **assertion function** throws if a condition is false, and tells the compiler that, if it returns normally, the condition holds for the rest of the scope. Its return type is written `asserts value is T` (narrow a parameter) or `asserts condition` (narrow whatever the condition expression checks). Unlike a type guard, you call it as a statement, not inside an `if`. Assertion functions must be declared with an explicit type annotation (a `function` declaration, or a `const` with an annotated function type). Node's `assert` module is typed this way.

ASSERTS VALUE IS T AND ASSERTS CONDITION main.ts

```ts
function assertIsString(value: unknown, name: string): asserts value is string {
  if (typeof value !== "string") throw new TypeError(`${name} must be a string, got ${typeof value}`);
}

function assert(condition: unknown, message: string): asserts condition {
  if (!condition) throw new Error(message);
}

function createAccount(input: Record<string, unknown>) {
  assertIsString(input.email, "email");
  const email = input.email;                        // email: string from here on
  assert(email.includes("@"), "email must contain @");
  const age = input.age;
  assert(typeof age === "number" && age >= 18, "age must be a number >= 18");
  return `account for ${email.toLowerCase()}, age ${age.toFixed(0)}`;   // age: number
}

for (const input of [{ email: "Asha@LBF.in", age: 21 }, { email: 42 }, { email: "ravi.lbf.in", age: 30
}, { email: "k@lbf.in", age: 15 }]) {
  try {
    console.log(createAccount(input));
  } catch (e) {
    console.log("rejected:", (e as Error).message);
  }
}
```

```
OUTPUT
account for asha@lbf.in, age 21
rejected: email must be a string, got number
rejected: email must contain @
rejected: age must be a number >= 18
```

Kind	Signature	Use as	Effect
type guard	`(x: unknown): x is T`	`if (isT(x)) { ... }`, `.filter(isT)`	narrows in the true branch, the false branch gets the rest
assertion (value)	`(x: unknown): asserts x is T`	`assertIsT(x);` as a statement	narrows for the rest of the scope, or throws
assertion (condition)	`(cond: unknown): asserts cond`	`assert(x > 0);`	everything the condition proves holds afterwards

8.4 Narrowing pitfalls

8.4.1 Narrowing lost in callbacks

Narrowing applies to the code that runs **right after** the check. A callback may run later, after the variable has changed, so TypeScript is conservative: narrowings of `let` variables and of object properties (`obj.prop`) are reset inside callbacks, and also after calls that might have mutated the object. The fixes: copy the narrowed value into a `const` before the callback, or narrow inside the callback. `const` variables keep their narrowing in callbacks because they cannot be reassigned, and since TypeScript 5.4 so do `let` variables that are never reassigned after the closure is created.

NARROWING DOES NOT SURVIVE INTO CALLBACKS WHEN THE VALUE COULD CHANGE main.ts

```
interface Config { theme?: { color: string } }

function apply(config: Config, items: string[]) {
  if (config.theme) {
    items.forEach((item) => {
      console.log(item, config.theme.color);        // property narrowing is lost in the callback
    });
  }

  let label: string | undefined = items[0];
  if (label) {
    setTimeout(() => console.log(label.toUpperCase()), 0);   // label may be reassigned later
  }
  label = undefined;
}
```

COMPILER ERRORS (tsc --strict)

```
main.ts(6,25): error TS18048: 'config.theme' is possibly 'undefined'.
main.ts(12,34): error TS18048: 'label' is possibly 'undefined'.
```

FIX: CAPTURE THE NARROWED VALUE IN A CONST main.ts

```
interface Config { theme?: { color: string } }

function apply(config: Config, items: string[]) {
  const theme = config.theme;                          // copy into a const
  if (theme) {
    items.forEach((item) => console.log(item, theme.color));   // OK: a const cannot change
  }
  const label = items[0];                              // const instead of let
  if (label) setTimeout(() => console.log(label.toUpperCase()), 0);
}

apply({ theme: { color: "navy" } }, ["header", "footer"]);
```

OUTPUT

```
header navy
footer navy
HEADER
```

8.4.2 Narrowing with unknown data from APIs

Data from `fetch`, `JSON.parse`, `localStorage`, files, environment variables and message queues has no compile-time guarantee. Typing it with an assertion (`as User`) only hides the risk (3.7.4). The safe approach is to type it as `unknown` and **narrow it step by step** with guards, or with a validation library such as Zod that produces the type and the checks from one schema (15.3). Validate once, at the boundary, and the rest of the program can trust the types.

VALIDATING UNKNOWN API DATA WITH SMALL TYPE GUARDS

main.ts

```
interface Course { code: string; title: string; fee: number; tags: string[] }

const isObject = (v: unknown): v is Record<string, unknown> => typeof v === "object" && v !== null &&
!Array.isArray(v);
const isStringArray = (v: unknown): v is string[] => Array.isArray(v) && v.every((x) => typeof x ===
"string");

function parseCourse(v: unknown): Course | string {
  if (!isObject(v)) return "not an object";
  if (typeof v.code !== "string" || !/^[A-Z]{2}\d{3}$/.test(v.code)) return "bad code";
  if (typeof v.title !== "string" || v.title.length === 0) return "bad title";
  if (typeof v.fee !== "number" || v.fee < 0) return "bad fee";
  if (!isStringArray(v.tags)) return "bad tags";
  return { code: v.code, title: v.title, fee: v.fee, tags: v.tags };
}

const responses = [
  '{"code": "TS101", "title": "TypeScript", "fee": 9999, "tags": ["web", "types"]}',
  '{"code": "TS101", "title": "TypeScript", "fee": "9999", "tags": []}',
  '{"code": "ts-1", "title": "", "fee": 1, "tags": []}',
  '[1, 2, 3]',
];

for (const body of responses) {
  const result = parseCourse(JSON.parse(body) as unknown);
  console.log(typeof result === "string" ? `invalid: ${result}` : `ok: ${result.title} Rs
${result.fee} [${result.tags.join(", ")}]`);
}
```

OUTPUT

```
ok: TypeScript Rs 9999 [web, types]
invalid: bad fee
invalid: bad code
invalid: not an object
```

Control-flow analysis narrows a variable along each path through the code. Built-in guards are typeof (beware typeof null === "object"), truthiness (which also removes 0 and ""), equality (x != null removes null and undefined), the in operator for object unions, instanceof for classes and errors, and Array.isArray. Discriminated unions share a literal tag such as kind; switching on it narrows to one member, and an assertNever(x: never) default makes the switch exhaustive so new members cause compile errors. User-defined type guards return x is T, TypeScript 5.5 infers simple predicates (filter(x => x != null)), and assertion functions return asserts x is T or asserts condition. Narrowing of let variables and properties is lost in callbacks, so copy values into const; treat external data as unknown and validate it at the boundary.

PRACTICE CHECKPOINT

A **Shape** discriminated union (**circle, square, triangle**) with an **exhaustive area function**, and a **type guard** that validates **unknown JSON as a User**.

SAMPLE SOLUTION: AN EXHAUSTIVE SHAPE AREA FUNCTION AND A USER TYPE GUARD

main.ts

```
// ---- Part 1: an exhaustive area function over a discriminated union
type Shape =
  | { kind: "circle"; radius: number }
  | { kind: "square"; side: number }
```

```
  | { kind: "triangle"; a: number; b: number; c: number };

function assertNever(x: never): never {
  throw new Error(`Unexpected shape: ${JSON.stringify(x)}`);
}

function area(shape: Shape): number {
  switch (shape.kind) {
    case "circle":
      return Math.PI * shape.radius ** 2;
    case "square":
      return shape.side ** 2;
    case "triangle": {
      const { a, b, c } = shape;
      const s = (a + b + c) / 2;                       // Heron's formula
      return Math.sqrt(s * (s - a) * (s - b) * (s - c));
    }
    default:
      return assertNever(shape);
  }
}

const shapes: Shape[] = [{ kind: "circle", radius: 2 }, { kind: "square", side: 4 }, { kind:
"triangle", a: 3, b: 4, c: 5 }];
for (const s of shapes) console.log(`${s.kind.padEnd(8)} area ${area(s).toFixed(2)}`);
console.log("total area", shapes.reduce((sum, s) => sum + area(s), 0).toFixed(2));

// ---- Part 2: a type guard that validates unknown JSON as a User
interface User {
  id: number;
  name: string;
  email: string;
  roles: ("student" | "mentor" | "admin")[];
  address?: { city: string };
}

const ROLES = ["student", "mentor", "admin"] as const;
const isRecord = (v: unknown): v is Record<string, unknown> => typeof v === "object" && v !== null &&
!Array.isArray(v);

function isUser(v: unknown): v is User {
  return (
    isRecord(v) &&
    Number.isInteger(v.id) &&
    typeof v.name === "string" && v.name.trim() !== "" &&
    typeof v.email === "string" && /^[^@\s]+@[^@\s]+\.[a-z]+$/i.test(v.email) &&
    Array.isArray(v.roles) && v.roles.every((r) => (ROLES as readonly unknown[]).includes(r)) &&
    (v.address === undefined || (isRecord(v.address) && typeof v.address.city === "string"))
  );
}

const payloads = [
  '{"id": 1, "name": "Asha", "email": "asha@lbf.in", "roles": ["student"], "address": {"city":
"Mysuru"}}',
  '{"id": 2, "name": "Ravi", "email": "ravi@lbf.in", "roles": ["mentor", "admin"]}',
  '{"id": "3", "name": "Meera", "email": "meera@lbf.in", "roles": []}',
  '{"id": 4, "name": "Kiran", "email": "kiran.lbf.in", "roles": ["student"]}',
  '{"id": 5, "name": "Zoya", "email": "zoya@lbf.in", "roles": ["superuser"]}',
];

for (const json of payloads) {
  const data: unknown = JSON.parse(json);
  if (isUser(data)) {
    console.log(`valid   #${data.id} ${data.name} <${data.email}> ${data.roles.join("+")}
```

```
${data.address?.city ?? ""}`);
  } else {
    console.log(`invalid ${json.slice(0, 48)}...`);
  }
}
```

OUTPUT

```
circle   area 12.57
square   area 16.00
triangle area 6.00
total area 34.57
valid   #1 Asha <asha@lbf.in> student Mysuru
valid   #2 Ravi <ravi@lbf.in> mentor+admin
invalid {"id": "3", "name": "Meera", "email": "meera@lbf...
invalid {"id": 4, "name": "Kiran", "email": "kiran.lbf.i...
invalid {"id": 5, "name": "Zoya", "email": "zoya@lbf.in"...
```

MODULE 09 · INTERMEDIATE

Classes and object-oriented TypeScript

Minimum duration **3 days** | Level **Intermediate**

TypeScript adds access modifiers, abstract classes and interface implementation on top of JavaScript classes. This module covers typed properties and constructors, strictPropertyInitialization and definite assignment, methods and this types, public, private and protected, TypeScript private versus JavaScript #private, readonly and parameter properties, getters, setters and static members, inheritance with super, override and noImplicitOverride, abstract classes, implementing interfaces, classes as types (instance types and typeof ClassName), structural typing with classes, and object-oriented design in TypeScript: the four pillars, composition over inheritance and the SOLID principles.

IN THIS MODULE

9.1	Class basics	9.5	Abstract classes
9.2	Access modifiers	9.6	Interfaces and classes
9.3	Getters, setters and static members	9.7	Classes as types
9.4	Inheritance	9.8	OOP design in TypeScript

9.1 Class basics

9.1.1 Typed properties and constructors

A TypeScript class looks like a JavaScript class with types added. **Properties** (fields) are declared in the class body with their types, optionally with an initial value; the **constructor** has typed parameters and assigns the fields. Declaring fields is required in TypeScript: assigning `this.name` in the constructor without declaring `name` is an error, which catches typos in field names.

A CLASS WITH TYPED FIELDS, A CONSTRUCTOR AND METHODS main.ts

```
class Student {
  name: string;
  roll: number;
  marks: number[] = [];              // field with an initial value
  readonly joined = new Date(2026, 6, 1);

  constructor(name: string, roll: number) {
    this.name = name;
    this.roll = roll;
  }

  addMark(mark: number): void {
    if (mark < 0 || mark > 100) throw new RangeError(`invalid mark ${mark}`);
    this.marks.push(mark);
  }

  average(): number {
    return this.marks.length ? this.marks.reduce((a, b) => a + b, 0) / this.marks.length : 0;
  }
}

const s = new Student("Asha", 7);
[91, 84, 77].forEach((m) => s.addMark(m));
console.log(s.name, s.roll, s.average().toFixed(1), s.joined.getFullYear());
```

OUTPUT

```
Asha 7 84.0 2026
```

FIELDS MUST BE DECLARED, AND CONSTRUCTOR CALLS ARE CHECKED main.ts

```
class Student {
  name: string;
  constructor(name: string, roll: number) {
    this.name = name;
    this.roll = roll;              // field not declared
  }
}
const s = new Student("Asha");     // missing argument
s.nmae = "Ravi";
```

COMPILER ERRORS (tsc --strict)

```
main.ts(5,10): error TS2339: Property 'roll' does not exist on type 'Student'.
main.ts(8,11): error TS2554: Expected 2 arguments, but got 1.
main.ts(9,3): error TS2339: Property 'nmae' does not exist on type 'Student'.
```

9.1.2 strictPropertyInitialization and definite assignment !

With `strictPropertyInitialization` (part of strict), every field whose type does not include `undefined` must be **definitely assigned**: by an initialiser or in the constructor on every path. This prevents objects with missing data. If a field is set somewhere TypeScript cannot see (an `init()` method called by a framework, dependency injection, a decorator), add the **definite assignment assertion** `!` after its name: `db!: Database`. As with the non-null assertion, you are promising something the compiler cannot verify, so prefer initialising in the constructor or making the field optional.

FIELDS MUST BE INITIALISED ON EVERY PATH main.ts

```
class Connection {
  host: string;
  port: number;                 // only assigned when a port is passed
  retries: number;

  constructor(host: string, port?: number) {
    this.host = host;
    if (port) this.port = port;  // not assigned on every path
    this.retries = 3;
  }
}
```

COMPILER ERRORS (tsc --strict)

```
main.ts(3,3): error TS2564: Property 'port' has no initializer and is not definitely assigned in the
constructor.
```

INITIALISERS, OPTIONAL FIELDS AND THE DEFINITE ASSIGNMENT ASSERTION main.ts

```
class Connection {
  host: string;
  port = 5432;                        // initialiser
  timeoutMs?: number;                 // optional: undefined allowed
  socket!: { send(msg: string): string };   // definite assignment: set by connect()

  constructor(host: string, port?: number) {
    this.host = host;
    if (port) this.port = port;
  }

  connect(): this {
    this.socket = { send: (msg) => `sent "${msg}" to ${this.host}:${this.port}` };
    return this;
  }
}

const c = new Connection("db.lbf.in").connect();
console.log(c.socket.send("SELECT 1"), c.timeoutMs ?? "default timeout");
```

OUTPUT

```
sent "SELECT 1" to db.lbf.in:5432 default timeout
```

9.1.3 Methods and this types

Inside methods, `this` has the type of the class instance. A method can return `this` (the type, not just the value) to support **fluent** chaining that keeps working in subclasses: a subclass that calls an inherited method returning `this` gets the subclass type back. Remember that JavaScript methods lose their `this` when detached (`const f = obj.method`); arrow-function fields keep it, at the cost of one function per instance.

FLUENT METHODS RETURNING THIS, AND DETACHED METHODS main.ts

```
class QueryBuilder {
  protected parts: string[] = [];
  select(...cols: string[]): this { this.parts.push(`SELECT ${cols.join(", ")}`); return this; }
  from(table: string): this { this.parts.push(`FROM ${table}`); return this; }
  build(): string { return this.parts.join(" "); }
}

class PagedQueryBuilder extends QueryBuilder {
  page(n: number, size = 10): this { this.parts.push(`LIMIT ${size} OFFSET ${(n - 1) * size}`); return
this; }
}

// select() and from() return this, so page() is still available after them
console.log(new PagedQueryBuilder().select("id", "name").from("students").page(3).build());

class Counter {
  count = 0;
  increment() { this.count++; return this.count; }
  incrementArrow = () => ++this.count;          // this is bound to the instance
}
const counter = new Counter();
const inc = counter.incrementArrow;
console.log(inc(), inc());
const broken = counter.increment;
try { broken(); } catch (e) { console.log("detached method:", (e as Error).message); }
```

```
OUTPUT
SELECT id, name FROM students LIMIT 10 OFFSET 20
1 2
detached method: Cannot read properties of undefined (reading 'count')
```

9.2 Access modifiers

9.2.1 public, private, protected

Access modifiers control **where** a member can be used. `public` (the default) members are accessible everywhere. `private` members are accessible only inside the class that declares them. `protected` members are accessible inside the class and its subclasses, but not from outside. They let you hide implementation details and expose a small, stable public API (encapsulation). TypeScript's modifiers are **compile-time only**: in the emitted JavaScript the fields are ordinary properties.

Modifier	Same class	Subclasses	Outside code
`public` (default)	yes	yes	yes
`protected`	yes	yes	no
`private`	yes	no	no
`#field` (JavaScript)	yes	no	no (enforced at run time)

ACCESS RULES FOR PUBLIC, PROTECTED AND PRIVATE

main.ts

```
class BankAccount {
  public owner: string;
  protected balance = 0;
  private pin: string;

  constructor(owner: string, pin: string) { this.owner = owner; this.pin = pin; }
  deposit(amount: number) { this.balance += amount; }
}

class SavingsAccount extends BankAccount {
  addInterest(rate: number) {
    this.balance *= 1 + rate;    // OK: protected is visible in subclasses
    console.log(this.pin);       // error: private to BankAccount
  }
}

const acc = new SavingsAccount("Asha", "1234");
acc.owner = "Ravi";              // OK: public
acc.balance = 1_000_000;         // error: protected
acc.pin;                         // error: private
```

COMPILER ERRORS (tsc --strict)

```
main.ts(13,22): error TS2341: Property 'pin' is private and only accessible within class
'BankAccount'.
main.ts(19,5): error TS2445: Property 'balance' is protected and only accessible within class
'BankAccount' and its subclasses.
main.ts(20,5): error TS2341: Property 'pin' is private and only accessible within class 'BankAccount'.
```

9.2.2 TypeScript private vs JavaScript #private

JavaScript has its own private fields, written with a `#` prefix (`#balance`). They are **enforced at run time**: code outside the class cannot read them at all, not even with bracket notation, `JSON.stringify` or `Object.keys`. TypeScript's `private` is only a compile-time check: `(acc as any).pin` or `acc["pin"]` still works, and the value appears in logs and JSON. Use `#private` when real privacy matters (secrets, library internals); TypeScript `private` is fine for ordinary encapsulation and is needed with some decorators and frameworks.

COMPILE-TIME PRIVATE VERSUS RUN-TIME #PRIVATE

main.ts

```
class Wallet {
  private pin = "1234";          // TypeScript-private: hidden from the type checker only
  #balance = 500;                // JavaScript-private: hidden at run time

  get balance() { return this.#balance; }
  spend(amount: number) {
    if (amount > this.#balance) throw new Error("insufficient balance");
    this.#balance -= amount;
  }
}

const w = new Wallet();
w.spend(120);
console.log(w.balance);
console.log(w["pin"], JSON.stringify(w), Object.keys(w));      // TS private leaks at run time
console.log((w as any)["#balance"]);                             // # fields cannot be reached
```

OUTPUT

```
380
1234 {"pin":"1234"} [ 'pin' ]
undefined
```

9.2.3 readonly properties

`readonly` fields can be assigned only in their declaration or in the constructor. They suit identifiers, configuration and values that define the object. Combine with `private` or `protected` as needed (`private readonly id`). As with object types, readonly is shallow: a readonly array field can still be mutated unless its type is `readonly T[]`.

READONLY FIELDS CAN ONLY BE SET DURING CONSTRUCTION

main.ts

```
class Invoice {
  readonly number: string;
  readonly items: readonly string[];
  constructor(n: string, items: string[]) { this.number = n; this.items = items; }
  rename() { this.number = "X"; }     // error: only in the constructor
}
const inv = new Invoice("INV-1", ["pen"]);
inv.number = "INV-2";
inv.items.push("book");
```

COMPILER ERRORS (tsc --strict)

```
main.ts(5,19): error TS2540: Cannot assign to 'number' because it is a read-only property.
main.ts(8,5): error TS2540: Cannot assign to 'number' because it is a read-only property.
main.ts(9,11): error TS2339: Property 'push' does not exist on type 'readonly string[]'.
```

9.2.4 Parameter properties constructor(private name: string)

A **parameter property** is a constructor parameter with a modifier (`public` , `private` , `protected` or `readonly`). It declares the field and assigns it in one step, removing the repetitive "declare, list as parameter, assign" boilerplate. They are popular in Angular and NestJS for dependency injection. Note that parameter properties generate code, so they are not allowed with `erasableSyntaxOnly` or Node.js type stripping (6.4.5).

PARAMETER PROPERTIES DECLARE AND ASSIGN FIELDS IN ONE STEP

main.ts

```
// Long form
class ProductA {
  readonly id: number;
  private price: number;
  constructor(id: number, price: number) {
    this.id = id;
    this.price = price;
  }
}

// Same class with parameter properties
class ProductB {
  constructor(public readonly id: number, private price: number, public title = "Untitled") {}
  priceWithGst(): number { return Math.round(this.price * 1.18); }
}

const p = new ProductB(7, 1000, "Notebook pack");
console.log(p.id, p.title, p.priceWithGst(), new ProductA(1, 50).id);
```

OUTPUT

```
7 Notebook pack 1180 1
```

9.3 Getters, setters and static members

Accessors (`get` and `set`) look like properties from outside but run code: computed values, validation and change tracking. A getter without a setter is read-only. Since TypeScript 5.1 the getter and setter may have unrelated types (for example a setter that accepts `string | number` and a getter that returns `number`). **Static** members belong to the class itself rather than to instances (`Temperature.fromFahrenheit`, counters, caches, constants); they can also be `private` or `readonly`, and a `static { }` block runs once when the class is defined.

GETTERS, SETTERS, STATIC FIELDS, STATIC METHODS AND A STATIC GETTER main.ts

```
class Temperature {
  static readonly ABSOLUTE_ZERO = -273.15;
  static #created = 0;
  #celsius = 0;

  constructor(celsius: number) { this.celsius = celsius; Temperature.#created++; }

  get celsius(): number { return this.#celsius; }
  set celsius(value: number) {
    if (value < Temperature.ABSOLUTE_ZERO) throw new RangeError(`${value} is below absolute zero`);
    this.#celsius = value;
  }

  get fahrenheit(): number { return (this.#celsius * 9) / 5 + 32; }     // computed, read-only

  static fromFahrenheit(f: number): Temperature { return new Temperature(((f - 32) * 5) / 9); }
  static get created(): number { return Temperature.#created; }
}

const t = new Temperature(25);
t.celsius = 37;
console.log(t.celsius, t.fahrenheit.toFixed(1), Temperature.fromFahrenheit(212).celsius,
Temperature.created);
try { t.celsius = -300; } catch (e) { console.log((e as Error).message); }
```

```
OUTPUT
37 98.6 100 2
-300 is below absolute zero
```

GETTER-ONLY PROPERTIES ARE READ-ONLY; STATICS ARE NOT ON INSTANCES main.ts

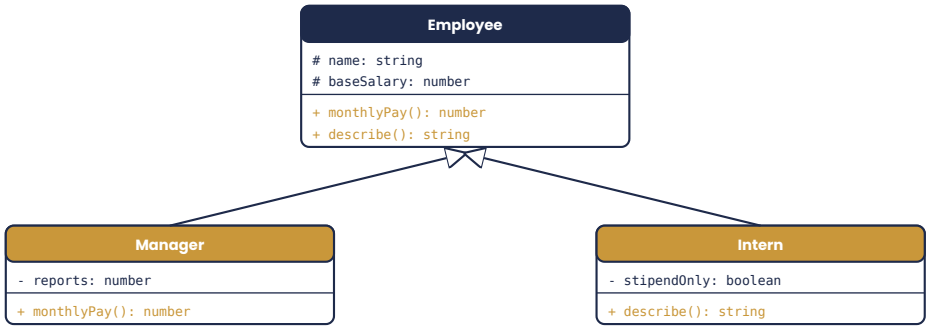
```
class Temperature {
  #c = 0;
  get fahrenheit() { return (this.#c * 9) / 5 + 32; }
  static count = 0;
}
const t = new Temperature();
t.fahrenheit = 100;          // no setter
t.count;                     // static members live on the class
Temperature.fahrenheit;
```

```
COMPILER ERRORS (tsc --strict)
main.ts(7,3): error TS2540: Cannot assign to 'fahrenheit' because it is a read-only property.
main.ts(8,3): error TS2576: Property 'count' does not exist on type 'Temperature'. Did you mean to
access the static member 'Temperature.count' instead?
main.ts(9,13): error TS2339: Property 'fahrenheit' does not exist on type 'typeof Temperature'.
```

9.4 Inheritance

9.4.1 extends and super

`class Child extends Parent` creates a subclass that inherits the parent's fields and methods. A subclass constructor must call `super(...)` with the parent's constructor arguments **before** using `this`; methods can call the parent's version with `super.method()`. A subclass instance can be used wherever the parent type is expected (it is a subtype).



A SMALL INHERITANCE HIERARCHY (UML: # PROTECTED, - PRIVATE, + PUBLIC)

SUBCLASSES WITH SUPER CALLS AND OVERRIDDEN METHODS

main.ts

```
class Employee {
  constructor(protected name: string, protected baseSalary: number) {}
  monthlyPay(): number { return this.baseSalary / 12; }
  describe(): string { return `${this.name} earns Rs
${Math.round(this.monthlyPay()).toLocaleString("en-IN")} a month`; }
}

class Manager extends Employee {
  constructor(name: string, baseSalary: number, private reports: number) {
    super(name, baseSalary);                        // must come before using this
  }
  override monthlyPay(): number {
    return super.monthlyPay() + this.reports * 2000;   // reuse the parent version
  }
}

class Intern extends Employee {
  override describe(): string { return `${super.describe()} (intern)`; }
}

const staff: Employee[] = [new Employee("Asha", 960000), new Manager("Ravi", 1800000, 6), new
Intern("Kiran", 240000)];
staff.forEach((e) => console.log(e.describe()));
```

OUTPUT

```
Asha earns Rs 80,000 a month
Ravi earns Rs 1,62,000 a month
Kiran earns Rs 20,000 a month (intern)
```

SUPER() MUST BE CALLED BEFORE THIS IS USED

main.ts

```
class Employee { constructor(protected name: string) {} }
class Manager extends Employee {
  reports: number;
  constructor(name: string) {
    this.reports = 0;            // this before super
    super(name);
  }
}
```

COMPILER ERRORS (tsc --strict)

```
main.ts(5,5): error TS17009: 'super' must be called before accessing 'this' in the constructor of a
derived class.
```

9.4.2 Method overriding and the override keyword

A subclass **overrides** a method by declaring one with the same name; its signature must be compatible with the parent's (same or more general parameters, same or more specific return type), so that subclass objects remain usable as parent objects. The `override` modifier (TypeScript 4.3+) states that a method is meant to replace a parent method: if the parent method is renamed or removed, `override` turns the silent bug into a compile error.

9.4.3 noImplicitOverride

With the compiler option `noImplicitOverride`, every method or property that overrides a parent member **must** be marked `override`. Together with the check above, this makes the relationship explicit in both directions: you cannot accidentally override a parent method by choosing the same name, and you cannot think you are overriding something that no longer exists.

SHAPES.TS — TypeScript

```
class Shape {
  area(): number { return 0; }
  describe(): string { return `area ${this.area().toFixed(2)}`; }
}

class Circle extends Shape {
  constructor(private r: number) { super(); }
  override area(): number { return Math.PI * this.r ** 2; }    // correct
  override perimeter(): number { return 2 * Math.PI * this.r; } // nothing to override
  describe(): string { return "circle"; }                       // override keyword missing
}

class Square extends Shape {
  constructor(private side: number) { super(); }
  override area(): string { return "big"; }                     // incompatible return type
}
```

OVERRIDE AND NOIMPLICITOVERRIDE CATCH BROKEN AND ACCIDENTAL OVERRIDES — terminal

```
npx tsc --noEmit --strict --noImplicitOverride --target es2022 shapes.ts
```

TERMINAL

```
$ npx tsc --noEmit --strict --noImplicitOverride --target es2022 shapes.ts
shapes.ts(9,12): error TS4113: This member cannot have an 'override' modifier because it is not
declared in the base class 'Shape'.
shapes.ts(10,3): error TS4114: This member must have an 'override' modifier because it overrides a
member in the base class 'Shape'.
shapes.ts(15,12): error TS2416: Property 'area' in type 'Square' is not assignable to the same
property in base type 'Shape'.
  Type '() => string' is not assignable to type '() => number'.
    Type 'string' is not assignable to type 'number'.
```

9.5 Abstract classes

9.5.1 Abstract methods and properties

An **abstract class** is a base class that cannot be instantiated directly; it exists to be extended. It can contain normal implemented members (shared code) and **abstract members**, declared without a body, which every concrete subclass must implement. Abstract classes are ideal for the **template method**

pattern: the base class defines the algorithm and calls abstract steps that subclasses fill in. Unlike interfaces, abstract classes exist at run time and can hold state and implementation.

AN ABSTRACT BASE CLASS WITH A TEMPLATE METHOD

main.ts

```
abstract class Report {
  constructor(protected title: string) {}

  protected abstract readonly format: "csv" | "text";
  protected abstract rows(): string[][];                  // each subclass supplies the data

  render(): string {                                      // template method: shared algorithm
    const body = this.rows().map((r) => (this.format === "csv" ? r.join(",") : r.join("  |  ")));
    return [`== ${this.title} (${this.format}) ==`, ...body].join("\n");
  }
}

class MarksReport extends Report {
  protected readonly format = "text";
  constructor(private marks: Record<string, number>) { super("Marks"); }
  protected rows() { return Object.entries(this.marks).map(([n, m]) => [n.padEnd(6), String(m)]); }
}

class FeesReport extends Report {
  protected readonly format = "csv";
  constructor(private fees: [string, number][]) { super("Fees"); }
  protected rows() { return [["name", "fee"], ...this.fees.map(([n, f]) => [n, String(f)])]; }
}

const reports: Report[] = [new MarksReport({ Asha: 91, Ravi: 76 }), new FeesReport([["Asha", 9999],
["Ravi", 14999]])];
reports.forEach((r) => console.log(r.render()));
```

OUTPUT

```
== Marks (text) ==
Asha    |  91
Ravi    |  76
== Fees (csv) ==
name,fee
Asha,9999
Ravi,14999
```

ABSTRACT CLASSES CANNOT BE CREATED, AND SUBCLASSES MUST IMPLEMENT ABSTRACT MEMBERS

main.ts

```
abstract class Report {
  abstract rows(): string[][];
  render() { return this.rows().length; }
}

const r = new Report();                    // cannot instantiate an abstract class

class EmptyReport extends Report {}        // abstract member not implemented
```

COMPILER ERRORS (tsc --strict)

```
main.ts(6,11): error TS2511: Cannot create an instance of an abstract class.
main.ts(8,7): error TS2515: Non-abstract class 'EmptyReport' does not implement inherited abstract
member rows from class 'Report'.
```

9.6 Interfaces and classes

9.6.1 implements

`class C implements I` asks the compiler to check that the class has every member the interface requires, with compatible types. It does not change the class or add anything to it (the class could satisfy the interface structurally anyway); it is a deliberate **contract** that produces clear errors at the class if a member is missing, and it documents the class's role.

CLASSES IMPLEMENTING AN INTERFACE main.ts

```
interface Payable {
  readonly payee: string;
  amountDue(): number;
}

class Invoice implements Payable {
  constructor(public readonly payee: string, private items: number[]) {}
  amountDue(): number { return this.items.reduce((a, b) => a + b, 0); }
}

class Salary implements Payable {
  constructor(public readonly payee: string, private monthly: number) {}
  amountDue(): number { return this.monthly; }
}

function payAll(bills: Payable[]): number {
  return bills.reduce((total, b) => {
    console.log(`  pay ${b.payee.padEnd(12)} Rs ${b.amountDue()}`);
    return total + b.amountDue();
  }, 0);
}

console.log("total", payAll([new Invoice("Office Mart", [1200, 450]), new Salary("Asha", 80000)]));
```

OUTPUT

```
  pay Office Mart  Rs 1650
  pay Asha         Rs 80000
total 81650
```

IMPLEMENTS REPORTS MISSING OR MISSPELLED MEMBERS AT THE CLASS main.ts

```
interface Payable { readonly payee: string; amountDue(): number }

class Refund implements Payable {
  constructor(public payee: string) {}
  amountdue() { return 100; }              // wrong name
}
```

COMPILER ERRORS (tsc --strict)

```
main.ts(3,7): error TS2420: Class 'Refund' incorrectly implements interface 'Payable'.
  Property 'amountDue' is missing in type 'Refund' but required in type 'Payable'.
```

9.6.2 Implementing multiple interfaces

A class can extend only one class, but it can implement **any number of interfaces**: `class C extends Base implements A, B, C`. This is how TypeScript (like Java and C#) models objects with several roles, and it encourages small, focused interfaces (the interface segregation principle, 9.8.3).

ONE CLASS IMPLEMENTING THREE INTERFACES main.ts

```
interface Printable { print(): string }
interface Serializable { toJSON(): object }
interface Comparable<T> { compareTo(other: T): number }

class Course implements Printable, Serializable, Comparable<Course> {
  constructor(public code: string, public fee: number) {}
  print() { return `${this.code} (Rs ${this.fee})`; }
  toJSON() { return { code: this.code, fee: this.fee }; }
  compareTo(other: Course) { return this.fee - other.fee; }
}

const courses = [new Course("PY201", 14999), new Course("TS101", 9999), new Course("JV301", 12999)];
courses.sort((a, b) => a.compareTo(b));
console.log(courses.map((c) => c.print()).join(", "));
console.log(JSON.stringify(courses));
```

OUTPUT

```
TS101 (Rs 9999), JV301 (Rs 12999), PY201 (Rs 14999)
[{"code":"TS101","fee":9999},{"code":"JV301","fee":12999},{"code":"PY201","fee":14999}]
```

9.6.3 Abstract class vs interface

	interface	abstract class
exists at run time	no (erased)	yes (a real class)
implementation code and state	no	yes: shared methods and fields
constructor	no	yes
a class can use	many (implements A, B)	one (extends)
access modifiers	all members public	public, protected, private
describes plain objects too	yes	only via its instance type
use when	defining a contract or role	sharing code among closely related classes

A common combination: an interface for the contract (what callers depend on) and an abstract class that implements it partially for convenience, which concrete classes extend.

9.7 Classes as types

9.7.1 Instance types and typeof ClassName

A class declaration creates two things: a **value** (the constructor function, used with `new` and for static members) and a **type** with the same name, the **instance type** describing objects created by the class. In `let s: Student`, `Student` is the instance type. To refer to the type of the class itself (the constructor, including its static members), write `typeof Student`. This is needed for factories, registries and plugin systems that receive classes as arguments.

THE INSTANCE TYPE VERSUS TYPEOF CLASSNAME main.ts

```
class Student {
  static school = "Learn Being Forward";
  constructor(public name: string) {}
  greet() { return `Hi, I am ${this.name}`; }
}

const s: Student = new Student("Asha");             // instance type
const Cls: typeof Student = Student;                 // the class itself
console.log(s.greet(), "|", Cls.school, "|", new Cls("Ravi").name);

// A registry of classes: the values are constructors, so the type uses typeof / construct signatures
class Circle { constructor(public size: number) {} area() { return Math.PI * this.size ** 2; } }
class Square { constructor(public size: number) {} area() { return this.size ** 2; } }
const registry: Record<string, typeof Circle | typeof Square> = { circle: Circle, square: Square };

for (const [name, Ctor] of Object.entries(registry)) {
  const shape = new Ctor(2);
  console.log(name, shape.area().toFixed(2), shape instanceof Circle);
}

type StudentInstance = InstanceType<typeof Student>;     // utility type: back to the instance type (12.4.3)
const again: StudentInstance = s;
console.log(again.name);
```

OUTPUT

```
Hi, I am Asha | Learn Being Forward | Ravi
circle 12.57 true
square 4.00 false
Asha
```

9.7.2 Structural typing with classes

Class instance types are compared **structurally**, like any other object type (5.7): an object literal or an instance of an unrelated class is accepted where a class type is expected if it has the same public members. The exception is classes with `private`, `protected` or `#private` members: such members make the class behave **nominally**, because only instances of that class (or its subclasses) can have them. This is one way to prevent accidental mixing of same-shaped types.

PUBLIC STRUCTURE IS ENOUGH; PRIVATE MEMBERS MAKE A CLASS NOMINAL main.ts

```
class Point { constructor(public x: number, public y: number) {} }
class Vector { constructor(public x: number, public y: number) {} }

const p: Point = new Vector(1, 2);          // OK: same public structure
const q: Point = { x: 3, y: 4 };            // OK: a plain object fits too

class UserId { private readonly brand = "UserId"; constructor(public value: number) {} }
class OrderId { private readonly brand = "OrderId"; constructor(public value: number) {} }

const u: UserId = new OrderId(7);           // error: private members make classes nominal
```

COMPILER ERRORS (tsc --strict)

```
main.ts(10,7): error TS2322: Type 'OrderId' is not assignable to type 'UserId'.
  Types have separate declarations of a private property 'brand'.
```

9.8 OOP design in TypeScript

9.8.1 Encapsulation, abstraction, inheritance, polymorphism

Encapsulation

Keep data and the code that changes it together; hide internals with `private` / `#` and expose a small public API with validated methods and accessors.

Abstraction

Depend on what something does, not how: interfaces and abstract classes describe capabilities (`Payable`, `Repository`) without implementation details.

Inheritance

Reuse and specialise behaviour with `extends`, `super` and `override`; keep hierarchies shallow.

Polymorphism

Treat different objects through one type: a `Shape[]` of circles and squares, each with its own `area()`, or a union narrowed by kind.

THE FOUR PILLARS IN ONE EXAMPLE

main.ts

```
interface Notifier { send(to: string, message: string): string }      // abstraction

class EmailNotifier implements Notifier {
  #sent = 0;                                                          // encapsulation
  send(to: string, message: string) { this.#sent++; return `email to ${to}: ${message}
(#${this.#sent})`; }
}
class SmsNotifier implements Notifier {
  send(to: string, message: string) { return `SMS to ${to}: ${message.slice(0, 20)}`; }
}
class UrgentSmsNotifier extends SmsNotifier {                          // inheritance
  override send(to: string, message: string) { return super.send(to, "URGENT " + message); }
}

const channels: Notifier[] = [new EmailNotifier(), new SmsNotifier(), new UrgentSmsNotifier()];
for (const ch of channels) console.log(ch.send("Asha", "Your TypeScript batch starts Monday"));   //
polymorphism
```

OUTPUT

```
email to Asha: Your TypeScript batch starts Monday (#1)
SMS to Asha: Your TypeScript batc
SMS to Asha: URGENT Your TypeScri
```

9.8.2 Composition over inheritance

Inheritance creates tight coupling: subclasses depend on the parent's internals, deep hierarchies become rigid, and a class can have only one parent. **Composition** builds objects from smaller objects that each do one job, usually referenced through interfaces and passed in through the constructor (**dependency injection**). Behaviour can then be mixed freely and swapped (for tests, for different environments) without new subclasses. Use inheritance for a genuine "is-a" relationship with shared behaviour; prefer composition ("has-a") otherwise.

COMPOSITION WITH INJECTED DEPENDENCIES main.ts

```
interface Logger { log(msg: string): void }
interface PriceRule { apply(amount: number): number }

class ConsoleLogger implements Logger { log(msg: string) { console.log(`  [log] ${msg}`); } }
class GstRule implements PriceRule { apply(a: number) { return a * 1.18; } }
class DiscountRule implements PriceRule { constructor(private pct: number) {} apply(a: number) {
return a * (1 - this.pct / 100); } }

class Checkout {                                    // composed from parts, not a subclass of them
  constructor(private rules: PriceRule[], private logger: Logger) {}
  total(amount: number): number {
    const result = this.rules.reduce((a, rule) => rule.apply(a), amount);
    this.logger.log(`${amount} -> ${result.toFixed(2)} after ${this.rules.length} rules`);
    return Math.round(result);
  }
}

const festive = new Checkout([new DiscountRule(20), new GstRule()], new ConsoleLogger());
const normal = new Checkout([new GstRule()], { log: () => {} });     // a silent logger for tests
console.log(festive.total(10000), normal.total(10000));
```

OUTPUT

```
  [log] 10000 -> 9440.00 after 2 rules
9440 11800
```

9.8.3 SOLID principles with interfaces

Principle	Meaning	In TypeScript
S: Single responsibility	a class has one reason to change	separate PriceCalculator, InvoiceRepository, EmailSender
O: Open/closed	open for extension, closed for modification	add a new PriceRule class instead of editing a switch
L: Liskov substitution	subtypes must work wherever the base type does	overrides keep compatible signatures and behaviour; override + strict checks help
I: Interface segregation	many small interfaces beat one large one	`Readable`, `Writable` rather than one `Storage` with 20 methods
D: Dependency inversion	depend on abstractions, not concrete classes	constructors take interfaces (`Logger`, `Repository<T>`); inject implementations

INTERFACE SEGREGATION AND DEPENDENCY INVERSION

main.ts

```
// I + D: small interfaces, and a service that depends only on them
interface Reader<T> { findById(id: number): T | undefined }
interface Writer<T> { save(item: T): void }

interface Student { id: number; name: string; fee: number }

class InMemoryStudents implements Reader<Student>, Writer<Student> {
  #rows = new Map<number, Student>();
  findById(id: number) { return this.#rows.get(id); }
  save(s: Student) { this.#rows.set(s.id, s); }
}

class FeeService {                                        // S: only fee logic
  constructor(private reader: Reader<Student>, private writer: Writer<Student>) {}
  applyScholarship(id: number, pct: number): string {
    const s = this.reader.findById(id);
    if (!s) return `student ${id} not found`;
    const updated = { ...s, fee: Math.round(s.fee * (1 - pct / 100)) };
    this.writer.save(updated);
    return `${s.name}: Rs ${s.fee} -> Rs ${updated.fee}`;
  }
}

const repo = new InMemoryStudents();
repo.save({ id: 1, name: "Asha", fee: 9999 });
const fees = new FeeService(repo, repo);
console.log(fees.applyScholarship(1, 25), "|", fees.applyScholarship(2, 10));
```

OUTPUT

```
Asha: Rs 9999 -> Rs 7499 | student 2 not found
```

Classes declare typed fields (initialised or definitely assigned, or marked with ! when set elsewhere), typed constructors and methods; methods can return this for fluent APIs. public, protected and private are compile-time checks, while #private is enforced at run time; readonly fields are set only during construction, and parameter properties declare and assign fields in one step (not erasable). Accessors compute or validate values; static members belong to the class. Subclasses call super() before this, override methods with compatible signatures, and override plus noImplicitOverride make overriding explicit. Abstract classes cannot be instantiated and force subclasses to implement abstract members. implements checks a class against one or more interfaces. A class name is both a value and an instance type; typeof Class is the constructor type. Instance types are structural unless the class has private members. Apply encapsulation, abstraction, inheritance and polymorphism, prefer composition with injected interfaces, and follow SOLID.

PRACTICE CHECKPOINT

An **employee payroll system** with an **abstract Employee**, **FullTime** and **Contract** subclasses, a **Payable** interface and **parameter properties**.

SAMPLE SOLUTION: A PAYROLL SYSTEM WITH AN ABSTRACT CLASS, SUBCLASSES, AN INTERFACE AND PARAMETER PROPERTIES

main.ts

```
interface Payable {
  readonly id: string;
  grossPay(month: string): number;
  deductions(gross: number): number;
```

```
}

abstract class Employee implements Payable {
  static #count = 0;
  readonly id: string;

  constructor(public readonly name: string, protected readonly department: string) {
    this.id = `E${String(++Employee.#count).padStart(3, "0")}`;
  }

  abstract grossPay(month: string): number;
  abstract readonly type: "full-time" | "contract";

  deductions(gross: number): number {
    return Math.round(gross * 0.1);                          // default: 10% TDS
  }

  payslip(month: string): string {
    const gross = this.grossPay(month);
    const ded = this.deductions(gross);
    return `${this.id} ${this.name.padEnd(6)} ${this.type.padEnd(9)} ${this.department.padEnd(9)}
gross ${String(gross).padStart(7)}  ded ${String(ded).padStart(6)}  net ${String(gross -
ded).padStart(7)}`;
  }

  static get headcount(): number { return Employee.#count; }
}

class FullTime extends Employee {
  readonly type = "full-time";
  constructor(name: string, department: string, private annualCtc: number, private bonusMonths:
string[] = []) {
    super(name, department);
  }
  override grossPay(month: string): number {
    const monthly = Math.round(this.annualCtc / 12);
    return this.bonusMonths.includes(month) ? monthly * 2 : monthly;
  }
  override deductions(gross: number): number {
    const pf = Math.round(Math.min(gross, 15000) * 0.12);    // provident fund
    return super.deductions(gross) + pf + 200;               // + professional tax
  }
}

class Contract extends Employee {
  readonly type = "contract";
  constructor(name: string, department: string, private hourlyRate: number, private hours:
Record<string, number>) {
    super(name, department);
  }
  override grossPay(month: string): number {
    return this.hourlyRate * (this.hours[month] ?? 0);
  }
}

class Payroll {
  constructor(private staff: Payable[]) {}
  run(month: string): number {
    return this.staff.reduce((total, p) => {
      const gross = p.grossPay(month);
      return total + gross - p.deductions(gross);
    }, 0);
  }
}
```

```
const staff: Employee[] = [
  new FullTime("Asha", "Training", 960000, ["Oct"]),
  new FullTime("Ravi", "Sales", 720000),
  new Contract("Kiran", "Design", 1200, { Sep: 80, Oct: 64 }),
];

for (const month of ["Sep", "Oct"]) {
  console.log(`--- Payroll ${month} 2026 ---`);
  staff.forEach((e) => console.log(e.payslip(month)));
  console.log(`total net payout: Rs ${new Payroll(staff).run(month).toLocaleString("en-IN")}`);
}
console.log("headcount:", Employee.headcount);
```

OUTPUT

```
--- Payroll Sep 2026 ---
E001 Asha   full-time Training  gross   80000  ded  10000  net   70000
E002 Ravi   full-time Sales     gross   60000  ded   8000  net   52000
E003 Kiran  contract  Design    gross   96000  ded   9600  net   86400
total net payout: Rs 2,08,400
--- Payroll Oct 2026 ---
E001 Asha   full-time Training  gross  160000  ded  18000  net  142000
E002 Ravi   full-time Sales     gross   60000  ded   8000  net   52000
E003 Kiran  contract  Design    gross   76800  ded   7680  net   69120
total net payout: Rs 2,63,120
headcount: 3
```

MODULE 10 · INTERMEDIATE

Generics

Minimum duration **3 days** | Level **Intermediate**

Generics let you write functions, classes and types that work with any type while keeping full type safety. This module explains the problem with any and type parameters as "type variables", generic functions (identity, inference of type arguments, multiple type parameters, generic arrow functions), constraints with extends and keyof, default and const type parameters, generic interfaces and type aliases such as ApiResponse<T>, generic function types, generic classes (Stack, Queue, Repository), the built-in generics Array, Promise, Map and Set, and best practices.

IN THIS MODULE

10.1 Why generics
10.2 Generic functions
10.3 Generic constraints
10.4 Generic interfaces and type aliases
10.5 Generic classes
10.6 Built-in generics
10.7 Generics best practices

10.1 Why generics

10.1.1 The problem with any

Suppose you want a function that returns the first element of an array, for an array of anything. Typing the parameter as `number[]` works only for numbers; writing one version per type duplicates code. Using `any[]` accepts every array, but the result is `any`: the information that you passed an array of strings is **lost**, so mistakes on the result are not caught and autocompletion disappears.

ANY ACCEPTS EVERYTHING BUT FORGETS THE TYPE — main.ts

```
function firstAny(items: any[]): any {
  return items[0];
}

const name = firstAny(["Asha", "Ravi"]);        // name: any
console.log(name.toUpperCase());                 // fine at run time
const mark = firstAny([91, 84]);                 // mark: any
console.log(mark.toUpperCase());                 // compiles, crashes at run time
```

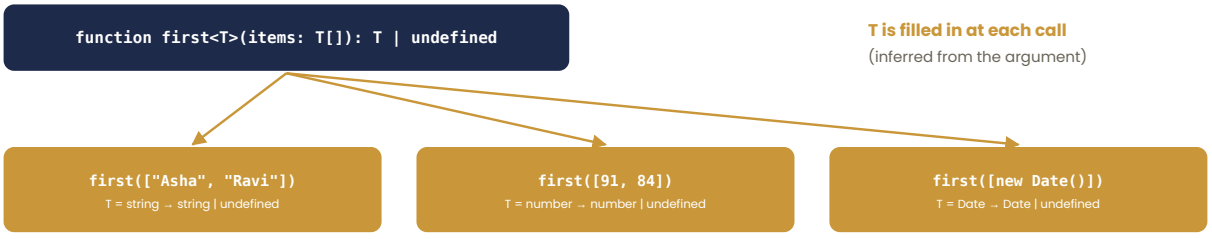
TERMINAL

```
$ tsc --strict main.ts
(no errors reported)
$ node main.js
ASHA
main.js:9
console.log(mark.toUpperCase()); // compiles, crashes at run time
                ^

TypeError: mark.toUpperCase is not a function
```

10.1.2 Type parameters as "type variables"

A **generic** function declares a **type parameter** in angle brackets, conventionally named `T` (for "type"): `function first<T>(items: T[]): T | undefined`. Think of `T` as a **variable that holds a type** instead of a value. Each time the function is called, `T` is filled in with a concrete type, either written explicitly (`first<string>(...)`) or, usually, **inferred** from the arguments. The relationship "the result has the same type as the elements of the input" is now part of the signature.



ONE GENERIC FUNCTION, A DIFFERENT T FOR EACH CALL

A GENERIC FUNCTION KEEPS THE TYPE OF ITS INPUT main.ts

```ts
function first<T>(items: T[]): T | undefined {
  return items[0];
}

const name = first(["Asha", "Ravi"]);         // T = string, name: string | undefined
const mark = first([91, 84]);                 // T = number
const day = first([new Date(2026, 0, 26)]);   // T = Date

console.log(name?.toUpperCase(), mark?.toFixed(1), day?.getFullYear());
```

```
OUTPUT
ASHA 91.0 2026
```

THE GENERIC VERSION CATCHES THE MISTAKE THAT ANY HID main.ts

```ts
function first<T>(items: T[]): T | undefined { return items[0]; }

const mark = first([91, 84]);
console.log(mark?.toUpperCase());
```

```
COMPILER ERRORS (tsc --strict)
main.ts(4,19): error TS2339: Property 'toUpperCase' does not exist on type 'number'.
```

10.2 Generic functions

10.2.1 function identity<T>(value: T): T

The simplest generic function, `identity`, returns its argument unchanged. It looks pointless, but it shows the essential idea: the type of the output is **linked** to the type of the input. More useful generic functions follow the same pattern: wrapping a value, picking from a collection, transforming with a callback, or caching results.

GENERIC FUNCTIONS THAT RELATE INPUT AND OUTPUT TYPES main.ts

```
function identity<T>(value: T): T {
  return value;
}

function wrapInArray<T>(value: T): T[] {
  return [value];
}

function lastOf<T>(items: readonly T[]): T | undefined {
  return items.at(-1);
}

function repeat<T>(value: T, times: number): T[] {
  return Array.from({ length: times }, () => value);
}

const a = identity("LBF");                  // string... in fact the literal type "LBF" for a const
const b = identity<number>(42);              // explicit type argument
const c = wrapInArray({ id: 1 });            // { id: number }[]
console.log(a, b, c, lastOf(["x", "y", "z"]), repeat("ab", 3));
```

OUTPUT

```
LBF 42 [ { id: 1 } ] z [ 'ab', 'ab', 'ab' ]
```

10.2.2 Inference of type arguments

You rarely write type arguments explicitly: TypeScript **infers** them from the arguments, the same way it infers variable types. Inference looks at every argument that mentions `T` and picks a type that fits all of them; if it cannot (for example, conflicting candidates), it reports an error. Write the type argument explicitly when there are no arguments to infer from (`new Map<string, number>()`, `useState<User | null>(null)`) or when inference picks a narrower type than you want.

INFERENCE FROM ARGUMENTS, AND EXPLICIT TYPE ARGUMENTS main.ts

```
function pair<T>(a: T, b: T): [T, T] {
  return [a, b];
}

const p1 = pair(1, 2);                       // T = number
const p2 = pair("x", "y");                   // T = string
const p3 = pair<string | number>(1, "two");  // explicit: allow both

function emptyList<T>(): T[] { return []; }
const names = emptyList<string>();           // nothing to infer from, so pass T
names.push("Asha");

console.log(p1, p2, p3, names);
```

OUTPUT

```
[ 1, 2 ] [ 'x', 'y' ] [ 1, 'two' ] [ 'Asha' ]
```

CONFLICTING INFERENCE CANDIDATES ARE AN ERROR main.ts

```
function pair<T>(a: T, b: T): [T, T] { return [a, b]; }
const p = pair(1, "two");
```

COMPILER ERRORS (tsc --strict)

```
main.ts(2,19): error TS2345: Argument of type 'string' is not assignable to parameter of type
'number'.
```

10.2.3 Multiple type parameters

A function can have several type parameters when it relates several independent types: `map<T, U>(items: T[], fn: (item: T) => U): U[]` transforms an array of `T` into an array of `U`, and both are inferred. Common names are `T`, `U`, `V`, or descriptive ones such as `K` (key), `V` (value), `TInput`, `TOutput` and `E` (error).

TWO AND THREE TYPE PARAMETERS main.ts

```
function mapAll<T, U>(items: readonly T[], fn: (item: T, index: number) => U): U[] {
  const out: U[] = [];
  items.forEach((item, i) => out.push(fn(item, i)));
  return out;
}

function zip<A, B>(as: readonly A[], bs: readonly B[]): [A, B][] {
  return as.slice(0, Math.min(as.length, bs.length)).map((a, i) => [a, bs[i]]);
}

function toMap<K, V>(entries: [K, V][]): Map<K, V> {
  return new Map(entries);
}

const lengths = mapAll(["Asha", "Ravi", "Meera"], (n) => n.length);        // T = string, U = number
const labels = mapAll([91, 84], (m, i) => `#${i + 1}: ${m}`);              // U = string
const zipped = zip(["Asha", "Ravi"], [91, 84, 77]);                         // [string, number][]
const marks = toMap(zipped);                                                // Map<string, number>
console.log(lengths, labels, zipped, marks.get("Ravi"));
```

OUTPUT

```
[ 4, 4, 5 ] [ '#1: 91', '#2: 84' ] [ [ 'Asha', 91 ], [ 'Ravi', 84 ] ] 84
```

10.2.4 Generic arrow functions

Arrow functions take type parameters before their parameter list: `const first = <T,>(items: T[]) => items[0]`. In `.ts` files you can write `<T>`; in `.tsx` files `<T>` would be read as a JSX tag, so write `<T,>` (with a trailing comma) or `<T extends unknown>`.

GENERIC ARROW FUNCTIONS

main.ts

```ts
const firstOf = <T,>(items: T[]): T | undefined => items[0];
const groupCount = <T, K extends string>(items: T[], key: (item: T) => K): Record<K, number> => {
  const out = {} as Record<K, number>;
  for (const item of items) {
    const k = key(item);
    out[k] = (out[k] ?? 0) + 1;
  }
  return out;
};

const students = [{ name: "Asha", city: "Mysuru" }, { name: "Ravi", city: "Pune" }, { name: "Kiran",
city: "Mysuru" }];
console.log(firstOf(students)?.name, groupCount(students, (s) => s.city));
```

OUTPUT

```
Asha { Mysuru: 2, Pune: 1 }
```

10.3 Generic constraints

10.3.1 T extends { length: number }

Inside a generic function, `T` could be **any** type, so you can only do things that work for every type (store it, return it, compare with `===`). To use a property such as `length`, **constrain** the type parameter with `extends`: `<T extends { length: number }>` means "T can be any type that has a numeric length". Strings, arrays and custom objects with a length all qualify, and the function still returns the caller's exact type.

AN UNCONSTRAINED T HAS NO KNOWN PROPERTIES

main.ts

```ts
function longest<T>(a: T, b: T): T {
  return a.length >= b.length ? a : b;
}
```

COMPILER ERRORS (tsc --strict)

```
main.ts(2,12): error TS2339: Property 'length' does not exist on type 'T'.
main.ts(2,24): error TS2339: Property 'length' does not exist on type 'T'.
```

CONSTRAINING T SO ITS PROPERTIES CAN BE USED

main.ts

```ts
function longest<T extends { length: number }>(a: T, b: T): T {
  return a.length >= b.length ? a : b;
}

const s = longest("TypeScript", "Java");              // string
const arr = longest([1, 2, 3], [4, 5]);               // number[]
const custom = longest({ length: 3, title: "A" }, { length: 9, title: "B" });   // keeps title
console.log(s, arr, custom.title);

function sortByKey<T extends { id: number }>(items: T[]): T[] {
  return [...items].sort((a, b) => a.id - b.id);
}
console.log(sortByKey([{ id: 3, name: "C" }, { id: 1, name: "A" }]).map((x) => x.name));
```

OUTPUT

```
TypeScript [ 1, 2, 3 ] B
[ 'A', 'C' ]
```

ARGUMENTS MUST SATISFY THE CONSTRAINT main.ts

```
function longest<T extends { length: number }>(a: T, b: T): T {
  return a.length >= b.length ? a : b;
}
longest(10, 20);
```

COMPILER ERRORS (tsc --strict)

```
main.ts(4,9): error TS2345: Argument of type 'number' is not assignable to parameter of type '{
length: number; }'.
```

10.3.2 K extends keyof T

`keyof T` is the union of the property names of `T` (14.1.1): for `{ name: string; age: number }` it is `"name" | "age"`. Constraining a second type parameter with `K extends keyof T` means "K must be one of T's keys", and `T[K]` is the type of that property (an **indexed access type**). Together they type functions that work with property names safely: `getProperty(obj, "name")` returns a string, and a misspelled key is a compile error.

TYPE-SAFE PROPERTY ACCESS WITH K EXTENDS KEYOF T main.ts

```
function getProperty<T, K extends keyof T>(obj: T, key: K): T[K] {
  return obj[key];
}

function pluck<T, K extends keyof T>(items: T[], key: K): T[K][] {
  return items.map((item) => item[key]);
}

function sortBy<T>(items: T[], key: keyof T): T[] {
  return [...items].sort((a, b) => (a[key] < b[key] ? -1 : a[key] > b[key] ? 1 : 0));
}

const student = { name: "Asha", age: 20, marks: [91, 84] };
const n = getProperty(student, "name");        // string
const m = getProperty(student, "marks");       // number[]
const people = [{ name: "Ravi", age: 22 }, { name: "Asha", age: 20 }];
console.log(n.toUpperCase(), m.length, pluck(people, "age"), sortBy(people, "name").map((p) =>
p.name));
```

OUTPUT

```
ASHA 2 [ 22, 20 ] [ 'Asha', 'Ravi' ]
```

UNKNOWN KEYS AND WRONG RESULT TYPES ARE CAUGHT main.ts

```
function getProperty<T, K extends keyof T>(obj: T, key: K): T[K] { return obj[key]; }
const student = { name: "Asha", age: 20 };
getProperty(student, "email");
const age: string = getProperty(student, "age");
```

COMPILER ERRORS (tsc --strict)

```
main.ts(3,22): error TS2345: Argument of type '"email"' is not assignable to parameter of type '"name"
| "age"'.
main.ts(4,7): error TS2322: Type 'number' is not assignable to type 'string'.
```

10.3.3 Default type parameters

A type parameter can have a **default**, used when the type argument is neither given nor inferable: `interface ApiResponse<T = unknown>`. Defaults make generic types easier to use in the common case, and they can refer to earlier parameters (`<T, U = T[]>`). Parameters with defaults must come after required ones.

DEFAULT TYPE PARAMETERS main.ts

```
interface ApiResponse<T = unknown, E = string> {
  ok: boolean;
  data?: T;
  error?: E;
}

const raw: ApiResponse = { ok: true, data: { anything: 1 } };          // T = unknown, E = string
const typed: ApiResponse<string[]> = { ok: true, data: ["TS101"] };
const failed: ApiResponse<never, { code: number }> = { ok: false, error: { code: 404 } };

function createStore<S = { count: number }>(initial: S) {
  let state = initial;
  return { get: () => state, set: (s: S) => { state = s; } };
}
const counter = createStore({ count: 0 });
counter.set({ count: 5 });
console.log(raw.ok, typed.data?.[0], failed.error?.code, counter.get());
```

```
OUTPUT
true TS101 404 { count: 5 }
```

10.3.4 Const type parameters

By default, inference widens literals in arrays and objects (`["a", "b"]` is inferred as `string[]`). Adding the `const` modifier to a type parameter (TypeScript 5.0+), `<const T>`, makes inference behave as if the caller had written `as const`: literal types and tuple shapes are preserved. It is useful for functions that build configuration, routes or option lists where the exact values matter. Use `readonly` in the constraint so that readonly arrays are accepted.

CONST TYPE PARAMETERS PRESERVE LITERAL TYPES main.ts

```
function routes<T extends readonly string[]>(paths: T): T { return paths; }
function routesConst<const T extends readonly string[]>(paths: T): T { return paths; }

const r1 = routes(["/home", "/courses"]);             // string[]
const r2 = routesConst(["/home", "/courses"]);        // readonly ["/home", "/courses"]

type Route = (typeof r2)[number];                       // "/home" | "/courses"
function go(route: Route) { return `navigating to ${route}`; }

console.log(r1.length, go("/courses"));
```

```
OUTPUT
2 navigating to /courses
```

WITH CONST T, ONLY THE EXACT LITERALS ARE ALLOWED main.ts

```
function routesConst<const T extends readonly string[]>(paths: T): T { return paths; }
const r2 = routesConst(["/home", "/courses"]);
type Route = (typeof r2)[number];
const bad: Route = "/about";
```

COMPILER ERRORS (tsc --strict)

```
main.ts(4,7): error TS2322: Type '"/about"' is not assignable to type '"/home" | "/courses"'.
```

10.4 Generic interfaces and type aliases

10.4.1 ApiResponse<T>

Interfaces and type aliases can have type parameters too. A generic type is a **template**: `ApiResponse<T>` is not a type by itself; `ApiResponse<User>` and `ApiResponse<Course[]>` are. Generic types remove duplication when many types share a structure (responses, pages of results, form state, events) and differ only in a payload.

GENERIC INTERFACES AND A GENERIC RESULT UNION main.ts

```
interface ApiResponse<T> {
  status: number;
  data: T;
  receivedAt: Date;
}

interface Page<T> {
  items: T[];
  page: number;
  totalPages: number;
}

type Result<T, E = Error> = { ok: true; value: T } | { ok: false; error: E };    // generic union

interface User { id: number; name: string }
interface Course { code: string; fee: number }

const userRes: ApiResponse<User> = { status: 200, data: { id: 1, name: "Asha" }, receivedAt: new
Date() };
const coursePage: ApiResponse<Page<Course>> = {
  status: 200, receivedAt: new Date(),
  data: { items: [{ code: "TS101", fee: 9999 }], page: 1, totalPages: 3 },
};

function divide(a: number, b: number): Result<number, string> {
  return b === 0 ? { ok: false, error: "division by zero" } : { ok: true, value: a / b };
}

console.log(userRes.data.name, coursePage.data.items[0].code, `page
${coursePage.data.page}/${coursePage.data.totalPages}`);
for (const r of [divide(10, 4), divide(1, 0)]) console.log(r.ok ? `value ${r.value}` : `error
${r.error}`);
```

OUTPUT

```
Asha TS101 page 1/3
value 2.5
error division by zero
```

10.4.2 Generic function types

Function types can be generic in two different ways, and the position of `<T>` matters. `type Mapper<T, U> = (item: T) => U` is a **generic type**: you choose `T` and `U` when you use the type, and the function then works only with those. `type Identity = <T>(value: T) => T` is a type of **generic function**: the function itself must work for every `T`, chosen at each call.

GENERIC TYPES OF FUNCTIONS, AND A GENERIC INTERFACE WITH METHODS main.ts

```
type Mapper<T, U> = (item: T) => U;                // generic type: T and U fixed when used
type Identity = <T>(value: T) => T;                  // a function that is itself generic
type Comparator<T> = (a: T, b: T) => number;

const toLength: Mapper<string, number> = (s) => s.length;
const id: Identity = (v) => v;                          // must work for any T
const byFee: Comparator<{ fee: number }> = (a, b) => a.fee - b.fee;

console.log(toLength("generics"), id(42), id("x"), [{ fee: 3 }, { fee: 1 }].sort(byFee));

interface Cache<K, V> {
  get(key: K): V | undefined;
  set(key: K, value: V): void;
  getOrCreate(key: K, create: (key: K) => V): V;
}

function memoryCache<K, V>(): Cache<K, V> {
  const m = new Map<K, V>();
  return {
    get: (k) => m.get(k),
    set: (k, v) => void m.set(k, v),
    getOrCreate(k, create) {
      if (!m.has(k)) m.set(k, create(k));
      return m.get(k)!;
    },
  };
}

const squares = memoryCache<number, number>();
console.log(squares.getOrCreate(12, (n) => n * n), squares.get(12), squares.get(5));
```

```
OUTPUT
8 42 x [ { fee: 1 }, { fee: 3 } ]
144 144 undefined
```

10.5 Generic classes

10.5.1 Stack<T>, Queue<T>, Repository<T>

Classes take type parameters after the class name, and the parameter is available to every field and method. Generic classes are how collections and services are written once and used for many element types: a `Stack<number>` accepts only numbers, a `Repository<Student>` stores only students. Static members cannot use the class's type parameter, because they belong to the class, not to one instance.

GENERIC STACK AND QUEUE CLASSES

main.ts

```
class Stack<T> {
  #items: T[] = [];
  push(item: T): this { this.#items.push(item); return this; }
  pop(): T | undefined { return this.#items.pop(); }
  peek(): T | undefined { return this.#items.at(-1); }
  get size(): number { return this.#items.length; }
}

class Queue<T> {
  #items: T[] = [];
  #head = 0;
  enqueue(item: T): void { this.#items.push(item); }
  dequeue(): T | undefined {
    if (this.#head >= this.#items.length) return undefined;
    return this.#items[this.#head++];               // O(1): no shifting
  }
  get length(): number { return this.#items.length - this.#head; }
}

const undo = new Stack<string>().push("type A").push("type B").push("delete");
console.log(undo.pop(), undo.peek(), undo.size);

const tickets = new Queue<{ id: number; issue: string }>();
tickets.enqueue({ id: 1, issue: "login fails" });
tickets.enqueue({ id: 2, issue: "fee receipt missing" });
console.log(tickets.dequeue()?.issue, "| waiting:", tickets.length);
```

OUTPUT

```
delete type B 2
login fails | waiting: 1
```

A STACK<NUMBER> ACCEPTS ONLY NUMBERS

main.ts

```
class Stack<T> {
  #items: T[] = [];
  push(item: T) { this.#items.push(item); }
}
const s = new Stack<number>();
s.push("3");
```

COMPILER ERRORS (tsc --strict)

```
main.ts(6,8): error TS2345: Argument of type 'string' is not assignable to parameter of type 'number'.
```

A GENERIC REPOSITORY<T EXTENDS ENTITY> AND A SPECIALISED SUBCLASS

main.ts

```
interface Entity { id: number }

class Repository<T extends Entity> {
  protected items = new Map<number, T>();

  add(item: T): T {
    if (this.items.has(item.id)) throw new Error(`duplicate id ${item.id}`);
    this.items.set(item.id, item);
    return item;
  }
  findById(id: number): T | undefined { return this.items.get(id); }
  findWhere(predicate: (item: T) => boolean): T[] { return [...this.items.values()].filter(predicate);
}
  update(id: number, changes: Partial<Omit<T, "id">>): T | undefined {
    const current = this.items.get(id);
    if (!current) return undefined;
    const updated = { ...current, ...changes };
    this.items.set(id, updated);
    return updated;
  }
  get count(): number { return this.items.size; }
}

interface Student extends Entity { name: string; city: string; fee: number }

class StudentRepository extends Repository<Student> {
  fromCity(city: string): Student[] { return this.findWhere((s) => s.city === city); }
}

const repo = new StudentRepository();
repo.add({ id: 1, name: "Asha", city: "Mysuru", fee: 9999 });
repo.add({ id: 2, name: "Ravi", city: "Pune", fee: 14999 });
repo.add({ id: 3, name: "Kiran", city: "Mysuru", fee: 12999 });
repo.update(2, { fee: 12000 });
console.log(repo.count, repo.findById(2)?.fee, repo.fromCity("Mysuru").map((s) => s.name));
```

OUTPUT

```
3 12000 [ 'Asha', 'Kiran' ]
```

10.6 Built-in generics

10.6.1 Array<T>, Promise<T>, Map<K, V>, Set<T>

You have been using generics since Module 3: the standard library is full of them. `Array<T>` (written `T[]`) has generic methods: `map<U>` returns `U[]`, `filter` keeps `T[]`, `reduce<U>` accumulates into `U`, `find` returns `T | undefined`. `Promise<T>` resolves to a `T`. `Map<K, V>` maps keys of type `K` to values of type `V` (any key type, unlike plain objects); `Set<T>` holds unique values. Others include `ReadonlyArray<T>`, `ReadonlyMap`, `WeakMap<K extends object, V>`, `Record<K, V>` and `Iterable<T>`.

ARRAY, MAP, SET AND PROMISE ARE GENERIC

main.ts

```ts
// Array<T>: generic methods infer their own type parameters
const fees: Array<number> = [9999, 14999, 12999];
const labels = fees.map((f) => `Rs ${f}`);                          // string[]
const total = fees.reduce<number>((sum, f) => sum + f, 0);          // explicit U
const byLength = ["TypeScript", "Go", "Python"].reduce<Record<number, string[]>>((acc, w) => {
  (acc[w.length] ??= []).push(w);
  return acc;
}, {});

// Map<K, V>: any key type, typed values
const enrolments = new Map<string, Set<string>>();
function enrol(student: string, course: string) {
  if (!enrolments.has(course)) enrolments.set(course, new Set());
  enrolments.get(course)!.add(student);
}
enrol("Asha", "TS101"); enrol("Ravi", "TS101"); enrol("Asha", "TS101"); enrol("Asha", "PY201");

// Promise<T>
const loadFee = (code: string): Promise<number> => Promise.resolve(code === "TS101" ? 9999 : 0);

async function main() {
  const [a, b] = await Promise.all([loadFee("TS101"), loadFee("XX")]);       // [number, number]
  console.log(labels, total, byLength);
  for (const [course, students] of enrolments) console.log(course, [...students], students.size);
  console.log(a, b);
}
main();
```

OUTPUT

```
[ 'Rs 9999', 'Rs 14999', 'Rs 12999' ] 37997 { '2': [ 'Go' ], '6': [ 'Python' ], '10': [ 'TypeScript' ]
}
TS101 [ 'Asha', 'Ravi' ] 2
PY201 [ 'Asha' ] 1
9999 0
```

THE TYPE ARGUMENTS OF BUILT-IN GENERICS ARE ENFORCED

main.ts

```ts
const enrolments = new Map<string, Set<string>>();
enrolments.set("TS101", ["Asha"]);
enrolments.set(101, new Set());
const ids = new Set<number>([1, 2]);
ids.add("3");
const p: Promise<number> = Promise.resolve("9999");
```

COMPILER ERRORS (tsc --strict)

```
main.ts(2,25): error TS2345: Argument of type 'string[]' is not assignable to parameter of type
'Set<string>'.
  Type 'string[]' is missing the following properties from type 'Set<string>': add, clear, delete,
has, and 2 more.
main.ts(3,16): error TS2345: Argument of type 'number' is not assignable to parameter of type
'string'.
main.ts(5,9): error TS2345: Argument of type 'string' is not assignable to parameter of type 'number'.
main.ts(6,7): error TS2322: Type 'Promise<string>' is not assignable to type 'Promise<number>'.
  Type 'string' is not assignable to type 'number'.
```

10.7 Generics best practices

10.7.1 Use as few type parameters as possible

Every type parameter should **relate** two or more things: inputs to outputs, or several inputs to each other. A type parameter that appears only once in the signature adds nothing but complexity: replace it with its constraint or `unknown`. Similarly, prefer constraining an existing parameter to adding a new one, and do not make a function generic just because it accepts several types (a union may be simpler).

REMOVE TYPE PARAMETERS THAT RELATE NOTHING main.ts

```
// Unnecessary: T appears only once, so it relates nothing
function logLengthBad<T extends { length: number }>(x: T): void { console.log(x.length); }
// Better: just use the constraint
function logLength(x: { length: number }): void { console.log(x.length); }

// Unnecessary second parameter: U is only used once
function filterBad<T, U extends (x: T) => boolean>(items: T[], pred: U): T[] { return
items.filter(pred); }
// Better
function filter<T>(items: T[], pred: (x: T) => boolean): T[] { return items.filter(pred); }

logLengthBad("abc"); logLength([1, 2]);
console.log(filterBad([1, 2, 3], (n) => n > 1), filter(["a", "bb"], (s) => s.length > 1));
```

```
OUTPUT
3
2
[ 2, 3 ] [ 'bb' ]
```

10.7.2 Let inference do the work

Design generic functions so that callers **never need to write type arguments**: put the type parameters where the arguments can supply them, and prefer inferring from the most direct parameter (for example `T[]` rather than a complex wrapper). Push type parameters "down": `<T>(items: T[])` infers better than `<A extends any[]>(items: A)`, whose element type is harder to reach. If callers often write type arguments, the signature can probably be improved. Explicit type arguments remain right for empty initial values (`new Map<K, V>()`, `useState<T>()`) and for functions whose return type cannot be inferred from arguments, such as `fetchJson<T>()`.

Guideline	Instead of	Prefer
relate types	`<T>(x: T): void`	`(x: unknown): void`
push parameters down	`<A extends any[]>(a: A): A[0]`	`<T>(a: T[]): T`
constrain, do not cast	`(x as any).length` inside	`<T extends { length: number }>`
let callers infer	`first<string>(names)`	`first(names)`
name meaningfully	`<A, B, C>` in complex code	`<TItem, TKey, TValue>`
avoid return-only generics without validation	`parse<T>(json): T` (an assertion in disguise)	return `unknown` and validate, or take a schema/guard

WATCH OUT

A generic whose type parameter appears **only in the return type**, such as `function parse<T>(json: string): T`, is an unchecked assertion: the caller picks any `T` and nothing verifies it. When the value comes from outside, accept a type guard or validator as an argument (`parse(json, isUser)`) so the check and the type stay together (8.4.2, 15.3).

Generics replace any with type parameters: type variables filled in (usually inferred) at each use, so input and output types stay linked. Generic functions can have several parameters and arrow functions use <T,> in .tsx. Constraints (T extends { length: number }, K extends keyof T with T[K]) allow using properties safely; defaults (T = unknown) simplify common cases; const type parameters preserve literal types. Interfaces, type aliases and unions can be generic (ApiResponse<T>, Page<T>, Result<T, E>); a generic type (Mapper<T, U>) differs from a type of generic function (<T>(x: T) => T). Classes such as Stack<T>, Queue<T> and Repository<T extends Entity> are written once for many element types. Array, Promise, Map, Set and Record are built-in generics. Use as few type parameters as possible, each relating several things, and design signatures so that inference does the work.

PRACTICE CHECKPOINT

A generic **Repository<T extends { id: number }>** with **CRUD** methods, a generic **fetchJson<T>** function, and a typed **groupBy** utility.

SAMPLE SOLUTION: REPOSITORY<T>, FETCHJSON<T> WITH VALIDATION, AND GROUPBY

main.ts

```ts
// ---------- a generic CRUD repository
class Repository<T extends { id: number }> {
  #rows = new Map<number, T>();
  #nextId = 1;

  create(data: Omit<T, "id">): T {
    const row = { ...data, id: this.#nextId++ } as T;
    this.#rows.set(row.id, row);
    return row;
  }
  read(id: number): T | undefined { return this.#rows.get(id); }
  list(filter?: (row: T) => boolean): T[] {
    const all = [...this.#rows.values()];
    return filter ? all.filter(filter) : all;
  }
  update(id: number, changes: Partial<Omit<T, "id">>): T {
    const row = this.#rows.get(id);
    if (!row) throw new Error(`no row with id ${id}`);
    const updated: T = { ...row, ...changes };
    this.#rows.set(id, updated);
    return updated;
  }
  delete(id: number): boolean { return this.#rows.delete(id); }
}

// ---------- a generic fetchJson<T> that validates instead of trusting
type Validator<T> = (value: unknown) => value is T;

const fakeServer: Record<string, string> = {
  "/api/courses": '[{"id": 1, "code": "TS101", "fee": 9999}, {"id": 2, "code": "PY201", "fee":
14999}]',
  "/api/broken": '{"oops": true}',
};
```

```
async function fetchJson<T>(url: string, isValid: Validator<T>): Promise<T> {
  await new Promise((r) => setTimeout(r, 10));                    // simulated network delay
  const body = fakeServer[url];
  if (body === undefined) throw new Error(`404 for ${url}`);
  const data: unknown = JSON.parse(body);
  if (!isValid(data)) throw new Error(`unexpected response shape from ${url}`);
  return data;
}

// ---------- a typed groupBy utility
function groupBy<T, K extends PropertyKey>(items: readonly T[], keyOf: (item: T) => K): Record<K, T[]>
{
  const groups = {} as Record<K, T[]>;
  for (const item of items) (groups[keyOf(item)] ??= []).push(item);
  return groups;
}

// ---------- using all three
interface Course { id: number; code: string; fee: number }
interface Student { id: number; name: string; city: string; courseCode: string }

const isCourseArray: Validator<Course[]> = (v): v is Course[] =>
  Array.isArray(v) && v.every((c) => typeof c?.id === "number" && typeof c?.code === "string" &&
typeof c?.fee === "number");

async function main() {
  const courses = await fetchJson("/api/courses", isCourseArray);             // T inferred as
Course[]
  console.log("courses:", courses.map((c) => `${c.code} Rs ${c.fee}`).join(", "));
  for (const url of ["/api/broken", "/api/missing"]) {
    try { await fetchJson(url, isCourseArray); } catch (e) { console.log("error:", (e as
Error).message); }
  }

  const students = new Repository<Student>();
  students.create({ name: "Asha", city: "Mysuru", courseCode: "TS101" });
  students.create({ name: "Ravi", city: "Pune", courseCode: "PY201" });
  const kiran = students.create({ name: "Kiran", city: "Mysuru", courseCode: "TS101" });
  students.update(kiran.id, { city: "Bengaluru" });
  students.delete(2);
  console.log("students:", students.list().map((s) => `${s.id}:${s.name}@${s.city}`).join(" "));

  const byCourse = groupBy(students.list(), (s) => s.courseCode);              // Record<string,
Student[]>
  const byFeeBand = groupBy(courses, (c) => (c.fee >= 12000 ? "premium" : "standard"));   //
Record<"premium" | "standard", Course[]>
  console.log(Object.fromEntries(Object.entries(byCourse).map(([k, v]) => [k, v.map((s) =>
s.name)])));
  console.log(byFeeBand.premium?.map((c) => c.code), byFeeBand.standard?.map((c) => c.code));
}
main();
```

OUTPUT

```
courses: TS101 Rs 9999, PY201 Rs 14999
error: unexpected response shape from /api/broken
error: 404 for /api/missing
students: 1:Asha@Mysuru 3:Kiran@Bengaluru
{ TS101: [ 'Asha', 'Kiran' ] }
[ 'PY201' ] [ 'TS101' ]
```

MODULE 11 · INTERMEDIATE

Modules, namespaces and declaration files

Minimum duration **2 days** | Level **Intermediate**

This module covers how TypeScript code is organised across files and how type information is shared with JavaScript. It explains ES module exports and imports, type-only imports and exports with import type and verbatimModuleSyntax, default versus named exports, module resolution (node16, nodenext and bundler), path aliases with paths and Node.js subpath imports, CommonJS and ESM interop with esModuleInterop, namespaces and why modules replaced them, declaration files (.d.ts): what they are, generating them with declaration: true, writing one for a JavaScript library, declare for globals and modules, module and global augmentation, ambient declarations for images, CSS and environment variables, and triple-slash directives.

IN THIS MODULE

11.1 ES modules in TypeScript
11.2 Module resolution
11.3 Namespaces
11.4 Declaration files
11.5 Triple-slash directives (overview)

11.1 ES modules in TypeScript

In TypeScript, as in modern JavaScript, every file that contains a top-level `import` or `export` is a **module**: its variables, functions, classes and types are private to the file unless exported. Files without imports or exports are **scripts** whose declarations are global, which easily causes name clashes; that is why the compiler option `moduleDetection: "force"` (set by `tsc --init`) treats every file as a module.

11.1.1 export and import

TypeScript uses standard ES module syntax, and **types can be exported and imported exactly like values**: interfaces, type aliases, enums and classes. Imports are checked: importing a name that is not exported, or using an export with the wrong type, is a compile error, and renaming an export with the editor updates every import.

SRC/MODELS.TS

TypeScript

```
export interface Course {
  code: string;
  title: string;
  fee: number;
}

export type Currency = "INR" | "USD";

export const GST_RATE = 0.18;
```

SRC/PRICING.TS

TypeScript

```
import { GST_RATE, type Course, type Currency } from "./models.js";

const USD_PER_INR = 0.012;

export function priceOf(course: Course, currency: Currency = "INR"): string {
  const total = course.fee * (1 + GST_RATE);
  return currency === "INR" ? `Rs ${Math.round(total)}` : `$${(total * USD_PER_INR).toFixed(2)}`;
}

export function cheapest(courses: Course[]): Course | undefined {
  return [...courses].sort((a, b) => a.fee - b.fee)[0];
}
```

SRC/MAIN.TS

TypeScript

```
import type { Course } from "./models.js";
import { priceOf, cheapest } from "./pricing.js";
import * as models from "./models.js";          // namespace import

const catalogue: Course[] = [
  { code: "TS101", title: "TypeScript", fee: 9999 },
  { code: "PY201", title: "Python", fee: 14999 },
];

for (const c of catalogue) console.log(c.code, priceOf(c), priceOf(c, "USD"));
console.log("cheapest:", cheapest(catalogue)?.title, "| GST", models.GST_RATE * 100 + "%");
```

A SMALL MULTI-MODULE PROJECT

terminal

```
npx tsc
node dist/main.js
```

TERMINAL

```
$ npx tsc
$ node dist/main.js
TS101 Rs 11799 $141.59
PY201 Rs 17699 $212.39
cheapest: TypeScript | GST 18%
```

11.1.2 Type-only imports and exports import type

Because types are erased, an import used only as a type disappears from the JavaScript. `import type { Course } from "./models.js"` states this explicitly: the import is guaranteed to be removed, it can never trigger a module's side effects, and it cannot accidentally be used as a value. You can also mark individual names: `import { priceOf, type Course }`. Similarly, `export type { Course }` re-exports only the type. Type-only imports also avoid problems with circular imports, and they are required by tools that compile files one at a time (esbuild, SWC, Babel, Node.js type stripping), which cannot know whether an imported name is a type.

SRC/REPORT.TS

TypeScript

```
import type { Student } from "./models.js";      // type only: removed
import { average } from "./models.js";             // value: kept

export function report(list: Student[]): string[] {
  return list.map((s) => `${s.name}: ${average(s).toFixed(1)}`);
}
```

IMPORT TYPE IS ALWAYS ERASED FROM THE OUTPUT terminal

```
npx tsc
cat dist/report.js
```

```
TERMINAL
$ npx tsc
$ cat dist/report.js
import { average } from "./models.js"; // value: kept
export function report(list) {
    return list.map((s) => `${s.name}: ${average(s).toFixed(1)}`);
}
```

SRC/APP.TS TypeScript

```
import type { Student } from "./models.js";

const s: Student = { name: "Asha" };   // fine: used as a type
const t = new Student();                // error: used as a value
```

A TYPE-ONLY IMPORT CANNOT BE USED AS A VALUE terminal

```
npx tsc --noEmit
```

```
TERMINAL
$ npx tsc --noEmit
src/app.ts(4,15): error TS1361: 'Student' cannot be used as a value because it was imported using
'import type'.
```

11.1.3 verbatimModuleSyntax

The option `verbatimModuleSyntax` (TypeScript 5.0+, set by `tsc --init`) makes the rule simple and predictable: **imports and exports without the `type` modifier are kept exactly as written**, and those with it are removed. If you import a type without `type`, tsc reports an error, because the kept import would fail at run time (there is no such value). It also forbids ES import syntax in files that compile to CommonJS. It replaces the older options `importsNotUsedAsValues`, `preserveValueImports` and the stricter half of `isolatedModules`.

SRC/APP.TS TypeScript

```
import { Student, PASS } from "./models.js";      // Student is a type: error
import { type Grade } from "./models.js";          // OK
export { Grade } from "./models.js";               // re-exporting a type: needs export type

const s: Student = { name: "Asha" };
const g: Grade = "A";
console.log(s, g, PASS);
```

VERBATIMMODULESYNTAX REQUIRES TYPE IMPORTS FOR TYPES

terminal

```
npx tsc --noEmit
```

```
TERMINAL
$ npx tsc --noEmit
src/app.ts(1,10): error TS1484: 'Student' is a type and must be imported using a type-only import when
'verbatimModuleSyntax' is enabled.
src/app.ts(3,10): error TS1205: Re-exporting a type when 'verbatimModuleSyntax' is enabled requires
using 'export type'.
```

11.1.4 Default vs named exports

A module can have one **default export** (`export default class Cart`), imported with any name you like (`import Cart from "./cart.js"`), and any number of **named exports**, imported by their exact names in braces. Many TypeScript teams prefer **named exports only**: the name is the same everywhere (easier search and refactoring), the editor can auto-import them reliably, typos are caught, and re-exporting from an index file is simpler. Default exports are still required by some frameworks (React lazy loading, Next.js pages, Vue single-file components, config files).

	Named export	Default export
syntax	`export function total() {}`	`export default function total() {}`
import	`import { total } from "./cart.js"`	`import total from "./cart.js"` (any name)
how many per module	any number	one
rename on import	`import { total as sum }`	just choose another name
refactoring and auto-import	reliable: one name everywhere	names can drift between files
typical use	utilities, types, components (most code)	framework conventions (pages, configs, lazy components)

SRC/CART/INDEX.TS

TypeScript

```
// a "barrel" file: one import location for the whole folder
export { default as Cart } from "./cart.js";
export * from "./discounts.js";
```

SRC/MAIN.TS

TypeScript

```
import { Cart, festiveDiscount, couponDiscount, type DiscountFn } from "./cart/index.js";

const cart = new Cart().add("Notebook", 120).add("Pen set", 280);
const rules: DiscountFn[] = [festiveDiscount, (a) => couponDiscount(a, "LBF10")];
console.log("total", cart.total, "after discounts", rules.reduce((a, f) => f(a), cart.total));
```

DEFAULT AND NAMED EXPORTS, AND A BARREL FILE THAT RE-EXPORTS

terminal

```
npx tsc
node dist/main.js
```

```
TERMINAL
$ npx tsc
$ node dist/main.js
total 400 after discounts 288
```

11.2 Module resolution

Module resolution is how the compiler finds the file behind an import specifier such as `"./pricing.js"`, `"zod"` or `"#utils/math.js"`, so that it can read its types. It must match how the code will actually be loaded at run time (by Node.js or by a bundler); otherwise code that type-checks fails to run, or the reverse.

11.2.1 moduleResolution: node16, nodenext, bundler

Setting	Use for	Key rules
`node16` / `nodenext` (with `module` of the same name)	code run directly by Node.js (servers, CLIs, libraries)	follows Node.js exactly: relative ESM imports need the extension (`./x.js`); respects package.json `"type"`, `"exports"` and `"imports"`; .mts/.cts files
`bundler` (with `module: "esnext"` or `"preserve"`)	code processed by Vite, webpack, esbuild, Bun	extensionless imports allowed (`./x`); respects `"exports"`; assumes the bundler resolves the rest
`node10` (formerly `node`)	old CommonJS projects	deprecated in TypeScript 6: does not understand package `"exports"`
`classic`	nothing modern	removed in TypeScript 6

In practice: Node.js projects use `"module": "nodenext"` (which implies `moduleResolution: nodenext`), and front-end projects built with Vite use `"module": "esnext", "moduleResolution": "bundler"` (Vite's templates set this up). Under `nodenext`, the `"type"` field of the nearest `package.json` decides whether a `.ts` file is compiled as an ES module or as CommonJS; `.mts` files are always ESM and `.cts` files always CommonJS.

SRC/APP.TS

TypeScript

```
import { grade } from "./grades";
console.log([91, 72, 40].map(grade).join(" "));
```

UNDER NODENEXT, RELATIVE IMPORTS NEED THE .JS EXTENSION terminal

```
npx tsc --noEmit -p . || true
sed -i 's|"./grades"|"./grades.js"|' src/app.ts
npx tsc -p . && node dist/app.js
```

```
TERMINAL
$ npx tsc --noEmit -p . || true
src/app.ts(1,23): error TS2835: Relative import paths need explicit file extensions in ECMAScript
imports when '--moduleResolution' is 'node16' or 'nodenext'. Did you mean './grades.js'?
$ sed -i 's|"./grades"|"./grades.js"|' src/app.ts
$ npx tsc -p . && node dist/app.js
A B C
```

11.2.2 Path aliases with paths and baseUrl

Deep relative imports such as `../../../utils/format.js` are hard to read and break when files move. The `paths` option maps **alias** patterns to locations: `"@utils/*": ["./src/utils/*"]` lets you write `import { add } from "@utils/math.js"`. Paths are resolved relative to the tsconfig file. The older `baseUrl` option, once required with `paths`, is **deprecated in TypeScript 6**; remove it. Crucially, **tsc does not rewrite aliases in the emitted JavaScript**: paths only tell the type checker where to look. The runtime must understand the alias too: bundlers (Vite `resolve.alias` or the `vite-tsconfig-paths` plugin) and tsx do; plain Node.js does not.

TSCONFIG.JSON JavaScript

```
{ "compilerOptions": { "strict": true, "target": "es2022", "module": "nodenext", "rootDir": "src",
"outDir": "dist", "paths": { "@utils/*": ["./src/utils/*"] }, "types": [] }, "include": ["src"] }
```

SRC/INDEX.TS TypeScript

```
import { add } from "@utils/math.js";
console.log(add(2, 3));
```

PATHS SATISFIES THE COMPILER, BUT NODE.JS CANNOT RESOLVE THE ALIAS IN THE OUTPUT terminal

```
npx tsc -p . && echo "type check OK"
node dist/index.js 2>&1 | grep "Cannot find package" || true
```

```
TERMINAL
$ npx tsc -p . && echo "type check OK"
type check OK
$ node dist/index.js 2>&1 | grep "Cannot find package" || true
Error [ERR_MODULE_NOT_FOUND]: Cannot find package '@utils/math.js' imported from dist/index.js
```

For Node.js projects the standard solution is **subpath imports**: an `"imports"` field in `package.json` whose keys start with `#`. Node.js resolves them at run time, and TypeScript (with `nodenext` or `bundler`) understands them too, even mapping `dist` back to `src` while type-checking. No tsconfig `paths` are needed.

PACKAGE.JSON JavaScript

```
{ "type": "module", "imports": { "#utils/*": "./dist/utils/*" } }
```

SRC/INDEX.TS

TypeScript

```
import { add, pct } from "#utils/math.js";
console.log(add(2, 3), pct(436, 500));
```

SUBPATH IMPORTS (#UTILS/*) WORK FOR BOTH TYPESCRIPT AND NODE.JS

terminal

```
npx tsc -p .
node dist/index.js
```

TERMINAL

```
$ npx tsc -p .
$ node dist/index.js
5 87.2%
```

11.2.3 CommonJS vs ESM interop and esModuleInterop

Node.js has two module systems: **CommonJS** (`require`, `module.exports`), still used by many npm packages, and **ES modules** (`import`/`export`). They export differently: a CommonJS module exports a single object (`module.exports = fn`), while ESM has named exports and an optional default. When ESM code imports a CommonJS module, Node.js treats `module.exports` as the **default** export. Historically, TypeScript compiled `import express from "express"` differently from Node.js and Babel; the option `esModuleInterop` (with `allowSyntheticDefaultImports`) fixes this by emitting helper code so default imports of CommonJS modules work the same everywhere. It is enabled by `tsc --init`, implied by `nodenext`, and in TypeScript 6 it can no longer be turned off.

NODE_MODULES/LEGACY-MATH/INDEX.D.TS

TypeScript

```
declare function square(n: number): number;
declare namespace square { function cube(n: number): number; }
export = square;
```

SRC/APP.TS

TypeScript

```
import square from "legacy-math";            // module.exports becomes the default import
console.log(square(7), square.cube(3));
```

IMPORTING A COMMONJS PACKAGE FROM AN ES MODULE

terminal

```
cat node_modules/legacy-math/index.js
npx tsc -p .
node dist/app.js
```

TERMINAL

```
$ cat node_modules/legacy-math/index.js
function square(n) { return n * n; }
square.cube = (n) => n * n * n;
module.exports = square;
$ npx tsc -p .
$ node dist/app.js
49 27
```

Note the declaration file's `export = square`: this is how `.d.ts` files describe a CommonJS module whose `module.exports` is a single value. In CommonJS TypeScript files you can also write `import square = require("legacy-math")`, which compiles to a plain `require`.

11.3 Namespaces

11.3.1 What namespaces are and why modules replaced them

Before ES modules existed (2015), TypeScript offered **namespaces** (originally "internal modules") to organise code and avoid polluting the global scope: `namespace Geometry { export function area(...) {} }` compiles to an object created by an immediately invoked function, and code refers to `Geometry.area`. Namespaces can be split across files and merged. Today **ES modules do the same job better**: every file has its own scope, dependencies are explicit imports, tools can tree-shake unused code, and the syntax is standard JavaScript. Namespaces are also not erasable (6.4.5). You will still meet them in old codebases and in declaration files, where `declare namespace` describes global libraries (such as `jQuery` or `google.maps`) or adds members to functions; but do not use them to organise new code.

GEOMETRY.TS TypeScript

```
namespace Geometry {
  export const PI = 3.14159;
  export function circleArea(r: number): number { return PI * r * r; }
  export namespace Units { export const cm = (m: number) => m * 100; }   // nested namespace
  const secret = "not exported";                                          // private to the namespace
}

console.log(Geometry.circleArea(2).toFixed(2), Geometry.Units.cm(1.5));
```

A NAMESPACE AND THE JAVASCRIPT IT GENERATES terminal

```
npx tsc --target es2022 --ignoreConfig geometry.ts
cat geometry.js
node geometry.js
```

TERMINAL

```
$ npx tsc --target es2022 --ignoreConfig geometry.ts
$ cat geometry.js
"use strict";
var Geometry;
(function (Geometry) {
    Geometry.PI = 3.14159;
    function circleArea(r) { return Geometry.PI * r * r; }
    Geometry.circleArea = circleArea;
    let Units;
    (function (Units) {
        Units.cm = (m) => m * 100;
    })(Units = Geometry.Units || (Geometry.Units = {})); // nested namespace
    const secret = "not exported"; // private to the namespace
})(Geometry || (Geometry = {}));
console.log(Geometry.circleArea(2).toFixed(2), Geometry.Units.cm(1.5));
$ node geometry.js
12.57 150
```

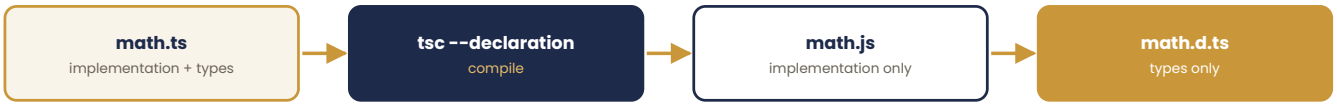
	namespace	ES module
scope	one global object holding the members	each file has its own scope
dependencies	implicit (load order, globals)	explicit imports
tree-shaking by bundlers	poor	yes
standard JavaScript	no (TypeScript-only, generates code)	yes

	namespace	ES module
still used for	declaration files for global libraries, legacy code	everything else

11.4 Declaration files

11.4.1 What .d.ts files are

A **declaration file** (`.d.ts`) contains **only type information**: the shapes of functions, classes, variables and modules, with no implementation. It is the bridge between JavaScript code and the type checker, like a C header file. Declaration files are never compiled to JavaScript. They come from three sources: generated by tsc from your own TypeScript (to publish a library), shipped with npm packages or `@types` packages (7.2), and written by hand for untyped JavaScript. The built-in types of JavaScript and the browser come from declaration files too: `lib.es2023.d.ts`, `lib.dom.d.ts` and so on, selected with the `lib` option.



TSC CAN SPLIT A TYPESCRIPT FILE INTO JAVASCRIPT AND A DECLARATION FILE

11.4.2 Generating declarations with declaration: true

With `"declaration": true`, tsc writes a `.d.ts` file next to every `.js` file it emits. This is what you publish with a library so that its users get types: set the `"types"` (or `"exports"` types condition) field in `package.json` to the main declaration file. `declarationMap` lets editors jump from the declarations to your original `.ts` source, and `emitDeclarationOnly` produces only declarations when another tool builds the JavaScript (17.3).

SRC/MONEY.TS

TypeScript

```
export type Currency = "INR" | "USD" | "EUR";

export interface Money {
  readonly amount: number;
  readonly currency: Currency;
}

const SYMBOLS: Record<Currency, string> = { INR: "Rs ", USD: "$", EUR: "€" };

export function money(amount: number, currency: Currency = "INR"): Money {
  return { amount: Math.round(amount * 100) / 100, currency };
}

export function format(m: Money): string {
  return SYMBOLS[m.currency] + m.amount.toLocaleString("en-IN");
}

export class Wallet {
  #balance: Money;
  constructor(currency: Currency) { this.#balance = money(0, currency); }
  deposit(amount: number): this { this.#balance = money(this.#balance.amount + amount,
this.#balance.currency); return this; }
  get balance(): Money { return this.#balance; }
}
```

TSC WRITES .D.TS FILES WITH DECLARATION: TRUE

terminal

```
npx tsc
cat dist/money.d.ts
cat dist/money.js
```

TERMINAL

```
$ npx tsc
$ cat dist/money.d.ts
export type Currency = "INR" | "USD" | "EUR";
export interface Money {
    readonly amount: number;
    readonly currency: Currency;
}
export declare function money(amount: number, currency?: Currency): Money;
export declare function format(m: Money): string;
export declare class Wallet {
    #private;
    constructor(currency: Currency);
    deposit(amount: number): this;
    get balance(): Money;
}
$ cat dist/money.js
const SYMBOLS = { INR: "Rs ", USD: "$", EUR: "€" };
export function money(amount, currency = "INR") {
    return { amount: Math.round(amount * 100) / 100, currency };
}
export function format(m) {
    return SYMBOLS[m.currency] + m.amount.toLocaleString("en-IN");
}
export class Wallet {
    #balance;
    constructor(currency) { this.#balance = money(0, currency); }
    deposit(amount) { this.#balance = money(this.#balance.amount + amount, this.#balance.currency);
return this; }
    get balance() { return this.#balance; }
}
```

11.4.3 Writing a declaration file for a JavaScript library

When a JavaScript library has no types, write a declaration file for the parts you use. Start from the library's documentation and source, declare each export with `export declare function`, `export declare const`, `export declare class` and interfaces for its options and results, and mirror the module format (`export =` for a CommonJS `module.exports` value, 11.2.3). Place it either next to the JavaScript file with the same name (`slugify.js` → `slugify.d.ts`) or in a `types/` folder wrapped in `declare module "name" { ... }`. Overloads, generics and literal unions make hand-written declarations precise.

SRC/LIB/TEXTKIT.D.TS TypeScript

```ts
/** Turns "Hello World!" into "hello-world". */
export declare function slugify(text: string, separator?: string): string;

export interface TruncateOptions {
  /** Text appended when the input is cut (default "..."). */
  ellipsis?: string;
}
export declare function truncate(text: string, max: number, options?: TruncateOptions): string;

export declare function wordCount(text: string): number;

export declare const VERSION: string;
```

SRC/APP.TS TypeScript

```ts
import { slugify, truncate, wordCount, VERSION } from "./lib/textkit.js";

const title = "  Learn TypeScript at Learn Being Forward!  ";
console.log(slugify(title), "|", slugify(title, "_"));
console.log(truncate(title.trim(), 20), "|", truncate("short", 20), "|", truncate(title.trim(), 15, { ellipsis: " >" }));
console.log(wordCount(title), "words, textkit", VERSION);
```

A HAND-WRITTEN .D.TS FOR AN UNTYPED JAVASCRIPT UTILITY terminal

```
npx tsc -p .
node dist/app.js
```

```
TERMINAL
$ npx tsc -p .
$ node dist/app.js
learn-typescript-at-learn-being-forward | learn_typescript_at_learn_being_forward
Learn TypeScript ... | short | Learn TypeScr >
6 words, textkit 1.4.0
```

11.4.4 declare for global variables and modules

The `declare` keyword tells TypeScript that something **exists at run time** but is defined elsewhere (a script tag, the runtime, a bundler), so no code is generated for it. `declare const VERSION: string` describes a global constant injected by a build tool; `declare function gtag(...args: unknown[]): void` describes a function loaded from a CDN; `declare module "name" { ... }` describes a whole module; and `declare global { ... }` inside a module adds global declarations. Such declarations without implementation are called **ambient declarations**.

AMBIENT DECLARATIONS FOR GLOBALS DEFINED OUTSIDE TYPESCRIPT main.ts

```
// Values provided by the environment (in a real app: injected by the build tool or a <script> tag)
declare const __APP_VERSION__: string;
declare function trackEvent(name: string, data?: Record<string, string | number>): void;

// For this demo we define them on globalThis so the program can run
(globalThis as any).__APP_VERSION__ = "2.3.1";
(globalThis as any).trackEvent = (name: string, data?: object) => console.log("track:", name,
JSON.stringify(data ?? {}));

console.log("version", __APP_VERSION__);
trackEvent("enrol_clicked", { course: "TS101", price: 9999 });
```

OUTPUT

```
version 2.3.1
track: enrol_clicked {"course":"TS101","price":9999}
```

AMBIENT DECLARATIONS ARE TYPE-CHECKED AT USE, AND CANNOT HAVE VALUES main.ts

```
declare function trackEvent(name: string, data?: Record<string, string | number>): void;
trackEvent("enrol", { course: "TS101", paid: true });
declare const VERSION: string = "1.0";
```

COMPILER ERRORS (tsc --strict)

```
main.ts(2,40): error TS2322: Type 'boolean' is not assignable to type 'string | number'.
main.ts(3,33): error TS1039: Initializers are not allowed in ambient contexts.
```

11.4.5 Module augmentation and global augmentation

Module augmentation adds declarations to an existing module's types, using interface declaration merging (5.3.5): write `declare module "express-serve-static-core" { interface Request { user?: User } }` in a file that is itself a module, and every Express `Request` gains a typed `user` property. **Global augmentation** uses `declare global { ... }` to add to global types such as `Window`, `Array` or `NodeJS.ProcessEnv`. Augmentation can only add members, not replace existing ones; the file must be included in the compilation (and contain at least one import or export).

SRC/TYPES/AUGMENT.D.TS TypeScript

```
import "tiny-router";

declare module "tiny-router" {
  interface Request {
    user?: { id: number; role: "student" | "admin" };     // added to the library's interface
  }
}

declare global {
  interface Array<T> {
    sum(this: number[]): number;                          // a new method on every array (use
sparingly!)
  }
}
```

SRC/MAIN.TS

TypeScript

```ts
import { createRequest } from "tiny-router";

Array.prototype.sum = function (this: number[]) { return this.reduce((a, b) => a + b, 0); };

const req = createRequest("/dashboard");
req.user = { id: 7, role: "student" };                    // typed thanks to the augmentation
console.log(req.path, req.user.role, [1, 2, 3].sum());
```

AUGMENTING A THIRD-PARTY MODULE AND THE GLOBAL SCOPE

terminal

```
npx tsc -p .
node dist/main.js
```

TERMINAL

```
$ npx tsc -p .
$ node dist/main.js
/dashboard student 6
```

WATCH OUT

Adding methods to built-in prototypes (`Array.prototype.sum`) is shown here only to demonstrate global augmentation. In real code it can clash with future JavaScript features and other libraries; prefer ordinary functions.

11.4.6 Ambient declarations for images, CSS and environment variables

Front-end bundlers let you import non-JavaScript files: `import logo from "./logo.png"` gives a URL string, `import styles from "./card.module.css"` gives an object of class names. TypeScript does not know these files, so you declare their module shapes with **wildcard module declarations** (`declare module "*.png"`). Vite ships these in `vite/client` types (referenced from `src/vite-env.d.ts`). Environment variables are typed by augmenting the right interface: `ImportMetaEnv` for Vite's `import.meta.env` , or `NodeJS.ProcessEnv` for `process.env` in Node.js (types only; they still need validating at run time, 15.3.3).

DECLARATIONS FOR ASSETS AND VITE ENVIRONMENT VARIABLES

TypeScript

```ts
// src/env.d.ts: ambient declarations for a front-end project
declare module "*.png" {
  const src: string;
  export default src;
}
declare module "*.svg" {
  const src: string;
  export default src;
}
declare module "*.module.css" {
  const classes: Readonly<Record<string, string>>;
  export default classes;
}

// Vite: typed import.meta.env (variables must start with VITE_)
interface ImportMetaEnv {
  readonly VITE_API_URL: string;
  readonly VITE_RAZORPAY_KEY?: string;
}
interface ImportMeta {
  readonly env: ImportMetaEnv;
}
```

SRC/ENV.D.TS TypeScript

```typescript
declare global {
  namespace NodeJS {
    interface ProcessEnv {
      API_URL: string;
      NODE_ENV?: "development" | "production" | "test";
      PORT?: string;
    }
  }
}
export {};
```

SRC/CONFIG.TS TypeScript

```typescript
const config = {
  apiUrl: process.env.API_URL,                    // string (declared, but still validate in real
apps)
  env: process.env.NODE_ENV ?? "development",     // "development" | "production" | "test"
  port: Number(process.env.PORT ?? 3000),
};
console.log(config);
```

TYPING PROCESS.ENV FOR NODE.JS terminal

```
npx tsc -p .
API_URL=https://api.lbf.in node dist/config.js
```

TERMINAL

```
$ npx tsc -p .
$ API_URL=https://api.lbf.in node dist/config.js
{ apiUrl: 'https://api.lbf.in', env: 'development', port: 3000 }
```

11.5 Triple-slash directives (overview)

Triple-slash directives are single-line comments at the very top of a file that give instructions to the compiler. `/// <reference types="vite/client" />` includes a package's type declarations (common in `vite-env.d.ts`); `/// <reference path="./globals.d.ts" />` includes another declaration file; `/// <reference lib="es2023.array" />` includes a built-in lib file. They predate ES modules and tsconfig options; in modern code prefer `import` statements and the `types`, `lib` and `include` options, and keep triple-slash directives for declaration files and the few framework conventions that still use them.

A TYPICAL FILE THAT USES TRIPLE-SLASH DIRECTIVES TypeScript

```typescript
/// <reference types="vite/client" />
/// <reference lib="dom" />

// vite-env.d.ts in a Vite project: brings in the types for import.meta.env,
// asset imports (*.png, *.svg, *.css) and hot module replacement (import.meta.hot).
```

Files with imports or exports are modules; types are exported and imported like values. import type and export type are always erased, and verbatimModuleSyntax requires them for types so the output is predictable. Prefer named exports; default exports suit framework conventions; barrel files re-export folders. Module resolution must match the runtime: nodenext for Node.js (relative imports need .js, package.json type, exports and imports fields) and bundler for Vite and other bundlers. paths aliases only affect type checking (baseUrl is deprecated in TypeScript 6), so use Node.js subpath imports (#utils/*) or bundler aliases at run time. esModuleInterop makes default imports of CommonJS modules work; .d.ts files describe them with export =. Namespaces are the pre-module way to organise code; avoid them in new code. Declaration files hold only types: generate them with declaration: true, write them for untyped JavaScript, use declare for ambient globals and modules, augment modules and the global scope via interface merging, and declare wildcard modules for assets and interfaces for environment variables. Triple-slash directives are a legacy mechanism kept mainly in declaration files.

PRACTICE CHECKPOINT

Split a project into **typed modules with path aliases**, and write a **.d.ts** file for a small **untyped JavaScript utility library**.

Sample solution. The project below has a domain module, a service module and an entry point, imported through the subpath alias `#app/*` (which works for both the compiler and Node.js), plus a small untyped JavaScript library `vendor/currency.js` described by a hand-written `currency.d.ts`.

PACKAGE.JSON JavaScript

```
{ "name": "lbf-checkout", "type": "module", "imports": { "#app/*": "./dist/app/*", "#vendor/*":
"./dist/vendor/*" } }
```

SRC/VENDOR/CURRENCY.D.TS TypeScript

```
/** Indian-style amount in words, e.g. 1.25 lakh. */
export declare function toWords(amount: number): string;
/** Round to the nearest step (default 1). */
export declare function round(amount: number, step?: number): number;
```

SRC/APP/CHECKOUT.TS TypeScript

```
import { GST, type Coupon, type LineItem } from "#app/domain.js";
import { round } from "#vendor/currency.js";

export function subtotal(items: readonly LineItem[]): number {
  return items.reduce((s, i) => s + i.price * i.qty, 0);
}

export function applyCoupon(amount: number, coupon?: Coupon): number {
  if (!coupon) return amount;
  return coupon.kind === "percent" ? amount * (1 - coupon.value / 100) : Math.max(0, amount -
coupon.value);
}

export function grandTotal(items: readonly LineItem[], coupon?: Coupon): number {
  return round(applyCoupon(subtotal(items), coupon) * (1 + GST), 1);
}
```

SRC/MAIN.TS TypeScript

```ts
import type { LineItem } from "#app/domain.js";
import { grandTotal, subtotal } from "#app/checkout.js";
import { toWords } from "#vendor/currency.js";

const cart: LineItem[] = [
  { sku: "LAP-14", title: "Laptop", price: 54990, qty: 1 },
  { sku: "BAG-02", title: "Laptop bag", price: 1499, qty: 2 },
];

const total = grandTotal(cart, { code: "LBF5", kind: "percent", value: 5 });
console.log(`subtotal Rs ${subtotal(cart)}, payable Rs ${total} (${toWords(total)})`);
```

SAMPLE SOLUTION: TYPED MODULES, AN ALIAS AND A DECLARATION FILE terminal

```
npx tsc -p .
find dist -name "*.js" | sort
node dist/main.js
```

TERMINAL

```
$ npx tsc -p .
$ find dist -name "*.js" | sort
dist/app/checkout.js
dist/app/domain.js
dist/main.js
dist/vendor/currency.js
$ node dist/main.js
subtotal Rs 57988, payable Rs 65005 (65.0 thousand)
```

MODULE 12 · INTERMEDIATE

Built-in utility types

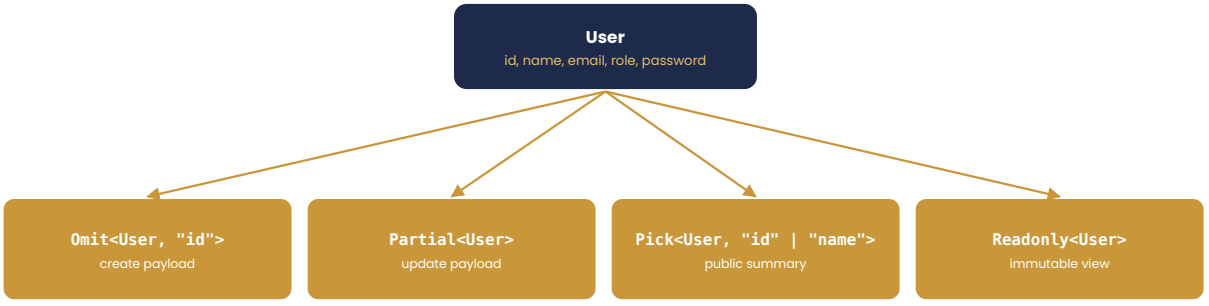
Minimum duration **2 days** | Level **Intermediate**

TypeScript ships with utility types that transform existing types, so you derive new types instead of rewriting them. This module covers the property modifiers Partial, Required and Readonly; Pick, Omit and Record; the union utilities Exclude, Extract and NonNullable; the function utilities Parameters, ReturnType, ConstructorParameters, InstanceType, ThisParameterType and OmitThisParameter; Awaited and the string utilities Uppercase, Lowercase, Capitalize and Uncapitalize; NoInfer; and real-world use for DTOs and form state.

IN THIS MODULE

12.1 Property modifiers
12.2 Picking and omitting
12.3 Union utilities
12.4 Function utilities
12.5 Async and string utilities
12.6 Other utilities
12.7 Real-world usage

Utility types are generic types defined in TypeScript's standard library (`lib.es5.d.ts`), built from the mapped and conditional types you will write yourself in Module 14. Their value is **a single source of truth**: define `User` once and derive "a user without an id", "a user where every field is optional", "only the public fields of a user". When `User` changes, every derived type follows automatically.



ONE SOURCE TYPE, MANY DERIVED TYPES

12.1 Property modifiers

12.1.1 Partial<T>

`Partial<T>` makes **every property optional**. It is the natural type for updates ("change any subset of these fields"), configuration overrides merged with defaults, and objects built step by step.

PARTIAL FOR OVERRIDES AND UPDATES main.ts

```
interface Settings {
  theme: "light" | "dark";
  fontSize: number;
  language: string;
  notifications: boolean;
}

const DEFAULTS: Settings = { theme: "light", fontSize: 14, language: "en", notifications: true };

function withDefaults(overrides: Partial<Settings>): Settings {
  return { ...DEFAULTS, ...overrides };
}

function updateSettings(current: Settings, changes: Partial<Settings>): Settings {
  return { ...current, ...changes };
}

console.log(withDefaults({ theme: "dark" }));
console.log(updateSettings(DEFAULTS, { fontSize: 18, language: "kn" }));
```

OUTPUT

```
{ theme: 'dark', fontSize: 14, language: 'en', notifications: true }
{ theme: 'light', fontSize: 18, language: 'kn', notifications: true }
```

PARTIAL STILL CHECKS THE TYPES AND NAMES OF THE PROPERTIES THAT ARE PRESENT main.ts

```
interface Settings { theme: "light" | "dark"; fontSize: number }
const p: Partial<Settings> = { fontSize: "18" };
const q: Partial<Settings> = { colour: "red" };
```

COMPILER ERRORS (tsc --strict)

```
main.ts(2,32): error TS2322: Type 'string' is not assignable to type 'number'.
main.ts(3,32): error TS2353: Object literal may only specify known properties, and 'colour' does not
exist in type 'Partial<Settings>'.
```

12.1.2 Required<T>

`Required<T>` is the opposite: it removes `?` from every property, so all of them must be present. Use it after defaults have been applied, for "fully resolved" configuration, or to check at compile time that a function fills in every optional field.

REQUIRED AFTER APPLYING DEFAULTS main.ts

```
interface Options { retries?: number; timeoutMs?: number; verbose?: boolean }

function resolve(opts: Options): Required<Options> {
  return { retries: opts.retries ?? 3, timeoutMs: opts.timeoutMs ?? 5000, verbose: opts.verbose ??
false };
}

const resolved = resolve({ timeoutMs: 2000 });
console.log(resolved.retries + 1, resolved.timeoutMs, resolved.verbose);    // no undefined checks
needed
```

OUTPUT

```
4 2000 false
```

FORGETTING AN OPTIONAL FIELD IS CAUGHT main.ts

```
interface Options { retries?: number; timeoutMs?: number; verbose?: boolean }
function resolve(opts: Options): Required<Options> {
  return { retries: opts.retries ?? 3, timeoutMs: opts.timeoutMs ?? 5000 };
}
```

COMPILER ERRORS (tsc --strict)

```
main.ts(3,3): error TS2741: Property 'verbose' is missing in type '{ retries: number; timeoutMs:
number; }' but required in type 'Required<Options>'.
```

12.1.3 Readonly<T>

`Readonly<T>` marks every property `readonly`. It is useful for function parameters that must not be modified, frozen configuration, and state objects in patterns such as Redux or React state, where you must create new objects instead of mutating. Like the `readonly` modifier, it is shallow; a `DeepReadonly` type is built in 14.5.2. `Object.freeze` returns a `Readonly<T>`.

READONLY FORBIDS MUTATION; CREATE NEW OBJECTS INSTEAD main.ts

```
interface CartState { items: string[]; total: number; coupon?: string }

function addItem(state: Readonly<CartState>, item: string, price: number): CartState {
  state.total += price;                          // error: mutation
  return { ...state, items: [...state.items, item], total: state.total + price };   // OK: new object
}

const config = Object.freeze({ apiUrl: "https://api.lbf.in", retries: 3 });   // Readonly<{...}>
config.retries = 5;
```

COMPILER ERRORS (tsc --strict)

```
main.ts(4,9): error TS2540: Cannot assign to 'total' because it is a read-only property.
main.ts(9,8): error TS2540: Cannot assign to 'retries' because it is a read-only property.
```

12.2 Picking and omitting

12.2.1 Pick<T, K>

`Pick<T, K>` builds a type with only the properties `K` (a union of keys of `T`). Use it to describe the subset of data a component or function needs (a summary, a card, a list row), so callers can pass a full object or just the needed fields.

PICK A SUBSET OF PROPERTIES main.ts

```
interface Course {
  id: number;
  code: string;
  title: string;
  fee: number;
  syllabus: string[];
  mentorEmail: string;
}

type CourseCard = Pick<Course, "code" | "title" | "fee">;

function renderCard(c: CourseCard): string {
  return `[${c.code}] ${c.title} - Rs ${c.fee}`;
}

const full: Course = { id: 1, code: "TS101", title: "TypeScript", fee: 9999, syllabus: ["types"],
mentorEmail: "lohit@lbf.in" };
console.log(renderCard(full));                                          // a full Course fits
console.log(renderCard({ code: "PY201", title: "Python", fee: 14999 }));   // so do just the needed
fields
```

OUTPUT

```
[TS101] TypeScript - Rs 9999
[PY201] Python - Rs 14999
```

PICK ONLY ACCEPTS EXISTING KEYS main.ts

```
interface Course { id: number; code: string; title: string }
type Bad = Pick<Course, "code" | "price">;
```

COMPILER ERRORS (tsc --strict)

```
main.ts(2,25): error TS2344: Type '"code" | "price"' does not satisfy the constraint 'keyof Course'.
  Type '"price"' is not assignable to type 'keyof Course'.
```

12.2.2 Omit<T, K>

`Omit<T, K>` is the complement: every property **except** `K`. Typical uses: a "create" payload without the server-generated `id` and timestamps, a public user without `password`, or props for a wrapper component that sets some of them itself. Note that `Omit` accepts any string keys (it does not check that `K` exists in `T`), so a misspelled key silently omits nothing; some teams define a stricter version (14.3.4).

OMIT FOR CREATE PAYLOADS AND SAFE RESPONSES

main.ts

```ts
interface User {
  id: number;
  name: string;
  email: string;
  passwordHash: string;
  createdAt: Date;
}

type NewUser = Omit<User, "id" | "createdAt">;          // what the client sends
type SafeUser = Omit<User, "passwordHash">;             // what the API returns

let nextId = 1;
function createUser(input: NewUser): User {
  return { ...input, id: nextId++, createdAt: new Date(Date.UTC(2026, 9, 7)) };
}

function toSafe({ passwordHash, ...rest }: User): SafeUser {
  return rest;
}

const u = createUser({ name: "Asha", email: "asha@lbf.in", passwordHash: "$argon2id$..." });
console.log(toSafe(u));
```

OUTPUT

```
{
  name: 'Asha',
  email: 'asha@lbf.in',
  id: 1,
  createdAt: 2026-10-07T00:00:00.000Z
}
```

12.2.3 Record<K, T>

`Record<K, T>` (previewed in 5.5.2) is an object type whose keys are `K` and whose values are `T`. With `K = string` it is a dictionary; with a union of literal keys it is an **exhaustive map**: the compiler insists on an entry for every key, which is perfect for lookup tables (labels, colours, prices, permissions per role) that must stay in sync with a union type.

RECORD AS AN EXHAUSTIVE LOOKUP TABLE AND AS A DICTIONARY

main.ts

```ts
type Role = "student" | "mentor" | "admin";
type Permission = "read" | "write" | "grade" | "manage";

const permissions: Record<Role, Permission[]> = {
  student: ["read"],
  mentor: ["read", "write", "grade"],
  admin: ["read", "write", "grade", "manage"],
};

const badge: Record<Role, { label: string; colour: string }> = {
  student: { label: "Learner", colour: "navy" },
  mentor: { label: "Mentor", colour: "gold" },
  admin: { label: "Admin", colour: "red" },
};

const can = (role: Role, p: Permission) => permissions[role].includes(p);
const scoresByStudent: Record<string, number[]> = { Asha: [91, 84], Ravi: [66] };

console.log(can("mentor", "grade"), can("student", "write"), badge.admin.label,
Object.keys(scoresByStudent));
```

OUTPUT

```
true false Admin [ 'Asha', 'Ravi' ]
```

12.3 Union utilities

These utilities filter the members of a union type. They are **distributive** conditional types (14.2.2), applied to each member separately.

12.3.1 Exclude<T, U>

`Exclude<T, U>` removes from the union `T` every member assignable to `U`: `Exclude<"a" | "b" | "c", "a">` is `"b" | "c"`.

12.3.2 Extract<T, U>

`Extract<T, U>` keeps only the members of `T` assignable to `U`. With discriminated unions it selects the member types that have a certain tag: `Extract<Event, { type: "click" }>`.

12.3.3 NonNullable<T>

`NonNullable<T>` removes `null` and `undefined` from `T`. It is used after a value has been checked, or to get the "present" version of an optional property type.

EXCLUDE, EXTRACT AND NONNULLABLE main.ts

```ts
type Status = "draft" | "review" | "published" | "archived";
type Editable = Exclude<Status, "published" | "archived">;        // "draft" | "review"
type Visible = Extract<Status, "published" | "review" | "deleted">; // "review" | "published"

type AppEvent =
  | { type: "click"; x: number; y: number }
  | { type: "key"; key: string }
  | { type: "scroll"; offset: number };
type PointerEvent_ = Extract<AppEvent, { type: "click" }>;          // the click member only
type NonClick = Exclude<AppEvent, { type: "click" }>;               // key | scroll

type MaybeEmail = string | null | undefined;
type Email = NonNullable<MaybeEmail>;                               // string

const e: Editable = "review";
const click: PointerEvent_ = { type: "click", x: 10, y: 20 };
const other: NonClick[] = [{ type: "key", key: "Enter" }, { type: "scroll", offset: 300 }];
const emails: MaybeEmail[] = ["a@lbf.in", null, undefined, "b@lbf.in"];
const valid: Email[] = emails.filter((m): m is Email => m != null);
console.log(e, click.x, other.map((o) => o.type), valid);
```

OUTPUT

```
review 10 [ 'key', 'scroll' ] [ 'a@lbf.in', 'b@lbf.in' ]
```

THE FILTERED UNIONS REJECT THE REMOVED MEMBERS main.ts

```ts
type Status = "draft" | "review" | "published" | "archived";
type Editable = Exclude<Status, "published" | "archived">;
const s: Editable = "published";
const n: NonNullable<string | null> = null;
```

COMPILER ERRORS (tsc --strict)

```
main.ts(3,7): error TS2322: Type '"published"' is not assignable to type 'Editable'.
main.ts(4,7): error TS2322: Type 'null' is not assignable to type 'string'.
```

12.4 Function utilities

These utilities extract parts of a **function type**, which is especially useful with functions you do not control (from libraries) or to keep wrapper functions in sync with the functions they wrap. Use `typeof fn` to get a function's type from its value.

12.4.1 Parameters<T>

12.4.2 ReturnType<T>

PARAMETERS AND RETURNTYPE DERIVED FROM AN EXISTING FUNCTION

main.ts

```
function createOrder(customerId: number, items: { sku: string; qty: number }[], coupon?: string) {
  return { orderId: 9001, customerId, lines: items.length, coupon: coupon ?? null, placedAt: new
Date(2026, 9, 7) };
}

type OrderArgs = Parameters<typeof createOrder>;      // [customerId: number, items: {...}[], coupon?:
string]
type Order = ReturnType<typeof createOrder>;          // { orderId: number; customerId: number; ... }
type ItemList = Parameters<typeof createOrder>[1];    // { sku: string; qty: number }[]

// A logging wrapper that stays in sync with createOrder automatically
function createOrderLogged(...args: OrderArgs): Order {
  console.log("createOrder called with", JSON.stringify(args));
  return createOrder(...args);
}

const items: ItemList = [{ sku: "NB-01", qty: 2 }];
const order: Order = createOrderLogged(42, items, "LBF10");
console.log(order.orderId, order.lines, order.coupon);
```

OUTPUT

```
createOrder called with [42,[{"sku":"NB-01","qty":2}],"LBF10"]
9001 1 LBF10
```

A generic version of this wrapper works for any function: `function logged<F extends (...args: any[]) => any>(fn: F) { return (...args: Parameters<F>): ReturnType<F> => fn(...args); }`. `ReturnType` is also the standard way to name the type of something a library function returns without exporting it, such as `ReturnType<typeof setTimeout>` (a number in browsers, a `Timeout` object in Node.js).

12.4.3 ConstructorParameters<T> and InstanceType<T>

For classes, `ConstructorParameters<typeof C>` gives the constructor's parameter types as a tuple, and `InstanceType<typeof C>` gives the type of objects created with `new C(...)`. They are used in factories, dependency-injection containers and plugin registries that work with classes as values.

CONSTRUCTORPARAMETERS AND INSTANCETYPE IN A GENERIC FACTORY main.ts

```
class Batch {
  constructor(public course: string, public startDate: Date, public seats = 30) {}
  describe() { return `${this.course} from ${this.startDate.toDateString()} (${this.seats} seats)`; }
}

type BatchArgs = ConstructorParameters<typeof Batch>;   // [course: string, startDate: Date, seats?:
number]
type BatchObj = InstanceType<typeof Batch>;             // Batch

function createMany<C extends new (...args: any[]) => any>(Cls: C, argList: ConstructorParameters<C>
[]): InstanceType<C>[] {
  return argList.map((args) => new Cls(...args));
}

const batches: BatchObj[] = createMany(Batch, [["TypeScript", new Date(2026, 9, 12)], ["Python", new
Date(2026, 10, 2), 24]]);
batches.forEach((b) => console.log(b.describe()));
const args: BatchArgs = ["Java", new Date(2026, 11, 1)];
console.log(new Batch(...args).seats);
```

OUTPUT

```
TypeScript from Mon Oct 12 2026 (30 seats)
Python from Mon Nov 02 2026 (24 seats)
30
```

12.4.4 ThisParameterType<T> and OmitThisParameter<T>

For functions that declare a `this` parameter (4.4.2), `ThisParameterType<F>` extracts the type of `this`, and `OmitThisParameter<F>` produces the function type without it, which is the type of the function after `bind`. These are rarely needed in application code, but they appear in libraries that bind methods.

THISPARAMETERTYPE AND OMITTHISPARAMETER main.ts

```
interface Counter { count: number; step: number }

function increment(this: Counter, times = 1): number {
  this.count += this.step * times;
  return this.count;
}

type Ctx = ThisParameterType<typeof increment>;          // Counter
type Bound = OmitThisParameter<typeof increment>;        // (times?: number) => number

const ctx: Ctx = { count: 0, step: 5 };
const inc: Bound = increment.bind(ctx);                   // after bind, no this is needed
console.log(inc(), inc(2), ctx.count);
```

OUTPUT

```
5 15 15
```

12.5 Async and string utilities

12.5.1 Awaited<T>

`Awaited<T>` computes the type you get from `await`: it unwraps `Promise<T>`, recursively (a promise of a promise), and leaves non-promise types unchanged. It is how the types of `Promise.all` and async

functions are defined, and it is useful combined with `ReturnType` to name the resolved result of an async function.

AWAITED UNWRAPS PROMISE TYPES main.ts

```
async function fetchProfile(id: number) {
  return { id, name: "Asha", courses: ["TS101", "PY201"] };
}

type ProfilePromise = ReturnType<typeof fetchProfile>;              // Promise<{ id: number; ... }>
type Profile = Awaited<ReturnType<typeof fetchProfile>>;            // { id: number; name: string;
courses: string[] }
type Nested = Awaited<Promise<Promise<number>>>;                    // number
type Plain = Awaited<string>;                                       // string

function summary(p: Profile): string { return `${p.name}: ${p.courses.length} courses`; }

async function main() {
  const promise: ProfilePromise = fetchProfile(1);
  const profile: Profile = await promise;
  const n: Nested = 42;
  const s: Plain = "plain";
  console.log(summary(profile), n, s);
}
main();
```

OUTPUT

```
Asha: 2 courses 42 plain
```

12.5.2 Uppercase, Lowercase, Capitalize, Uncapitalize

Four intrinsic types transform **string literal types**: `Uppercase<"get">` is `"GET"`, `Lowercase` does the reverse, `Capitalize<"name">` is `"Name"` and `Uncapitalize` lowers the first letter. They work on unions too, and they shine in **template literal types** (14.4), for example to turn a property name into a getter name or an event handler name.

STRING LITERAL UTILITIES WITH TEMPLATE LITERAL TYPES main.ts

```
type Method = "get" | "post" | "delete";
type UpperMethod = Uppercase<Method>;                               // "GET" | "POST" | "DELETE"
type Field = "name" | "email";
type Getter = `get${Capitalize<Field>}`;                            // "getName" | "getEmail"
type Handler = `on${Capitalize<"click" | "submit">}`;               // "onClick" | "onSubmit"

function request(method: UpperMethod, url: string) { return `${method} ${url}`; }
const getters: Record<Getter, () => string> = { getName: () => "Asha", getEmail: () => "asha@lbf.in"
};

console.log(request("POST", "/api/enrol"), getters.getName(), getters.getEmail());
const h: Handler = "onSubmit";
const lower: Lowercase<"ADMIN" | "USER"> = "admin";
const unc: Uncapitalize<"Hello"> = "hello";
console.log(h, lower, unc);
```

OUTPUT

```
POST /api/enrol Asha asha@lbf.in
onSubmit admin hello
```

THE TRANSFORMED LITERALS ARE CHECKED EXACTLY

main.ts

```ts
type UpperMethod = Uppercase<"get" | "post">;
const m: UpperMethod = "get";
type Getter = `get${Capitalize<"name">}`;
const g: Getter = "getname";
```

COMPILER ERRORS (tsc --strict)

```
main.ts(2,7): error TS2820: Type '"get"' is not assignable to type '"GET" | "POST"'. Did you mean
'"GET"'?
main.ts(4,7): error TS2322: Type '"getname"' is not assignable to type '"getName"'.
```

12.6 Other utilities

12.6.1 NoInfer<T>

`NoInfer<T>` (TypeScript 5.4+) marks a position where a type parameter should **not** be inferred from. Consider `function pickColour<C extends string>(options: C[], fallback: C)`: TypeScript infers `C` from both arguments, so a typo in `fallback` simply widens `C` and is accepted. Wrapping the second parameter as `fallback: NoInfer<C>` tells TypeScript to infer `C` only from `options` and then **check** `fallback` against it. Module 15 shows more uses.

NOINFER STOPS A PARAMETER FROM WIDENING THE INFERRED TYPE

main.ts

```ts
function pickWithoutNoInfer<C extends string>(options: C[], fallback: C): C { return options[0] ??
fallback; }
function pick<C extends string>(options: C[], fallback: NoInfer<C>): C { return options[0] ??
fallback; }

pickWithoutNoInfer(["navy", "gold"], "blue");    // accepted: C widened to "navy" | "gold" | "blue"
pick(["navy", "gold"], "blue");                  // error: "blue" is not one of the options
```

COMPILER ERRORS (tsc --strict)

```
main.ts(5,24): error TS2345: Argument of type '"blue"' is not assignable to parameter of type '"navy"
| "gold"'.
```

12.7 Real-world usage

12.7.1 DTOs for create and update operations

A **DTO** (data transfer object) is the shape of data sent between layers, typically between a client and an API. Deriving DTOs from the entity type with utilities keeps them consistent: the create DTO omits generated fields, the update DTO makes the editable fields optional, and the response DTO hides private fields. The table summarises a typical set.

DTO	Definition	Meaning
`Entity`	`interface Course { id; code; title; fee; createdAt; updatedAt }`	the full stored record
`CreateCourseDto`	`Omit<Course, "id" | "createdAt" | "updatedAt">`	everything the client must provide
`UpdateCourseDto`	`Partial<CreateCourseDto>`	any subset of the editable fields

DTO	Definition	Meaning
`CourseSummary`	`Pick<Course, "id" | "code" | "title">`	list views
`CourseResponse`	`Readonly<Omit<Course, "updatedAt">>`	what the API returns, immutable
`CoursesById`	`Record<Course["id"], Course>`	a normalised cache

CREATE, UPDATE AND SUMMARY DTOS DERIVED FROM ONE ENTITY TYPE

main.ts

```
interface Course {
  id: number;
  code: string;
  title: string;
  fee: number;
  tags: string[];
  createdAt: Date;
  updatedAt: Date;
}

type CreateCourseDto = Omit<Course, "id" | "createdAt" | "updatedAt">;
type UpdateCourseDto = Partial<CreateCourseDto>;
type CourseSummary = Pick<Course, "id" | "code" | "title">;

const db = new Map<number, Course>();
let seq = 100;
const now = () => new Date(Date.UTC(2026, 9, 7));

function create(dto: CreateCourseDto): Course {
  const course: Course = { ...dto, id: ++seq, createdAt: now(), updatedAt: now() };
  db.set(course.id, course);
  return course;
}

function update(id: number, dto: UpdateCourseDto): Course | undefined {
  const existing = db.get(id);
  if (!existing) return undefined;
  const updated = { ...existing, ...dto, updatedAt: now() };
  db.set(id, updated);
  return updated;
}

const list = (): CourseSummary[] => [...db.values()].map(({ id, code, title }) => ({ id, code, title
}));

const ts101 = create({ code: "TS101", title: "TypeScript", fee: 9999, tags: ["web"] });
create({ code: "PY201", title: "Python", fee: 14999, tags: ["data"] });
update(ts101.id, { fee: 8999, tags: ["web", "types"] });
console.log(list());
console.log(db.get(ts101.id)?.fee, db.get(ts101.id)?.tags);
```

OUTPUT

```
[
  { id: 101, code: 'TS101', title: 'TypeScript' },
  { id: 102, code: 'PY201', title: 'Python' }
]
8999 [ 'web', 'types' ]
```

DTO MISTAKES CAUGHT AT COMPILE TIME

main.ts

```ts
interface Course { id: number; code: string; title: string; fee: number; createdAt: Date }
type CreateCourseDto = Omit<Course, "id" | "createdAt">;
type UpdateCourseDto = Partial<CreateCourseDto>;

const c: CreateCourseDto = { id: 5, code: "TS101", title: "TypeScript", fee: 9999 };   // id not allowed
const u: UpdateCourseDto = { fee: "free" };                                            // wrong type
const d: CreateCourseDto = { code: "JV301", title: "Java" };                           // fee missing
```

COMPILER ERRORS (tsc --strict)

```
main.ts(5,30): error TS2353: Object literal may only specify known properties, and 'id' does not exist in type 'CreateCourseDto'.
main.ts(6,30): error TS2322: Type 'string' is not assignable to type 'number'.
main.ts(7,7): error TS2741: Property 'fee' is missing in type '{ code: string; title: string; }' but required in type 'CreateCourseDto'.
```

12.7.2 Form state types

Forms need several types derived from the same data: the **values** (often all strings while the user types), the **errors** (an optional message per field), and the **touched** flags (a boolean per field). Mapping over the keys of one type with `Record<keyof T, ...>` and `Partial` keeps all of them in sync: adding a field to the form type updates every derived type, and the compiler points out the code that must handle it.

FORM VALUES, ERRORS AND TOUCHED FLAGS DERIVED FROM ONE FORM TYPE

main.ts

```
interface EnrolmentForm {
  fullName: string;
  email: string;
  phone: string;
  course: "TS101" | "PY201" | "JV301";
}

type FormErrors<T> = Partial<Record<keyof T, string>>;
type FormTouched<T> = Record<keyof T, boolean>;

interface FormState<T> {
  values: T;
  errors: FormErrors<T>;
  touched: FormTouched<T>;
  submitting: boolean;
}

function validate(v: EnrolmentForm): FormErrors<EnrolmentForm> {
  const errors: FormErrors<EnrolmentForm> = {};
  if (v.fullName.trim().length < 3) errors.fullName = "Enter your full name";
  if (!/^[^@\s]+@[^@\s]+\.[a-z]{2,}$/i.test(v.email)) errors.email = "Enter a valid email";
  if (!/^[6-9]\d{9}$/.test(v.phone)) errors.phone = "Enter a 10-digit mobile number";
  return errors;
}

function touchAll<T extends object>(values: T): FormTouched<T> {
  return Object.fromEntries(Object.keys(values).map((k) => [k, true])) as FormTouched<T>;
}

const values: EnrolmentForm = { fullName: "As", email: "asha@lbf", phone: "9972934418", course:
"TS101" };
const state: FormState<EnrolmentForm> = { values, errors: validate(values), touched: touchAll(values),
submitting: false };

for (const field of Object.keys(state.values) as (keyof EnrolmentForm)[]) {
  const msg = state.touched[field] ? state.errors[field] : undefined;
  console.log(`${field.padEnd(9)} ${String(state.values[field]).padEnd(12)} ${msg ?? "ok"}`);
}
console.log("can submit:", Object.keys(state.errors).length === 0);
```

OUTPUT

```
fullName  As           Enter your full name
email     asha@lbf     Enter a valid email
phone     9972934418   ok
course    TS101        ok
can submit: false
```

Utility	Result	Typical use
`Partial<T>` / `Required<T>` / `Readonly<T>`	all properties optional / required / readonly	updates, resolved config, immutable state
`Pick<T, K>` / `Omit<T, K>`	only / all except the keys K	summaries, create DTOs, hiding fields
`Record<K, T>`	object with keys K and values T	lookup tables, dictionaries
`Exclude` / `Extract` / `NonNullable`	filter union members	states, event types, present values

Utility	Result	Typical use
`Parameters` / `ReturnType`	a function's argument tuple / result	wrappers, library return types
`ConstructorParameters` / `InstanceType`	a class's constructor args / instance	factories, DI containers
`ThisParameterType` / `OmitThisParameter`	the this type / function without it	binding methods
`Awaited<T>`	the resolved type of a promise	async results
`Uppercase`, `Lowercase`, `Capitalize`, `Uncapitalize`	transformed string literals	names in template literal types
`NoInfer<T>`	blocks inference at that position	defaults and fallbacks checked against T

Utility types derive new types from existing ones so each concept is defined once. Partial, Required and Readonly change the optional and readonly modifiers of every property; Pick and Omit select or remove keys (Omit does not check its keys); Record<K, T> builds dictionaries and exhaustive lookup tables. Exclude, Extract and NonNullable filter union members. Parameters, ReturnType, ConstructorParameters, InstanceType, ThisParameterType and OmitThisParameter extract parts of function and class types (use typeof value). Awaited unwraps promises, the intrinsic string types transform literal types, and NoInfer stops a parameter from influencing inference. In real projects they produce create, update and response DTOs and form state types that stay in sync with the entity.

PRACTICE CHECKPOINT

From one **User** type, derive **CreateUserDto** (Omit id), **UpdateUserDto** (Partial), **PublicUser** (Pick) and a **Record of users by role**.

SAMPLE SOLUTION: DTOS AND A RECORD BY ROLE DERIVED FROM ONE USER TYPE

main.ts

```ts
type Role = "student" | "mentor" | "admin";

interface User {
  id: number;
  name: string;
  email: string;
  role: Role;
  passwordHash: string;
  city?: string;
  joinedAt: Date;
}

type CreateUserDto = Omit<User, "id" | "joinedAt">;
type UpdateUserDto = Partial<Omit<CreateUserDto, "passwordHash">>;   // password changes go elsewhere
type PublicUser = Pick<User, "id" | "name" | "role" | "city">;
type UsersByRole = Record<Role, PublicUser[]>;

class UserStore {
  #users = new Map<number, User>();
  #seq = 0;

  create(dto: CreateUserDto): PublicUser {
    const user: User = { ...dto, id: ++this.#seq, joinedAt: new Date(Date.UTC(2026, 9, this.#seq)) };
    this.#users.set(user.id, user);
```

```
    return this.toPublic(user);
  }

  update(id: number, dto: UpdateUserDto): PublicUser | undefined {
    const user = this.#users.get(id);
    if (!user) return undefined;
    const updated: User = { ...user, ...dto };
    this.#users.set(id, updated);
    return this.toPublic(updated);
  }

  byRole(): UsersByRole {
    const groups: UsersByRole = { student: [], mentor: [], admin: [] };     // every role must be
present
    for (const u of this.#users.values()) groups[u.role].push(this.toPublic(u));
    return groups;
  }

  private toPublic({ id, name, role, city }: User): PublicUser {
    return city === undefined ? { id, name, role } : { id, name, role, city };
  }
}

const store = new UserStore();
const asha = store.create({ name: "Asha", email: "asha@lbf.in", role: "student", passwordHash: "h1",
city: "Mysuru" });
store.create({ name: "Lohit", email: "lohit@lbf.in", role: "admin", passwordHash: "h2" });
const ravi = store.create({ name: "Ravi", email: "ravi@lbf.in", role: "student", passwordHash: "h3"
});
store.update(ravi.id, { role: "mentor", city: "Pune" });

console.log("created:", asha);
const grouped = store.byRole();
for (const role of Object.keys(grouped) as Role[]) {
  console.log(role.padEnd(8), grouped[role].map((u) => `${u.name}${u.city ? " (" + u.city + ")" :
""}`).join(", ") || "-");
}
```

OUTPUT

```
created: { id: 1, name: 'Asha', role: 'student', city: 'Mysuru' }
student  Asha (Mysuru)
mentor   Ravi (Pune)
admin    Lohit
```

MODULE 13 · INTERMEDIATE

TypeScript in the browser and with asynchronous code

Minimum duration **2 days** | Level **Intermediate**

This module applies TypeScript to DOM manipulation, events and API calls, where many real-world type issues appear. It covers the lib setting and the DOM type declarations, element types such as HTMLElement, HTMLInputElement and HTMLButtonElement, querySelector return types and null checks, casting DOM elements safely, event types (MouseEvent, KeyboardEvent, SubmitEvent, Event) and typed handlers, typing promises, async functions and fetch responses, treating API data as unknown and validating it, error handling with catch (e: unknown), and typed localStorage wrappers.

IN THIS MODULE

13.1 DOM types
13.2 Event types
13.3 Async TypeScript
13.4 Browser storage with types

NOTE

Browser examples in this module were compiled with `tsc --strict` (with the DOM library) and the resulting JavaScript was then run in a real headless Chromium browser against the HTML shown. User actions such as clicks and typing were simulated, and API responses came from a mock server. The output box shows what the browser console printed.

13.1 DOM types

13.1.1 lib setting and dom types

The types for everything the browser provides (`document`, `window`, elements, events, `fetch`, `localStorage`) come from the declaration file `lib.dom.d.ts`, which ships with TypeScript. The `lib` compiler option chooses which built-in declaration files are loaded: `["es2023", "dom", "dom.iterable"]` is typical for front-end code (Vite's templates use `["ES2022", "DOM", "DOM.Iterable"]`). If `lib` is not set, it defaults to the ES version of your `target` plus DOM. Node.js projects usually leave `dom` out, so that browser-only globals such as `document` are reported as errors, and get Node.js types from `@types/node` instead.

PAGE.TS

TypeScript

```
const heading = document.querySelector("h1");
window.addEventListener("resize", () => console.log(window.innerWidth));
localStorage.setItem("visited", "yes");
```

WITHOUT THE DOM LIB, BROWSER GLOBALS ARE UNKNOWN

terminal

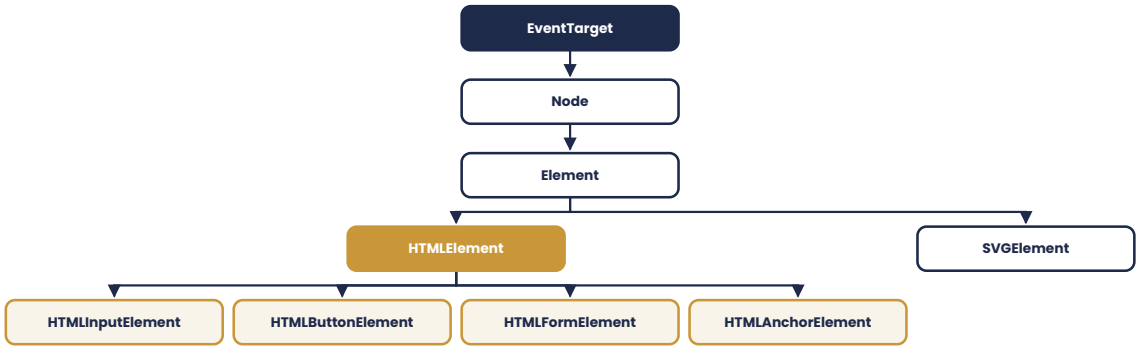
```
npx tsc --noEmit --strict --target es2022 --lib es2023 --ignoreConfig page.ts || true
npx tsc --noEmit --strict --target es2022 --lib es2023,dom --ignoreConfig page.ts && echo "OK with the dom lib"
```

TERMINAL

```
$ npx tsc --noEmit --strict --target es2022 --lib es2023 --ignoreConfig page.ts || true
page.ts(1,17): error TS2584: Cannot find name 'document'. Do you need to change your target library?
Try changing the 'lib' compiler option to include 'dom'.
page.ts(2,1): error TS2304: Cannot find name 'window'.
page.ts(2,41): error TS2584: Cannot find name 'console'. Do you need to change your target library?
Try changing the 'lib' compiler option to include 'dom'.
page.ts(2,53): error TS2304: Cannot find name 'window'.
page.ts(3,1): error TS2304: Cannot find name 'localStorage'.
$ npx tsc --noEmit --strict --target es2022 --lib es2023,dom --ignoreConfig page.ts && echo "OK with the dom lib"
OK with the dom lib
```

13.1.2 HTMLElement, HTMLInputElement, HTMLButtonElement

The DOM types form a class hierarchy that mirrors the browser's: `EventTarget` → `Node` → `Element` → `HTMLElement` → specific element types such as `HTMLInputElement` (with `value`, `checked`, `valueAsNumber`, `files`), `HTMLButtonElement` (`disabled`, `type`), `HTMLAnchorElement` (`href`), `HTMLImageElement` (`src`, `alt`), `HTMLFormElement` (`elements`, `reset()`), `HTMLSelectElement` and `HTMLCanvasElement`. Only the specific type has the specific properties: a plain `HTMLElement` has no `value`.



PART OF THE DOM TYPE HIERARCHY

A GENERAL ELEMENT TYPE DOES NOT HAVE SPECIFIC PROPERTIES

main.ts

```
const input = document.getElementById("email");      // HTMLElement | null
console.log(input.value);

const el = document.body.firstElementChild;           // Element | null
el?.click();
```

COMPILER ERRORS (tsc --strict)

```
main.ts(2,13): error TS18047: 'input' is possibly 'null'.
main.ts(2,19): error TS2339: Property 'value' does not exist on type 'HTMLElement'.
main.ts(5,5): error TS2339: Property 'click' does not exist on type 'Element'.
```

13.1.3 querySelector return types and null checks

`document.querySelector` and `getElementById` can find nothing, so their return types include `null`, and strict null checks make you handle it. `querySelector` is smart about **tag selectors**: `querySelector("input")` returns `HTMLInputElement | null` and `querySelector("a")` returns

`HTMLAnchorElement | null`, because the tag determines the type. For class or id selectors (`"#email"`, `".card"`) it can only return `Element | null`. `querySelectorAll` returns a `NodeListOf<...>`, which is never null (it may be empty) and is iterable with `for...of` (with `dom.iterable`).

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY) HTML

```
<input placeholder="Your email">
<p>Total: <span id="total">Rs 9,999</span></p>
<a href="/courses">Courses</a> <a href="/contact">Contact</a>
```

ELEMENT TYPES FROM TAG SELECTORS, AND NULL CHECKS main.ts

```
const firstInput = document.querySelector("input");          // HTMLInputElement | null (from the tag
name)
const byId = document.querySelector("#total");                // Element | null
const links = document.querySelectorAll("a");                 // NodeListOf<HTMLAnchorElement>

if (firstInput) {
  console.log("placeholder:", firstInput.placeholder);         // narrowed to HTMLInputElement
}
console.log("total text:", byId?.textContent ?? "(missing)");
for (const link of links) console.log("link:", link.textContent, "->", link.getAttribute("href"));
console.log("missing element:", document.querySelector(".does-not-exist"));
```

BROWSER CONSOLE

```
placeholder: Your email
total text: Rs 9,999
link: Courses -> /courses
link: Contact -> /contact
missing element: null
```

13.1.4 Casting DOM elements safely

When you select by id or class, TypeScript cannot know the element type, so you must tell it. There are three ways, from most to least safe. (1) A **runtime check** with `instanceof` narrows correctly and fails gracefully if the HTML changes. (2) A **generic type argument** `querySelector<HTMLInputElement>("#email")` keeps the `null` but trusts the element type. (3) An **assertion** `as HTMLInputElement` (possibly with `!`) trusts both. A small helper that checks with `instanceof` and throws a clear error gives safety with short call sites.

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY) HTML

```
<input id="email" value="asha@lbf.in">
<select id="course"><option value="TS101" selected>TypeScript</option><option
value="PY201">Python</option></select>
<button id="enrol">Enrol</button>
```

THREE WAYS TO GET A SPECIFIC ELEMENT TYPE, AND A CHECKED HELPER main.ts

```
// 1. instanceof: checked at run time
const email = document.getElementById("email");
if (email instanceof HTMLInputElement) console.log("1:", email.value);

// 2. generic type argument: trusted element type, null still handled
const phone = document.querySelector<HTMLInputElement>("#phone");
console.log("2:", phone?.value ?? "no phone field");

// 3. assertion: trusts everything (avoid unless certain)
const course = document.querySelector("#course") as HTMLSelectElement;
console.log("3:", course.value);

// A safe helper: one place that checks and fails loudly
function $<T extends Element>(selector: string, type: new () => T): T {
  const el = document.querySelector(selector);
  if (!(el instanceof type)) throw new Error(`${selector} is missing or is not a ${type.name}`);
  return el;
}

const button = $("#enrol", HTMLButtonElement);       // HTMLButtonElement, guaranteed
button.disabled = true;
console.log("button disabled:", button.disabled);
try {
  $("#email", HTMLButtonElement);                    // wrong element type
} catch (e) {
  console.log((e as Error).message);
}
```

BROWSER CONSOLE

```
1: asha@lbf.in
2: no phone field
3: TS101
button disabled: true
#email is missing or is not a HTMLButtonElement
```

13.2 Event types

13.2.1 MouseEvent, KeyboardEvent, SubmitEvent, Event

Every DOM event has a type in `lib.dom.d.ts`: `MouseEvent` (`clientX`, `button`, `shiftKey`) for mouse events (`click` is typed as `PointerEvent`, a subtype of `MouseEvent`, in current DOM types), `KeyboardEvent` (`key`, `code`, `ctrlKey`) for key presses, `InputEvent` for typing, `SubmitEvent` (`submitter`) for form submissions, `FocusEvent`, `PointerEvent`, `DragEvent`, `ClipboardEvent`, and the base `Event` for generic events such as `change`. When you call `addEventListener` with a known event name, TypeScript picks the right type automatically through the `HTMLElementEventMap` interface, so `el.addEventListener("keydown", (e) => ...)` gives `e` the type `KeyboardEvent`.

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY) HTML

```
<div id="box" style="width:200px;height:80px;background:#eee">click me</div>
<input id="search">
```

EVENT PARAMETER TYPES ARE INFERRED FROM THE EVENT NAME

main.ts

```
const box = document.querySelector<HTMLDivElement>("#box")!;
const search = document.querySelector<HTMLInputElement>("#search")!;

box.addEventListener("click", (e) => {                    // e: PointerEvent (a MouseEvent), inferred
from "click"
  console.log(`click at (${e.offsetX}, ${e.offsetY}), button ${e.button}, shift ${e.shiftKey}`);
});

search.addEventListener("keydown", (e) => {               // e: KeyboardEvent
  if (e.key === "Enter") console.log("search for:", search.value);
});

search.addEventListener("input", (e) => {                 // e: Event (InputEvent in practice)
  const target = e.target as HTMLInputElement;            // e.target is EventTarget | null
  console.log("typed, length now", target.value.length);
});
```

BROWSER CONSOLE

```
click at (100, 40), button 0, shift false
typed, length now 3
search for: tsx
```

Simulated user actions, in order: clicks `#box`; types "tsx" into `#search`; presses Enter in `#search`.

Notice `e.target`: its type is `EventTarget | null`, not the element, because events bubble and the target may be a child element. Use `e.currentTarget` (the element the listener is attached to), the variable you already have, or an `instanceof` check.

13.2.2 Typing event handlers

When you write a handler as a separate function, annotate its parameter with the event type. Handlers registered with `addEventListener` can declare `this` as the element. For forms, listen to `submit` on the form, call `e.preventDefault()`, and read values through `FormData` or the form's typed elements. Event delegation (one listener on a parent) is typed by checking `e.target` with `instanceof` or `closest`.

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY)

HTML

```
<form id="enrol-form">
  <input name="name"> <input name="email">
  <select name="course"><option value="TS101">TypeScript</option><option value="PY201">Python</option>
</select>
  <label><input type="radio" name="batch" value="weekday" checked> Weekday</label>
  <label><input type="radio" name="batch" value="weekend"> Weekend</label>
  <button type="submit">Enrol now</button>
</form>
<ul id="courses"><li data-code="TS101">TypeScript</li><li data-code="JV301"><b>Java</b></li></ul>
```

NAMED, TYPED HANDLERS FOR A FORM AND FOR EVENT DELEGATION

main.ts

```
interface Enrolment { name: string; email: string; course: string; batch: "weekday" | "weekend" }

const form = document.querySelector<HTMLFormElement>("#enrol-form")!;
const list = document.querySelector<HTMLUListElement>("#courses")!;

function handleSubmit(e: SubmitEvent): void {
  e.preventDefault();                                          // stop the page reload
  const data = new FormData(form);
  const enrolment: Enrolment = {
    name: String(data.get("name") ?? "").trim(),
    email: String(data.get("email") ?? "").trim(),
    course: String(data.get("course")),
    batch: data.get("batch") === "weekend" ? "weekend" : "weekday",
  };
  const submitter = e.submitter as HTMLButtonElement | null;
  console.log(`submitted by "${submitter?.textContent}":`, JSON.stringify(enrolment));
}

function handleListClick(e: MouseEvent): void {                 // event delegation
  if (!(e.target instanceof HTMLElement)) return;
  const item = e.target.closest<HTMLLIElement>("li[data-code]");
  if (item) console.log("selected course", item.dataset.code, "-", item.textContent);
}

form.addEventListener("submit", handleSubmit);
list.addEventListener("click", handleListClick);
```

BROWSER CONSOLE

```
submitted by "Enrol now": {"name":"Asha Rao","email":"asha@lbf.in","course":"TS101","batch":"weekend"}
selected course JV301 - Java
```

Simulated user actions, in order: types "Asha Rao" into `input[name=name]`; types "asha@lbf.in" into `input[name=email]`; clicks `input[value=weekend]`; clicks `button`; clicks `li[data-code=JV301] b`.

WRONG EVENT PROPERTIES AND MISMATCHED HANDLERS ARE CAUGHT main.ts

```ts
const button = document.querySelector("button")!;
button.addEventListener("click", (e) => {
  console.log(e.key);                       // MouseEvent has no key
});
function onKey(e: KeyboardEvent) { console.log(e.key); }
button.addEventListener("click", onKey);    // a KeyboardEvent handler for a click
```

COMPILER ERRORS (tsc --strict)

```
main.ts(3,17): error TS2339: Property 'key' does not exist on type 'PointerEvent'.
main.ts(6,34): error TS2769: No overload matches this call.
  Overload 1 of 2, '(type: "click", listener: (this: HTMLButtonElement, ev: PointerEvent) => any,
options?: boolean | AddEventListenerOptions | undefined): void', gave the following error.
    Argument of type '(e: KeyboardEvent) => void' is not assignable to parameter of type '(this:
HTMLButtonElement, ev: PointerEvent) => any'.
      Types of parameters 'e' and 'ev' are incompatible.
        Type 'PointerEvent' is missing the following properties from type 'KeyboardEvent': charCode,
code, isComposing, key, and 8 more.
  Overload 2 of 2, '(type: string, listener: EventListenerOrEventListenerObject, options?: boolean |
AddEventListenerOptions | undefined): void', gave the following error.
    Argument of type '(e: KeyboardEvent) => void' is not assignable to parameter of type
'EventListenerOrEventListenerObject'.
      Type '(e: KeyboardEvent) => void' is not assignable to type 'EventListener'.
        Types of parameters 'e' and 'evt' are incompatible.
          Type 'Event' is missing the following properties from type 'KeyboardEvent': altKey,
charCode, code, ctrlKey, and 17 more.
```

13.3 Async TypeScript

13.3.1 Typing promises and async functions

Async code is typed with `Promise<T>` (4.6). Two details matter in the browser. First, when you create a promise yourself, give the type argument to the constructor (`new Promise<number>(...)`) or annotate the function's return type, because there is nothing to infer it from. Second, timers return `number` in the browser but a `Timeout` object in Node.js; write `ReturnType<typeof setTimeout>` to stay portable. Utility functions such as a typed `delay`, `timeout` or `retry` are easy to write once and reuse.

TYPED PROMISE HELPERS: DELAY, TIMEOUT AND RETRY main.ts

```ts
const delay = (ms: number) => new Promise<void>((resolve) => setTimeout(resolve, ms));

function withTimeout<T>(promise: Promise<T>, ms: number): Promise<T> {
  let timer: ReturnType<typeof setTimeout> | undefined;
  const timeout = new Promise<never>((_, reject) => {
    timer = setTimeout(() => reject(new Error(`timed out after ${ms} ms`)), ms);
  });
  return Promise.race([promise, timeout]).finally(() => clearTimeout(timer));
}

async function retry<T>(task: () => Promise<T>, attempts: number, waitMs = 10): Promise<T> {
  let lastError: unknown;
  for (let i = 1; i <= attempts; i++) {
    try {
      return await task();
    } catch (e) {
      lastError = e;
      console.log(`  attempt ${i} failed: ${(e as Error).message}`);
      await delay(waitMs * i);
    }
```

```
  }
  throw lastError;
}

let calls = 0;
const flaky = async (): Promise<string> => {
  await delay(5);
  if (++calls < 3) throw new Error("network error");
  return "seat booked";
};

async function main() {
  console.log(await retry(flaky, 5));
  try {
    await withTimeout(delay(200).then(() => "slow result"), 50);
  } catch (e) {
    console.log((e as Error).message);
  }
  console.log(await withTimeout(Promise.resolve(42), 50));
}
main();
```

OUTPUT

```
  attempt 1 failed: network error
  attempt 2 failed: network error
seat booked
timed out after 50 ms
42
```

13.3.2 Typing fetch responses

`fetch` returns `Promise<Response>`, and `response.json()` returns `Promise<any>`: the compiler has no idea what the server sent. Assigning that `any` to a typed variable compiles, but it is an unchecked assertion. Remember also that `fetch` only rejects on network failures: an HTTP 404 or 500 is a successful promise with `response.ok === false`, so check it. A small typed wrapper centralises these rules.

A FETCH WRAPPER THAT CHECKS RESPONSE.OK AND RETURNS UNKNOWN

main.ts

```
interface Course { id: number; code: string; title: string; fee: number }

class HttpError extends Error {
  constructor(public status: number, message: string) { super(message); }
}

async function getJson(url: string): Promise<unknown> {
  const response = await fetch(url, { headers: { Accept: "application/json" } });
  if (!response.ok) throw new HttpError(response.status, `${response.status} ${response.statusText}
for ${url}`);
  return response.json() as Promise<unknown>;               // any -> unknown: callers must validate
}

async function main() {
  const data = await getJson("/api/courses");
  const courses = data as Course[];                         // trusting the server (see 13.3.3 for
checking)
  console.log(courses.map((c) => `${c.code}: Rs ${c.fee}`).join(", "));
  try {
    await getJson("/api/courses/99");
  } catch (e) {
    if (e instanceof HttpError) console.log("HTTP error", e.status, "-", e.message);
  }
}
main();
```

BROWSER CONSOLE

```
TS101: Rs 9999, PY201: Rs 14999
HTTP error 404 - 404 Not Found for /api/courses/99
```

13.3.3 Treating API data as unknown and validating it

The response body is external data, so the safe pattern is: get it as `unknown`, **validate** it, and only then give it a type (8.4.2). Validation can be hand-written type guards, as below, or a schema library such as Zod, which defines the checks and the TypeScript type together (15.3). Combine validation with the fetch wrapper by passing the guard as an argument: `fetchJson(url, isCourseList)` returns `Promise<Course[]>` and throws a clear error when the server sends something unexpected, instead of crashing later in unrelated code.

FETCHING WITH VALIDATION: UNKNOWN IN, TYPED DATA OUT

main.ts

```
interface Course { id: number; code: string; title: string; fee: number }
type Guard<T> = (value: unknown) => value is T;

const isCourse: Guard<Course> = (v): v is Course =>
  typeof v === "object" && v !== null &&
  typeof (v as Course).id === "number" && typeof (v as Course).code === "string" &&
  typeof (v as Course).title === "string" && typeof (v as Course).fee === "number";

const arrayOf = <T,>(guard: Guard<T>): Guard<T[]> => (v): v is T[] => Array.isArray(v) &&
v.every(guard);

async function fetchJson<T>(url: string, guard: Guard<T>): Promise<T> {
  const res = await fetch(url);
  if (!res.ok) throw new Error(`HTTP ${res.status} for ${url}`);
  const body: unknown = await res.json();
  if (!guard(body)) throw new TypeError(`unexpected data from ${url}: ${JSON.stringify(body).slice(0,
60)}`);
  return body;
}

async function main() {
  const courses = await fetchJson("/api/courses", arrayOf(isCourse));        // Course[]
  console.log("valid:", courses.map((c) => c.title).join(", "));
  try {
    await fetchJson("/api/v2/courses", arrayOf(isCourse));                   // the API changed shape
  } catch (e) {
    console.log((e as Error).name + ":", (e as Error).message);
  }
}
main();
```

BROWSER CONSOLE

```
valid: TypeScript
TypeError: unexpected data from /api/v2/courses: {"items":[{"id":"1","name":"TypeScript"}]}
```

13.3.4 Error handling: catch (e: unknown)

JavaScript can `throw` any value, not just `Error` objects, so TypeScript cannot know what a `catch` clause receives. With `useUnknownInCatchVariables` (part of strict, TypeScript 4.4+), the catch variable has type `unknown`, and you must narrow it before reading `.message`. You may annotate it as `catch (e: unknown)` for clarity, but not as `Error` or a specific class. The usual tools: `instanceof Error` (and your own error subclasses), a helper that converts anything to a message, and for promises, the same narrowing in `.catch(e => ...)`, whose parameter is `any` (so be disciplined).

NARROWING THE UNKNOWN VALUE IN A CATCH CLAUSE main.ts

```
class ValidationError extends Error {
  constructor(public field: string, message: string) { super(message); this.name = "ValidationError";
}
}

function errorMessage(e: unknown): string {
  if (e instanceof Error) return `${e.name}: ${e.message}`;
  if (typeof e === "string") return e;
  try { return JSON.stringify(e); } catch { return String(e); }
}

const failures: (() => void)[] = [
  () => { throw new ValidationError("email", "email is required"); },
  () => { JSON.parse("{bad json"); },
  () => { throw "a plain string was thrown"; },
  () => { throw { code: 503, reason: "maintenance" }; },
];

for (const fail of failures) {
  try {
    fail();
  } catch (e: unknown) {
    if (e instanceof ValidationError) console.log(`fix field "${e.field}": ${e.message}`);
    else console.log(errorMessage(e));
  }
}
```

OUTPUT

```
fix field "email": email is required
SyntaxError: Expected property name or '}' in JSON at position 1 (line 1 column 2)
a plain string was thrown
{"code":503,"reason":"maintenance"}
```

THE CATCH VARIABLE IS UNKNOWN UNDER STRICT MODE main.ts

```
try {
  JSON.parse("{bad");
} catch (e) {
  console.log(e.message);              // e is unknown
}
try {
  JSON.parse("{bad");
} catch (e: Error) {                    // only any or unknown is allowed here
  console.log(e.message);
}
```

COMPILER ERRORS (tsc --strict)

```
main.ts(4,15): error TS18046: 'e' is of type 'unknown'.
main.ts(8,13): error TS1196: Catch clause variable type annotation must be 'any' or 'unknown' if
specified.
```

13.4 Browser storage with types

13.4.1 Typed localStorage wrappers

`localStorage` and `sessionStorage` store only **strings**: `getItem` returns `string | null`, so objects must be serialised with `JSON.stringify` and parsed back, and the parsed value is untyped. The stored data may

also be from an older version of your app, edited by the user, or missing (private browsing, storage cleared), and `setItem` can throw when storage is full. A small typed wrapper solves all of this in one place: a schema of keys and their types, a validator or default per key, and safe parse and save functions.

A TYPED WRAPPER AROUND LOCALSTORAGE

main.ts

```
interface Preferences { theme: "light" | "dark"; fontSize: number }
interface CartItem { sku: string; qty: number }

// One map of every key the app stores, and the type of its value
interface StorageSchema {
  prefs: Preferences;
  cart: CartItem[];
  lastVisit: string;
}

const defaults: StorageSchema = { prefs: { theme: "light", fontSize: 14 }, cart: [], lastVisit:
"never" };

class TypedStorage<S extends object> {
  constructor(private prefix: string, private fallback: S, private store: Storage = localStorage) {}

  get<K extends keyof S & string>(key: K): S[K] {
    const raw = this.store.getItem(this.prefix + key);
    if (raw === null) return this.fallback[key];
    try {
      return JSON.parse(raw) as S[K];
    } catch {
      console.log(`  corrupt value for "${key}", using default`);
      return this.fallback[key];
    }
  }

  set<K extends keyof S & string>(key: K, value: S[K]): boolean {
    try {
      this.store.setItem(this.prefix + key, JSON.stringify(value));
      return true;
    } catch {
      return false;                                  // quota exceeded or storage disabled
    }
  }

  remove(key: keyof S & string): void { this.store.removeItem(this.prefix + key); }
}

const storage = new TypedStorage<StorageSchema>("lbf:", defaults);

console.log("first load:", storage.get("prefs"), storage.get("cart").length);
storage.set("prefs", { theme: "dark", fontSize: 16 });
storage.set("cart", [{ sku: "TS101", qty: 1 }]);
storage.set("lastVisit", "2026-10-07");
console.log("after saving:", storage.get("prefs").theme, storage.get("cart")[0].sku,
storage.get("lastVisit"));

localStorage.setItem("lbf:prefs", "{not json");          // simulate a corrupted value
console.log("after corruption:", storage.get("prefs"));
console.log("raw keys:", Object.keys(localStorage).sort());
```

BROWSER CONSOLE

```
first load: {theme: light, fontSize: 14} 0
after saving: dark TS101 2026-10-07
  corrupt value for "prefs", using default
after corruption: {theme: light, fontSize: 14}
raw keys: [lbf:cart, lbf:lastVisit, lbf:prefs]
```

THE SCHEMA MAKES EVERY KEY AND VALUE TYPE-CHECKED main.ts

```ts
interface StorageSchema { prefs: { theme: "light" | "dark" }; cart: { sku: string; qty: number }[] }
declare const storage: {
  get<K extends keyof StorageSchema>(key: K): StorageSchema[K];
  set<K extends keyof StorageSchema>(key: K, value: StorageSchema[K]): boolean;
};

storage.set("prefs", { theme: "blue" });
storage.set("cart", [{ sku: "TS101", qty: "1" }]);
storage.get("settings");
const n: number = storage.get("cart");
```

COMPILER ERRORS (tsc --strict)

```
main.ts(7,24): error TS2322: Type '"blue"' is not assignable to type '"light" | "dark"'.
main.ts(8,38): error TS2322: Type 'string' is not assignable to type 'number'.
main.ts(9,13): error TS2345: Argument of type '"settings"' is not assignable to parameter of type
'keyof StorageSchema'.
main.ts(10,7): error TS2322: Type '{ sku: string; qty: number; }[]' is not assignable to type
'number'.
```

Browser types come from lib.dom.d.ts, selected with the lib option. Elements form a hierarchy (Element, HTMLElement, HTMLInputElement, HTMLButtonElement…), and only specific types have specific properties. querySelector returns T | null, inferring T from tag selectors; for ids and classes use an instanceof check, querySelector<T>, or (least safe) an assertion, ideally inside one checked helper. Event types (MouseEvent, KeyboardEvent, SubmitEvent, Event) are inferred from the event name in addEventListener; e.target is EventTarget | null, so use currentTarget, the element variable or instanceof. Type promises you create, use ReturnType<typeof setTimeout> for timers, check response.ok, and treat response.json() as unknown that a guard or schema validates. Catch variables are unknown under strict mode, so narrow them. Wrap localStorage in a typed class driven by a schema interface, with JSON parsing, defaults and error handling.

PRACTICE CHECKPOINT

A **typed to-do app** with **DOM manipulation**, **typed events**, data loaded from a **mock API with validation**, and **typed localStorage persistence**.

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY) HTML

```html
<form id="add"><input id="title" placeholder="New task">
  <select id="priority"><option>low</option><option selected>normal</option><option>high</option>
</select>
  <button>Add</button></form>
<p>Pending: <span id="count">0</span></p>
<ul id="list"></ul>
```

SAMPLE SOLUTION: A TYPED TO-DO APP WITH FETCH VALIDATION AND LOCALSTORAGE

main.ts

```
type Priority = "low" | "normal" | "high";
interface Todo { id: number; title: string; done: boolean; priority: Priority }

// ---------- validation of API data
const PRIORITIES: readonly Priority[] = ["low", "normal", "high"];
function isTodo(v: unknown): v is Todo {
  if (typeof v !== "object" || v === null) return false;
  const t = v as Record<string, unknown>;
  return typeof t.id === "number" && typeof t.title === "string" && typeof t.done === "boolean" &&
    PRIORITIES.includes(t.priority as Priority);
}

async function loadTodos(url: string): Promise<Todo[]> {
  const res = await fetch(url);
  if (!res.ok) throw new Error(`HTTP ${res.status}`);
  const body: unknown = await res.json();
  if (!Array.isArray(body)) throw new TypeError("expected an array");
  const valid = body.filter(isTodo);
  if (valid.length < body.length) console.log(`skipped ${body.length - valid.length} invalid todo(s)
from the API`);
  return valid;
}

// ---------- typed persistence
const KEY = "lbf:todos";
const save = (todos: Todo[]): void => localStorage.setItem(KEY, JSON.stringify(todos));
function restore(): Todo[] | null {
  const raw = localStorage.getItem(KEY);
  if (raw === null) return null;
  try {
    const parsed: unknown = JSON.parse(raw);
    return Array.isArray(parsed) ? parsed.filter(isTodo) : null;
  } catch {
    return null;
  }
}

// ---------- DOM elements (checked once)
function el<T extends HTMLElement>(selector: string, type: new () => T): T {
  const found = document.querySelector(selector);
  if (!(found instanceof type)) throw new Error(`missing ${selector}`);
  return found;
}
const form = el("#add", HTMLFormElement);
const titleInput = el("#title", HTMLInputElement);
const prioritySelect = el("#priority", HTMLSelectElement);
const list = el("#list", HTMLUListElement);
const counter = el("#count", HTMLSpanElement);

let todos: Todo[] = [];

function render(): void {
  list.replaceChildren(...todos.map((t) => {
    const li = document.createElement("li");
    li.dataset.id = String(t.id);
    li.className = t.done ? "done" : "";
    li.textContent = `${t.done ? "[x]" : "[ ]"} ${t.title} (${t.priority})`;
    return li;
  }));
  counter.textContent = String(todos.filter((t) => !t.done).length);
  save(todos);
}
```

```
form.addEventListener("submit", (e: SubmitEvent) => {
  e.preventDefault();
  const title = titleInput.value.trim();
  if (!title) return;
  const priority = PRIORITIES.find((p) => p === prioritySelect.value) ?? "normal";
  todos = [...todos, { id: Math.max(0, ...todos.map((t) => t.id)) + 1, title, done: false, priority
}];
  titleInput.value = "";
  render();
  console.log(`added "${title}" (${priority}); pending: ${counter.textContent}`);
});

list.addEventListener("click", (e: MouseEvent) => {
  const li = e.target instanceof HTMLElement ? e.target.closest("li") : null;
  if (!li) return;
  todos = todos.map((t) => (t.id === Number(li.dataset.id) ? { ...t, done: !t.done } : t));
  render();
  const changed = todos.find((t) => t.id === Number(li.dataset.id));
  console.log(`marked "${changed?.title}" as ${changed?.done ? "done" : "pending"}; pending:
${counter.textContent}`);
  showState();
});

function showState(): void {
  console.log("--- list on screen ---\n" + [...list.querySelectorAll("li")].map((li) =>
li.textContent).join("\n"));
  console.log("saved in localStorage:", (JSON.parse(localStorage.getItem(KEY) ?? "[]") as
Todo[]).length, "todos");
}

async function start(): Promise<void> {
  const saved = restore();
  try {
    todos = saved ?? (await loadTodos("/api/todos"));
    console.log(saved ? "restored from localStorage" : `loaded ${todos.length} todos from the API`);
  } catch (e: unknown) {
    console.log("could not load todos:", e instanceof Error ? e.message : String(e));
  }
  render();
}
start();
```

BROWSER CONSOLE

```
skipped 1 invalid todo(s) from the API
loaded 2 todos from the API
added "Build the to-do app" (normal); pending: 2
marked "Revise generics" as done; pending: 1
--- list on screen ---
[x] Revise generics (high)
[x] Watch narrowing video (normal)
[ ] Build the to-do app (normal)
saved in localStorage: 3 todos
```

Simulated user actions, in order: waits 200 ms; types "Build the to-do app" into `#title`; clicks `button`; clicks `#list li[data-id="1"]`.

LEVEL 3

Advanced level

Outcome: build advanced and type-level types, use decorators, configure the compiler and tooling, apply TypeScript in React and Node.js, and migrate real projects.

6 MODULES · 18 DAYS MINIMUM

Module 14 · Advanced types	**4 days**
Module 15 · Type-level patterns and advanced techniques	**3 days**
Module 16 · Decorators	**2 days**
Module 17 · Compiler configuration and tooling	**2 days**
Module 18 · TypeScript with frameworks and back ends	**4 days**
Module 19 · Migration, projects and interview preparation	**3 days**

MODULE 14 · ADVANCED

Advanced types

Minimum duration **4 days** Level **Advanced**

These type operators let you create new types from existing ones; they are how libraries such as React, Zod, Prisma and tRPC achieve strong type safety, and how the utility types of Module 12 are built. This module covers keyof, typeof in type positions and indexed access types; conditional types, distribution, preventing distribution and infer; mapped types with modifiers and key remapping, rebuilding Partial, Pick and Record; template literal types for event names and routes, including inference; recursive types (a JSON type, DeepPartial, DeepReadonly); and variadic tuple types.

IN THIS MODULE

14.1 Type operators
14.2 Conditional types
14.3 Mapped types
14.4 Template literal types
14.5 Recursive types
14.6 Variadic tuple types

In this module you program at the **type level**: types are computed from other types, much as values are computed from values. Most examples use a small helper, `Expect<Equal<A, B>>`, that makes compilation fail if two types differ; this is the technique used by the type-challenges collection (19.3) and by libraries to test their types. When an example compiles and runs, every `Expect` in it has passed.

TYPE-LEVEL ASSERTIONS: EXPECT FAILS TO COMPILE IF A AND B DIFFER

TypeScript

```
// Type-level test helpers used in this module
type Equal<A, B> = (<T>() => T extends A ? 1 : 2) extends (<T>() => T extends B ? 1 : 2) ? true :
false;
type Expect<T extends true> = T;
```

14.1 Type operators

14.1.1 keyof

`keyof T` produces a **union of the property names** of an object type T, as string (or number and symbol) literal types. If T has an index signature, keyof gives the key type (`string`, or `string | number` for string index signatures, because numeric keys are converted to strings). keyof is the foundation of type-safe property access (10.3.2) and of mapped types (14.3).

KEYOF GIVES THE UNION OF AN OBJECT TYPE'S KEYS

main.ts

```
type Equal<A, B> = (<T>() => T extends A ? 1 : 2) extends (<T>() => T extends B ? 1 : 2) ? true :
false;
type Expect<T extends true> = T;
interface Course { code: string; title: string; fee: number }
type CourseKey = keyof Course;                              // "code" | "title" | "fee"
type DictKey = keyof { [k: string]: boolean };              // string | number
type ArrKey = keyof string[];                               // number | "length" | "push" | ... (all
array members)

type T1 = Expect<Equal<CourseKey, "code" | "title" | "fee">>;
type T2 = Expect<Equal<DictKey, string | number>>;

function get<T, K extends keyof T>(obj: T, key: K): T[K] { return obj[key]; }
const ts101: Course = { code: "TS101", title: "TypeScript", fee: 9999 };
const keys = Object.keys(ts101) as CourseKey[];             // Object.keys returns string[]
console.log(keys, get(ts101, "fee"));
```

OUTPUT

```
[ 'code', 'title', 'fee' ] 9999
```

NOTE

`Object.keys(obj)` returns `string[]`, not `(keyof T)[]`, on purpose: because typing is structural, an object may have **more** properties at run time than its type lists. Cast to `(keyof T)[]` only when you know the object has exactly those keys.

14.1.2 typeof in type positions

In a **type position** (after a colon, in a type alias), `typeof x` gives the **TypeScript type of the variable x**, not the JavaScript string that the runtime `typeof` returns. It lets you derive types from values: from a configuration object, a constant array (with `as const`), a function (for `ReturnType`) or a class (`typeof ClassName`). This "value first, type derived" style keeps one source of truth.

TYPEOF DERIVES TYPES FROM VALUES

main.ts

```
type Equal<A, B> = (<T>() => T extends A ? 1 : 2) extends (<T>() => T extends B ? 1 : 2) ? true :
false;
type Expect<T extends true> = T;
const config = {
  apiUrl: "https://api.lbf.in",
  retries: 3,
  features: { darkMode: true, beta: false },
};
type Config = typeof config;                       // { apiUrl: string; retries: number; features:
{...} }

const LEVELS = ["beginner", "intermediate", "advanced"] as const;
type Level = (typeof LEVELS)[number];              // "beginner" | "intermediate" | "advanced"

function makeUser(name: string, level: Level) { return { name, level, joined: new Date() }; }
type User = ReturnType<typeof makeUser>;           // { name: string; level: Level; joined: Date }

type T1 = Expect<Equal<Level, "beginner" | "intermediate" | "advanced">>;
type T2 = Expect<Equal<Config["features"], { darkMode: boolean; beta: boolean }>>;

const override: Partial<Config> = { retries: 5 };
const u: User = makeUser("Asha", "advanced");
console.log(typeof config, override, u.level);      // runtime typeof prints "object"
```

OUTPUT

```
object { retries: 5 } advanced
```

14.1.3 Indexed access types T["name"] and T[number]

An **indexed access type** looks up the type of a property: `Course["fee"]` is `number`. The index can be a union (`Course["code" | "fee"]` is `string | number`) or `keyof T` (all property types). On arrays and tuples, `T[number]` gives the element type, which is how `(typeof ARR)[number]` turns a constant array into a union. Indexed access works on nested types too: `Order["customer"]["address"]["city"]`.

INDEXED ACCESS TYPES, INCLUDING T[NUMBER] AND NESTED LOOKUPS main.ts

```ts
type Equal<A, B> = (<T>() => T extends A ? 1 : 2) extends (<T>() => T extends B ? 1 : 2) ? true :
false;
type Expect<T extends true> = T;
interface Order {
  id: number;
  customer: { name: string; address: { city: string; pin: string } };
  items: { sku: string; qty: number; price: number }[];
  status: "placed" | "shipped" | "delivered";
}

type Status = Order["status"];                       // "placed" | "shipped" | "delivered"
type City = Order["customer"]["address"]["city"];    // string
type Item = Order["items"][number];                  // { sku: string; qty: number; price: number }
type IdOrStatus = Order["id" | "status"];            // number | "placed" | "shipped" | "delivered"
type AnyValue = Order[keyof Order];                  // the union of all property types

type T1 = Expect<Equal<Item, { sku: string; qty: number; price: number }>>;
type T2 = Expect<Equal<IdOrStatus, number | Status>>;

const PRIORITY = [{ level: "low", sla: 72 }, { level: "high", sla: 4 }] as const;
type PriorityLevel = (typeof PRIORITY)[number]["level"];   // "low" | "high"

function lineTotal(item: Item): number { return item.qty * item.price; }
const next: Record<Status, Status | null> = { placed: "shipped", shipped: "delivered", delivered: null
};
const p: PriorityLevel = "high";
console.log(lineTotal({ sku: "NB", qty: 3, price: 60 }), next.placed, p);
```

OUTPUT

```
180 shipped high
```

14.2 Conditional types

14.2.1 T extends U ? X : Y

A **conditional type** chooses between two types based on assignability: `T extends U ? X : Y` reads "if T is assignable to U, then X, otherwise Y". It is the type-level `if`/ternary. Conditional types are mostly used inside generic types, where T is a type parameter, to compute different results for different inputs; with concrete types they are resolved immediately.



A CONDITIONAL TYPE IS AN IF/ELSE OVER TYPES

CONDITIONAL TYPES, INCLUDING A CHAIN AND A CONDITIONAL RETURN TYPE main.ts

```
type Equal<A, B> = (<T>() => T extends A ? 1 : 2) extends (<T>() => T extends B ? 1 : 2) ? true :
false;
type Expect<T extends true> = T;
type IsString<T> = T extends string ? "yes" : "no";
type A = IsString<"hello">;                 // "yes"
type B = IsString<42>;                      // "no"

type Description<T> =
  T extends string ? "text" :
  T extends number ? "number" :
  T extends boolean ? "flag" :
  T extends readonly unknown[] ? "list" :
  T extends (...args: any[]) => any ? "function" :
  "object";

type T1 = Expect<Equal<Description<string[]>, "list">>;
type T2 = Expect<Equal<Description<() => void>, "function">>;
type T3 = Expect<Equal<Description<{ id: 1 }>, "object">>;

// A conditional return type that depends on the argument type
function parseInput<T extends string | number>(value: T): T extends string ? number : string {
  return (typeof value === "string" ? Number(value) : String(value)) as T extends string ? number :
string;
}

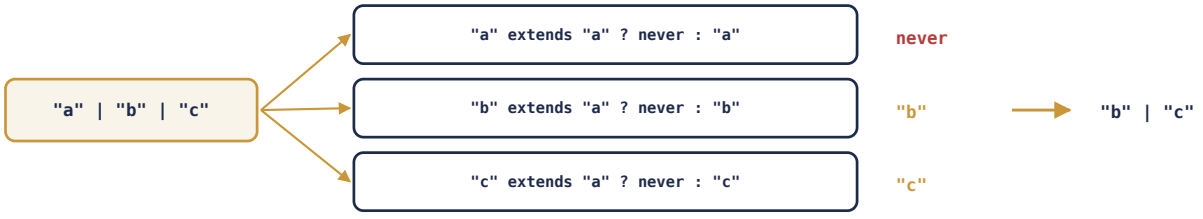
const n = parseInput("42");                 // number
const s = parseInput(42);                   // string
const a: A = "yes", b: B = "no";
console.log(n + 1, s.padStart(5, "0"), a, b);
```

OUTPUT

```
43 00042 yes no
```

14.2.2 Distributive conditional types

When the checked type is a **naked type parameter** and it is instantiated with a **union**, the conditional type is applied to **each member separately** and the results are combined into a union. This is called **distribution**, and it is what makes `Exclude`, `Extract` and `NonNullable` work: `Exclude<"a" | "b" | "c", "a">` evaluates `"a" extends "a" ? never : "a"`, then the same for "b" and "c", giving `never | "b" | "c"`, which simplifies to `"b" | "c"` (never disappears from unions).



DISTRIBUTION: MYEXCLUDE<"A" | "B" | "C", "A"> IS EVALUATED MEMBER BY MEMBER

REBUILDING EXCLUDE, EXTRACT AND NONNULLABLE WITH DISTRIBUTION main.ts

```
type Equal<A, B> = (<T>() => T extends A ? 1 : 2) extends (<T>() => T extends B ? 1 : 2) ? true :
false;
type Expect<T extends true> = T;
type MyExclude<T, U> = T extends U ? never : T;
type MyExtract<T, U> = T extends U ? T : never;
type MyNonNullable<T> = T extends null | undefined ? never : T;
type ToArray<T> = T extends unknown ? T[] : never;        // distributes over unions

type T1 = Expect<Equal<MyExclude<"a" | "b" | "c", "a">, "b" | "c">>;
type T2 = Expect<Equal<MyExtract<string | number | boolean, number | boolean>, number | boolean>>;
type T3 = Expect<Equal<MyNonNullable<string | null | undefined>, string>>;
type T4 = Expect<Equal<ToArray<string | number>, string[] | number[]>>;   // not (string | number)[]

type Events = { type: "click"; x: number } | { type: "key"; key: string } | { type: "scroll"; y:
number };
type EventByType<T extends Events["type"]> = Extract<Events, { type: T }>;
const e: EventByType<"key"> = { type: "key", key: "Enter" };
console.log("all type assertions passed:", e.key);
```

OUTPUT

```
all type assertions passed: Enter
```

14.2.3 Preventing distribution with [T]

Distribution is not always wanted. Wrapping both sides of `extends` in square brackets, `[T] extends [U]`, makes T no longer a naked type parameter, so the union is checked **as a whole**. A classic use is testing for `never`: since `never` is an empty union, a distributive conditional over it produces `never` rather than true or false.

DISTRIBUTION VERSUS [T] EXTENDS [U] main.ts

```
type Equal<A, B> = (<T>() => T extends A ? 1 : 2) extends (<T>() => T extends B ? 1 : 2) ? true :
false;
type Expect<T extends true> = T;
type ToArrayDist<T> = T extends unknown ? T[] : never;
type ToArrayWhole<T> = [T] extends [unknown] ? T[] : never;

type IsNeverWrong<T> = T extends never ? true : false;
type IsNever<T> = [T] extends [never] ? true : false;

type IsUnion<T, C = T> = T extends unknown ? ([C] extends [T] ? false : true) : never;

type T1 = Expect<Equal<ToArrayDist<string | number>, string[] | number[]>>;
type T2 = Expect<Equal<ToArrayWhole<string | number>, (string | number)[]>>;
type T3 = Expect<Equal<IsNeverWrong<never>, never>>;          // surprising: distributes over nothing
type T4 = Expect<Equal<IsNever<never>, true>>;
type T5 = Expect<Equal<IsUnion<"a" | "b">, true>>;
type T6 = Expect<Equal<IsUnion<"a">, false>>;
console.log("distribution assertions passed");
```

OUTPUT

```
distribution assertions passed
```

14.2.4 The infer keyword

Inside the `extends` clause of a conditional type, `infer X` declares a type variable that TypeScript fills in by **pattern matching**: "if T looks like `Promise<something>`, call that something X". The inferred variable is

available in the true branch. This is how `ReturnType`, `Parameters` and `Awaited` are written. `infer X extends C` adds a constraint to the inferred type.

PATTERN MATCHING WITH INFER

main.ts

```
type Equal<A, B> = (<T>() => T extends A ? 1 : 2) extends (<T>() => T extends B ? 1 : 2) ? true :
false;
type Expect<T extends true> = T;
type MyReturnType<F> = F extends (...args: any[]) => infer R ? R : never;
type MyParameters<F> = F extends (...args: infer P) => any ? P : never;
type UnwrapPromise<T> = T extends Promise<infer V> ? UnwrapPromise<V> : T;      // recursive
type ElementOf<T> = T extends readonly (infer E)[] ? E : never;
type FirstArg<F> = F extends (first: infer A, ...rest: any[]) => any ? A : never;
type PropType<T, K extends string> = T extends { [P in K]: infer V } ? V : never;
type NumericString<T> = T extends `${infer N extends number}` ? N : never;      // infer with a
constraint

function enrol(student: { name: string }, course: string, paid: boolean) { return { ok: true, course
}; }

type T1 = Expect<Equal<MyReturnType<typeof enrol>, { ok: boolean; course: string }>>;
type T2 = Expect<Equal<MyParameters<typeof enrol>, [student: { name: string }, course: string, paid:
boolean]>>;
type T3 = Expect<Equal<UnwrapPromise<Promise<Promise<number[]>>>, number[]>>;
type T4 = Expect<Equal<ElementOf<readonly ["a", "b"]>, "a" | "b">>;
type T5 = Expect<Equal<FirstArg<typeof enrol>, { name: string }>>;
type T6 = Expect<Equal<PropType<{ fee: number }, "fee">, number>>;
type T7 = Expect<Equal<NumericString<"42">, 42>>;
console.log("infer assertions passed");
```

OUTPUT

```
infer assertions passed
```

14.3 Mapped types

14.3.1 { [K in keyof T]: … }

A **mapped type** builds an object type by iterating over a union of keys: `{ [K in Keys]: SomeType }`. Most often the keys are `keyof T`, giving the type-level equivalent of `Object.fromEntries(Object.entries(obj).map(...))`: the same keys with transformed value types. Inside, `T[K]` refers to the original property type. Mapped types over `keyof T` are **homomorphic**: they preserve the optional and readonly modifiers of T.

MAPPED TYPES TRANSFORM EVERY PROPERTY main.ts

```
type Equal<A, B> = (<T>() => T extends A ? 1 : 2) extends (<T>() => T extends B ? 1 : 2) ? true :
false;
type Expect<T extends true> = T;
interface Course { code: string; fee: number; published: boolean }

type Stringify<T> = { [K in keyof T]: string };                // every value becomes a string
type Nullable<T> = { [K in keyof T]: T[K] | null };             // allow null for every property
type Validators<T> = { [K in keyof T]: (value: T[K]) => boolean };
type Flags<Keys extends string> = { [K in Keys]: boolean };      // keys from any union

type T1 = Expect<Equal<Stringify<Course>, { code: string; fee: string; published: string }>>;
type T2 = Expect<Equal<Nullable<Course>["fee"], number | null>>;

const rules: Validators<Course> = {
  code: (v) => /^[A-Z]{2}\d{3}$/.test(v),
  fee: (v) => v >= 0 && v <= 100_000,
  published: () => true,
};
const features: Flags<"darkMode" | "beta"> = { darkMode: true, beta: false };

const draft: Course = { code: "TS101", fee: -5, published: false };
const errors = (Object.keys(rules) as (keyof Course)[]).filter((k) => !(rules[k] as (v: unknown) =>
boolean)(draft[k]));
console.log("invalid fields:", errors, "| features:", features);
```

OUTPUT

```
invalid fields: [ 'fee' ] | features: { darkMode: true, beta: false }
```

14.3.2 Modifiers: adding and removing readonly and ?

Inside a mapped type you can add or remove the `readonly` and `?` modifiers: `readonly` or `+readonly` adds it, `-readonly` removes it; `?` or `+?` makes properties optional, `-?` makes them required (and removes `undefined` that came only from optionality). This is exactly how `Partial`, `Required` and `Readonly` are defined, and you can write the missing `Mutable` yourself.

ADDING AND REMOVING MODIFIERS: REBUILDING PARTIAL, REQUIRED AND READONLY main.ts

```
type Equal<A, B> = (<T>() => T extends A ? 1 : 2) extends (<T>() => T extends B ? 1 : 2) ? true :
false;
type Expect<T extends true> = T;
type MyPartial<T> = { [K in keyof T]?: T[K] };
type MyRequired<T> = { [K in keyof T]-?: T[K] };
type MyReadonly<T> = { readonly [K in keyof T]: T[K] };
type Mutable<T> = { -readonly [K in keyof T]: T[K] };            // not built in

interface Settings { readonly id: number; theme?: "light" | "dark"; fontSize: number }

type T1 = Expect<Equal<MyPartial<Settings>, Partial<Settings>>>;
type T2 = Expect<Equal<MyRequired<Settings>, Required<Settings>>>;
type T3 = Expect<Equal<MyReadonly<Settings>, Readonly<Settings>>>;
type T4 = Expect<Equal<Mutable<Settings>, { id: number; theme?: "light" | "dark"; fontSize: number
}>>;

const s: Mutable<Settings> = { id: 1, fontSize: 14 };
s.id = 2;                                                        // allowed: readonly removed
console.log(s);
```

OUTPUT

```
{ id: 2, fontSize: 14 }
```

14.3.3 Key remapping with as

An `as` clause in a mapped type (TypeScript 4.1+) **renames** or **filters** keys: `{ [K in keyof T as NewKey]: ... }`. The new key is usually built with a template literal type (`` `get${Capitalize<K & string>}` ``); mapping a key to `never` removes it, which gives a way to filter properties by their value type.

KEY REMAPPING: GETTERS, SETTERS AND FILTERING BY VALUE TYPE main.ts

```
type Equal<A, B> = (<T>() => T extends A ? 1 : 2) extends (<T>() => T extends B ? 1 : 2) ? true :
false;
type Expect<T extends true> = T;
interface Student { name: string; age: number; email: string; active: boolean }

type Getters<T> = { [K in keyof T as `get${Capitalize<K & string>}`]: () => T[K] };
type Setters<T> = { [K in keyof T as `set${Capitalize<K & string>}`]: (value: T[K]) => void };
type PickByValue<T, V> = { [K in keyof T as T[K] extends V ? K : never]: T[K] };
type OmitByKeyPrefix<T, P extends string> = { [K in keyof T as K extends `${P}${string}` ? never : K]:
T[K] };

type T1 = Expect<Equal<PickByValue<Student, string>, { name: string; email: string }>>;
type T2 = Expect<Equal<keyof Getters<Student>, "getName" | "getAge" | "getEmail" | "getActive">>;
type T3 = Expect<Equal<OmitByKeyPrefix<{ _secret: 1; _id: 2; name: 3 }, "_">, { name: 3 }>>;

function withAccessors<T extends object>(obj: T): T & Getters<T> & Setters<T> {
  const out: any = { ...obj };
  for (const key of Object.keys(obj)) {
    const cap = key[0].toUpperCase() + key.slice(1);
    out[`get${cap}`] = () => out[key];
    out[`set${cap}`] = (v: unknown) => { out[key] = v; };
  }
  return out;
}

const st = withAccessors<Student>({ name: "Asha", age: 20, email: "asha@lbf.in", active: true });
st.setAge(21);
console.log(st.getName(), st.getAge(), st.getActive());
```

OUTPUT

```
Asha 21 true
```

14.3.4 Rebuilding Partial, Pick and Record yourself

With mapped types, conditional types and keyof you can now read the definitions of the utility types in `lib.es5.d.ts`, and write stricter or extended versions. For example, the built-in `Omit` accepts any string as a key (12.2.2); a `StrictOmit` constrains the keys to `keyof T`.

UTILITY TYPES REBUILT FROM MAPPED AND CONDITIONAL TYPES main.ts

```
type Equal<A, B> = (<T>() => T extends A ? 1 : 2) extends (<T>() => T extends B ? 1 : 2) ? true :
false;
type Expect<T extends true> = T;
type MyPick<T, K extends keyof T> = { [P in K]: T[P] };
type MyRecord<K extends keyof any, V> = { [P in K]: V };
type MyOmit<T, K extends keyof any> = MyPick<T, Exclude<keyof T, K>>;          // like the built-in
Omit
type StrictOmit<T, K extends keyof T> = MyPick<T, Exclude<keyof T, K>>;       // keys must exist
type PartialBy<T, K extends keyof T> = Omit<T, K> & Partial<Pick<T, K>>;       // make only some keys
optional

interface User { id: number; name: string; email: string; role: "admin" | "student" }

type T1 = Expect<Equal<MyPick<User, "id" | "name">, Pick<User, "id" | "name">>>;
type T2 = Expect<Equal<MyRecord<"a" | "b", number>, Record<"a" | "b", number>>>;
type T3 = Expect<Equal<MyOmit<User, "email">, Omit<User, "email">>>;

type NewUser = PartialBy<User, "id" | "role">;      // id and role optional, the rest required
const u: NewUser = { name: "Asha", email: "asha@lbf.in" };
const typo: MyOmit<User, "emial"> = { id: 1, name: "R", email: "r@lbf.in", role: "student" };   //
Omit ignores the typo
console.log(u, typo.email);
```

OUTPUT

```
{ name: 'Asha', email: 'asha@lbf.in' } r@lbf.in
```

STRICTOMIT CATCHES MISSPELLED KEYS THAT THE BUILT-IN OMIT IGNORES main.ts

```
type StrictOmit<T, K extends keyof T> = Pick<T, Exclude<keyof T, K>>;
interface User { id: number; name: string; email: string }
type Bad = StrictOmit<User, "emial">;
```

COMPILER ERRORS (tsc --strict)

```
main.ts(3,29): error TS2344: Type '"emial"' does not satisfy the constraint 'keyof User'.
```

14.4 Template literal types

14.4.1 Building string types from other types

Template literal types (TypeScript 4.1+) use JavaScript's template string syntax at the type level: `` `Hello ${Name}` `` where `Name` is a type. When an interpolated type is a union, the result is the **cross product** of all combinations, so a few small unions describe a large set of valid strings precisely. Combined with the intrinsic string types (12.5.2) they can transform names.

TEMPLATE LITERAL TYPES BUILD EXACT STRING PATTERNS main.ts

```
type Equal<A, B> = (<T>() => T extends A ? 1 : 2) extends (<T>() => T extends B ? 1 : 2) ? true :
false;
type Expect<T extends true> = T;
type Size = "sm" | "md" | "lg";
type Colour = "navy" | "gold";
type ButtonClass = `btn-${Colour}-${Size}`;                // 2 x 3 = 6 combinations
type CssUnit = `${number}${"px" | "rem" | "%"}`;           // "12px", "1.5rem", "50%" ...
type Version = `v${number}.${number}.${number}`;

type T1 = Expect<Equal<ButtonClass, "btn-navy-sm" | "btn-navy-md" | "btn-navy-lg" | "btn-gold-sm" |
"btn-gold-md" | "btn-gold-lg">>;

function button(label: string, cls: ButtonClass, width: CssUnit): string {
  return `<button class="${cls}" style="width:${width}">${label}</button>`;
}
const v: Version = "v6.0.3";
console.log(button("Enrol", "btn-gold-lg", "12rem"), v);
```

OUTPUT

```
<button class="btn-gold-lg" style="width:12rem">Enrol</button> v6.0.3
```

STRINGS OUTSIDE THE PATTERN ARE REJECTED main.ts

```
type Colour = "navy" | "gold";
type Size = "sm" | "md" | "lg";
type ButtonClass = `btn-${Colour}-${Size}`;
type CssUnit = `${number}${"px" | "rem" | "%"}`;

const a: ButtonClass = "btn-red-sm";
const b: ButtonClass = "btn-navy-xl";
const c: CssUnit = "12em";
const d: CssUnit = "wide%";
```

COMPILER ERRORS (tsc --strict)

```
main.ts(6,7): error TS2322: Type '"btn-red-sm"' is not assignable to type '"btn-navy-sm" | "btn-navy-
md" | "btn-navy-lg" | "btn-gold-sm" | "btn-gold-md" | "btn-gold-lg"'.
main.ts(7,7): error TS2820: Type '"btn-navy-xl"' is not assignable to type '"btn-navy-sm" | "btn-navy-
md" | "btn-navy-lg" | "btn-gold-sm" | "btn-gold-md" | "btn-gold-lg"'. Did you mean '"btn-navy-lg"'?
main.ts(8,7): error TS2322: Type '"12em"' is not assignable to type '`${number}px` | `${number}rem` |
`${number}%`'.
main.ts(9,7): error TS2322: Type '"wide%"' is not assignable to type '`${number}px` | `${number}rem` |
`${number}%`'.
```

14.4.2 Event name and route patterns

Template literal types shine for **naming conventions** that libraries rely on: event handler names (`on${Capitalize<Event>}`), change events derived from property names (`${Prop}Changed`), CSS variables, and URL routes. A typed event emitter or router can then accept only valid names and give each handler the right argument types.

A TYPED OBSERVABLE WHOSE EVENT NAMES COME FROM PROPERTY NAMES

main.ts

```
type Equal<A, B> = (<T>() => T extends A ? 1 : 2) extends (<T>() => T extends B ? 1 : 2) ? true :
false;
type Expect<T extends true> = T;
// Events derived from a model's properties: "nameChanged", "feeChanged", ...
type Course = { name: string; fee: number; published: boolean };
type ChangeEvents<T> = { [K in keyof T as `${K & string}Changed`]: (newValue: T[K], oldValue: T[K]) =>
void };

class Observable<T extends object> {
  #listeners: Partial<Record<string, ((n: any, o: any) => void)[]>> = {};
  constructor(private state: T) {}
  on<E extends keyof ChangeEvents<T> & string>(event: E, handler: ChangeEvents<T>[E]): void {
    (this.#listeners[event] ??= []).push(handler as (n: any, o: any) => void);
  }
  set<K extends keyof T & string>(key: K, value: T[K]): void {
    const old = this.state[key];
    this.state[key] = value;
    this.#listeners[`${key}Changed`]?.forEach((h) => h(value, old));
  }
}

const course = new Observable<Course>({ name: "TypeScript", fee: 9999, published: false });
course.on("feeChanged", (now, before) => console.log(`fee ${before} -> ${now} (${now < before ?
"discount" : "increase"})`));
course.on("publishedChanged", (now) => console.log("published:", now));
course.set("fee", 8999);
course.set("published", true);

type T1 = Expect<Equal<keyof ChangeEvents<Course>, "nameChanged" | "feeChanged" |
"publishedChanged">>;
```

OUTPUT

```
fee 9999 -> 8999 (discount)
published: true
```

14.4.3 Inference with template literals

`infer` works inside template literal types, so you can **parse strings at the type level**: split `"user.address.city"` on dots, extract `:id` parameters from a route such as `"/courses/:courseId/batches/:batchId"`, or strip a prefix. Typed routers (Express types, React Router, tRPC, Hono) use exactly this technique to give route handlers a typed `params` object.

PARSING ROUTE PARAMETERS AND DOTTED PATHS AT THE TYPE LEVEL main.ts

```
type Equal<A, B> = (<T>() => T extends A ? 1 : 2) extends (<T>() => T extends B ? 1 : 2) ? true :
false;
type Expect<T extends true> = T;
type RouteParams<Path extends string> =
  Path extends `${string}:${infer Param}/${infer Rest}`
    ? { [K in Param | keyof RouteParams<`/${Rest}`>]: string }
    : Path extends `${string}:${infer Param}`
      ? { [K in Param]: string }
      : {};

type Split<S extends string, D extends string> = S extends `${infer Head}${D}${infer Tail}` ? [Head,
...Split<Tail, D>] : [S];
type TrimPrefix<S extends string, P extends string> = S extends `${P}${infer Rest}` ? Rest : S;

type T1 = Expect<Equal<RouteParams<"/courses/:courseId/batches/:batchId">, { courseId: string;
batchId: string }>>;
type T2 = Expect<Equal<Split<"user.address.city", ".">, ["user", "address", "city"]>>;
type T3 = Expect<Equal<TrimPrefix<"VITE_API_URL", "VITE_">, "API_URL">>;

function route<P extends string>(path: P, handler: (params: RouteParams<P>) => string) {
  return (url: string) => {
    const names = [...path.matchAll(/:(\w+)/g)].map((m) => m[1]);
    const pattern = new RegExp("^" + path.replace(/:\w+/g, "([^/]+)") + "$");
    const match = url.match(pattern);
    if (!match) return "404";
    return handler(Object.fromEntries(names.map((n, i) => [n, match[i + 1]]))) as RouteParams<P>);
  };
}

const showBatch = route("/courses/:courseId/batches/:batchId", (p) => `course ${p.courseId}, batch
${p.batchId}`);
console.log(showBatch("/courses/TS101/batches/12"), "|", showBatch("/about"));
```

OUTPUT

```
course TS101, batch 12 | 404
```

THE HANDLER CAN ONLY USE PARAMETERS THAT APPEAR IN THE ROUTE main.ts

```
type RouteParams<Path extends string> =
  Path extends `${string}:${infer Param}/${infer Rest}` ? { [K in Param | keyof
RouteParams<`/${Rest}`>]: string }
  : Path extends `${string}:${infer Param}` ? { [K in Param]: string } : {};

declare function route<P extends string>(path: P, handler: (params: RouteParams<P>) => string): void;
route("/courses/:courseId", (p) => p.batchId);
```

COMPILER ERRORS (tsc --strict)

```
main.ts(6,38): error TS2339: Property 'batchId' does not exist on type '{ courseId: string; }'.
```

14.5 Recursive types

A type alias can refer to **itself**, directly or through other types, to describe recursive data such as trees, nested comments, menus, file systems and JSON. Recursive conditional and mapped types also allow deep transformations. TypeScript limits recursion depth (around 50 levels for conditional types, more with tail-recursive patterns) to keep checking fast.

14.5.1 JSON value type

A RECURSIVE JSON TYPE AND A RECURSIVE MENU INTERFACE main.ts

```
type JsonPrimitive = string | number | boolean | null;
type JsonValue = JsonPrimitive | JsonValue[] | { [key: string]: JsonValue };

const config: JsonValue = {
  app: "LBF portal",
  version: 3,
  features: ["courses", "payments", { name: "beta", enabled: false }],
  limits: { maxUsers: 5000, regions: ["IN", "AE"], fallback: null },
};

function countLeaves(v: JsonValue): number {
  if (v === null || typeof v !== "object") return 1;
  const children = Array.isArray(v) ? v : Object.values(v);
  return children.reduce((sum: number, c) => sum + countLeaves(c), 0);
}

interface MenuItem { label: string; href?: string; children?: MenuItem[] }   // a recursive interface
const menu: MenuItem[] = [{ label: "Courses", children: [{ label: "TypeScript", href: "/ts" }, {
label: "Python", href: "/py" }] }, { label: "Contact", href: "/contact" }];
const flatten = (items: MenuItem[], depth = 0): string[] =>
  items.flatMap((i) => [`${"  ".repeat(depth)}- ${i.label}`, ...flatten(i.children ?? [], depth +
1)]);

console.log("leaf values in config:", countLeaves(config));
console.log(flatten(menu).join("\n"));
```

OUTPUT

```
leaf values in config: 10
- Courses
  - TypeScript
  - Python
- Contact
```

DATES AND FUNCTIONS ARE NOT VALID JSON VALUES main.ts

```
type JsonValue = string | number | boolean | null | JsonValue[] | { [key: string]: JsonValue };
const bad: JsonValue = { created: new Date(), handler: () => 1 };
```

COMPILER ERRORS (tsc --strict)

```
main.ts(2,7): error TS2322: Type '{ created: Date; handler: () => number; }' is not assignable to type
'JsonValue'.
  Types of property 'created' are incompatible.
    Type 'Date' is not assignable to type 'JsonValue | undefined'.
      Type 'Date' is not assignable to type '{ [key: string]: JsonValue; }'.
        Index signature for type 'string' is missing in type 'Date'.
```

14.5.2 DeepPartial and DeepReadonly

The built-in `Partial` and `Readonly` are shallow. Recursive mapped types apply the transformation at every level. Arrays and functions need special handling: functions should be left alone, and arrays should become readonly arrays of transformed elements.

DEEPREADONLY AND DEEPPARTIAL, USED TO MERGE CONFIGURATION

main.ts

```
type Equal<A, B> = (<T>() => T extends A ? 1 : 2) extends (<T>() => T extends B ? 1 : 2) ? true :
false;
type Expect<T extends true> = T;
type DeepReadonly<T> =
  T extends (...args: any[]) => any ? T :
  T extends readonly (infer E)[] ? readonly DeepReadonly<E>[] :
  T extends object ? { readonly [K in keyof T]: DeepReadonly<T[K]> } :
  T;

type DeepPartial<T> =
  T extends (...args: any[]) => any ? T :
  T extends readonly (infer E)[] ? DeepPartial<E>[] :
  T extends object ? { [K in keyof T]?: DeepPartial<T[K]> } :
  T;

interface AppConfig {
  server: { host: string; port: number; tls: { enabled: boolean; cert?: string } };
  features: { name: string; enabled: boolean }[];
}

type T1 = Expect<Equal<DeepReadonly<AppConfig>["server"]["tls"], { readonly enabled: boolean; readonly
cert?: string }>>;

function merge<T>(base: T, patch: DeepPartial<T>): T {
  if (typeof base !== "object" || base === null || Array.isArray(base)) return (patch ?? base) as T;
  const out: any = { ...base };
  for (const [k, v] of Object.entries(patch as object)) out[k] = v !== undefined && typeof v ===
"object" && !Array.isArray(v) ? merge(out[k], v) : v;
  return out;
}

const defaults: DeepReadonly<AppConfig> = { server: { host: "localhost", port: 3000, tls: { enabled:
false } }, features: [] };
const prod = merge<AppConfig>(defaults as AppConfig, { server: { host: "api.lbf.in", tls: { enabled:
true } } });
console.log(JSON.stringify(prod));
```

OUTPUT

```
{"server":{"host":"api.lbf.in","port":3000,"tls":{"enabled":true}},"features":[]}
```

DEEPREADONLY BLOCKS MUTATION AT EVERY LEVEL

main.ts

```
type DeepReadonly<T> = T extends (...args: any[]) => any ? T : T extends readonly (infer E)[] ?
readonly DeepReadonly<E>[] : T extends object ? { readonly [K in keyof T]: DeepReadonly<T[K]> } : T;
interface AppConfig { server: { host: string; tls: { enabled: boolean } }; features: string[] }
declare const cfg: DeepReadonly<AppConfig>;

cfg.server.tls.enabled = true;
cfg.features.push("beta");
```

COMPILER ERRORS (tsc --strict)

```
main.ts(5,16): error TS2540: Cannot assign to 'enabled' because it is a read-only property.
main.ts(6,14): error TS2339: Property 'push' does not exist on type 'readonly string[]'.
```

14.6 Variadic tuple types

Variadic tuple types (TypeScript 4.0+) let a tuple type contain a **spread of a generic tuple**: `[...T, ...U]`. This makes it possible to type functions that concatenate tuples, add or remove elements at either end, or pass through an unknown list of arguments while adding or removing some, such as `bind`-style partial application, middleware and `Promise.all`. Combined with labelled elements (3.5.3), the resulting signatures stay readable in the editor.

VARIADIC TUPLES: CONCAT, TAIL, LAST AND TYPED PARTIAL APPLICATION main.ts

```
type Equal<A, B> = (<T>() => T extends A ? 1 : 2) extends (<T>() => T extends B ? 1 : 2) ? true :
false;
type Expect<T extends true> = T;
type Concat<A extends readonly unknown[], B extends readonly unknown[]> = [...A, ...B];
type Push<T extends readonly unknown[], V> = [...T, V];
type Tail<T extends readonly unknown[]> = T extends readonly [unknown, ...infer Rest] ? Rest : [];
type Last<T extends readonly unknown[]> = T extends readonly [...unknown[], infer L] ? L : never;

type T1 = Expect<Equal<Concat<[1, 2], ["a"]>, [1, 2, "a"]>>;
type T2 = Expect<Equal<Tail<[string, number, boolean]>, [number, boolean]>>;
type T3 = Expect<Equal<Last<[1, 2, 3]>, 3>>;

function concat<A extends unknown[], B extends unknown[]>(a: [...A], b: [...B]): [...A, ...B] {
  return [...a, ...b];
}

// Partial application: fix the first arguments, return a function taking the rest
function partial<Fixed extends unknown[], Rest extends unknown[], R>(fn: (...args: [...Fixed,
...Rest]) => R, ...fixed: Fixed) {
  return (...rest: Rest): R => fn(...fixed, ...rest);
}

function fee(course: string, base: number, gstPct: number, discountPct: number): string {
  return `${course}: Rs ${Math.round(base * (1 + gstPct / 100) * (1 - discountPct / 100))}`;
}

const tuple = concat([1, "two"], [true, 4n]);          // [number, string, boolean, bigint]
const tsFee = partial(fee, "TypeScript", 9999);        // (gstPct: number, discountPct: number) =>
string
console.log(tuple, tsFee(18, 10), tsFee(18, 0));
```

OUTPUT

```
[ 1, 'two', true, 4n ] TypeScript: Rs 10619 TypeScript: Rs 11799
```

THE REMAINING PARAMETERS ARE CHECKED PRECISELY

main.ts

```
function partial<Fixed extends unknown[], Rest extends unknown[], R>(fn: (...args: [...Fixed,
...Rest]) => R, ...fixed: Fixed) {
  return (...rest: Rest): R => fn(...fixed, ...rest);
}
function fee(course: string, base: number, gstPct: number, discountPct: number) { return course; }

const tsFee = partial(fee, "TypeScript", 9999);
tsFee(18);                      // missing discountPct
tsFee(18, "10");                // wrong type
```

COMPILER ERRORS (tsc --strict)

```
main.ts(7,1): error TS2554: Expected 2 arguments, but got 1.
main.ts(8,11): error TS2345: Argument of type 'string' is not assignable to parameter of type
'number'.
```

keyof T gives a union of keys, typeof x in a type position gives a value's type, and T[K] / T[number] look up property and element types. Conditional types (T extends U ? X : Y) choose types; with a naked type parameter they distribute over unions (Exclude, Extract), [T] extends [U] prevents distribution, and infer extracts parts by pattern matching (ReturnType, Awaited). Mapped types ({ [K in keyof T]: … }) transform every property, add or remove readonly and ? with + and -, and rename or filter keys with as; they define Partial, Required, Readonly, Pick and Record. Template literal types build and, with infer, parse string patterns such as class names, event names and route parameters. Recursive types describe JSON, trees and deep transformations such as DeepReadonly and DeepPartial. Variadic tuple types ([…A, …B]) type concatenation and partial application. Test type-level code with Expect.

PRACTICE CHECKPOINT

Implement your own **MyPartial**, **MyPick**, **MyReturnType**, **DeepReadonly** and a **Getters<T>** type that turns `{ name: string }` into `{ getName(): string }`.

SAMPLE SOLUTION: FIVE TYPE-LEVEL UTILITIES WITH TESTS AND USAGE

main.ts

```
type Equal<A, B> = (<T>() => T extends A ? 1 : 2) extends (<T>() => T extends B ? 1 : 2) ? true :
false;
type Expect<T extends true> = T;
// ---------- the five types
type MyPartial<T> = { [K in keyof T]?: T[K] };
type MyPick<T, K extends keyof T> = { [P in K]: T[P] };
type MyReturnType<F extends (...args: any) => any> = F extends (...args: any) => infer R ? R : never;
type DeepReadonly<T> =
  T extends (...args: any[]) => any ? T :
  T extends readonly (infer E)[] ? readonly DeepReadonly<E>[] :
  T extends object ? { readonly [K in keyof T]: DeepReadonly<T[K]> } :
  T;
type Getters<T> = { [K in keyof T as `get${Capitalize<K & string>}`]: () => T[K] };

// ---------- type-level tests (compilation fails if any is wrong)
interface Student { name: string; age: number; address: { city: string; tags: string[] }; greet():
string }

type T1 = Expect<Equal<MyPartial<Student>, Partial<Student>>>;
type T2 = Expect<Equal<MyPick<Student, "name" | "age">, { name: string; age: number }>>;
type T3 = Expect<Equal<MyReturnType<() => Promise<number>>, Promise<number>>>;
type T4 = Expect<Equal<MyReturnType<typeof JSON.parse>, any>>;
type T5 = Expect<Equal<DeepReadonly<Student>["address"], { readonly city: string; readonly tags:
readonly string[] }>>;
```

```
type T6 = Expect<Equal<Getters<{ name: string }>, { getName: () => string }>>;
type T7 = Expect<Equal<keyof Getters<Student>, "getName" | "getAge" | "getAddress" | "getGreet">>;

// ---------- the types in use
function toGetters<T extends object>(obj: T): Getters<T> {
  const out: Record<string, () => unknown> = {};
  for (const [k, v] of Object.entries(obj)) out[`get${k[0].toUpperCase()}${k.slice(1)}`] = () => v;
  return out as Getters<T>;
}

function freeze<T>(obj: T): DeepReadonly<T> {
  if (obj && typeof obj === "object") {
    Object.values(obj).forEach((v) => freeze(v));
    Object.freeze(obj);
  }
  return obj as DeepReadonly<T>;
}

const student: Student = { name: "Asha", age: 20, address: { city: "Mysuru", tags: ["hostel"] },
greet() { return `Hi, ${this.name}`; } };
const getters = toGetters({ name: student.name, age: student.age });
const frozen = freeze(structuredClone({ name: student.name, address: student.address }));
const update: MyPartial<Student> = { age: 21 };
const card: MyPick<Student, "name" | "age"> = { name: student.name, age: update.age ?? student.age };
let greeting: MyReturnType<Student["greet"]> = student.greet();

console.log(getters.getName(), getters.getAge(), card, greeting);
try {
  (frozen.address.tags as string[]).push("day-scholar");    // DeepReadonly forbids this at compile
time; freeze at run time
} catch (e) {
  console.log("run-time freeze:", (e as Error).message);
}
```

OUTPUT

```
Asha 20 { name: 'Asha', age: 21 } Hi, Asha
run-time freeze: Cannot add property 1, object is not extensible
```

MODULE 15 · ADVANCED

Type-level patterns and advanced techniques

Minimum duration **3 days** | Level **Advanced**

This module covers the patterns used in professional codebases to make illegal states unrepresentable and to keep types honest at the boundaries of a program. It covers controlling literal inference with as const, the satisfies operator and NoInfer; branded (nominal) types for IDs and currency; modelling state with discriminated unions; Result and Option types for error handling; runtime validation with Zod and inferring types from schemas; typing environment variables; variance (covariance, contravariance, in and out annotations, strictFunctionTypes); declaration merging and augmentation in depth; and keeping type checking fast.

IN THIS MODULE

15.1 Literal and inference control
15.2 Modelling data safely
15.3 Type safety at boundaries
15.4 Variance
15.5 Declaration merging and augmentation in depth
15.6 Type performance

15.1 Literal and inference control

Much of advanced TypeScript is about controlling **how wide** an inferred type is. Too wide (`string` instead of `"GET"`) loses information; too narrow, or annotated with a broad type, loses the precise knowledge the compiler had. Three tools give fine control: `as const`, `satisfies` and `NoInfer`.

15.1.1 as const assertions

`as const` (6.3.3) infers the narrowest possible type for a literal expression: literal types for every value, `readonly` properties, and readonly tuples for arrays. It is the basis of the "values first, types derived" style: define data once as a constant, and derive unions and lookup types from it, so adding a value updates the types automatically.

DERIVING UNION TYPES FROM A CONSTANT ARRAY

main.ts

```
const COURSES = [
  { code: "TS101", title: "TypeScript", level: "advanced", fee: 9999 },
  { code: "PY201", title: "Python", level: "beginner", fee: 14999 },
  { code: "JV301", title: "Java", level: "intermediate", fee: 12999 },
] as const;

type Course = (typeof COURSES)[number];
type CourseCode = Course["code"];                 // "TS101" | "PY201" | "JV301"
type Level = Course["level"];                     // "advanced" | "beginner" | "intermediate"

function feeOf(code: CourseCode): number {
  return COURSES.find((c) => c.code === code)!.fee;
}

const byLevel = {} as Record<Level, CourseCode[]>;
for (const c of COURSES) (byLevel[c.level] ??= []).push(c.code);
console.log(feeOf("PY201"), byLevel, COURSES.length);
```

OUTPUT

```
14999 {
  advanced: [ 'TS101' ],
  beginner: [ 'PY201' ],
  intermediate: [ 'JV301' ]
} 3
```

15.1.2 The satisfies operator

Annotating a variable (`const routes: Record<string, Route> = {...}`) checks the value, but then the variable **has** the broad annotated type: the specific keys and literal values are forgotten. Not annotating keeps the precise inferred type but checks nothing. The `satisfies` operator (TypeScript 4.9+) gives both: `expr satisfies Type` checks that the expression conforms to `Type` (including excess property checks), but the expression keeps its own, more precise inferred type. It is ideal for configuration objects, lookup tables and theme definitions. Combined with `as const` (`as const satisfies T`) it keeps literal types too.



ANNOTATION, INFERENCE AND SATISFIES COMPARED

SATISFIES CHECKS A VALUE BUT KEEPS ITS PRECISE TYPE main.ts

```
type Colour = string | [r: number, g: number, b: number];
type Theme = Record<"primary" | "accent" | "text", Colour>;

const annotated: Theme = { primary: "#1E2A4A", accent: [201, 151, 58], text: "#222" };
const checked = { primary: "#1E2A4A", accent: [201, 151, 58], text: "#222" } satisfies Theme;

// annotated.accent is Colour (string | tuple): must narrow before use
const a = annotated.accent;
console.log(typeof a === "string" ? a : a.join(","));

// checked.accent is known to be the tuple, checked.primary a string: no narrowing needed
console.log(checked.accent.map((v) => v.toString(16)).join(""), checked.primary.toLowerCase());

const ROUTES = {
  home: { path: "/", auth: false },
  dashboard: { path: "/dashboard", auth: true },
} as const satisfies Record<string, { path: `/${string}`; auth: boolean }>;
type RouteName = keyof typeof ROUTES;                    // "home" | "dashboard", not string
const go = (r: RouteName) => ROUTES[r].path;
console.log(go("dashboard"));
```

OUTPUT

```
201,151,58
c9973a #1e2a4a
/dashboard
```

SATISFIES REPORTS MISSING KEYS, EXTRA KEYS AND WRONG VALUES main.ts

```
type Theme = Record<"primary" | "accent" | "text", string>;

const theme = { primary: "#1E2A4A", accent: "#C9973A" } satisfies Theme;          // missing text
const theme2 = { primary: "#1E2A4A", accent: "#C9973A", text: "#222", bg: "#fff" } satisfies Theme;
// extra key
const ROUTES = { home: { path: "home" } } satisfies Record<string, { path: `/${string}` }>;
// bad path
```

COMPILER ERRORS (tsc --strict)

```
main.ts(3,57): error TS1360: Type '{ primary: string; accent: string; }' does not satisfy the expected
type 'Theme'.
  Property 'text' is missing in type '{ primary: string; accent: string; }' but required in type
'Theme'.
main.ts(4,71): error TS2353: Object literal may only specify known properties, and 'bg' does not exist
in type 'Theme'.
main.ts(5,26): error TS2322: Type '"home"' is not assignable to type '`/${string}`'.
```

15.1.3 NoInfer

When a generic function infers a type parameter from several arguments, a "secondary" argument (a default, a fallback, an initial selection) can widen the inference and let a mistake through. `NoInfer<T>` (12.6.1) marks such positions so that `T` is inferred only from the primary arguments and the secondary ones are **checked** against it. Typical uses: default values, the initial state of a state machine, and a selected item that must be one of the options.

NOINFER KEEPS THE INITIAL STATE AND TRANSITIONS CHECKED AGAINST THE DECLARED STATES main.ts

```
function createMachine<S extends string>(config: { states: S[]; initial: NoInfer<S>; transitions:
Partial<Record<S, NoInfer<S>[]>> }) {
  let current: S = config.initial;
  return {
    get state() { return current; },
    send(to: S) {
      if (!config.transitions[current]?.includes(to)) return `cannot go from ${current} to ${to}`;
      current = to;
      return `now ${current}`;
    },
  };
}

const order = createMachine({
  states: ["cart", "paid", "shipped", "cancelled"],
  initial: "cart",
  transitions: { cart: ["paid", "cancelled"], paid: ["shipped", "cancelled"] },
});

console.log(order.send("shipped"), "|", order.send("paid"), "|", order.send("shipped"), "|",
order.state);
```

OUTPUT

```
cannot go from cart to shipped | now paid | now shipped | shipped
```

WITHOUT NOINFER THESE TYPOS WOULD SILENTLY BECOME NEW STATES main.ts

```
function createMachine<S extends string>(config: { states: S[]; initial: NoInfer<S>; transitions:
Partial<Record<S, NoInfer<S>[]>> }) { return config; }

createMachine({
  states: ["cart", "paid", "shipped"],
  initial: "checkout",                                  // not a declared state
  transitions: { cart: ["paid"], paid: ["delivered"] }, // "delivered" is not a state
});
```

COMPILER ERRORS (tsc --strict)

```
main.ts(5,3): error TS2322: Type '"checkout"' is not assignable to type '"cart" | "paid" | "shipped"'.
main.ts(6,41): error TS2322: Type '"delivered"' is not assignable to type '"cart" | "paid" |
"shipped"'.
```

15.2 Modelling data safely

15.2.1 Branded (nominal) types for IDs and currency

Structural typing (5.7) means that `type UserId = number` and `type OrderId = number` are the same type: passing an order ID where a user ID is expected compiles. **Branded types** add a phantom property that exists only at the type level, making otherwise identical types incompatible: `type UserId = number & { readonly __brand: "UserId" }`. Values are branded in one place, usually a constructor function that validates the raw value, and are plain numbers or strings at run time. Use brands for identifiers, currencies and units, validated strings (`Email`, `Slug`) and sanitised HTML.

BRANDED IDS, CURRENCY AND VALIDATED EMAILS

main.ts

```ts
declare const brand: unique symbol;
type Brand<T, B extends string> = T & { readonly [brand]: B };

type UserId = Brand<number, "UserId">;
type OrderId = Brand<number, "OrderId">;
type Rupees = Brand<number, "Rupees">;
type Email = Brand<string, "Email">;

const UserId = (n: number) => n as UserId;              // the only places that create branded values
const OrderId = (n: number) => n as OrderId;
const rupees = (n: number) => Math.round(n * 100) / 100 as Rupees;
function Email(s: string): Email {
  if (!/^[^@\s]+@[^@\s]+\.[a-z]{2,}$/i.test(s)) throw new Error(`invalid email: ${s}`);
  return s.toLowerCase() as Email;
}

function findOrders(user: UserId): OrderId[] { return [OrderId(9001), OrderId(9002)]; }
function refund(order: OrderId, amount: Rupees, notify: Email): string { return `refund Rs ${amount}
on #${order}, mail ${notify}`; }
const addRupees = (a: Rupees, b: Rupees) => rupees(a + b);

const me = UserId(7);
const [first] = findOrders(me);
console.log(refund(first, addRupees(rupees(499.5), rupees(100)), Email("Asha@LBF.in")));
console.log(typeof me, me + 1);                          // still a plain number at run time
try { Email("not-an-email"); } catch (e) { console.log((e as Error).message); }
```

OUTPUT

```
refund Rs 599.5 on #9001, mail asha@lbf.in
number 8
invalid email: not-an-email
```

BRANDS STOP IDS AND AMOUNTS FROM BEING MIXED UP

main.ts

```ts
declare const brand: unique symbol;
type Brand<T, B extends string> = T & { readonly [brand]: B };
type UserId = Brand<number, "UserId">;
type OrderId = Brand<number, "OrderId">;
type Rupees = Brand<number, "Rupees">;

declare function findOrders(user: UserId): OrderId[];
declare const order: OrderId;
declare const price: Rupees;

findOrders(order);              // an OrderId is not a UserId
findOrders(42);                 // a raw number is not a UserId
const total: Rupees = price * 2; // arithmetic returns a plain number
```

COMPILER ERRORS (tsc --strict)

```
main.ts(11,12): error TS2345: Argument of type 'OrderId' is not assignable to parameter of type
'UserId'.
  Type 'OrderId' is not assignable to type '{ readonly [brand]: "UserId"; }'.
    Types of property '[brand]' are incompatible.
      Type '"OrderId"' is not assignable to type '"UserId"'.
main.ts(12,12): error TS2345: Argument of type 'number' is not assignable to parameter of type
'UserId'.
  Type 'number' is not assignable to type '{ readonly [brand]: "UserId"; }'.
main.ts(13,7): error TS2322: Type 'number' is not assignable to type 'Rupees'.
  Type 'number' is not assignable to type '{ readonly [brand]: "Rupees"; }'.
```

15.2.2 Making illegal states unrepresentable with discriminated unions

A common source of bugs is a type that allows **combinations that should never happen**: `{ loading: boolean; data?: T; error?: string }` allows "loading with an error and data at the same time". Model each valid state as its own member of a discriminated union, carrying exactly the data that state needs. Then impossible states cannot be constructed, every state must be handled (8.2.3), and data is only accessible where it exists. The same idea applies to forms (editing, submitting, submitted), payments, connections and wizards.

REPLACING A LOOSE STATE OBJECT WITH A DISCRIMINATED UNION main.ts

```
// Bad: 2^3 = 8 combinations, most of them meaningless
interface FetchStateLoose<T> { loading: boolean; data?: T; error?: string }
const nonsense: FetchStateLoose<string[]> = { loading: true, data: ["x"], error: "failed" };   //
compiles!

// Good: exactly the four valid states
type FetchState<T> =
  | { status: "idle" }
  | { status: "loading"; startedAt: number }
  | { status: "success"; data: T; receivedAt: number }
  | { status: "error"; error: string; retries: number };

function view(s: FetchState<string[]>): string {
  switch (s.status) {
    case "idle": return "Press load";
    case "loading": return "Loading...";
    case "success": return `${s.data.length} courses: ${s.data.join(", ")}`;
    case "error": return `Failed (${s.error}), retried ${s.retries}x`;
  }
}

// Transitions are functions from one state to the next
const start = (): FetchState<string[]> => ({ status: "loading", startedAt: 0 });
const succeed = (data: string[]): FetchState<string[]> => ({ status: "success", data, receivedAt: 1
});

console.log(nonsense.loading, "|", [{ status: "idle" } as const, start(), succeed(["TS101", "PY201"]),
{ status: "error", error: "timeout", retries: 2 } as const].map(view).join(" | "));
```

OUTPUT

```
true | Press load | Loading... | 2 courses: TS101, PY201 | Failed (timeout), retried 2x
```

IMPOSSIBLE STATES CAN NO LONGER BE WRITTEN

main.ts

```
type FetchState<T> =
  | { status: "loading" }
  | { status: "success"; data: T }
  | { status: "error"; error: string };

const s1: FetchState<string[]> = { status: "loading", data: ["x"] };          // loading has no data
const s2: FetchState<string[]> = { status: "error" };                          // error needs a
message
const s3: FetchState<string[]> = { status: "success", data: [], error: "x" }; // success has no error
```

COMPILER ERRORS (tsc --strict)

```
main.ts(6,55): error TS2353: Object literal may only specify known properties, and 'data' does not
exist in type '{ status: "loading"; }'.
main.ts(7,7): error TS2322: Type '{ status: "error"; }' is not assignable to type
'FetchState<string[]>'.
  Property 'error' is missing in type '{ status: "error"; }' but required in type '{ status: "error";
error: string; }'.
main.ts(8,65): error TS2353: Object literal may only specify known properties, and 'error' does not
exist in type '{ status: "success"; data: string[]; }'.
```

15.2.3 Result and Option types for error handling

Exceptions are invisible in function signatures: nothing in `function parsePrice(s: string): number` warns callers that it may throw. A **Result** type makes failure part of the return type: `Result<number, ParseError>` is either `{ ok: true, value }` or `{ ok: false, error }`, and the compiler forces callers to check before using the value. An **Option** (or Maybe) type does the same for "a value or nothing", although in TypeScript `T | undefined` with strict null checks usually serves that purpose. Use Results for **expected** failures (validation, parsing, business rules) and keep exceptions for unexpected bugs. Libraries such as neverthrow and Effect provide richer versions.

A RESULT TYPE WITH TYPED ERRORS AND A MAP COMBINATOR

main.ts

```
type Result<T, E = string> = { ok: true; value: T } | { ok: false; error: E };
const Ok = <T,>(value: T): Result<T, never> => ({ ok: true, value });
const Err = <E,>(error: E): Result<never, E> => ({ ok: false, error });

type ParseError = { kind: "empty" } | { kind: "notANumber"; input: string } | { kind: "outOfRange";
value: number };

function parseMarks(input: string): Result<number, ParseError> {
  const s = input.trim();
  if (s === "") return Err({ kind: "empty" });
  const n = Number(s);
  if (Number.isNaN(n)) return Err({ kind: "notANumber", input: s });
  if (n < 0 || n > 100) return Err({ kind: "outOfRange", value: n });
  return Ok(n);
}

// Combinators keep code linear: map transforms success, errors pass through
const map = <T, U, E>(r: Result<T, E>, fn: (v: T) => U): Result<U, E> => (r.ok ? Ok(fn(r.value)) : r);
const grade = (m: number) => (m >= 85 ? "A" : m >= 60 ? "B" : "C");

function explain(e: ParseError): string {
  switch (e.kind) {
    case "empty": return "please enter marks";
    case "notANumber": return `"${e.input}" is not a number`;
    case "outOfRange": return `${e.value} is outside 0-100`;
  }
}

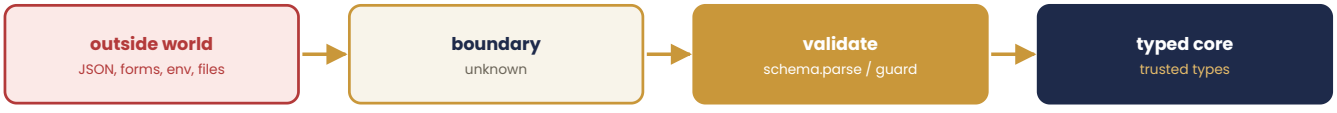
for (const input of ["91", " 72 ", "", "ninety", "120"]) {
  const r = map(parseMarks(input), grade);
  console.log(JSON.stringify(input).padEnd(9), r.ok ? `grade ${r.value}` : `error:
${explain(r.error)}`);
}
```

OUTPUT

```
"91"      grade A
" 72 "    grade B
""        error: please enter marks
"ninety"  error: "ninety" is not a number
"120"     error: 120 is outside 0-100
```

15.3 Type safety at boundaries

Inside your program, types are guaranteed by the compiler. At the **boundaries** (HTTP requests and responses, JSON files, environment variables, URL parameters, `localStorage`, messages from workers or queues) data arrives untyped, and the compiler can only trust what you tell it. The rule: **validate at the boundary, trust inside**. Hand-written guards work (8.4.2) but duplicate the type and the checks; schema libraries define both from one source.



VALIDATE ONCE AT THE BOUNDARY; TRUST THE TYPES INSIDE

15.3.1 Runtime validation with Zod (or Valibot)

Zod (`npm install zod`) is the most popular TypeScript-first validation library. You build a **schema** from small pieces (`z.string()`, `z.number().int().min(0)`, `z.email()`, `z.enum([...])`, `z.object({...})`, `z.array()`, `z.union()`), then call `schema.parse(data)`, which returns typed data or throws a `ZodError` describing every problem, or `schema.safeParse(data)`, which returns a Result-like object. Schemas can also **transform** (coerce strings to numbers, trim, set defaults). **Valibot** offers the same ideas with a smaller, tree-shakable bundle; ArkType and TypeBox are other options. The examples below need the package installed, so their output is shown rather than generated.

VALIDATING DATA WITH A ZOD SCHEMA — main.ts

```ts
import { z } from "zod";

const CourseSchema = z.object({
  code: z.string().regex(/^[A-Z]{2}\d{3}$/, "code must look like TS101"),
  title: z.string().min(2),
  fee: z.number().int().nonnegative(),
  level: z.enum(["beginner", "intermediate", "advanced"]),
  tags: z.array(z.string()).default([]),
  mentorEmail: z.email().optional(),
});

const good = CourseSchema.parse({ code: "TS101", title: "TypeScript", fee: 9999, level: "advanced" });
console.log(good);                                  // typed, with the default tags: []

const result = CourseSchema.safeParse({ code: "ts-1", title: "T", fee: -5, level: "expert" });
if (!result.success) {
  for (const issue of result.error.issues) console.log(`${issue.path.join(".")}: ${issue.message}`);
}
```

```
OUTPUT
{ code: 'TS101', title: 'TypeScript', fee: 9999, level: 'advanced', tags: [] }
code: code must look like TS101
title: Too small: expected string to have >=2 characters
fee: Too small: expected number to be >=0
level: Invalid option: expected one of "beginner"|"intermediate"|"advanced"
```

Requires `npm install zod` (Zod 4). Error message wording varies slightly between versions.

15.3.2 Inferring types from schemas z.infer

The key benefit of a schema library is that the TypeScript type is **derived from the schema**: `type Course = z.infer<typeof CourseSchema>`. You never write the interface separately, so the type and the runtime checks cannot drift apart. For schemas that transform data, `z.input<typeof S>` gives the type before parsing and `z.output` (the same as `z.infer`) the type after. Schemas compose like types: `.extend()`, `.pick()`, `.omit()`, `.partial()` mirror the utility types of Module 12.

TYPES DERIVED FROM SCHEMAS WITH Z.INFER main.ts

```
import { z } from "zod";

const UserSchema = z.object({
  id: z.number().int().positive(),
  name: z.string().trim().min(1),
  email: z.email(),
  role: z.enum(["student", "mentor", "admin"]),
  joinedAt: z.coerce.date(),                    // accepts an ISO string, outputs a Date
});

type User = z.infer<typeof UserSchema>;          // { id: number; name: string; email: string; role:
...; joinedAt: Date }
type UserInput = z.input<typeof UserSchema>;     // joinedAt is unknown before coercion

const CreateUserSchema = UserSchema.omit({ id: true, joinedAt: true });
const UpdateUserSchema = CreateUserSchema.partial();
type UpdateUser = z.infer<typeof UpdateUserSchema>;   // all fields optional

async function getUser(id: number): Promise<User> {
  const res = await fetch(`https://api.lbf.in/users/${id}`);
  return UserSchema.parse(await res.json());     // unknown in, User out (or a ZodError)
}

const u = UserSchema.parse({ id: 7, name: "  Asha ", email: "asha@lbf.in", role: "student", joinedAt:
"2026-07-01" });
console.log(u.name, u.joinedAt instanceof Date, UpdateUserSchema.parse({ role: "mentor" }));
```

```
OUTPUT
Asha true { role: 'mentor' }
```

Requires `npm install zod`.

To see the principle without installing anything, the program below implements a tiny schema library in about 40 lines. Each schema is an object with a `parse` method, and an `Infer` type extracts the output type from a schema type with `infer` (14.2.4). Real libraries add error paths, transformations and many more validators, but the type-level idea is exactly this.

A TINY SCHEMA LIBRARY: ONE DEFINITION GIVES BOTH THE CHECKS AND THE TYPE main.ts

```
interface Schema<T> { parse(value: unknown, path?: string): T }
type Infer<S> = S extends Schema<infer T> ? T : never;

class ValidationError extends Error {}
const fail = (path: string, expected: string, got: unknown): never => {
  throw new ValidationError(`${path || "value"}: expected ${expected}, got ${JSON.stringify(got)}`);
};

const z = {
  string: (opts: { min?: number } = {}): Schema<string> => ({
    parse: (v, p = "") => (typeof v === "string" && v.length >= (opts.min ?? 0) ? v : fail(p,
`string(min ${opts.min ?? 0})`, v)),
  }),
  number: (opts: { min?: number; max?: number } = {}): Schema<number> => ({
    parse: (v, p = "") => (typeof v === "number" && v >= (opts.min ?? -Infinity) && v <= (opts.max ??
Infinity) ? v : fail(p, "number in range", v)),
  }),
  enum: <const T extends readonly string[]>(values: T): Schema<T[number]> => ({
    parse: (v, p = "") => (values.includes(v as string) ? (v as T[number]) : fail(p, values.join("|"),
v)),
  }),
  array: <T,>(item: Schema<T>): Schema<T[]> => ({
```

```
    parse: (v, p = "") => (Array.isArray(v) ? v.map((x, i) => item.parse(x, `${p}[${i}]`)) : fail(p,
"array", v)),
  }),
  optional: <T,>(inner: Schema<T>): Schema<T | undefined> => ({ parse: (v, p) => (v === undefined ?
undefined : inner.parse(v, p)) }),
  object: <S extends Record<string, Schema<unknown>>>(shape: S): Schema<{ [K in keyof S]: Infer<S[K]>
}> => ({
    parse: (v, p = "") => {
      if (typeof v !== "object" || v === null || Array.isArray(v)) return fail(p, "object", v);
      const out: Record<string, unknown> = {};
      for (const key in shape) out[key] = shape[key].parse((v as Record<string, unknown>)[key], p ?
`${p}.${key}` : key);
      return out as { [K in keyof S]: Infer<S[K]> };
    },
  }),
};

const CourseSchema = z.object({
  code: z.string({ min: 5 }),
  fee: z.number({ min: 0 }),
  level: z.enum(["beginner", "advanced"]),
  tags: z.array(z.string()),
  mentor: z.optional(z.object({ name: z.string(), rating: z.number({ min: 0, max: 5 }) })),
});
type Course = Infer<typeof CourseSchema>;     // derived: { code: string; fee: number; level:
"beginner" | "advanced"; ... }

const inputs: unknown[] = [
  { code: "TS101", fee: 9999, level: "advanced", tags: ["web"], mentor: { name: "Lohit", rating: 4.9 }
},
  { code: "TS101", fee: 9999, level: "expert", tags: [] },
  { code: "PY201", fee: 14999, level: "beginner", tags: ["data", 7] },
];
for (const input of inputs) {
  try {
    const c: Course = CourseSchema.parse(input);
    console.log("valid:", c.code, c.level, c.mentor?.name ?? "no mentor");
  } catch (e) {
    console.log("invalid:", (e as Error).message);
  }
}
```

```
OUTPUT
valid: TS101 advanced Lohit
invalid: level: expected beginner|advanced, got "expert"
invalid: tags[1]: expected string(min 0), got 7
```

15.3.3 Typing environment variables

`process.env` is typed as `Record<string, string | undefined>`: every variable may be missing, and all values are strings. Declaring them with interface augmentation (11.4.6) only changes what the compiler believes. The robust approach is to **parse the environment once at startup** with a schema, converting types (ports to numbers, flags to booleans), applying defaults and failing fast with a clear message, and to export a typed, frozen `env` object that the rest of the app imports. Libraries such as `@t3-oss/env-core` and `envalid` package this pattern; with Zod it is `z.object({...}).parse(process.env)`.

PARSING ENVIRONMENT VARIABLES ONCE, INTO A TYPED AND FROZEN OBJECT main.ts

```
type EnvSpec = Record<string, { type: "string" | "number" | "boolean" | "url"; default?: string;
values?: readonly string[] }>;
type EnvValue<S extends EnvSpec[string]> =
  S["values"] extends readonly (infer V)[] ? V :
  S["type"] extends "number" ? number : S["type"] extends "boolean" ? boolean : string;
type Env<S extends EnvSpec> = { readonly [K in keyof S]: EnvValue<S[K]> };

function loadEnv<const S extends EnvSpec>(spec: S, source: Record<string, string | undefined>): Env<S>
{
  const out: Record<string, unknown> = {};
  const problems: string[] = [];
  for (const [name, rule] of Object.entries(spec)) {
    const raw = source[name] ?? rule.default;
    if (raw === undefined) { problems.push(`${name} is required`); continue; }
    if (rule.values && !rule.values.includes(raw)) { problems.push(`${name} must be one of
${rule.values.join(", ")}`); continue; }
    if (rule.type === "number" && Number.isNaN(Number(raw))) { problems.push(`${name} must be a
number`); continue; }
    if (rule.type === "url" && !URL.canParse(raw)) { problems.push(`${name} must be a URL`); continue;
}
    out[name] = rule.type === "number" ? Number(raw) : rule.type === "boolean" ? raw === "true" : raw;
  }
  if (problems.length) throw new Error("Invalid environment:\n  " + problems.join("\n  "));
  return Object.freeze(out) as Env<S>;
}

const spec = {
  NODE_ENV: { type: "string", values: ["development", "production", "test"], default: "development" },
  PORT: { type: "number", default: "3000" },
  DATABASE_URL: { type: "url" },
  ENABLE_PAYMENTS: { type: "boolean", default: "false" },
} as const;

const env = loadEnv(spec, { DATABASE_URL: "postgres://localhost:5432/lbf", PORT: "8080", NODE_ENV:
"production" });
const port: number = env.PORT;                                    // number, not string | undefined
const mode: "development" | "production" | "test" = env.NODE_ENV;
console.log(port + 1, mode, env.ENABLE_PAYMENTS, env.DATABASE_URL);

try {
  loadEnv(spec, { PORT: "eighty", NODE_ENV: "staging" });
} catch (e) {
  console.log((e as Error).message);
}
```

OUTPUT

```
8081 production false postgres://localhost:5432/lbf
Invalid environment:
  NODE_ENV must be one of development, production, test
  PORT must be a number
  DATABASE_URL is required
```

15.4 Variance

Variance describes how subtyping of a generic type relates to subtyping of its type argument. If `Dog` is a subtype of `Animal`, is `Box<Dog>` a subtype of `Box<Animal>`? It depends on how `Box` uses its type parameter. Understanding variance explains many confusing assignability errors, especially with callbacks.

15.4.1 Covariance and contravariance

Variance	Rule (Dog extends Animal)	When	Example
covariant	`F<Dog>` is assignable to `F<Animal>`	T is only **produced** (output, read)	return types, readonly arrays, `Promise<T>`
contravariant	`F<Animal>` is assignable to `F<Dog>`	T is only **consumed** (input, written)	function parameters, event handlers, comparators
invariant	neither direction	T is both read and written	mutable boxes, `(x: T) => T`
bivariant	both directions (unsafe)	method parameters (legacy rule)	methods declared with method syntax

The intuition: a function that **produces** dogs can be used where something that produces animals is needed (every dog is an animal), so producers are covariant. A function that **consumes** animals (handles any animal) can be used where a dog-handler is needed, but a dog-handler cannot be used where any animal may be passed in (it might receive a cat), so consumers are contravariant.

COVARIANCE OF OUTPUTS AND CONTRAVARIANCE OF INPUTS main.ts

```ts
interface Animal { name: string }
interface Dog extends Animal { bark(): string }
interface Cat extends Animal { meow(): string }

const rex: Dog = { name: "Rex", bark: () => "woof" };

// Covariant: producers of Dog can stand in for producers of Animal
const getDog = (): Dog => rex;
const getAnimal: () => Animal = getDog;                    // OK
const dogs: readonly Dog[] = [rex];
const animals: readonly Animal[] = dogs;                    // OK (readonly arrays are covariant)

// Contravariant: consumers of Animal can stand in for consumers of Dog
const describeAnimal = (a: Animal) => `animal ${a.name}`;
const describeDog: (d: Dog) => string = describeAnimal;     // OK: it can handle any animal, so dogs too

console.log(getAnimal().name, animals.length, describeDog(rex));
```

OUTPUT

```
Rex 1 animal Rex
```

PASSING A DOG HANDLER WHERE ANY ANIMAL MAY ARRIVE IS REJECTED main.ts

```
interface Animal { name: string }
interface Dog extends Animal { bark(): string }
interface Cat extends Animal { meow(): string }

const handleDog = (d: Dog) => d.bark();
const handleAnimal: (a: Animal) => string = handleDog;   // unsafe: a Cat could be passed in

const dogs: Dog[] = [];
const animals: Animal[] = dogs;                          // allowed (arrays are treated covariantly)
...
const tom: Cat = { name: "Tom", meow: () => "meow" };
animals.push(tom);                                       // ... so a Cat ends up in dogs: a known
unsoundness
```

COMPILER ERRORS (tsc --strict)

```
main.ts(6,7): error TS2322: Type '(d: Dog) => string' is not assignable to type '(a: Animal) =>
string'.
  Types of parameters 'd' and 'a' are incompatible.
    Property 'bark' is missing in type 'Animal' but required in type 'Dog'.
```

NOTE

Mutable arrays are checked covariantly for convenience, which is technically unsound: the last two lines compile and put a cat into the `dogs` array. Use `readonly` arrays in parameters to make such code safe, because a readonly array cannot be pushed to.

15.4.2 in and out variance annotations

TypeScript normally **measures** variance by examining how a generic type uses its parameter. For complex recursive types this can be slow or imprecise, so TypeScript 4.7 added optional annotations: `out T` declares a type parameter covariant, `in T` contravariant, and `in out T` invariant. The compiler checks that the annotation matches the actual use, and then uses it directly. They are mainly useful to library authors and to document intent; most application code never needs them.

VARIANCE ANNOTATIONS ARE CHECKED AGAINST THE ACTUAL USE main.ts

```
interface Producer<out T> { get(): T }
interface Consumer<in T> { accept(value: T): void }
interface Cell<in out T> { get(): T; set(value: T): void }

interface Wrong<out T> { accept(value: T): void }          // annotation contradicts the use

interface Animal { name: string }
interface Dog extends Animal { bark(): string }
declare const dogProducer: Producer<Dog>;
declare const animalConsumer: Consumer<Animal>;
declare const dogCell: Cell<Dog>;

const p: Producer<Animal> = dogProducer;     // OK: covariant
const c: Consumer<Dog> = animalConsumer;     // OK: contravariant
const cell: Cell<Animal> = dogCell;          // error: invariant
```

COMPILER ERRORS (tsc --strict)

```
main.ts(15,7): error TS2322: Type 'Cell<Dog>' is not assignable to type 'Cell<Animal>'.
  Property 'bark' is missing in type 'Animal' but required in type 'Dog'.
```

15.4.3 strictFunctionTypes

`strictFunctionTypes` (part of strict) makes **function type** parameters contravariant, catching the unsafe assignment shown in 15.4.1. For historical compatibility it does **not** apply to parameters declared with **method syntax** (`handle(a: Animal): void` inside an interface), which stay bivariant. This is why some style guides (and the ESLint rule `@typescript-eslint/method-signature-style`) prefer property syntax (`handle: (a: Animal) => void`) for callbacks: it gets the stricter check.

METHOD SYNTAX ESCAPES STRICTFUNCTIONTYPES; PROPERTY SYNTAX DOES NOT main.ts

```ts
interface Animal { name: string }
interface Dog extends Animal { bark(): string }

interface HandlerMethod { handle(a: Animal): void }         // method syntax: bivariant (lenient)
interface HandlerProp { handle: (a: Animal) => void }       // property syntax: contravariant (strict)

const dogOnly = { handle: (d: Dog) => console.log(d.bark()) };

const m: HandlerMethod = dogOnly;     // accepted, although unsafe
const f: HandlerProp = dogOnly;       // rejected
```

COMPILER ERRORS (tsc --strict)

```
main.ts(10,7): error TS2322: Type '{ handle: (d: Dog) => void; }' is not assignable to type
'HandlerProp'.
  Types of property 'handle' are incompatible.
    Type '(d: Dog) => void' is not assignable to type '(a: Animal) => void'.
      Types of parameters 'd' and 'a' are incompatible.
        Property 'bark' is missing in type 'Animal' but required in type 'Dog'.
```

15.5 Declaration merging and augmentation in depth

Declaration merging combines several declarations with the same name into one entity. Knowing the rules explains how libraries are typed and how to extend them safely (11.4.5). Interfaces merge their members (later declarations' overloads take precedence). A namespace can merge with a function, class or enum to add static members. A class and an interface with the same name merge, adding members to the class's instance type (without implementing them). Type aliases never merge. Module augmentation (`declare module "lib"`) and global augmentation (`declare global`) are declaration merging applied to other files.

Declarations with the same name	Result
interface + interface	one interface with all members (the most common case)
namespace + function / class / enum	the function or class gains properties (e.g. `jQuery.ajax`, `square.cube`)
class + interface	the interface's members are added to the class instance type (mixins)
namespace + namespace	members combined (exported members are shared)
type alias + anything	error: aliases cannot merge
declare module "x" in a module file	augments module x (adds, never replaces)
declare global in a module file	augments global types (Window, Array, NodeJS.ProcessEnv)

FUNCTION + NAMESPACE, CLASS + INTERFACE, AND MERGED OVERLOADS main.ts

```
// 1. Function + namespace: a callable with properties
function currency(amount: number): string { return `Rs ${currency.format(amount)}`; }
namespace currency {
  export const symbol = "Rs";
  export function format(n: number) { return n.toLocaleString("en-IN"); }
}

// 2. Class + interface: a mixin adds methods to the class type
class Course { constructor(public title: string) {} }
interface Course { timestamp(): string }                    // merged into Course's instance type
Object.assign(Course.prototype, { timestamp(this: Course) { return `${this.title} @ 2026-10-07`; } });

// 3. Interface + interface, including overloads (later declarations come first)
interface Formatter { format(value: number): string }
interface Formatter { format(value: Date): string }
const fmt: Formatter = { format: (v: number | Date) => (v instanceof Date ? v.toISOString().slice(0,
10) : v.toFixed(2)) };

console.log(currency(1250000), currency.symbol, new Course("TypeScript").timestamp(),
fmt.format(3.14159), fmt.format(new Date(Date.UTC(2026, 0, 26))));
```

OUTPUT

```
Rs 12,50,000 Rs TypeScript @ 2026-10-07 3.14 2026-01-26
```

WATCH OUT

Class and interface merging adds members to the **type** only; you must provide the implementation yourself (here with `Object.assign` on the prototype). If you forget, the compiler will not warn you. Prefer composition or explicit mixin functions for new code.

15.6 Type performance

15.6.1 Avoiding overly complex types

Type checking can become slow in large codebases, mostly because of a few expensive types. Guidelines from the TypeScript team: prefer **interfaces that extend** over large intersections (interfaces are cached and checked once; intersections are re-checked at each use); **name complex types** with aliases or interfaces instead of repeating inline types (names enable caching and give shorter errors); add **explicit return types** to exported functions, especially those returning complex inferred types (the declaration emitter and other files then need not re-infer them); avoid very large unions (thousands of members, often generated by template literal types) and deeply recursive conditional types; and keep generic utilities simple. Clever type-level code is valuable in libraries, but application code should favour clarity.

Slow pattern	Faster alternative
`type A = B & C & D & { ... }` used everywhere	`interface A extends B, C, D { ... }`
the same large inline object type in many signatures	one named interface
exported function with a huge inferred return type	an explicit return type annotation
template literal type producing 10,000+ string members	a broader type (`` `btn-${string}` ``) plus runtime checks
deeply recursive conditional types on large inputs	limit depth, use tail recursion, or simplify

Slow pattern	Faster alternative
type-checking all of node_modules declaration files	`skipLibCheck: true` (17.1.3)

15.6.2 Diagnosing slow type checking

When builds or the editor feel slow, measure before changing anything. `tsc --extendedDiagnostics` prints the time spent in parsing, binding, checking and emitting, the number of files, types and instantiations, and memory use. `tsc --listFiles` or `--explainFiles` shows which files are included and why (often unexpected files from `node_modules`). `tsc --generateTrace traceDir` records a detailed trace that you open in a Chromium browser's performance tool (or analyse with the `@typescript/analyze-trace` package) to find the specific types that are expensive. In VS Code, *TypeScript: Open TS Server log* helps with editor slowness. Splitting a large codebase with project references (17.2) and incremental builds also reduces work, and the native TypeScript 7 compiler is about ten times faster overall.

TSCONFIG.JSON JavaScript

```
{ "compilerOptions": { "strict": true, "noEmit": true, "target": "es2022", "types": [] }, "include":
["src"] }
```

SRC/TYPES.TS TypeScript

```
type Digit = "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9";
export type ThreeDigits = `${Digit}${Digit}${Digit}`;          // 1,000 members
export type Code = `${"TS" | "PY" | "JV"}${ThreeDigits}`;         // 3,000 members
export const isCode = (s: string): s is Code => /^(TS|PY|JV)\d{3}$/.test(s);
```

MEASURING A TYPE-CHECK WITH --EXTENDEDDIAGNOSTICS terminal

```
npx tsc -p . --extendedDiagnostics | grep -E "^(Files|Lines of TypeScript|Types|Instantiations|Memory
used|Check time|Total time)"
```

TERMINAL

```
$ npx tsc -p . --extendedDiagnostics | grep -E "^(Files|Lines of
TypeScript|Types|Instantiations|Memory used|Check time|Total time)"
Files:                            64
Lines of TypeScript:               5
Types:                         32079
Instantiations:                27046
Memory used:                  113147K
Check time:                     5.02s
Total time:                     7.12s
```

Times and memory differ between machines; compare the numbers before and after a change on the same machine.

as const keeps literal types; satisfies checks a value against a type without widening it (as const satisfies T does both); NoInfer stops secondary arguments from widening inference. Branded types (T & { [brand]: "Name" }) make same-shaped IDs, currencies and validated strings incompatible. Discriminated unions make illegal states unrepresentable, and Result<T, E> puts expected failures in the return type. Validate external data at the boundary with schemas (Zod, Valibot) and derive types with z.infer; parse environment variables once into a typed, frozen object. Producers are covariant, consumers contravariant, and read-write types invariant; in/out annotations declare variance, and strictFunctionTypes applies only to property-syntax function types, not methods. Declaration merging combines interfaces, adds namespaces to functions and classes, and powers module and global augmentation. Keep types fast: interfaces over intersections, named types, explicit return types on exports, small unions; measure with --extendedDiagnostics and --generateTrace.

PRACTICE CHECKPOINT

A **typed configuration object using satisfies**, **branded UserId and OrderId** types that cannot be mixed up, and an **API layer validated** with Zod (here, with a small schema library so the program runs without installing packages).

SAMPLE SOLUTION: SATISFIES, BRANDED IDS AND A VALIDATED API LAYER

main.ts

```
// ---------- 1. configuration checked with satisfies (keys and literal values stay precise)
type Plan = { priceInr: number; seats: number; features: readonly string[] };
const PLANS = {
  free: { priceInr: 0, seats: 1, features: ["videos"] },
  pro: { priceInr: 999, seats: 5, features: ["videos", "mentor chat"] },
  team: { priceInr: 4999, seats: 50, features: ["videos", "mentor chat", "reports"] },
} as const satisfies Record<string, Plan>;
type PlanName = keyof typeof PLANS;                       // "free" | "pro" | "team"

// ---------- 2. branded ids
declare const brand: unique symbol;
type Brand<T, B> = T & { readonly [brand]: B };
type UserId = Brand<number, "UserId">;
type OrderId = Brand<number, "OrderId">;

// ---------- 3. a tiny schema layer (stands in for Zod: same parse / infer idea)
type Schema<T> = { parse(v: unknown, path: string): T };
type Infer<S> = S extends Schema<infer T> ? T : never;
const bad = (path: string, msg: string): never => { throw new Error(`${path}: ${msg}`); };
const s = {
  int: <B,>(): Schema<Brand<number, B>> => ({ parse: (v, p) => (Number.isInteger(v) && (v as number) >
0 ? (v as Brand<number, B>) : bad(p, "positive integer")) }),
  plan: (): Schema<PlanName> => ({ parse: (v, p) => (typeof v === "string" && v in PLANS ? (v as
PlanName) : bad(p, `one of ${Object.keys(PLANS).join("|")}`)) }),
  str: (): Schema<string> => ({ parse: (v, p) => (typeof v === "string" && v.length > 0 ? v : bad(p,
"non-empty string")) }),
  object: <S extends Record<string, Schema<unknown>>>(shape: S): Schema<{ [K in keyof S]: Infer<S[K]>
}> => ({
    parse: (v, p) => {
      if (typeof v !== "object" || v === null) return bad(p, "object");
      return Object.fromEntries(Object.entries(shape).map(([k, sch]) => [k, sch.parse((v as any)[k],
`${p}.${k}`)])) as any;
    },
  }),
};

const OrderSchema = s.object({ id: s.int<"OrderId">(), userId: s.int<"UserId">(), plan: s.plan(),
coupon: s.str() });
```

```
type Order = Infer<typeof OrderSchema>;                    // { id: OrderId; userId: UserId; plan:
PlanName; coupon: string }

// ---------- the API layer: unknown in, validated and branded data out
const fakeApi: Record<string, unknown> = {
  "/orders/9001": { id: 9001, userId: 7, plan: "pro", coupon: "LBF10" },
  "/orders/9002": { id: 9002, userId: "7", plan: "enterprise", coupon: "" },
};
async function getOrder(id: OrderId): Promise<Order> {
  const body = fakeApi[`/orders/${id}`] ?? null;
  return OrderSchema.parse(body, "order");
}

function invoice(order: Order, forUser: UserId): string {
  if (order.userId !== forUser) return "not your order";
  const plan = PLANS[order.plan];
  return `order #${order.id}: ${order.plan} plan, Rs ${plan.priceInr}, ${plan.seats} seats
(${plan.features.join(", ")}), coupon ${order.coupon}`;
}

async function main() {
  const user = 7 as UserId;
  console.log(invoice(await getOrder(9001 as OrderId), user));
  try {
    await getOrder(9002 as OrderId);
  } catch (e) {
    console.log("rejected:", (e as Error).message);
  }
  // invoice(await getOrder(9001 as OrderId), 9001 as OrderId);   // compile error: OrderId is not a
UserId
  console.log("team features:", PLANS.team.features.length, "| plan names:",
Object.keys(PLANS).join(", "));
}
main();
```

OUTPUT

```
order #9001: pro plan, Rs 999, 5 seats (videos, mentor chat), coupon LBF10
rejected: order.userId: positive integer
team features: 3 | plan names: free, pro, team
```

MODULE 16 · ADVANCED

Decorators

Minimum duration **2 days** | Level **Advanced**

Decorators attach reusable behaviour to classes and their members, and they are used heavily by Angular, NestJS and TypeORM. This module explains what decorators are and the difference between standard ECMAScript decorators (TypeScript 5.0+) and the legacy experimentalDecorators; class, method, accessor and field decorators and their context objects; decorator factories, composition and evaluation order; practical decorators such as @log, @measure, @memoize and @readonly; and how decorators are used in Angular, NestJS and TypeORM.

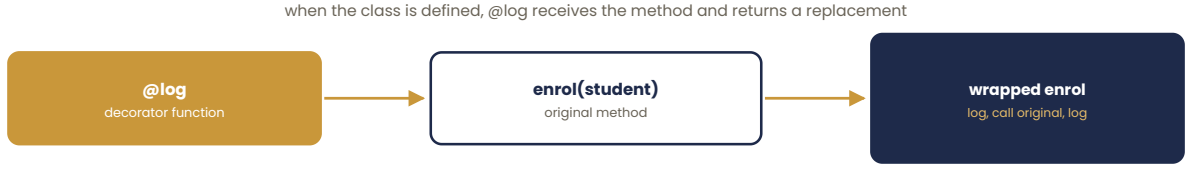
IN THIS MODULE

16.1 Introduction
16.2 Types of decorators
16.3 Decorator factories
16.4 Practical decorators
16.5 Decorators in frameworks

16.1 Introduction

16.1.1 What decorators are

A **decorator** is a function, written with `@name` just before a class or class member, that the runtime calls when the class is defined. It receives the decorated thing (the class, the method function, the accessor or the field) and a **context** object describing it, and it can observe it, **replace** it with a wrapped version, or register extra initialisation. Decorators separate **cross-cutting concerns** (logging, timing, caching, validation, access control, dependency injection, routing metadata) from business logic: you write the behaviour once and apply it declaratively wherever needed.



A METHOD DECORATOR RECEIVES THE ORIGINAL METHOD AND RETURNS A WRAPPED VERSION

A FIRST DECORATOR: LOGGING EVERY CALL OF A METHOD

main.ts

```ts
function log(original: (...args: any[]) => any, context: ClassMethodDecoratorContext) {
  const name = String(context.name);
  return function (this: unknown, ...args: unknown[]) {
    console.log(`-> ${name}(${args.map((a) => JSON.stringify(a)).join(", ")})`);
    const result = original.apply(this, args);
    console.log(`<- ${name} returned ${JSON.stringify(result)}`);
    return result;
  };
}

class Batch {
  #students: string[] = [];
  constructor(public course: string) {}

  @log
  enrol(name: string): number {
    this.#students.push(name);
    return this.#students.length;
  }
}

const b = new Batch("TypeScript");
b.enrol("Asha");
b.enrol("Ravi");
```

OUTPUT

```
-> enrol("Asha")
<- enrol returned 1
-> enrol("Ravi")
<- enrol returned 2
```

16.1.2 Standard (ECMAScript) decorators vs legacy experimentalDecorators

TypeScript has supported decorators since 2015 behind the `experimentalDecorators` flag, based on an early proposal. The JavaScript standard later went a different way, and TypeScript 5.0 (2023) implemented the **standard ECMAScript decorators** (TC39 stage 3), which are enabled by default with no flag. The two systems are **not compatible**: the function signatures differ, standard decorators do not support **parameter decorators** or the `emitDecoratorMetadata` option, and they receive a typed `context` object instead of a target, key and property descriptor. Angular, NestJS and TypeORM were built on the legacy version, so their projects set `"experimentalDecorators": true` (and often `emitDecoratorMetadata`). For new code that is not tied to those frameworks, use standard decorators.

	Standard decorators (TS 5.0+)	Legacy experimentalDecorators
enabled by	default (no flag)	`"experimentalDecorators": true`
method signature	`(value, context) => replacement | void`	`(target, key, descriptor) => descriptor | void`
context information	typed `context`: kind, name, static, private, access, addInitializer, metadata	target object and property key only
parameter decorators	not supported	supported (`constructor(@Inject() x)`)
type metadata (reflect-metadata)	not supported (use `context.metadata`)	`emitDecoratorMetadata`

	Standard decorators (TS 5.0+)	Legacy experimentalDecorators
used by	new libraries, Lit, MobX 6.10+	Angular, NestJS, TypeORM, class-validator, InversifyJS
standardised	yes (ECMAScript proposal, stage 3)	no (TypeScript only)

WATCH OUT
Do not mix the two: with `experimentalDecorators` enabled, the compiler uses the legacy semantics for every decorator in the project, and decorators written for one system will not work with the other. Check which one your framework expects.

16.2 Types of decorators

Standard decorators can be applied to classes, methods, getters and setters, auto-accessors (`accessor` fields) and fields. Each kind receives a different first argument and context type, all declared in TypeScript's `lib.decorators.d.ts`: `ClassDecoratorContext`, `ClassMethodDecoratorContext`, `ClassGetterDecoratorContext`, `ClassSetterDecoratorContext`, `ClassAccessorDecoratorContext` and `ClassFieldDecoratorContext`. Every context has `kind`, `name`, `static`, `private`, `metadata` and `addInitializer`.

Kind	Receives	May return	Typical uses
class	the class constructor	a replacement class (often a subclass)	registration, sealing, adding behaviour
method	the method function	a replacement function	logging, timing, caching, auth checks, binding
getter / setter	the accessor function	a replacement accessor function	validation, lazy values
accessor (auto-accessor)	`{ get, set }`	`{ get?, set?, init? }`	observable or validated properties
field	`undefined`	an initialiser function `(initial) => value`	defaults, transformations, registration

16.2.1 Class decorators

A class decorator receives the class itself. It can record the class somewhere (a registry of plugins, routes or entities), freeze or seal it, or return a **new class** that extends the original to add behaviour. `context.addInitializer` runs code once, after the class is fully defined.

CLASS DECORATORS FOR REGISTRATION AND FOR ADDING FIELDS main.ts

```
const registry = new Map<string, new (...args: any[]) => unknown>();

function register(id: string) {
  return function <T extends new (...args: any[]) => any>(cls: T, context: ClassDecoratorContext<T>) {
    context.addInitializer(() => registry.set(id, cls));
  };
}

function withTimestamps<T extends new (...args: any[]) => object>(Base: T, _context:
ClassDecoratorContext<T>) {
  return class extends Base {
    createdAt = new Date(Date.UTC(2026, 9, 7));
  };
}

@register("course")
@withTimestamps
class Course {
  constructor(public code: string, public fee: number) {}
}

@register("batch")
class Batch {
  constructor(public course: string, public seats = 30) {}
}

const c = new Course("TS101", 9999) as Course & { createdAt: Date };
console.log(c.code, c.createdAt.toISOString().slice(0, 10));
console.log([...registry.keys()], new (registry.get("batch")!)("Python"));
```

```
OUTPUT
TS101 2026-10-07
[ 'course', 'batch' ] Batch { course: 'Python', seats: 30 }
```

16.2.2 Method decorators

A method decorator receives the original method and a `ClassMethodDecoratorContext`. To change the behaviour, return a new function that calls the original with the same `this` (`original.call(this, ...args)`). To be fully typed, make the decorator generic over `this`, the arguments and the return type: `function log<This, Args extends unknown[], R>(original: (this: This, ...args: Args) => R, context: ClassMethodDecoratorContext<This, (this: This, ...args: Args) => R>)`. A classic method decorator is `@bound`, which uses `addInitializer` to bind the method to each instance so it can be passed around as a callback.

TYPED METHOD DECORATORS: @BOUND AND AN AUTHORISATION CHECK main.ts

```
function bound<This extends object, Args extends unknown[], R>(
  original: (this: This, ...args: Args) => R,
  context: ClassMethodDecoratorContext<This, (this: This, ...args: Args) => R>,
) {
  context.addInitializer(function (this: This) {
    (this as any)[context.name] = original.bind(this);     // runs for every new instance
  });
}

function requireRole(role: "admin" | "mentor") {
  return function <This extends { role: string }, Args extends unknown[], R>(
    original: (this: This, ...args: Args) => R,
    context: ClassMethodDecoratorContext<This, (this: This, ...args: Args) => R>,
  ) {
    return function (this: This, ...args: Args): R {
      if (this.role !== role) throw new Error(`${String(context.name)} requires ${role}, current role
is ${this.role}`);
      return original.call(this, ...args);
    };
  };
}

class Dashboard {
  constructor(public user: string, public role: string) {}

  @bound
  greet() { return `Welcome, ${this.user}`; }

  @requireRole("admin")
  deleteCourse(code: string) { return `${code} deleted by ${this.user}`; }
}

const admin = new Dashboard("Lohit", "admin");
const learner = new Dashboard("Asha", "student");
const greet = learner.greet;                        // detached, but still bound
console.log(greet(), "|", admin.deleteCourse("JV301"));
try { learner.deleteCourse("TS101"); } catch (e) { console.log((e as Error).message); }
```

```
OUTPUT
Welcome, Asha | JV301 deleted by Lohit
deleteCourse requires admin, current role is student
```

16.2.3 Accessor decorators

Standard decorators work on getters and setters (receiving the accessor function) and on **auto-accessors**, a new kind of class member declared with the `accessor` keyword: `accessor fee = 0` creates a private storage slot with a getter and setter. An auto-accessor decorator receives `{ get, set }` and can return replacements plus an `init` function, which makes it the natural place for validation, change tracking and observable properties.

AUTO-ACCESSOR DECORATORS FOR VALIDATION AND CHANGE TRACKING main.ts

```
function range(min: number, max: number) {
  return function <This>(target: ClassAccessorDecoratorTarget<This, number>, context:
ClassAccessorDecoratorContext<This, number>): ClassAccessorDecoratorResult<This, number> {
    const check = (v: number) => {
      if (v < min || v > max) throw new RangeError(`${String(context.name)} must be between ${min} and
${max}, got ${v}`);
      return v;
    };
    return {
      init: (initial) => check(initial),
      set(value) { target.set.call(this, check(value)); },
    };
  };
}

function tracked<This, V>(target: ClassAccessorDecoratorTarget<This, V>, context:
ClassAccessorDecoratorContext<This, V>): ClassAccessorDecoratorResult<This, V> {
  return {
    set(value) {
      console.log(`  ${String(context.name)}: ${JSON.stringify(target.get.call(this))} ->
${JSON.stringify(value)}`);
      target.set.call(this, value);
    },
  };
}

class Enrolment {
  @range(0, 100) accessor discountPct = 0;
  @tracked accessor status: "pending" | "paid" | "cancelled" = "pending";
  constructor(public fee: number) {}
  get payable() { return Math.round(this.fee * (1 - this.discountPct / 100)); }
}

const e = new Enrolment(9999);
e.discountPct = 20;
e.status = "paid";
console.log("payable", e.payable);
try { e.discountPct = 150; } catch (err) { console.log((err as Error).message); }
```

```
OUTPUT
  status: "pending" -> "paid"
payable 7999
discountPct must be between 0 and 100, got 150
```

16.2.4 Field decorators

A field decorator receives `undefined` as its first argument (the field does not exist yet when the class is defined) and may return an **initialiser**: a function that receives the field's initial value for each new instance and returns the value to use. Field decorators can transform defaults, register field names in `context.metadata` for later use (for example, a list of fields to serialise or validate), or use `addInitializer` to run per-instance code.

FIELD DECORATORS THAT TRANSFORM INITIAL VALUES AND RECORD METADATA main.ts

```
function trimmed<This>(_: undefined, _context: ClassFieldDecoratorContext<This, string>) {
  return (initial: string) => initial.trim();
}

function upper<This>(_: undefined, _context: ClassFieldDecoratorContext<This, string>) {
  return (initial: string) => initial.toUpperCase();
}

// Collect field names in decorator metadata (shared by all decorators of the class)
(Symbol as any).metadata ??= Symbol("Symbol.metadata");            // polyfill for older runtimes
function serialise<This, V>(_: undefined, context: ClassFieldDecoratorContext<This, V>) {
  const meta = context.metadata!;                                  // defined once Symbol.metadata
exists
  const fields = ((meta.serialise as string[] | undefined) ??= []);
  fields.push(String(context.name));
}

function toJson(obj: object): string {
  const meta = (obj.constructor as any)[(Symbol as any).metadata] as { serialise?: string[] } |
undefined;
  return JSON.stringify(Object.fromEntries((meta?.serialise ?? []).map((k) => [k, (obj as any)[k]])));
}

class Student {
  @serialise @trimmed name = "  Asha Rao  ";
  @serialise @upper code = "ts101";
  @serialise marks = 91;
  passwordHash = "secret";                                         // not serialised
}

const s = new Student();
console.log(JSON.stringify(s.name), s.code, toJson(s));
```

OUTPUT

```
"Asha Rao" TS101 {"name":"Asha Rao","code":"TS101","marks":91}
```

16.3 Decorator factories

16.3.1 Decorators with arguments

To configure a decorator (`@retry(3)`, `@range(0, 100)`, `@route("/courses")`), write a **decorator factory**: an ordinary function that takes the options and **returns** the decorator. Note the difference between `@log` (the decorator itself) and `@log()` (calling a factory that returns a decorator); writing the wrong one is a common mistake that TypeScript reports as a type error.

A DECORATOR FACTORY: @RETRY(TIMES, DELAY)

main.ts

```
function retry(times: number, delayMs = 0) {
  return function <This, Args extends unknown[], R>(
    original: (this: This, ...args: Args) => Promise<R>,
    context: ClassMethodDecoratorContext<This, (this: This, ...args: Args) => Promise<R>>,
  ) {
    return async function (this: This, ...args: Args): Promise<R> {
      for (let attempt = 1; ; attempt++) {
        try {
          return await original.call(this, ...args);
        } catch (e) {
          if (attempt >= times) throw e;
          console.log(`  ${String(context.name)} failed (attempt ${attempt}/${times}): ${(e as
Error).message}`);
          await new Promise((r) => setTimeout(r, delayMs));
        }
      }
    };
  };
}

class PaymentGateway {
  #calls = 0;
  @retry(3, 10)
  async charge(amount: number): Promise<string> {
    if (++this.#calls < 3) throw new Error("gateway timeout");
    return `charged Rs ${amount} on call ${this.#calls}`;
  }
}

new PaymentGateway().charge(9999).then(console.log);
```

OUTPUT

```
  charge failed (attempt 1/3): gateway timeout
  charge failed (attempt 2/3): gateway timeout
charged Rs 9999 on call 3
```

USING A FACTORY WITHOUT CALLING IT IS A TYPE ERROR

main.ts

```
function retry(times: number) {
  return function (original: (...args: any[]) => any, context: ClassMethodDecoratorContext) { return
original; };
}

class Gateway {
  @retry                        // forgot the parentheses: retry itself is not a decorator
  charge() { return 1; }
}
```

COMPILER ERRORS (tsc --strict)

```
main.ts(6,4): error TS1241: Unable to resolve signature of method decorator when called as an
expression.
  Argument of type '() => number' is not assignable to parameter of type 'number'.
main.ts(6,4): error TS1270: Decorator function return type '(original: (...args: any[]) => any,
context: ClassMethodDecoratorContext) => (...args: any[]) => any' is not assignable to type 'void |
(() => number)'.
  Type '(original: (...args: any[]) => any, context: ClassMethodDecoratorContext) => (...args: any[])
=> any' is not assignable to type '() => number'.
    Target signature provides too few arguments. Expected 2 or more, but got 0.
```

16.3.2 Composing decorators and evaluation order

Several decorators can be stacked on one declaration. The decorator **expressions** (including factory calls) are evaluated **top to bottom**, but the decorators are **applied bottom to top**, closest to the declaration first, like function composition: `@a @b method` behaves like `a(b(method))`. At run time the outer wrapper (a) therefore runs first and calls the inner one. Across a class, member decorators are applied before the class decorator. Order matters: for example, a caching decorator outside a logging decorator hides cached calls from the log.

FACTORIES ARE EVALUATED TOP-DOWN; DECORATORS ARE APPLIED BOTTOM-UP

main.ts

```
function tag(name: string) {
  console.log(`evaluate factory ${name}`);
  return function (original: (...args: any[]) => any, context: ClassMethodDecoratorContext) {
    console.log(`apply decorator ${name} to ${String(context.name)}`);
    return function (this: unknown, ...args: unknown[]) {
      console.log(`  enter ${name}`);
      const r = original.apply(this, args);
      console.log(`  exit ${name}`);
      return r;
    };
  };
}

function classTag(cls: Function, context: ClassDecoratorContext) {
  console.log(`apply class decorator to ${context.name}`);
}

@classTag
class Report {
  @tag("outer")
  @tag("inner")
  build() { console.log("    building report"); return "done"; }
}

console.log("--- calling build() ---");
new Report().build();
```

OUTPUT

```
evaluate factory outer
evaluate factory inner
apply decorator inner to build
apply decorator outer to build
apply class decorator to Report
--- calling build() ---
  enter outer
  enter inner
    building report
  exit inner
  exit outer
```



EVALUATION ORDER OF STACKED DECORATORS

16.4 Practical decorators

16.4.1 @log, @measure, @memoize, @readonly

Four classic decorators show the main techniques: **@log** wraps a method to record arguments and results; **@measure** times it with `performance.now()` (supporting async methods); **@memoize** caches results per instance and arguments in a `WeakMap` so cached data is released with the object; and **@readonly** makes a field non-writable by defining it with `Object.defineProperty` in an initializer. All are typed generically so they keep the decorated method's signature.

@LOG, @MEASURE (SYNC AND ASYNC), @MEMOIZE AND @READONLY main.ts

```
type Method<This, Args extends unknown[], R> = (this: This, ...args: Args) => R;
type Ctx<This, Args extends unknown[], R> = ClassMethodDecoratorContext<This, Method<This, Args, R>>;

function log<This, Args extends unknown[], R>(fn: Method<This, Args, R>, ctx: Ctx<This, Args, R>) {
  return function (this: This, ...args: Args): R {
    const r = fn.call(this, ...args);
    console.log(`[log] ${String(ctx.name)}(${args.join(", ")}) = ${JSON.stringify(r)}`);
    return r;
  };
}

function measure<This, Args extends unknown[], R>(fn: Method<This, Args, R>, ctx: Ctx<This, Args, R>)
{
  return function (this: This, ...args: Args): R {
    const start = performance.now();
    const report = () => console.log(`[measure] ${String(ctx.name)} took ${performance.now() - start <
1000 ? "under 1 s" : "over 1 s"}`);
    const r = fn.call(this, ...args);
    if (r instanceof Promise) return r.finally(report) as R;
    report();
    return r;
  };
}

function memoize<This extends object, Args extends unknown[], R>(fn: Method<This, Args, R>, _ctx:
Ctx<This, Args, R>) {
  const caches = new WeakMap<This, Map<string, R>>();
  return function (this: This, ...args: Args): R {
    let cache = caches.get(this);
    if (!cache) caches.set(this, (cache = new Map()));
    const key = JSON.stringify(args);
    if (!cache.has(key)) cache.set(key, fn.call(this, ...args));
    return cache.get(key)!;
  };
}

function readonly<This, V>(_: undefined, context: ClassFieldDecoratorContext<This, V>) {
  context.addInitializer(function (this: This) {
    const value = (this as any)[context.name];
    Object.defineProperty(this, context.name, { value, writable: false, enumerable: true,
configurable: false });
  });
}

class FeeCalculator {
  @readonly currency = "INR";
  calls = 0;

  @log
  @memoize
```

```
  slowFee(base: number, months: number): number {
    this.calls++;
    let total = 0;
    for (let i = 0; i < 2_000_000; i++) total += base / 2_000_000;   // pretend this is expensive
    return Math.round((total * months) / 12);
  }

  @measure
  async fetchRate(code: string): Promise<number> {
    await new Promise((r) => setTimeout(r, 20));
    return code === "USD" ? 0.012 : 1;
  }
}

const calc = new FeeCalculator();
calc.slowFee(12000, 6);
calc.slowFee(12000, 6);                  // served from the cache (still logged: @log is outside)
console.log("real computations:", calc.calls);
try { (calc as any).currency = "USD"; } catch (e) { console.log("readonly:", (e as Error).message); }
calc.fetchRate("USD").then((r) => console.log("rate", r, calc.currency));
```

OUTPUT

```
[log] slowFee(12000, 6) = 6000
[log] slowFee(12000, 6) = 6000
real computations: 1
readonly: Cannot assign to read only property 'currency' of object '#<FeeCalculator>'
[measure] fetchRate took under 1 s
rate 0.012 INR
```

16.5 Decorators in frameworks

Large frameworks use decorators to attach **metadata** to classes, which the framework reads at startup to wire the application together: which class is a component and what its template is, which methods handle which HTTP routes, which classes are database tables. These frameworks use the **legacy** decorators, so their projects set `"experimentalDecorators": true` and usually `"emitDecoratorMetadata": true`, which makes the compiler emit the constructor parameter types so the dependency-injection container can create services automatically. The examples below need each framework installed; they show typical code rather than generated output.

16.5.1 Angular: @Component, @Injectable, @Input

ANGULAR COMPONENTS AND SERVICES ARE CONFIGURED WITH DECORATORS

main.ts

```ts
import { Component, Injectable, Input, inject } from "@angular/core";

@Injectable({ providedIn: "root" })              // a singleton service available everywhere
export class CourseService {
  private courses = [{ code: "TS101", title: "TypeScript", fee: 9999 }];
  all() { return this.courses; }
}

@Component({
  selector: "lbf-course-card",
  standalone: true,
  template: `<h3>{{ title }}</h3><p>Fee: Rs {{ fee }}</p>`,
})
export class CourseCardComponent {
  @Input({ required: true }) title!: string;      // set by the parent: <lbf-course-card [title]="..."
/>
  @Input() fee = 0;
  private service = inject(CourseService);         // dependency injection
}
```

Angular projects (Angular CLI) configure the compiler for Angular's decorators automatically. Newer Angular versions also offer function-based APIs such as `input()` and `inject()`.

16.5.2 NestJS: @Controller, @Get, @Injectable

A NESTJS CONTROLLER AND SERVICE main.ts

```
import { Body, Controller, Get, Injectable, Module, Param, ParseIntPipe, Post } from "@nestjs/common";

@Injectable()
export class CoursesService {
  private courses = [{ id: 1, code: "TS101", fee: 9999 }];
  findAll() { return this.courses; }
  findOne(id: number) { return this.courses.find((c) => c.id === id); }
  create(dto: { code: string; fee: number }) { const c = { id: this.courses.length + 1, ...dto };
this.courses.push(c); return c; }
}

@Controller("courses")                          // routes start with /courses
export class CoursesController {
  constructor(private readonly service: CoursesService) {}   // injected using emitted type metadata

  @Get()                                        // GET /courses
  findAll() { return this.service.findAll(); }

  @Get(":id")                                   // GET /courses/1
  findOne(@Param("id", ParseIntPipe) id: number) { return this.service.findOne(id); }

  @Post()                                       // POST /courses
  create(@Body() dto: { code: string; fee: number }) { return this.service.create(dto); }
}

@Module({ controllers: [CoursesController], providers: [CoursesService] })
export class CoursesModule {}
```

HTTP REQUESTS AND RESPONSES

```
GET  /courses     -> [{"id":1,"code":"TS101","fee":9999}]
GET  /courses/1   -> {"id":1,"code":"TS101","fee":9999}
POST /courses     {"code":"PY201","fee":14999} -> {"id":2,"code":"PY201","fee":14999}
```

Note the parameter decorators (`@Param`, `@Body`), which exist only in legacy decorators.

16.5.3 TypeORM: @Entity, @Column

TYPEORM ENTITIES: TABLES AND COLUMNS DESCRIBED BY DECORATORS main.ts

```
import { Column, CreateDateColumn, Entity, ManyToOne, OneToMany, PrimaryGeneratedColumn } from
"typeorm";

@Entity("courses")                              // maps the class to the "courses" table
export class Course {
  @PrimaryGeneratedColumn() id!: number;
  @Column({ unique: true, length: 10 }) code!: string;
  @Column() title!: string;
  @Column("int") fee!: number;
  @OneToMany(() => Enrolment, (e) => e.course) enrolments!: Enrolment[];
  @CreateDateColumn() createdAt!: Date;
}

@Entity("enrolments")
export class Enrolment {
  @PrimaryGeneratedColumn() id!: number;
  @Column() studentName!: string;
  @ManyToOne(() => Course, (c) => c.enrolments) course!: Course;
}

// const ts101 = await dataSource.getRepository(Course).findOneBy({ code: "TS101" });
```

The definite assignment assertions (`!` , 9.1.2) are needed because TypeORM, not the constructor, sets the fields. Prisma and Drizzle (18.3) take a different, decorator-free approach.

Decorators are functions applied with @ to classes and members when the class is defined; they receive the decorated value and a typed context and may replace it, which moves cross-cutting concerns out of business logic. TypeScript 5.0+ implements standard ECMAScript decorators (no flag, context objects, no parameter decorators); Angular, NestJS and TypeORM use legacy experimentalDecorators (with emitDecoratorMetadata), and the two are incompatible. Class decorators can register or subclass; method decorators wrap functions (type them with This, Args and R); auto-accessor decorators (accessor x) validate and track values; field decorators return initialisers and can record context.metadata; addInitializer runs per-instance or per-class setup. Factories (@retry(3)) return decorators; stacked decorators are evaluated top-down and applied bottom-up. Practical examples are @log, @measure, @memoize (WeakMap caches) and @readonly.

PRACTICE CHECKPOINT

Write **@log** and **@measure** method decorators and a **@validateRange(min, max)** decorator factory.

SAMPLE SOLUTION: @LOG, @MEASURE AND @VALIDATERANGE(MIN, MAX) main.ts

```
type Fn<This, Args extends unknown[], R> = (this: This, ...args: Args) => R;
type MethodCtx<This, Args extends unknown[], R> = ClassMethodDecoratorContext<This, Fn<This, Args,
R>>;

const lines: string[] = [];
const out = (s: string) => { lines.push(s); console.log(s); };

// ---------- @log: arguments, result and errors
function log<This, Args extends unknown[], R>(fn: Fn<This, Args, R>, ctx: MethodCtx<This, Args, R>):
Fn<This, Args, R> {
  const name = String(ctx.name);
```

```
  return function (this: This, ...args: Args): R {
    try {
      const result = fn.call(this, ...args);
      out(`[log] ${name}(${args.map((a) => JSON.stringify(a)).join(", ")}) ->
${JSON.stringify(result)}`);
      return result;
    } catch (e) {
      out(`[log] ${name}(${args.map((a) => JSON.stringify(a)).join(", ")}) threw ${(e as
Error).message}`);
      throw e;
    }
  };
}

// ---------- @measure: duration, for sync and async methods
function measure<This, Args extends unknown[], R>(fn: Fn<This, Args, R>, ctx: MethodCtx<This, Args,
R>): Fn<This, Args, R> {
  return function (this: This, ...args: Args): R {
    const t0 = performance.now();
    const done = () => out(`[measure] ${String(ctx.name)} finished (${performance.now() - t0 >= 0 ?
"timed" : "?"})`);
    const result = fn.call(this, ...args);
    if (result instanceof Promise) return result.finally(done) as R;
    done();
    return result;
  };
}

// ---------- @validateRange(min, max): checks every numeric argument
function validateRange(min: number, max: number) {
  return function <This, Args extends number[], R>(fn: Fn<This, Args, R>, ctx: MethodCtx<This, Args,
R>): Fn<This, Args, R> {
    return function (this: This, ...args: Args): R {
      args.forEach((value, i) => {
        if (!Number.isFinite(value) || value < min || value > max) {
          throw new RangeError(`${String(ctx.name)}: argument ${i + 1} must be within ${min}..${max},
got ${value}`);
        }
      });
      return fn.call(this, ...args);
    };
  };
}

class Gradebook {
  #marks: number[] = [];

  @log
  @validateRange(0, 100)
  add(...marks: number[]): number {
    this.#marks.push(...marks);
    return this.#marks.length;
  }

  @log
  @measure
  average(): number {
    return Number((this.#marks.reduce((a, b) => a + b, 0) / this.#marks.length).toFixed(2));
  }

  @measure
  async publish(): Promise<string> {
    await new Promise((r) => setTimeout(r, 15));
    return `published ${this.#marks.length} marks`;
```

```
  }
}

const book = new Gradebook();
book.add(91, 84);
book.add(77);
try { book.add(65, 120); } catch (e) { console.log("rejected:", (e as Error).message); }
book.average();
book.publish().then((msg) => console.log(msg, "| log lines:", lines.length));
```

OUTPUT

```
[log] add(91, 84) -> 2
[log] add(77) -> 3
[log] add(65, 120) threw add: argument 2 must be within 0..100, got 120
rejected: add: argument 2 must be within 0..100, got 120
[measure] average finished (timed)
[log] average() -> 84
[measure] publish finished (timed)
published 3 marks | log lines: 6
```

MODULE 17 · ADVANCED

Compiler configuration and tooling

Minimum duration **2 days** Level **Advanced**

Real projects depend on the right compiler settings and toolchain. This module covers tsconfig in depth: the strict family (strict, noImplicitAny, strictNullChecks, noUncheckedIndexedAccess, exactOptionalPropertyTypes), output options (target, lib, sourceMap, declaration), isolatedModules, skipLibCheck and noEmit, extending shared base configs; project references, tsc --build and incremental builds for monorepos; tsc for type checking versus bundlers for building (Vite, esbuild, SWC, tsup); ESLint with typescript-eslint and Prettier; running TypeScript on the server with Node.js type stripping, erasableSyntaxOnly and tsx; and an overview of the compiler's internals and the native Go-based compiler.

IN THIS MODULE

17.1 tsconfig in depth
17.2 Project references and monorepos
17.3 Build tools
17.4 Code quality
17.5 Running TypeScript on the server
17.6 Compiler internals (overview)

17.1 tsconfig in depth

`tsconfig.json` has two kinds of settings: **which files** belong to the project (`include`, `exclude`, `files`, `references`, 2.5.5) and **compilerOptions**, which fall into groups: type checking, modules, emit (output), JavaScript support, interop and projects. `tsc --showConfig` prints the final configuration after defaults and `extends` are applied, and `tsc --explainFiles` explains why each file is included. The TSConfig reference at typescriptlang.org/tsconfig documents every option.

17.1.1 Strict family: strict, noImplicitAny, strictNullChecks, noUncheckedIndexedAccess, exactOptionalPropertyTypes

`"strict": true` turns on a family of checks (2.5.4) and automatically includes new strict checks added in future versions. You can turn individual members off (`"strict": true, "strictPropertyInitialization": false`) during a migration. Two valuable checks are **not** part of strict, because they require more code changes: `noUncheckedIndexedAccess` (index access returns `T | undefined`, 5.5.3) and `exactOptionalPropertyTypes`, which distinguishes a **missing** optional property from one explicitly set to `undefined`. Further "linting" options catch common mistakes: `noImplicitReturns`, `noFallthroughCasesInSwitch`, `noImplicitOverride` (9.4.3), `noUnusedLocals`, `noUnusedParameters` and `noPropertyAccessFromIndexSignature`.

Option	In strict?	What it catches
`noImplicitAny`	yes	variables and parameters whose type would silently be any
`strictNullChecks`	yes	using values that may be null or undefined
`strictFunctionTypes`	yes	unsafe function parameter types (15.4.3)

Option	In strict?	What it catches
`strictBindCallApply`	yes	wrong arguments to bind, call and apply
`strictPropertyInitialization`	yes	class fields not initialised (9.1.2)
`noImplicitThis`, `alwaysStrict`	yes	this of type any; emits "use strict"
`useUnknownInCatchVariables`	yes	catch variables typed unknown (13.3.4)
`noUncheckedIndexedAccess`	no	arr[i] and dict[key] may be undefined
`exactOptionalPropertyTypes`	no	assigning undefined to an optional property that does not allow it
`noImplicitReturns`	no	code paths that fall off the end without returning
`noFallthroughCasesInSwitch`	no	switch cases without break or return
`noUnusedLocals` / `noUnusedParameters`	no	dead variables, imports and parameters

SRC/CHECKS.TS TypeScript

```
const debugMode = true;                              // declared but never used

interface Profile { name: string; bio?: string }
const p: Profile = { name: "Asha", bio: undefined };   // bio may be missing, but not explicitly
undefined

function label(level: number): string {
  if (level > 5) return "expert";
  if (level > 2) return "intermediate";
}                                                    // falls off the end

function describe(kind: "a" | "b"): string {
  let text = "";
  switch (kind) {
    case "a":
      text = "first";
    case "b":                                        // fallthrough from "a"
      text += "second";
      break;
  }
  return text;
}
export { p, label, describe };
```

CHECKS BEYOND STRICT: EXACTOPTIONALPROPERTYTYPES, NOIMPLICITRETURNS, NOFALLTHROUGHCASESINSWITCH, NOUNUSEDLOCALS

terminal

```
npx tsc --noEmit -p .
```

TERMINAL

```
$ npx tsc --noEmit -p .
src/checks.ts(1,7): error TS6133: 'debugMode' is declared but its value is never read.
src/checks.ts(4,7): error TS2375: Type '{ name: string; bio: undefined; }' is not assignable to type
'Profile' with 'exactOptionalPropertyTypes: true'. Consider adding 'undefined' to the types of the
target's properties.
  Types of property 'bio' are incompatible.
    Type 'undefined' is not assignable to type 'string'.
src/checks.ts(6,32): error TS2366: Function lacks ending return statement and return type does not
include 'undefined'.
src/checks.ts(14,5): error TS7029: Fallthrough case in switch.
```

17.1.2 Output options: target, lib, sourceMap, declaration

`target` (2.5.2) chooses the JavaScript syntax version of the output; `lib` chooses which built-in APIs the type checker assumes exist (`es2023`, `dom`, `dom.iterable`, `webworker`; it defaults from `target`), independently of syntax. They must match your runtime: a high `lib` with an old browser lets you call methods that do not exist there. `sourceMap` writes `.js.map` files so that debuggers and error stack traces point to the original `.ts` lines (Node.js: `node --enable-source-maps`); `inlineSourceMap` embeds them. `declaration` (11.4.2) and `declarationMap` produce `.d.ts` files; `outDir`, `rootDir`, `removeComments`, `newLine` and `noEmitOnError` control the rest of the output.

SRC/MAIN.TS

TypeScript

```
interface Order { id: number; total: number }

function checkTotal(order: Order): number {
  if (order.total < 0) {
    throw new Error(`order ${order.id} has a negative total`);    // line 5 of main.ts
  }
  return order.total;
}

checkTotal({ id: 7, total: -10 });
```

SOURCE MAPS MAKE STACK TRACES POINT TO THE TYPESCRIPT SOURCE

terminal

```
npx tsc -p .
ls dist
node --enable-source-maps dist/main.js 2>&1 | grep -E "Error|main.ts" | head -n 3
```

TERMINAL

```
$ npx tsc -p .
$ ls dist
main.d.ts
main.js
main.js.map
$ node --enable-source-maps dist/main.js 2>&1 | grep -E "Error|main.ts" | head -n 3
src/main.ts:5
    throw new Error(`order ${order.id} has a negative total`);    // line 5 of main.ts
Error: order 7 has a negative total
```

17.1.3 isolatedModules, skipLibCheck, noEmit

`isolatedModules` makes tsc report code that cannot be compiled **one file at a time**, which is how esbuild, SWC, Babel and Node.js type stripping work (they do not look at other files, so they cannot know whether an imported name is a type). It flags re-exporting types without `export type`, using `const enum` values across files, and files that are not modules. Enable it whenever another tool does the compiling (Vite, Next.js); `verbatimModuleSyntax` (11.1.3) implies most of it. `skipLibCheck` skips type-checking all `.d.ts` files (your own and those in `node_modules`), which speeds up builds a lot and avoids errors from conflicting third-party declarations; it is enabled in almost every real project. `noEmit` makes tsc a pure type checker that writes nothing, the usual setting when a bundler builds the code.

SRC/INDEX.TS TypeScript

```ts
export { Course } from "./types";          // re-exporting a type needs export type
import { Level } from "./types";
console.log(Level.Advanced);               // const enum value from another file
```

ISOLATEDMODULES FLAGS CODE THAT SINGLE-FILE COMPILERS CANNOT HANDLE terminal

```
npx tsc --noEmit -p .
```

```
TERMINAL
$ npx tsc --noEmit -p .
src/index.ts(1,10): error TS1205: Re-exporting a type when 'isolatedModules' is enabled requires using
'export type'.
```

17.1.4 Extending configs and shared base configs

`"extends": "./tsconfig.base.json"` inherits all options from another file, and the current file overrides or adds to them (`include`, `files` and `exclude` are replaced, not merged; paths are resolved relative to the file that defines them). Since TypeScript 5.0, `extends` can be an array, merged in order. Monorepos keep one base config with the team's strictness settings and give each package a small config that extends it. The community **@tsconfig/bases** packages provide recommended bases for each runtime and framework (`@tsconfig/node22`, `@tsconfig/strictest`, `@tsconfig/vite-react`).

TSCONFIG.BASE.JSON JavaScript

```json
{
  "compilerOptions": {
    "strict": true,
    "noUncheckedIndexedAccess": true,
    "target": "es2022",
    "module": "nodenext",
    "skipLibCheck": true,
    "declaration": true,
    "types": []
  }
}
```

A PACKAGE CONFIG EXTENDING A SHARED BASE, AND THE RESOLVED RESULT

terminal

```
cat packages/api/tsconfig.json
npx tsc --showConfig -p packages/api
```

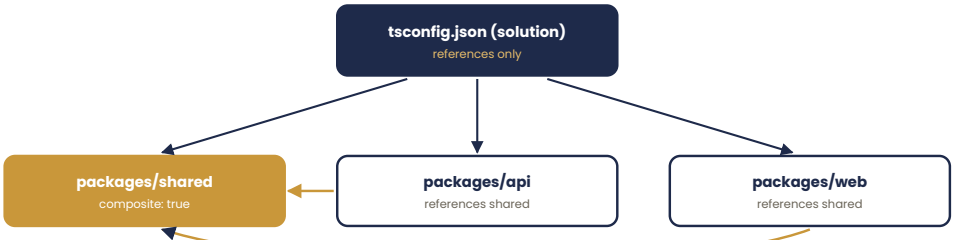
TERMINAL

```
$ cat packages/api/tsconfig.json
{
  "extends": "../../tsconfig.base.json",
  "compilerOptions": {
    "rootDir": "src",
    "outDir": "dist",
    "noImplicitReturns": true
  },
  "include": ["src"]
}
$ npx tsc --showConfig -p packages/api
{
    "compilerOptions": {
        "strict": true,
        "noUncheckedIndexedAccess": true,
        "target": "es2022",
        "module": "nodenext",
        "skipLibCheck": true,
        "declaration": true,
        "types": [],
        "rootDir": "./src",
        "outDir": "./dist",
        "noImplicitReturns": true,
        "moduleResolution": "nodenext",
        "moduleDetection": "force"
    },
    "files": [
        "./src/index.ts"
    ],
    "include": [
        "src"
    ],
    "exclude": [
        "packages/api/dist"
    ]
}
```

17.2 Project references and monorepos

17.2.1 references and tsc --build

A large codebase or a **monorepo** (several packages in one repository, typically managed with npm, pnpm or Yarn workspaces, plus Turborepo or Nx) is better split into several TypeScript **projects** that reference each other. A project that others depend on sets `"composite": true` (which requires `declaration` and makes all files explicit), and dependants list it in `"references": [{ "path": "../shared" }]`. Then `tsc --build` (or `tsc -b`) builds the projects **in dependency order**, uses each project's `.d.ts` output instead of re-checking its sources, and **skips** projects that are up to date. Editors also understand references, so "go to definition" jumps into the other package's source. Use `tsc -b --verbose` to see what is rebuilt and why, and `tsc -b --clean` to delete outputs.



A MONOREPO WITH PROJECT REFERENCES: SHARED IS BUILT FIRST, THEN API AND WEB

TSCONFIG.JSON JavaScript

```
{ "files": [], "references": [{ "path": "packages/shared" }, { "path": "packages/api" }] }
```

PACKAGES/API/TSCONFIG.JSON JavaScript

```
{ "extends": "../../tsconfig.base.json", "compilerOptions": { "rootDir": "src", "outDir": "dist" },
"include": ["src"], "references": [{ "path": "../shared" }] }
```

TSC --BUILD BUILDS REFERENCED PROJECTS IN ORDER AND SKIPS UP-TO-DATE ONES terminal

```
npx tsc -b --verbose 2>&1 | grep -E "Building|up to date" | sed 's/^.*- //'
node packages/api/dist/index.js
echo "--- second build: nothing changed ---"
npx tsc -b --verbose 2>&1 | grep -E "Building|up to date" | sed 's/^.*- //'
sed -i 's/18/12/' packages/shared/src/index.ts
echo "--- after editing shared ---"
npx tsc -b --verbose 2>&1 | grep -E "Building|up to date" | sed 's/^.*- //'
node packages/api/dist/index.js
```

TERMINAL

```
$ npx tsc -b --verbose 2>&1 | grep -E "Building|up to date" | sed 's/^.*- //'
Building project 'packages/shared/tsconfig.json'...
Building project 'packages/api/tsconfig.json'...
$ node packages/api/dist/index.js
TS101: Rs 11799 (GST 18%)
$ echo "--- second build: nothing changed ---"
--- second build: nothing changed ---
$ npx tsc -b --verbose 2>&1 | grep -E "Building|up to date" | sed 's/^.*- //'
Project 'packages/shared/tsconfig.json' is up to date because newest input
'packages/shared/src/index.ts' is older than output 'packages/shared/tsconfig.tsbuildinfo'
Project 'packages/api/tsconfig.json' is up to date because newest input 'packages/api/src/index.ts' is
older than output 'packages/api/tsconfig.tsbuildinfo'
$ sed -i 's/18/12/' packages/shared/src/index.ts
$ echo "--- after editing shared ---"
--- after editing shared ---
$ npx tsc -b --verbose 2>&1 | grep -E "Building|up to date" | sed 's/^.*- //'
Building project 'packages/shared/tsconfig.json'...
Building project 'packages/api/tsconfig.json'...
$ node packages/api/dist/index.js
TS101: Rs 11199 (GST 12%)
```

In a real workspace, api would import `"@lbf/shared"` through the package manager's workspace link; the relative import keeps this example self-contained.

17.2.2 Incremental builds

With `"incremental": true` (implied by `composite`), tsc saves information about the last build in a `.tsbuildinfo` file and, on the next run, re-checks only the files affected by changes. Combined with project references, it makes rebuilds of large codebases much faster. `tsc --watch` is incremental in

memory. In CI, cache the `.tsbuildinfo` files between runs to benefit there too; delete them (or run `tsc -b --clean`) if a build ever seems stale.

17.3 Build tools

17.3.1 tsc for type checking vs bundlers for building

Modern projects usually split two jobs that `tsc` can do alone. **Type checking** is done by `tsc --noEmit` (in the editor, in a pre-commit hook and in CI). **Building** (removing types, bundling modules, minifying, handling CSS and assets, dev server with hot reload) is done by a fast tool written in Go or Rust that simply strips types without checking them: esbuild, SWC, Rolldown/Oxc, or a bundler built on them such as Vite. They are 10 to 100 times faster than tsc at emitting, which keeps the dev loop instant, but because they never report type errors, the separate type-check step is essential. Libraries still use tsc (or a dedicated tool) to produce `.d.ts` files.



TYPE CHECKING AND BUILDING ARE SEPARATE STEPS IN MODERN TOOLCHAINS

17.3.2 Vite, esbuild and SWC

Vite is the standard front-end build tool: `npm create vite@latest my-app -- --template react-ts` (or `vanilla-ts`, `vue-ts`, `svelte-ts`) creates a TypeScript project with a dev server that serves source files on demand with instant hot module replacement, and a production build that bundles with Rollup (Rolldown in Vite 7+). Its templates set `"moduleResolution": "bundler"`, `"noEmit": true` and `"isolatedModules": true`, and the build script runs `tsc -b && vite build` so type errors fail the build. **esbuild** (Go) is an extremely fast bundler and transpiler used inside Vite and many tools; **SWC** (Rust) is a compiler used by Next.js, Jest (via `@swc/jest`) and others. Both only strip types.

PACKAGE.JSON OF A VITE + TYPESCRIPT PROJECT: TYPE-CHECK, THEN BUILD — JavaScript

```
{
  "name": "lbf-portal",
  "private": true,
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "tsc -b && vite build",
    "typecheck": "tsc -b --noEmit",
    "preview": "vite preview",
    "lint": "eslint ."
  },
  "devDependencies": {
    "@vitejs/plugin-react": "^5.0.0",
    "typescript": "^6.0.0",
    "vite": "^7.0.0"
  }
}
```

BUNDLING A TYPESCRIPT ENTRY POINT WITH ESBUILD terminal

```
npx esbuild src/index.ts --bundle --platform=node --format=esm --outfile=dist/index.js --sourcemap --
minify
```

```
TERMINAL
  dist/index.js      2.1kb
  dist/index.js.map  6.4kb

Done in 4ms
```

Requires `npm install -D esbuild`. esbuild bundled and minified the code in milliseconds, but it did not check types: run `tsc --noEmit` as well.

17.3.3 Building libraries with tsup

Publishing a TypeScript library to npm means producing JavaScript for both ESM and (often) CommonJS consumers, `.d.ts` declarations, source maps, and a `package.json` whose `"exports"` field points each condition to the right file. **tsup** (built on esbuild) does this with one command; alternatives include **tsdown** (built on Rolldown), **unbuild** and plain `tsc` for ESM-only packages. Check the result with `npx @arethetypeswrong/cli`, which detects broken type resolution for different module settings.

A TSUP CONFIGURATION AND THE MATCHING PACKAGE.JSON EXPORTS TypeScript

```ts
// tsup.config.ts
import { defineConfig } from "tsup";

export default defineConfig({
  entry: ["src/index.ts"],
  format: ["esm", "cjs"],       // dist/index.js and dist/index.cjs
  dts: true,                    // dist/index.d.ts and dist/index.d.cts
  sourcemap: true,
  clean: true,
});

// package.json (relevant fields)
// "type": "module",
// "exports": {
//   ".": {
//     "import": { "types": "./dist/index.d.ts", "default": "./dist/index.js" },
//     "require": { "types": "./dist/index.d.cts", "default": "./dist/index.cjs" }
//   }
// },
// "files": ["dist"],
// "scripts": { "build": "tsup", "prepublishOnly": "npm run build" }
```

17.4 Code quality

17.4.1 ESLint with typescript-eslint

The compiler checks types; a **linter** checks for bug-prone patterns and team conventions that are type-correct but unwise: floating promises that are never awaited, `any` creeping in, unnecessary conditions, misused promises in callbacks, non-null assertions, unused variables. **ESLint** with **typescript-eslint** is the standard. Its **typed linting** mode gives rules access to the type checker, enabling powerful rules such as `no-floating-promises` and `no-unsafe-assignment`. Configuration uses the flat config file `eslint.config.js`. Biome and Oxlint are fast Rust-based alternatives with many of the same rules.

AN ESLINT FLAT CONFIG WITH TYPED LINTING TypeScript

```ts
// eslint.config.js
import eslint from "@eslint/js";
import tseslint from "typescript-eslint";

export default tseslint.config(
  eslint.configs.recommended,
  tseslint.configs.strictTypeChecked,          // recommended + strict rules that use type information
  tseslint.configs.stylisticTypeChecked,
  {
    languageOptions: { parserOptions: { projectService: true, tsconfigRootDir: import.meta.dirname } }
},
    rules: {
      "@typescript-eslint/no-floating-promises": "error",
      "@typescript-eslint/consistent-type-definitions": ["error", "interface"],
      "@typescript-eslint/no-non-null-assertion": "warn",
    },
  },
  { ignores: ["dist/**"] },
);
```

PROBLEMS THAT TYPE-CHECK BUT THAT TYPESCRIPT-ESLINT REPORTS main.ts

```ts
// src/enrol.ts
export async function saveEnrolment(id: number): Promise<void> { /* ... */ }

export function onSubmit(id: number) {
  saveEnrolment(id);                           // promise never awaited or handled
  const data: any = JSON.parse("{}");
  const name: string = data.name;              // any leaks into a typed variable
  if (id !== null) console.log(name);          // id can never be null: unnecessary condition
}
```

npx eslint src

```
src/enrol.ts
  5:3   error  Promises must be awaited, end with a call to .catch, or be explicitly marked as ignored
with the void operator  @typescript-eslint/no-floating-promises
  6:13  error  Unexpected any. Specify a different type
@typescript-eslint/no-explicit-any
  7:9   error  Unsafe assignment of an any value
@typescript-eslint/no-unsafe-assignment
  8:7   error  Unnecessary conditional, the types have no overlap
@typescript-eslint/no-unnecessary-condition

4 problems (4 errors, 0 warnings)
```

Requires `npm install -D eslint @eslint/js typescript-eslint`.

17.4.2 Prettier

Prettier is an opinionated code formatter: it reprints your code in a consistent style (indentation, quotes, semicolons, line breaks), so formatting is never discussed in code reviews. Configure it once (`.prettierrc`), run it on save in the editor and check it in CI (`prettier --check .`). Leave formatting to Prettier and code-quality rules to ESLint (the typescript-eslint stylistic configs do not conflict with it, and `eslint-config-prettier` turns off any remaining overlapping rules). Biome combines a formatter and linter in one fast tool.

MESSY.TS TypeScript

```
interface Course{code:string,fee:number;tags?:string[]}
const courses:Course[]=[{code:"TS101",fee:9999,tags:['web','types']},{code:'PY201',fee:14999}]
export function cheapest(list:Course[]):Course|undefined{return [...list].sort((a,b)=>a.fee-b.fee)[0]}
```

PRETTIER REFORMATS CODE CONSISTENTLY terminal

```
cat messy.ts
prettier --print-width 90 messy.ts
```

```
TERMINAL
$ cat messy.ts
interface Course{code:string,fee:number;tags?:string[]}
const courses:Course[]=[{code:"TS101",fee:9999,tags:['web','types']},{code:'PY201',fee:14999}]
export function cheapest(list:Course[]):Course|undefined{return [...list].sort((a,b)=>a.fee-b.fee)[0]}
$ prettier --print-width 90 messy.ts
interface Course {
  code: string;
  fee: number;
  tags?: string[];
}
const courses: Course[] = [
  { code: "TS101", fee: 9999, tags: ["web", "types"] },
  { code: "PY201", fee: 14999 },
];
export function cheapest(list: Course[]): Course | undefined {
  return [...list].sort((a, b) => a.fee - b.fee)[0];
}
```

17.5 Running TypeScript on the server

17.5.1 Node.js type stripping and erasableSyntaxOnly

Node.js 22.18+ and 23.6+ run `.ts` files directly by **type stripping** (2.4.2): the type annotations are replaced by spaces, so line and column numbers stay identical and no source maps are needed. It supports only erasable syntax, and it does not read `tsconfig.json` or check types. To develop comfortably with it: set `"erasableSyntaxOnly": true` so tsc rejects enums, namespaces and parameter properties (6.4.5); set `"verbatimModuleSyntax": true` so type imports are explicit; write imports with `.ts` extensions and enable `"allowImportingTsExtensions"` (with `noEmit`) or `"rewriteRelativeImportExtensions"` (which rewrites `.ts` to `.js` when tsc does emit); and keep `tsc --noEmit` for checking. TypeScript in `node_modules` is not stripped: published packages must still ship JavaScript.

TSCONFIG.JSON JavaScript

```
{
  "compilerOptions": {
    "strict": true,
    "target": "esnext",
    "module": "nodenext",
    "noEmit": true,
    "erasableSyntaxOnly": true,
    "verbatimModuleSyntax": true,
    "allowImportingTsExtensions": true,
    "types": ["node"],
    "skipLibCheck": true
  },
  "include": ["src"]
}
```

SRC/SERVER.TS TypeScript

```
import { createServer } from "node:http";
import { courses, Level, type Course } from "./courses.ts";      // .ts extension, type-only import

const summary = (list: Course[]) => list.map((c) => `${c.code}=${c.fee}`).join(", ");
console.log("courses:", summary(courses), "| level:", Level.Advanced);
const server = createServer((_req, res) => res.end(JSON.stringify(courses)));
console.log("server object created:", typeof server.listen);
```

A NODE.JS APP RUN DIRECTLY FROM .TS FILES, TYPE-CHECKED SEPARATELY terminal

```
npx tsc -p . && echo "type check passed"
node src/server.ts
```

TERMINAL

```
$ npx tsc -p . && echo "type check passed"
type check passed
$ node src/server.ts
courses: TS101=9999, PY201=14999 | level: advanced
server object created: function
```

17.5.2 tsx for development

tsx (2.4.1) runs TypeScript with esbuild and supports everything type stripping does not: enums, namespaces, parameter properties, decorators, `paths` aliases from tsconfig, extensionless imports and JSX. `tsx watch src/server.ts` restarts the process on every change, which makes it the most common way to run a Node.js TypeScript server during development. For production, either compile with tsc (or a bundler) and run the JavaScript, or, for erasable-only code, run with Node.js directly. Like type stripping, tsx does not type-check.

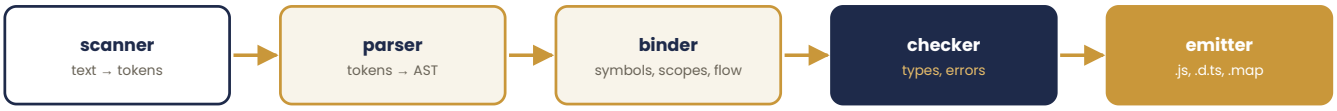
Approach	Speed	Type checks	Supports	Use for
`tsc` then `node dist/`	slower build	yes	everything	production builds, libraries
`node file.ts` (type stripping)	instant	no	erasable syntax only	scripts, simple services, tooling

Approach	Speed	Type checks	Supports	Use for
`tsx watch file.ts`	very fast	no	almost everything (esbuild)	development servers, CLIs
bundler (esbuild, tsup, Vite)	very fast	no	everything + bundling	deployable bundles, serverless

17.6 Compiler internals (overview)

17.6.1 Scanner, parser, binder, checker, emitter

The TypeScript compiler works in five main stages. The **scanner** (lexer) turns source text into **tokens** (keywords, identifiers, punctuation, literals). The **parser** builds an **abstract syntax tree** (AST) from the tokens and reports syntax errors. The **binder** walks the AST, creates **symbols** for declarations, links each name to its declaration through scopes, and builds the control-flow graph used for narrowing. The **checker**, by far the largest part (tens of thousands of lines), computes the type of every expression, checks assignability and reports semantic errors. The **emitter** produces JavaScript, declaration files and source maps, applying transformations for the target (down-levelling). The editor uses the same pipeline through the language service, which adds completion, quick fixes and refactorings on top of the checker.



THE STAGES OF THE TYPESCRIPT COMPILER

The compiler is also a library: `import ts from "typescript"` exposes the **Compiler API**, used by linters, code generators, documentation tools and editors. The program below uses it to show each stage on a small source file.

INSPECT.MJS

JavaScript

```
import ts from "typescript";

const source = 'const fee: number = 9999;\nconst total: string = fee * 1.18;\n';

// 1. scanner: text -> tokens
const scanner = ts.createScanner(ts.ScriptTarget.Latest, true, ts.LanguageVariant.Standard, source);
const tokens = [];
for (let t = scanner.scan(); t !== ts.SyntaxKind.EndOfFileToken; t = scanner.scan())
tokens.push(ts.SyntaxKind[t]);
console.log("tokens:", tokens.slice(0, 9).join(" "), "...");

// 2. parser: tokens -> AST
const file = ts.createSourceFile("demo.ts", source, ts.ScriptTarget.Latest, true);
const show = (node, depth = 0) => {
  if (depth < 4) console.log("  ".repeat(depth) + ts.SyntaxKind[node.kind]);
  node.forEachChild((child) => show(child, depth + 1));
};
show(file.statements[0]);

// 3 + 4. binder and checker: build a program and ask for diagnostics and types
const host = ts.createCompilerHost({ strict: true });
const original = host.getSourceFile;
host.getSourceFile = (name, ...rest) => (name === "demo.ts" ? file : original(name, ...rest));
const program = ts.createProgram(["demo.ts"], { strict: true, noEmit: true, types: [] }, host);
const checker = program.getTypeChecker();
const second = file.statements[1].declarationList.declarations[0].initializer;
console.log("type of `fee * 1.18`:", checker.typeToString(checker.getTypeAtLocation(second)));
for (const d of ts.getPreEmitDiagnostics(program, file)) console.log("diagnostic:",
ts.flattenDiagnosticMessageText(d.messageText, " "));

// 5. emitter: transpile to JavaScript (types removed)
console.log("emitted:", ts.transpileModule(source, { compilerOptions: { target: ts.ScriptTarget.ES2022
} }).outputText.trim().replace(/\n/g, " "));
```

LOOKING INSIDE THE COMPILER WITH THE TYPESCRIPT COMPILER API

terminal

```
node inspect.mjs
```

TERMINAL

```
$ node inspect.mjs
tokens: ConstKeyword Identifier ColonToken NumberKeyword FirstAssignment FirstLiteralToken
SemicolonToken ConstKeyword Identifier ...
FirstStatement
  VariableDeclarationList
    VariableDeclaration
      Identifier
      NumberKeyword
      FirstLiteralToken
type of `fee * 1.18`: number
diagnostic: Type 'number' is not assignable to type 'string'.
emitted: "use strict"; const fee = 9999; const total = fee * 1.18;
```

17.6.2 The native Go-based compiler and its speed gains

As introduced in 1.2.4, TypeScript 7 ("Project Corsa") is a port of the compiler from TypeScript to **Go**. It is a careful line-by-line port rather than a rewrite, so it produces the same types and errors. The speed comes from two sources: native code instead of JavaScript running on a JIT (about 3 to 4 times faster), and **shared-memory parallelism**: parsing, binding and checking of different files run on multiple CPU cores at once (a further 2 to 3 times). Microsoft reported full builds of large projects such as VS Code

dropping from over a minute to under ten seconds, with lower memory use and much faster editor start-up. It is available as `@typescript/native-preview` (the `tsgo` command, and a VS Code extension). TypeScript 6.0 is the bridge release: it deprecates options that 7.0 removes (such as `baseUrl`, `moduleResolution: node10`, `target: es5` and AMD/UMD/System modules), so a project that builds cleanly on 6.0 without `ignoreDeprecations` is ready for 7.0. The JavaScript-based Compiler API remains available for tools during the transition.

COMPARING TSC AND THE NATIVE PREVIEW COMPILER ON A LARGE PROJECT terminal

```
npm install -D @typescript/native-preview
npx tsgo --version
time npx tsc -p . --noEmit
time npx tsgo -p . --noEmit
```

```
TERMINAL
Version 7.0.0-dev.20260915.1
real    0m41.8s     (tsc)
real    0m4.6s      (tsgo)
```

Illustrative timings for a large codebase; the speed-up depends on project size and the number of CPU cores.

tsconfig selects files and compilerOptions. strict enables the core checks; add noUncheckedIndexedAccess, exactOptionalPropertyTypes, noImplicitReturns, noFallthroughCasesInSwitch and noUnused* for more safety. target sets output syntax, lib the assumed APIs, sourceMap maps errors back to .ts, declaration emits .d.ts. isolatedModules keeps code compatible with single-file compilers, skipLibCheck skips .d.ts checking, noEmit makes tsc a pure checker; extends shares base configs. Monorepos use composite projects, references and tsc --build with incremental builds. Bundlers and transpilers (Vite, esbuild, SWC, tsup) build fast but never type-check, so run tsc --noEmit too. typescript-eslint (with typed linting) and Prettier keep quality and formatting consistent. Node.js type stripping runs erasable TypeScript directly (pair it with erasableSyntaxOnly and verbatimModuleSyntax); tsx covers everything else in development. The compiler has scanner, parser, binder, checker and emitter stages, exposed by the Compiler API, and TypeScript 7 ports it to Go for about ten times faster builds.

PRACTICE CHECKPOINT

Set up a **strict monorepo-style project** with **two packages**, **project references**, **ESLint** and a **Vite build**.

Sample solution. The repository has a shared base config, a `shared` library package and a `web` application package that references it. The build below type-checks and builds both projects with `tsc -b` (the part that runs here); in the real project, Vite then bundles the web package and ESLint lints everything, using the configuration files shown.

PACKAGE.JSON JavaScript

```
{ "name": "lbf-monorepo", "private": true, "type": "module", "workspaces": ["packages/*"], "scripts":
{ "build": "tsc -b && npm run build -w web", "lint": "eslint .", "typecheck": "tsc -b" } }
```

TSCONFIG.BASE.JSON JavaScript

```
{
  "compilerOptions": {
    "strict": true,
    "noUncheckedIndexedAccess": true,
    "exactOptionalPropertyTypes": true,
    "noImplicitReturns": true,
    "noFallthroughCasesInSwitch": true,
    "verbatimModuleSyntax": true,
    "target": "es2022",
    "module": "nodenext",
    "composite": true,
    "declaration": true,
    "sourceMap": true,
    "skipLibCheck": true,
    "types": []
  }
}
```

TSCONFIG.JSON JavaScript

```
{ "files": [], "references": [{ "path": "packages/shared" }, { "path": "packages/web" }] }
```

PACKAGES/WEB/TSCONFIG.JSON JavaScript

```
{ "extends": "../../tsconfig.base.json", "compilerOptions": { "rootDir": "src", "outDir": "dist" },
"include": ["src"], "references": [{ "path": "../shared" }] }
```

ESLINT.CONFIG.JS JavaScript

```
import eslint from "@eslint/js";
import tseslint from "typescript-eslint";
export default tseslint.config(eslint.configs.recommended, tseslint.configs.strictTypeChecked,
  { languageOptions: { parserOptions: { projectService: true } } }, { ignores: ["**/dist/**"] });
```

SAMPLE SOLUTION: A STRICT TWO-PACKAGE WORKSPACE BUILT WITH PROJECT REFERENCES terminal

```
npx tsc -b --verbose 2>&1 | grep -E "Building project|is up to date" | sed 's/^.*- //'
find . -name "*.d.ts" -path "*dist*" | sort
node packages/web/dist/main.js
```

TERMINAL

```
$ npx tsc -b --verbose 2>&1 | grep -E "Building project|is up to date" | sed 's/^.*- //'
Building project 'packages/shared/tsconfig.json'...
Building project 'packages/web/tsconfig.json'...
$ find . -name "*.d.ts" -path "*dist*" | sort
./packages/shared/dist/index.d.ts
./packages/web/dist/main.d.ts
$ node packages/web/dist/main.js
TS101 TypeScript Rs 11799 / $141.59
PY201 Python     Rs 17699 / $212.39
```

With the workspace installed (`npm install`), the web package would import `"@lbf/shared"` and `npm run build` would run `tsc -b` followed by `vite build`; `npm run lint` runs ESLint with typed linting across both packages.

MODULE 18 · ADVANCED

TypeScript with frameworks and back ends

Minimum duration **4 days** | Level **Advanced**

Most TypeScript is written inside frameworks. This module shows the typing patterns used with React (props and children, useState, useRef and useReducer, events in JSX, context and custom hooks, generic components), with Node.js and Express (typed requests, responses and middleware, typed routes and DTOs, validating input with Zod, an overview of NestJS), with databases through Prisma or Drizzle, end-to-end type safety by sharing types between front end and back end (tRPC and OpenAPI-generated clients), and testing TypeScript with Vitest or Jest, typed mocks and type tests with expectTypeOf.

IN THIS MODULE

18.1 TypeScript with React
18.2 TypeScript with Node.js and Express
18.3 TypeScript with databases
18.4 End-to-end type safety
18.5 Testing TypeScript

NOTE

Framework examples need their packages installed (React, Express, Prisma, Zod, tRPC, Vitest), which is not possible while typesetting these notes, so their boxes show the code with the output or behaviour it produces. The Node.js examples that use only built-in modules were compiled and run for real.

18.1 TypeScript with React

React projects use `.tsx` files (TypeScript plus JSX) and the type declarations in `@types/react` and `@types/react-dom`. Create one with `npm create vite@latest my-app -- --template react-ts` (or Next.js, which uses TypeScript by default). The tsconfig sets `"jsx": "react-jsx"`. Most React typing is about three things: the props a component accepts, the state and refs it holds, and the events it handles.

18.1.1 Typing props and children

Describe a component's props with an interface or type alias and annotate the destructured parameter. Optional props get `?` and usually a default in the destructuring; callback props are function types; union props (`variant: "primary" | "ghost"`) restrict values. For content passed between the tags, use `children: React.ReactNode` (anything renderable: elements, strings, numbers, arrays, null). The older `React.FC` type is no longer recommended; a plain function with typed props is simpler. To accept all the attributes of a native element, extend `React.ComponentProps<"button">`.

TYPED PROPS, OPTIONAL PROPS WITH DEFAULTS, CHILDREN AND NATIVE ELEMENT PROPS main.ts

```
import type { ComponentProps, ReactNode } from "react";

interface CourseCardProps {
  code: string;
  title: string;
  fee: number;
  level?: "beginner" | "intermediate" | "advanced";
  tags?: readonly string[];
  onEnrol: (code: string) => void;          // callback prop
  children?: ReactNode;                      // anything renderable between the tags
}

export function CourseCard({ code, title, fee, level = "beginner", tags = [], onEnrol, children }:
CourseCardProps) {
  return (
    <article className={`card card--${level}`}>
      <h3>{title} <small>{code}</small></h3>
      <p>Rs {fee.toLocaleString("en-IN")}</p>
      {tags.length > 0 && <ul>{tags.map((t) => <li key={t}>{t}</li>)}</ul>}
      {children}
      <Button variant="primary" onClick={() => onEnrol(code)}>Enrol</Button>
    </article>
  );
}

// Extending a native element's props: every <button> attribute is accepted and typed
interface ButtonProps extends ComponentProps<"button"> { variant: "primary" | "ghost" }
export function Button({ variant, className = "", ...rest }: ButtonProps) {
  return <button className={`btn btn--${variant} ${className}`} {...rest} />;
}

// Usage: <CourseCard code="TS101" title="TypeScript" fee={9999} onEnrol={(c) => console.log(c)}>New
batch!</CourseCard>
// <CourseCard code="TS101" title="TypeScript" fee="9999" />   error: fee must be a number; onEnrol is
missing
```

18.1.2 Typing useState, useRef, useReducer

useState infers its type from the initial value, so `useState(0)` is a number. Pass a type argument when the initial value does not reveal the full type: `useState<User | null>(null)` or `useState<string[]>([])`. **useRef** has two uses: a reference to a DOM element, typed with the element type and initialised to `null` (`useRef<HTMLInputElement>(null)`; check `ref.current` before use), and a mutable box for a value that does not trigger re-renders (`useRef<number | undefined>(undefined)` for a timer id). **useReducer** is where TypeScript shines: model the actions as a discriminated union (8.2) and the reducer's `switch` narrows each action, with an exhaustiveness check.

USESTATE, USEREF AND A USEREDUCER WITH A DISCRIMINATED UNION OF ACTIONS main.ts

```
import { useEffect, useReducer, useRef, useState } from "react";

interface Course { code: string; title: string; fee: number }

type CartState = { items: Course[]; coupon: string | null };
type CartAction =
  | { type: "add"; course: Course }
  | { type: "remove"; code: string }
  | { type: "applyCoupon"; coupon: string }
  | { type: "clear" };
```

```
function cartReducer(state: CartState, action: CartAction): CartState {
  switch (action.type) {
    case "add": return { ...state, items: [...state.items, action.course] };
    case "remove": return { ...state, items: state.items.filter((c) => c.code !== action.code) };
    case "applyCoupon": return { ...state, coupon: action.coupon };
    case "clear": return { items: [], coupon: null };
    default: { const never: never = action; return never; }     // exhaustiveness check
  }
}

export function Checkout({ catalogue }: { catalogue: Course[] }) {
  const [query, setQuery] = useState("");                          // string (inferred)
  const [selected, setSelected] = useState<Course | null>(null);  // explicit: starts as null
  const searchRef = useRef<HTMLInputElement>(null);               // DOM ref
  const timer = useRef<ReturnType<typeof setTimeout> | undefined>(undefined);   // mutable box
  const [cart, dispatch] = useReducer(cartReducer, { items: [], coupon: null });

  useEffect(() => { searchRef.current?.focus(); }, []);           // current may be null

  const total = cart.items.reduce((s, c) => s + c.fee, 0) * (cart.coupon === "LBF10" ? 0.9 : 1);
  return (
    <section>
      <input ref={searchRef} value={query} onChange={(e) => setQuery(e.target.value)} />
      {catalogue.filter((c) => c.title.toLowerCase().includes(query.toLowerCase())).map((c) => (
        <button key={c.code} onClick={() => { setSelected(c); dispatch({ type: "add", course: c });
}}>{c.title}</button>
      ))}
      <p>{selected ? `Last added: ${selected.title}` : "Nothing selected"} | Total Rs
{Math.round(total)}</p>
      <button onClick={() => dispatch({ type: "applyCoupon", coupon: "LBF10" })}>Apply LBF10</button>
      {/* dispatch({ type: "add" })  error: course is required for "add" */}
    </section>
  );
}
```

18.1.3 Typing events in JSX

React wraps DOM events in its own **synthetic event** types, generic over the element: `React.ChangeEvent<HTMLInputElement>`, `React.MouseEvent<HTMLButtonElement>`, `React.FormEvent<HTMLFormElement>`, `React.KeyboardEvent`. Inline handlers (`onChange={(e) => ...}`) get the type inferred from the JSX attribute, so annotations are only needed for handlers defined separately; alternatively type the whole handler with `React.ChangeEventHandler<HTMLInputElement>`. Unlike `e.target` in the DOM (13.2), `e.currentTarget` in React is typed as the element the handler is attached to.

TYPED EVENT HANDLERS FOR CHANGE, SUBMIT AND KEYBOARD EVENTS

main.ts

```
import { useState, type ChangeEvent, type FormEvent, type KeyboardEvent } from "react";

interface EnrolForm { name: string; email: string; batch: "weekday" | "weekend" }

export function EnrolmentForm({ onSubmit }: { onSubmit: (data: EnrolForm) => Promise<void> }) {
  const [form, setForm] = useState<EnrolForm>({ name: "", email: "", batch: "weekday" });

  // one handler for every text input, typed by the input element
  function handleChange(e: ChangeEvent<HTMLInputElement | HTMLSelectElement>) {
    const { name, value } = e.currentTarget;
    setForm((f) => ({ ...f, [name]: value }));
  }

  async function handleSubmit(e: FormEvent<HTMLFormElement>) {
    e.preventDefault();
    await onSubmit(form);
  }

  const blockDigits = (e: KeyboardEvent<HTMLInputElement>) => { if (/\d/.test(e.key))
e.preventDefault(); };

  return (
    <form onSubmit={handleSubmit}>
      <input name="name" value={form.name} onChange={handleChange} onKeyDown={blockDigits} />
      <input name="email" type="email" value={form.email} onChange={handleChange} />
      <select name="batch" value={form.batch} onChange={handleChange}>
        <option value="weekday">Weekday</option>
        <option value="weekend">Weekend</option>
      </select>
      <button type="submit" disabled={!form.name || !form.email}>Enrol</button>
    </form>
  );
}
```

18.1.4 Typing context and custom hooks

`createContext<T>(defaultValue)` needs a type for the context value. When there is no sensible default, create it with `undefined` (`createContext<AuthContextValue | undefined>(undefined)`) and wrap `useContext` in a **custom hook** that throws if the provider is missing, so components get a non-undefined value. Custom hooks are ordinary functions; annotate their return type, and return a tuple `as const` (or an object) so that destructuring keeps each element's type.

A TYPED CONTEXT WITH A SAFE CUSTOM HOOK, AND A GENERIC USELOCALSTORAGE HOOK main.ts

```
import { createContext, useCallback, useContext, useEffect, useState, type ReactNode } from "react";

interface User { id: number; name: string; role: "student" | "mentor" | "admin" }
interface AuthContextValue { user: User | null; login: (email: string, password: string) =>
Promise<void>; logout: () => void }

const AuthContext = createContext<AuthContextValue | undefined>(undefined);

export function AuthProvider({ children }: { children: ReactNode }) {
  const [user, setUser] = useState<User | null>(null);
  const login = useCallback(async (email: string, _password: string) => {
    setUser({ id: 1, name: email.split("@")[0], role: "student" });     // call the real API here
  }, []);
  const logout = useCallback(() => setUser(null), []);
  return <AuthContext.Provider value={{ user, login, logout }}>{children}</AuthContext.Provider>;
}

export function useAuth(): AuthContextValue {
  const ctx = useContext(AuthContext);
  if (!ctx) throw new Error("useAuth must be used inside <AuthProvider>");
  return ctx;                                          // never undefined for callers
}

// A generic custom hook returning a typed tuple
export function useLocalStorage<T>(key: string, initial: T) {
  const [value, setValue] = useState<T>(() => {
    const raw = localStorage.getItem(key);
    return raw === null ? initial : (JSON.parse(raw) as T);
  });
  useEffect(() => localStorage.setItem(key, JSON.stringify(value)), [key, value]);
  return [value, setValue] as const;                   // readonly [T, Dispatch<SetStateAction<T>>]
}

// In a component:
// const { user, logout } = useAuth();
// const [theme, setTheme] = useLocalStorage<"light" | "dark">("theme", "light");
```

18.1.5 Generic components

Components can be generic just like functions (10.2). A generic list, table, select or autocomplete component takes `items: T[]` and render or key callbacks that receive `T`, so each use gets full type safety for its own item type with no casts. In `.tsx` files write the type parameter as `<T,>` for arrow functions, or use a function declaration.

A GENERIC DATATABLE COMPONENT main.ts

```
import type { ReactNode } from "react";

interface Column<T> { header: string; render: (row: T) => ReactNode; align?: "left" | "right" }

interface DataTableProps<T> {
  rows: readonly T[];
  columns: readonly Column<T>[];
  getKey: (row: T) => string | number;
  onRowClick?: (row: T) => void;
  empty?: ReactNode;
}

export function DataTable<T>({ rows, columns, getKey, onRowClick, empty = "No data" }:
DataTableProps<T>) {
```

```
  if (rows.length === 0) return <p>{empty}</p>;
  return (
    <table>
      <thead><tr>{columns.map((c) => <th key={c.header} align={c.align}>{c.header}</th>)}</tr></thead>
      <tbody>
        {rows.map((row) => (
          <tr key={getKey(row)} onClick={() => onRowClick?.(row)}>
            {columns.map((c) => <td key={c.header} align={c.align}>{c.render(row)}</td>)}
          </tr>
        ))}
      </tbody>
    </table>
  );
}

// T is inferred as Student from rows; every callback receives a Student
interface Student { id: number; name: string; marks: number }
const students: Student[] = [{ id: 1, name: "Asha", marks: 91 }, { id: 2, name: "Ravi", marks: 76 }];

export const MarksTable = () => (
  <DataTable
    rows={students}
    getKey={(s) => s.id}
    columns={[{ header: "Name", render: (s) => s.name }, { header: "Marks", render: (s) => s.marks,
align: "right" }]}
    onRowClick={(s) => alert(s.name)}
  />
);
// render: (s) => s.email      error: Property 'email' does not exist on type 'Student'
```

18.2 TypeScript with Node.js and Express

Express remains the most widely used Node.js web framework. Express 5 is written in JavaScript; its types come from `@types/express` (install with `npm i express` and `npm i -D @types/express @types/node`). The core types are `Request`, `Response`, `NextFunction` and `RequestHandler`. Newer frameworks such as **Fastify**, **Hono** and **NestJS** are written in TypeScript and infer more types automatically.

18.2.1 Typing requests, responses and middleware

`Request` is generic: `Request<Params, ResBody, ReqBody, Query>`, so you can state the shape of `req.params`, the JSON body and the query string, and `Response<ResBody>` checks what you send. Remember these are **claims**, not checks: the body arrives as untyped JSON, so it must still be validated (18.2.3). Middleware has the type `(req, res, next) => void`; information that middleware adds to the request (the logged-in user, a request id) is typed by **augmenting** Express's `Request` interface (11.4.5).

TYPED MIDDLEWARE AND AUGMENTING THE EXPRESS REQUEST

main.ts

```
// src/types/express.d.ts: middleware adds these to every request
import "express";
declare global {
  namespace Express {
    interface Request { user?: { id: number; role: "student" | "admin" }; requestId: string }
  }
}

// src/middleware.ts
import type { NextFunction, Request, Response } from "express";
import { randomUUID } from "node:crypto";

export function requestId(req: Request, _res: Response, next: NextFunction): void {
  req.requestId = randomUUID();
  next();
}

export function requireAdmin(req: Request, res: Response, next: NextFunction): void {
  if (req.user?.role !== "admin") {
    res.status(403).json({ error: "admin only", requestId: req.requestId });
    return;
  }
  next();
}

// Typed error-handling middleware: four parameters
export function errorHandler(err: unknown, req: Request, res: Response, _next: NextFunction): void {
  const message = err instanceof Error ? err.message : "unknown error";
  res.status(500).json({ error: message, requestId: req.requestId });
}
```

18.2.2 Typed routes and DTOs

TYPED EXPRESS ROUTES WITH DTOS main.ts

```
import express, { type Request, type Response } from "express";
import { requestId, requireAdmin, errorHandler } from "./middleware.js";

interface Course { id: number; code: string; title: string; fee: number }
type CreateCourseDto = Omit<Course, "id">;                    // DTOs derived from the entity (12.7)
type UpdateCourseDto = Partial<CreateCourseDto>;
type ErrorBody = { error: string };

const courses: Course[] = [{ id: 1, code: "TS101", title: "TypeScript", fee: 9999 }];
const app = express();
app.use(express.json(), requestId);

// GET /courses?maxFee=10000          Request<Params, ResBody, ReqBody, Query>
app.get("/courses", (req: Request<{}, Course[], never, { maxFee?: string }>, res: Response<Course[]>)
=> {
  const max = req.query.maxFee ? Number(req.query.maxFee) : Infinity;
  res.json(courses.filter((c) => c.fee <= max));
});

// GET /courses/:id                    req.params.id is a string (URLs are text)
app.get("/courses/:id", (req: Request<{ id: string }>, res: Response<Course | ErrorBody>) => {
  const course = courses.find((c) => c.id === Number(req.params.id));
  if (!course) return void res.status(404).json({ error: "course not found" });
  res.json(course);
});

// POST /courses (admin only)
app.post("/courses", requireAdmin, (req: Request<{}, Course, CreateCourseDto>, res: Response<Course>)
=> {
  const course: Course = { id: courses.length + 1, ...req.body };     // body must still be validated!
(18.2.3)
  courses.push(course);
  res.status(201).json(course);
});

app.use(errorHandler);
app.listen(3000, () => console.log("API on http://localhost:3000"));
```

TERMINAL

```
$ curl localhost:3000/courses?maxFee=10000
[{"id":1,"code":"TS101","title":"TypeScript","fee":9999}]
$ curl localhost:3000/courses/9
{"error":"course not found"}
```

Without any framework, Node.js's built-in `http` module is fully typed by `@types/node`. The runnable example below builds a tiny typed router on it: route handlers declare the shapes they return, and a small validation function turns the untyped JSON body into a typed DTO or a 400 error. The same structure, with Express or Fastify instead of the hand-written router, is how real APIs are organised.

SRC/SERVER.TS TypeScript

```
import { createServer, type IncomingMessage, type ServerResponse } from "node:http";

interface Course { id: number; code: string; title: string; fee: number }
type CreateCourseDto = Omit<Course, "id">;
type Json = Course | Course[] | { error: string; details?: string[] };
type Handler = (params: Record<string, string>, body: unknown) => { status: number; body: Json };
```

```
const courses: Course[] = [{ id: 1, code: "TS101", title: "TypeScript", fee: 9999 }];

function validateCreate(body: unknown): CreateCourseDto | string[] {
  const b = (typeof body === "object" && body !== null ? body : {}) as Record<string, unknown>;
  const errors: string[] = [];
  if (typeof b.code !== "string" || !/^[A-Z]{2}\d{3}$/.test(b.code)) errors.push("code must look like
TS101");
  if (typeof b.title !== "string" || b.title.length < 2) errors.push("title is required");
  if (typeof b.fee !== "number" || b.fee < 0) errors.push("fee must be a non-negative number");
  return errors.length ? errors : { code: b.code as string, title: b.title as string, fee: b.fee as
number };
}

const routes: [method: string, pattern: RegExp, handler: Handler][] = [
  ["GET", /^\/courses$/, () => ({ status: 200, body: courses })],
  ["GET", /^\/courses\/(?<id>\d+)$/, (p) => {
    const c = courses.find((x) => x.id === Number(p.id));
    return c ? { status: 200, body: c } : { status: 404, body: { error: "course not found" } };
  }],
  ["POST", /^\/courses$/, (_p, body) => {
    const dto = validateCreate(body);
    if (Array.isArray(dto)) return { status: 400, body: { error: "invalid course", details: dto } };
    const course: Course = { id: courses.length + 1, ...dto };
    courses.push(course);
    return { status: 201, body: course };
  }],
];

async function readJson(req: IncomingMessage): Promise<unknown> {
  let raw = "";
  for await (const chunk of req) raw += chunk;
  return raw ? JSON.parse(raw) : undefined;
}

function send(res: ServerResponse, status: number, body: Json): void {
  res.writeHead(status, { "Content-Type": "application/json" }).end(JSON.stringify(body) + "\n");
}

createServer(async (req, res) => {
  const url = new URL(req.url ?? "/", "http://localhost");
  for (const [method, pattern, handler] of routes) {
    const m = url.pathname.match(pattern);
    if (req.method === method && m) {
      const { status, body } = handler(m.groups ?? {}, await readJson(req));
      return send(res, status, body);
    }
  }
  send(res, 404, { error: "no such route" });
}).listen(3000, () => console.log("listening on 3000"));
```

A TYPED JSON API ON NODE.JS'S HTTP MODULE, WITH BODY VALIDATION

terminal

```
npx tsc -p .
node dist/server.js &
sleep 1
curl -s localhost:3000/courses
curl -s localhost:3000/courses/1
curl -s -X POST localhost:3000/courses -H "Content-Type: application/json" -d
'{"code":"PY201","title":"Python","fee":14999}'
curl -s -X POST localhost:3000/courses -H "Content-Type: application/json" -d
'{"code":"x","fee":"free"}'
curl -s localhost:3000/courses/42
```

TERMINAL

```
$ npx tsc -p .
$ node dist/server.js &
listening on 3000
$ sleep 1
$ curl -s localhost:3000/courses
[{"id":1,"code":"TS101","title":"TypeScript","fee":9999}]
$ curl -s localhost:3000/courses/1
{"id":1,"code":"TS101","title":"TypeScript","fee":9999}
$ curl -s -X POST localhost:3000/courses -H "Content-Type: application/json" -d
'{"code":"PY201","title":"Python","fee":14999}'
{"id":2,"code":"PY201","title":"Python","fee":14999}
$ curl -s -X POST localhost:3000/courses -H "Content-Type: application/json" -d
'{"code":"x","fee":"free"}'
{"error":"invalid course","details":["code must look like TS101","title is required","fee must be a
non-negative number"]}
$ curl -s localhost:3000/courses/42
{"error":"course not found"}
```

18.2.3 Validating input with Zod

Because request bodies, params and query strings are untyped at run time, real APIs validate them with a schema (15.3) and derive the DTO types from the schemas, so one definition drives both the checks and the types. A small generic middleware validates `req.body` against any schema and replaces it with the parsed (typed, coerced, trimmed) value, returning 400 with field-level messages on failure.

A REUSABLE ZOD VALIDATION MIDDLEWARE FOR EXPRESS

main.ts

```
import { z } from "zod";
import type { NextFunction, Request, RequestHandler, Response } from "express";

export const CreateCourseSchema = z.object({
  code: z.string().regex(/^[A-Z]{2}\d{3}$/),
  title: z.string().trim().min(2),
  fee: z.number().int().nonnegative(),
  tags: z.array(z.string()).max(10).default([]),
});
export const CourseQuerySchema = z.object({ maxFee: z.coerce.number().positive().optional() });
export type CreateCourseDto = z.infer<typeof CreateCourseSchema>;

export function validateBody<S extends z.ZodType>(schema: S): RequestHandler<any, any, z.infer<S>> {
  return (req: Request, res: Response, next: NextFunction) => {
    const result = schema.safeParse(req.body);
    if (!result.success) {
      res.status(400).json({ error: "validation failed", issues: result.error.issues.map((i) =>
`${i.path.join(".")}: ${i.message}`) });
      return;
    }
    req.body = result.data;          // now typed and cleaned
    next();
  };
}

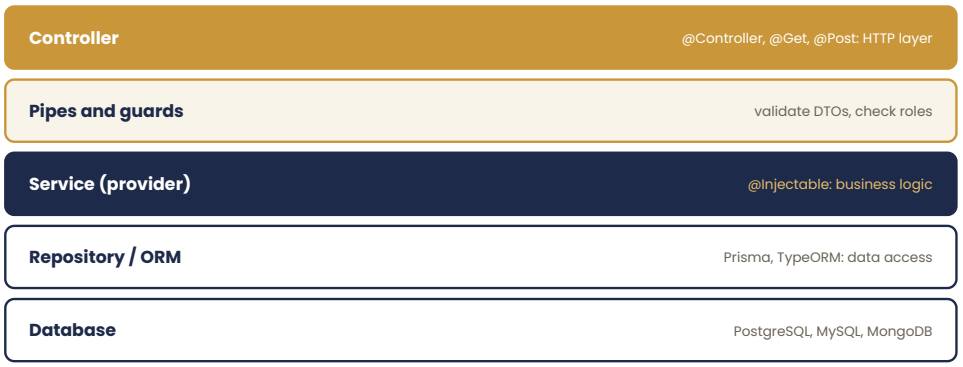
// app.post("/courses", requireAdmin, validateBody(CreateCourseSchema), (req, res) => {
//   const dto = req.body;           // CreateCourseDto: validated
//   ...
// });
```

TERMINAL

```
$ curl -X POST localhost:3000/courses -d '{"code":"ts","title":"T","fee":-1}' -H "Content-Type:
application/json"
{"error":"validation failed","issues":["code: Invalid string: must match pattern /^[A-Z]
{2}\d{3}$/","title: Too small: expected string to have >=2 characters","fee: Too small: expected
number to be >=0"]}
```

18.2.4 NestJS overview

NestJS is a structured, TypeScript-first framework for large back ends, inspired by Angular (16.5.2). An application is organised into **modules**; each module has **controllers** (routes, declared with decorators), **providers** (services, repositories) wired by **dependency injection**, and cross-cutting **pipes** (validation and transformation, often with class-validator or Zod), **guards** (authorisation), **interceptors** (logging, caching) and **exception filters**. It runs on Express or Fastify, and has first-class support for TypeORM, Prisma, GraphQL, WebSockets, microservices, OpenAPI documentation and testing. Its opinionated structure suits teams and enterprise projects; Express, Fastify or Hono suit smaller services.



THE LAYERS OF A NESTJS APPLICATION

18.3 TypeScript with databases

18.3.1 Prisma or Drizzle ORM with generated types

Raw SQL drivers return untyped rows. Modern TypeScript ORMs make database access type-safe end to end. **Prisma** uses its own schema file (`schema.prisma`) from which `prisma generate` produces a fully typed client: `prisma.course.findMany({ where, include, select })` returns objects whose type follows exactly the fields you selected and the relations you included, and `prisma migrate` creates SQL migrations. **Drizzle** defines tables in TypeScript code and offers a SQL-like query builder whose results are inferred from the table definitions, with no generation step. Both catch misspelled columns, wrong value types and missing relations at compile time. Kysely (a typed query builder) and TypeORM (decorators, 16.5.3) are other options.

A PRISMA SCHEMA: THE SINGLE SOURCE OF TRUTH FOR TABLES AND TYPES — text

```
// prisma/schema.prisma
model Course {
  id         Int         @id @default(autoincrement())
  code       String      @unique
  title      String
  fee        Int
  level      Level       @default(BEGINNER)
  enrolments Enrolment[]
  createdAt  DateTime    @default(now())
}

model Student {
  id         Int         @id @default(autoincrement())
  name       String
  email      String      @unique
  enrolments Enrolment[]
}

model Enrolment {
  id        Int     @id @default(autoincrement())
  course    Course  @relation(fields: [courseId], references: [id])
  courseId  Int
  student   Student @relation(fields: [studentId], references: [id])
  studentId Int
  paid      Boolean @default(false)
}

enum Level { BEGINNER INTERMEDIATE ADVANCED }
```

TYPE-SAFE QUERIES WITH PRISMA CLIENT

main.ts

```
import { PrismaClient, type Prisma } from "./generated/prisma/client.js";

const prisma = new PrismaClient();

// The result type follows the query: Course & { enrolments: (Enrolment & { student: { name: string } })[] }
const ts101 = await prisma.course.findUnique({
  where: { code: "TS101" },
  include: { enrolments: { include: { student: { select: { name: true } } } } },
});
console.log(ts101?.title, ts101?.enrolments.map((e) => `${e.student.name}${e.paid ? "" : " (unpaid)"}`));

// select narrows the result type to exactly these fields: { code: string; fee: number }[]
const cheap = await prisma.course.findMany({ where: { fee: { lt: 12000 } }, select: { code: true, fee: true } });

// Inputs are typed too
const data: Prisma.CourseCreateInput = { code: "JV301", title: "Java", fee: 12999, level: "INTERMEDIATE" };
await prisma.course.create({ data });

// await prisma.course.findMany({ where: { price: 100 } });        error: 'price' does not exist in CourseWhereInput
// cheap[0].title;                                                  error: title was not selected
```

OUTPUT

```
TypeScript [ 'Asha', 'Ravi (unpaid)' ]
```

Requires `npm i @prisma/client`, `npm i -D prisma`, a database URL and `npx prisma migrate dev`.

DRIZZLE: TABLES DEFINED IN TYPESCRIPT, TYPES INFERRED

main.ts

```
import { drizzle } from "drizzle-orm/node-postgres";
import { pgTable, serial, text, integer, boolean } from "drizzle-orm/pg-core";
import { eq, lt } from "drizzle-orm";

export const courses = pgTable("courses", {
  id: serial("id").primaryKey(),
  code: text("code").notNull().unique(),
  title: text("title").notNull(),
  fee: integer("fee").notNull(),
  published: boolean("published").default(false).notNull(),
});
export type Course = typeof courses.$inferSelect;       // { id: number; code: string; ...; published: boolean }
export type NewCourse = typeof courses.$inferInsert;     // id and published optional

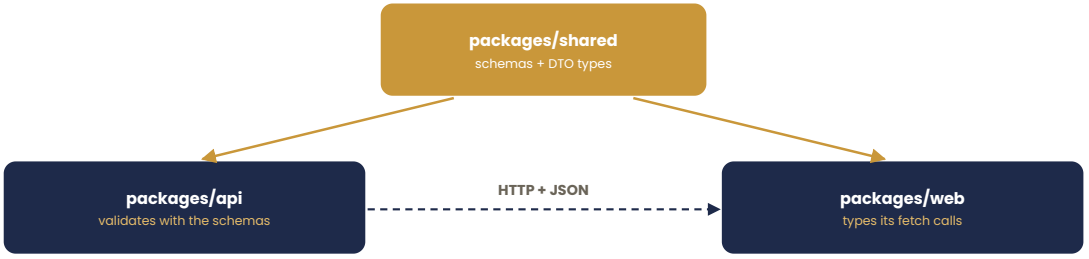
const db = drizzle(process.env.DATABASE_URL!);
const cheap = await db.select({ code: courses.code, fee: courses.fee
}).from(courses).where(lt(courses.fee, 12000));
await db.update(courses).set({ published: true }).where(eq(courses.code, "TS101"));
```

Requires `npm i drizzle-orm pg` and `npm i -D drizzle-kit`.

18.4 End-to-end type safety

18.4.1 Sharing types between front end and back end

In a full-stack TypeScript project the front end and the back end must agree on the shape of every request and response. The simplest way is a **shared package** in a monorepo (17.2) that contains the DTO types (or, better, the Zod schemas from which both the server validation and the client types are derived). The server imports them to type and validate its handlers; the client imports them to type its API calls. A change to a DTO then produces compile errors on **both** sides wherever the code no longer fits. Shared code must contain only types and runtime-neutral logic (no Node.js or browser APIs).



ONE SHARED PACKAGE KEEPS THE CLIENT AND THE SERVER IN SYNC

PACKAGES/SHARED/SRC/CONTRACT.TS

TypeScript

```
// The API contract: every endpoint, its input and its output
export interface Course { id: number; code: string; title: string; fee: number }
export interface Enrolment { id: number; courseCode: string; student: string; payable: number }

export interface Api {
  "GET /courses": { input: void; output: Course[] };
  "POST /enrolments": { input: { courseCode: string; student: string; coupon?: string }; output:
Enrolment };
}
export type Endpoint = keyof Api;
export type Input<E extends Endpoint> = Api[E]["input"];
export type Output<E extends Endpoint> = Api[E]["output"];
```

PACKAGES/WEB/SRC/CLIENT.TS

TypeScript

```
import type { Endpoint, Input, Output } from "../../shared/dist/contract.js";

// One typed function for every endpoint: input and output types come from the contract
async function call<E extends Endpoint>(endpoint: E, ...args: Input<E> extends void ? [] :
[Input<E>]): Promise<Output<E>> {
  const [method, path] = endpoint.split(" ");
  const res = await fetch(`http://localhost:3000${path}`, { method, body: args.length ?
JSON.stringify(args[0]) : undefined });
  const data = await res.json();
  if (!res.ok) throw new Error(data.error);
  return data as Output<E>;
}

const courses = await call("GET /courses");                                    // Course[]
console.log(courses.map((c) => `${c.code} Rs ${c.fee}`).join(", "));
const enrolment = await call("POST /enrolments", { courseCode: "TS101", student: "Asha", coupon:
"LBF10" });   // Enrolment
console.log(`enrolment #${enrolment.id}: ${enrolment.student} pays Rs ${enrolment.payable}`);
try {
  await call("POST /enrolments", { courseCode: "XX999", student: "Ravi" });
} catch (e) {
  console.log("server said:", (e as Error).message);
}
// call("POST /enrolments", { course: "TS101" });     compile error: wrong input shape
process.exit(0);
```

A RUNNABLE FULL-STACK EXAMPLE: ONE SHARED CONTRACT USED BY THE SERVER AND THE CLIENT

terminal

```
npx tsc -b
node packages/api/dist/server.js &
sleep 1
node packages/web/dist/client.js
```

TERMINAL

```
$ npx tsc -b
$ node packages/api/dist/server.js &
$ sleep 1
$ node packages/web/dist/client.js
TS101 Rs 9999, PY201 Rs 14999
enrolment #1: Asha pays Rs 8999
server said: unknown course XX999
```

18.4.2 tRPC and OpenAPI-generated clients (overview)

Two popular tools automate the shared contract. **tRPC** (for TypeScript on both sides) lets you define server **procedures** with input schemas (usually Zod); the client imports only the router's **type**, and calls such as `trpc.course.byCode.query({ code: "TS101" })` are fully typed, including errors, with no code generation and no hand-written fetch calls. It is ideal for full-stack TypeScript apps (it is part of the "T3 stack" with Next.js and Prisma). **OpenAPI** (Swagger) is the language-neutral alternative: the API is described in an OpenAPI document (written by hand or generated by frameworks such as NestJS, FastAPI or Spring), and tools such as `openapi-typescript` with `openapi-fetch`, Orval or OpenAPI Generator produce typed TypeScript clients. Choose tRPC when you own both ends and both are TypeScript; choose OpenAPI for public APIs, other back-end languages or several client teams.

TRPC: THE CLIENT IS TYPED FROM THE SERVER ROUTER'S TYPE main.ts

```
// server/router.ts
import { initTRPC, TRPCError } from "@trpc/server";
import { z } from "zod";

const t = initTRPC.create();
export const appRouter = t.router({
  courseByCode: t.procedure
    .input(z.object({ code: z.string().regex(/^[A-Z]{2}\d{3}$/) }))
    .query(async ({ input }) => {
      const course = await db.course.findUnique({ where: { code: input.code } });
      if (!course) throw new TRPCError({ code: "NOT_FOUND" });
      return course;                                  // its type flows to the client
    }),
  enrol: t.procedure
    .input(z.object({ code: z.string(), student: z.string().min(2) }))
    .mutation(({ input }) => enrolStudent(input.code, input.student)),
});
export type AppRouter = typeof appRouter;             // only the TYPE is shared

// client/main.ts
import { createTRPCClient, httpBatchLink } from "@trpc/client";
import type { AppRouter } from "../server/router";

const trpc = createTRPCClient<AppRouter>({ links: [httpBatchLink({ url: "/api/trpc" })] });
const course = await trpc.courseByCode.query({ code: "TS101" });     // typed result
await trpc.enrol.mutate({ code: "TS101", student: "Asha" });
// await trpc.enrol.mutate({ code: "TS101" });         error: student is missing
```

18.5 Testing TypeScript

18.5.1 Vitest or Jest with TypeScript

Vitest is the modern default for TypeScript projects: it runs `.ts` test files directly (through Vite's esbuild transform), has a Jest-compatible API (`describe`, `it`, `expect`, `vi.fn`), watch mode, coverage and built-in type testing. **Jest** is still widespread and works with TypeScript through `ts-jest` or `@swc/jest`. Like other fast transpilers, test runners do **not** type-check, so keep `tsc --noEmit` in CI. Node.js also has a built-in test runner, `node:test`, which together with type stripping (17.5.1) runs TypeScript tests with no dependencies at all, as the runnable example below shows.

A VITEST TEST FILE

main.ts

```
// src/pricing.test.ts (Vitest)
import { describe, expect, it } from "vitest";
import { applyCoupon, gst, type Coupon } from "./pricing";

describe("pricing", () => {
  it("adds 18% GST and rounds", () => {
    expect(gst(9999)).toBe(11799);
  });

  it.each<[number, Coupon, number]>([
    [1000, { kind: "percent", value: 10 }, 900],
    [1000, { kind: "flat", value: 250 }, 750],
    [100, { kind: "flat", value: 250 }, 0],
  ])("applyCoupon(%i, %o) = %i", (amount, coupon, expected) => {
    expect(applyCoupon(amount, coupon)).toBe(expected);
  });

  it("rejects negative amounts", () => {
    expect(() => gst(-1)).toThrowError(/negative/);
  });
});
```

```
npx vitest run
 ✓ src/pricing.test.ts (5 tests) 4ms
   ✓ pricing > adds 18% GST and rounds
   ✓ pricing > applyCoupon(1000, { kind: 'percent', value: 10 }) = 900
   ✓ pricing > applyCoupon(1000, { kind: 'flat', value: 250 }) = 750
   ✓ pricing > applyCoupon(100, { kind: 'flat', value: 250 }) = 0
   ✓ pricing > rejects negative amounts

 Test Files  1 passed (1)
      Tests  5 passed (5)
```

SRC/PRICING.TEST.TS

TypeScript

```
import { describe, it } from "node:test";
import assert from "node:assert/strict";
import { applyCoupon, gst, type Coupon } from "./pricing.ts";

describe("pricing", () => {
  it("adds 18% GST and rounds", () => {
    assert.equal(gst(9999), 11799);
  });

  const cases: [number, Coupon, number][] = [
    [1000, { kind: "percent", value: 10 }, 900],
    [1000, { kind: "flat", value: 250 }, 750],
    [100, { kind: "flat", value: 250 }, 0],
  ];
  for (const [amount, coupon, expected] of cases) {
    it(`applyCoupon(${amount}, ${coupon.kind} ${coupon.value}) = ${expected}`, () => {
      assert.equal(applyCoupon(amount, coupon), expected);
    });
  }

  it("rejects negative amounts", () => {
    assert.throws(() => gst(-1), /negative/);
  });
});
```

RUNNING TYPESCRIPT TESTS WITH NODE.JS'S BUILT-IN RUNNER (NO DEPENDENCIES)

terminal

```
npx tsc --noEmit -p . && echo "types OK"
node --test --test-reporter=spec src/*.test.ts 2>&1 | grep -vE "^\s*$|duration_ms|i
(todo|skipped|cancelled)" | sed -E "s/ \([0-9.]+ms\)//"
```

TERMINAL

```
$ npx tsc --noEmit -p . && echo "types OK"
types OK
$ node --test --test-reporter=spec src/*.test.ts 2>&1 | grep -vE "^\s*$|duration_ms|i
(todo|skipped|cancelled)" | sed -E "s/ \([0-9.]+ms\)//"
▶ pricing
  ✔ adds 18% GST and rounds
  ✔ applyCoupon(1000, percent 10) = 900
  ✔ applyCoupon(1000, flat 250) = 750
  ✔ applyCoupon(100, flat 250) = 0
  ✔ rejects negative amounts
✔ pricing
ℹ tests 5
ℹ suites 1
ℹ pass 5
ℹ fail 0
```

18.5.2 Typing mocks

A **mock** replaces a dependency (a repository, an HTTP client, an email sender, the clock) with a fake whose behaviour the test controls and whose calls it can inspect. With TypeScript, mocks should satisfy the **same interface** as the real dependency, so that a test cannot pass with a fake that no longer matches. Two techniques: write small typed fakes by hand (implementing the interface, often with `Partial` plus a cast for large interfaces), or use the framework's typed mock functions: `vi.fn<(id: number) => Promise<User>>()`, `vi.mocked(fn)` to get the mock type of a mocked import, or Node's `mock.fn()`. Designing code around injected interfaces (9.8.3) makes it easy to test.

SRC/ENROLMENT.TEST.TS

TypeScript

```ts
import { describe, it, mock } from "node:test";
import assert from "node:assert/strict";
import { EnrolmentService, type Course, type CourseRepo, type Mailer } from "./enrolment.ts";

function fakeRepo(courses: Course[]): CourseRepo {          // a typed hand-written fake
  return {
    find: async (code) => courses.find((c) => c.code === code),
    reserveSeat: mock.fn(async (_code: string) => {}),
  };
}

describe("EnrolmentService", () => {
  it("reserves a seat and sends a welcome mail", async () => {
    const repo = fakeRepo([{ code: "TS101", fee: 9999, seatsLeft: 3 }]);
    const send = mock.fn<Mailer["send"]>(async () => {});      // typed mock function
    const service = new EnrolmentService(repo, { send });

    assert.equal(await service.enrol("TS101", "asha@lbf.in"), "enrolled, pay Rs 9999");
    assert.equal(send.mock.callCount(), 1);
    assert.deepEqual(send.mock.calls[0]?.arguments, ["asha@lbf.in", "Welcome to TS101"]);
  });

  it("does not mail anyone when the batch is full", async () => {
    const send = mock.fn<Mailer["send"]>(async () => {});
    const service = new EnrolmentService(fakeRepo([{ code: "PY201", fee: 14999, seatsLeft: 0 }]), {
send });
    assert.equal(await service.enrol("PY201", "ravi@lbf.in"), "batch full");
    assert.equal(send.mock.callCount(), 0);
  });

  it("reports unknown courses", async () => {
    const service = new EnrolmentService(fakeRepo([]), { send: async () => {} });
    assert.equal(await service.enrol("XX000", "x@lbf.in"), "unknown course");
  });
});
```

A SERVICE TESTED WITH TYPED FAKES AND NODE:TEST MOCK FUNCTIONS

terminal

```
node --test --test-reporter=spec src/*.test.ts 2>&1 | grep -vE "^\s*$|duration_ms|i
(todo|skipped|cancelled)" | sed -E "s/ \([0-9.]+ms\)//"
```

TERMINAL

```
$ node --test --test-reporter=spec src/*.test.ts 2>&1 | grep -vE "^\s*$|duration_ms|i
(todo|skipped|cancelled)" | sed -E "s/ \([0-9.]+ms\)//"
▶ EnrolmentService
  ✔ reserves a seat and sends a welcome mail
  ✔ does not mail anyone when the batch is full
  ✔ reports unknown courses
✔ EnrolmentService
ℹ tests 3
ℹ suites 1
ℹ pass 3
ℹ fail 0
```

18.5.3 Testing types with expectTypeOf

Libraries and shared utilities have **types** worth testing too: that a function returns the expected type, that a generic infers correctly, that invalid calls are **rejected**. Vitest provides `expectTypeOf` (and `assertType`) for this; tests in `*.test-d.ts` files are checked by the compiler with `vitest --typecheck`. The `expect-type`

and `tsd` packages offer the same outside Vitest. Without any package, the `Expect<Equal<A, B>>` helper from Module 14 plus `// @ts-expect-error` lines (7.3.3) give the same guarantees with plain `tsc`.

TYPE TESTS WITH VITEST'S EXPECTTYPEOF main.ts

```ts
// src/pricing.test-d.ts (vitest --typecheck)
import { expectTypeOf, test } from "vitest";
import { applyCoupon, groupBy, type Coupon } from "./pricing";

test("applyCoupon types", () => {
  expectTypeOf(applyCoupon).parameter(1).toEqualTypeOf<Coupon | undefined>();
  expectTypeOf(applyCoupon(100)).toBeNumber();
  // @ts-expect-error  coupons must have a kind
  applyCoupon(100, { value: 10 });
});

test("groupBy infers the key union", () => {
  const groups = groupBy([{ level: "a" as const }, { level: "b" as const }], (x) => x.level);
  expectTypeOf(groups).toEqualTypeOf<Record<"a" | "b", { level: "a" | "b" }[]>>();
});
```

```
npx vitest --typecheck run
 ✓ src/pricing.test-d.ts (2 tests)
 Type Errors  no errors
```

TYPE TESTS WITH PLAIN TSC: A FAILING ASSERTION AND AN UNUSED @TS-EXPECT-ERROR main.ts

```ts
type Equal<A, B> = (<T>() => T extends A ? 1 : 2) extends (<T>() => T extends B ? 1 : 2) ? true :
false;
type Expect<T extends true> = T;

type Coupon = { kind: "percent" | "flat"; value: number };
declare function applyCoupon(amount: number, coupon?: Coupon): number;
declare function groupBy<T, K extends PropertyKey>(items: T[], key: (t: T) => K): Record<K, T[]>;

type T1 = Expect<Equal<ReturnType<typeof applyCoupon>, number>>;              // passes
type T2 = Expect<Equal<Parameters<typeof applyCoupon>[1], Coupon>>;           // fails: it is Coupon
| undefined
// @ts-expect-error  coupons must have a kind
applyCoupon(100, { value: 10 });
// @ts-expect-error  this call is valid, so the directive itself is reported
applyCoupon(100);
```

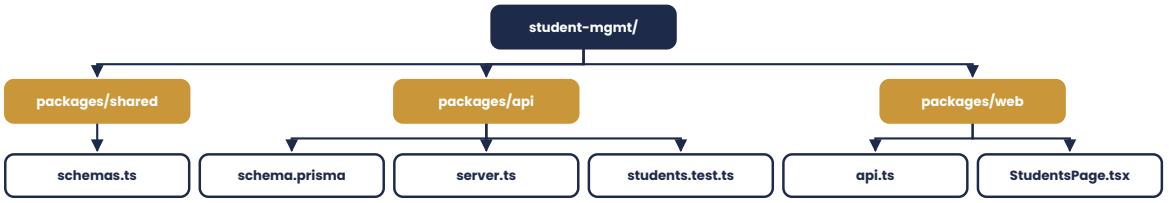
```
COMPILER ERRORS (tsc --strict)
main.ts(9,18): error TS2344: Type 'false' does not satisfy the constraint 'true'.
main.ts(12,1): error TS2578: Unused '@ts-expect-error' directive.
```

React components type their props (interfaces, optional props with defaults, children: ReactNode, ComponentProps<"button"> for native attributes); useState infers from the initial value or takes a type argument, useRef<HTMLElement>(null) types DOM refs, and useReducer pairs well with discriminated-union actions; JSX events use ChangeEvent, FormEvent, MouseEvent generics; context is created with T | undefined and read through a custom hook that throws; components can be generic. Express types Request<Params, ResBody, ReqBody, Query>, Response<Body> and middleware, augments Request for added fields, derives DTOs with utility types and validates bodies with Zod middleware; NestJS adds modules, DI, pipes and guards. Prisma (generated client) and Drizzle (inferred from TypeScript tables) make database queries type-safe. Share contracts or schemas between client and server in a monorepo, or use tRPC or OpenAPI-generated clients. Test with Vitest, Jest or node:test (tsc --noEmit still needed), use mocks that satisfy the real interfaces, and test types with expectTypeOf or Expect<Equal> and @ts-expect-error.

PRACTICE CHECKPOINT

A **full-stack typed Student Management app**: an Express + Prisma API with Zod validation and a React front end sharing the same types, with Vitest tests.

Sample solution (outline and key files). The project is a monorepo with three packages. `packages/shared` holds the Zod schemas and the types inferred from them; `packages/api` is an Express server using Prisma and the shared schemas for validation; `packages/web` is a Vite + React app that calls the API through a typed client built from the same types. Because a real install of these frameworks is needed, the code is shown with the behaviour it produces; the runnable examples in 18.2.2, 18.4.1 and 18.5 demonstrate the same structure with built-in modules.



PROJECT LAYOUT

SHARED: SCHEMAS AND INFERRED TYPES (THE SINGLE SOURCE OF TRUTH) main.ts

```
// packages/shared/src/schemas.ts
import { z } from "zod";

export const StudentInput = z.object({
  name: z.string().trim().min(2),
  email: z.email(),
  course: z.enum(["TS101", "PY201", "JV301"]),
  feePaid: z.number().int().nonnegative().default(0),
});
export const Student = StudentInput.extend({ id: z.number().int(), createdAt: z.coerce.date() });

export type StudentInput = z.input<typeof StudentInput>;
export type Student = z.infer<typeof Student>;
```

API: EXPRESS + PRISMA, VALIDATED WITH THE SHARED SCHEMA

main.ts

```ts
// packages/api/src/server.ts
import express from "express";
import { PrismaClient } from "./generated/prisma/client.js";
import { StudentInput, type Student } from "@lbf/shared";

const prisma = new PrismaClient();
export const app = express().use(express.json());

app.get("/students", async (_req, res: express.Response<Student[]>) => {
  res.json(await prisma.student.findMany({ orderBy: { name: "asc" } }));
});

app.post("/students", async (req, res: express.Response<Student | { issues: string[] }>) => {
  const parsed = StudentInput.safeParse(req.body);
  if (!parsed.success) return void res.status(400).json({ issues: parsed.error.issues.map((i) =>
`${i.path}: ${i.message}`) });
  res.status(201).json(await prisma.student.create({ data: parsed.data }));
});

app.delete("/students/:id", async (req: express.Request<{ id: string }>, res) => {
  await prisma.student.delete({ where: { id: Number(req.params.id) } });
  res.status(204).end();
});
```

BEHAVIOUR

```
POST /students {"name":"Asha","email":"asha@lbf.in","course":"TS101"}  -> 201 {"id":1,...,"feePaid":0}
POST /students {"name":"A","email":"nope","course":"GO101"}         -> 400 {"issues":["name: Too
small...","email: Invalid email address","course: Invalid option..."]}
GET  /students                                                       -> 200
[{"id":1,"name":"Asha",...}]
```

WEB: A TYPED API CLIENT AND A REACT PAGE BUILT ON THE SHARED TYPES

main.ts

```
// packages/web/src/api.ts
import { Student, type StudentInput } from "@lbf/shared";

export async function listStudents(): Promise<Student[]> {
  const res = await fetch("/api/students");
  return Student.array().parse(await res.json());       // validated on the client too
}
export async function addStudent(input: StudentInput): Promise<Student> {
  const res = await fetch("/api/students", { method: "POST", headers: { "Content-Type":
"application/json" }, body: JSON.stringify(input) });
  if (!res.ok) throw new Error((await res.json()).issues.join("; "));
  return Student.parse(await res.json());
}

// packages/web/src/StudentsPage.tsx
import { useEffect, useState } from "react";
import { addStudent, listStudents } from "./api";
import type { Student, StudentInput } from "@lbf/shared";

export function StudentsPage() {
  const [students, setStudents] = useState<Student[]>([]);
  const [error, setError] = useState<string | null>(null);
  useEffect(() => { listStudents().then(setStudents).catch((e: unknown) => setError(String(e))); },
[]);

  async function handleAdd(input: StudentInput) {
    try { const s = await addStudent(input); setStudents((list) => [...list, s]); }
    catch (e) { setError(e instanceof Error ? e.message : String(e)); }
  }
  return (
    <main>
      {error && <p role="alert">{error}</p>}
      <ul>{students.map((s) => <li key={s.id}>{s.name} ({s.course}) paid Rs {s.feePaid}</li>)}</ul>
      <button onClick={() => handleAdd({ name: "Ravi", email: "ravi@lbf.in", course: "PY201" })}>Add
Ravi</button>
    </main>
  );
}
```

VITEST TESTS FOR THE API WITH A MOCKED PRISMA CLIENT

main.ts

```
// packages/api/src/students.test.ts
import { describe, expect, it, vi, beforeEach } from "vitest";
import request from "supertest";
import { app } from "./server";

vi.mock("./generated/prisma/client.js", () => {
  const rows: any[] = [];
  const student = {
    findMany: vi.fn(async () => rows),
    create: vi.fn(async ({ data }) => { const s = { id: rows.length + 1, createdAt: new Date(),
...data }; rows.push(s); return s; }),
    delete: vi.fn(),
  };
  return { PrismaClient: vi.fn(() => ({ student })) };
});

describe("students API", () => {
  it("creates a valid student with defaults", async () => {
    const res = await request(app).post("/students").send({ name: "Asha", email: "asha@lbf.in",
course: "TS101" });
    expect(res.status).toBe(201);
    expect(res.body).toMatchObject({ id: 1, name: "Asha", feePaid: 0 });
  });
  it("rejects invalid input with field messages", async () => {
    const res = await request(app).post("/students").send({ name: "A", email: "nope", course: "GO101"
});
    expect(res.status).toBe(400);
    expect(res.body.issues).toHaveLength(3);
  });
  it("lists students", async () => {
    expect((await request(app).get("/students")).body).toHaveLength(1);
  });
});
```

```
npx vitest run
 ✓ src/students.test.ts (3 tests) 41ms
   ✓ students API > creates a valid student with defaults
   ✓ students API > rejects invalid input with field messages
   ✓ students API > lists students

 Test Files  1 passed (1)
      Tests  3 passed (3)
```

MODULE 19 · ADVANCED

Migration, projects and interview preparation

Minimum duration **3 days** Level **Advanced**

The final module turns your TypeScript knowledge into practical results. It covers a step-by-step strategy for migrating JavaScript projects to TypeScript, starting loose and tightening strictness over time, handling third-party libraries without types; the best practices that experienced teams follow (prefer unknown to any, let inference work and annotate boundaries, model domain data with unions and literal types, keep types close to their code); practising with the type-challenges collection; four capstone projects for your portfolio; and interview preparation: a core-topics checklist, implementing utility types live, and explaining your projects.

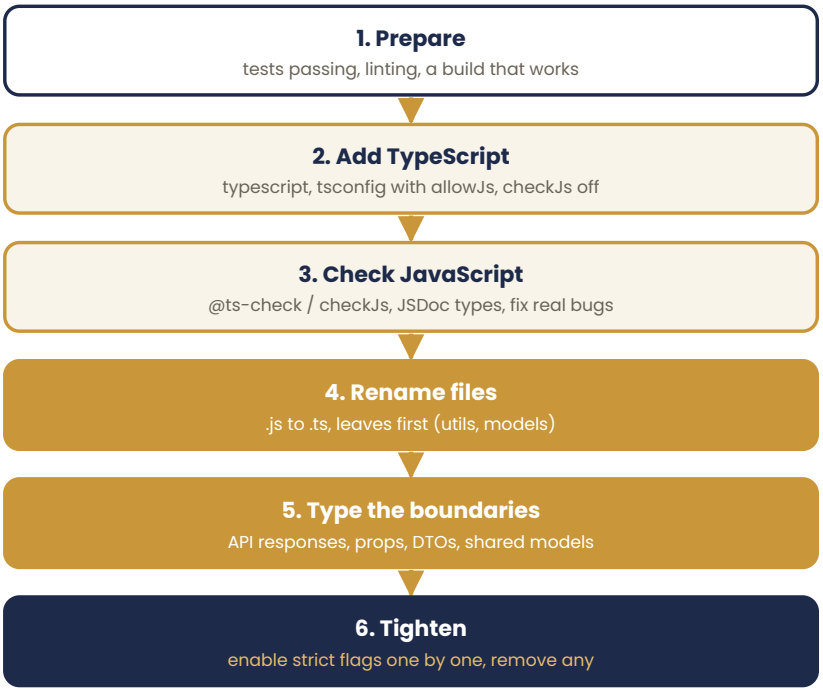
IN THIS MODULE

19.1 Migrating JavaScript to TypeScript
19.2 Best practices
19.3 Type challenges
19.4 Capstone projects
19.5 Interview preparation

19.1 Migrating JavaScript to TypeScript

Most TypeScript jobs involve existing code, and much of it started as JavaScript. Large companies (Airbnb, Slack, Stripe, Bloomberg, Microsoft's own VS Code team) have published how they migrated millions of lines. The lessons are consistent: migrate **incrementally** while the product keeps shipping, start with loose settings and tighten them, convert the most depended-upon code first, and measure progress.

19.1.1 Step-by-step migration strategy



AN INCREMENTAL MIGRATION PATH

1. **Prepare.** Make sure the project builds and its tests pass, so you can tell when the migration breaks something. Add ESLint if it is missing.
2. **Add TypeScript alongside JavaScript.** Install `typescript` and the `@types` packages for your runtime and libraries, and add a tsconfig with `"allowJs": true`, `"checkJs": false`, `"noEmit": true` (if a bundler builds the code) and loose strictness. Add a `typecheck` script and run it in CI from day one.
3. **Check JavaScript first (optional but valuable).** Turn on `checkJs` or add `// @ts-check` to individual files, add JSDoc types where inference fails, and fix the real bugs that appear (7.3).
4. **Rename files to .ts / .tsx, starting with the leaves** of the dependency graph: utilities, constants, models and API clients that many other files import. Each converted leaf gives types to everything that uses it. Convert in small pull requests; renaming and refactoring in the same change makes review hard.
5. **Type the boundaries**: domain models, API responses (with validation), component props and function signatures of shared modules. Internal code often needs few annotations thanks to inference.
6. **Tighten the configuration** step by step (19.1.2) until `strict` is on, and track the remaining `any`, `@ts-expect-error` and `.js` files.

TSCONFIG.JSON JavaScript

```
{ "compilerOptions": { "allowJs": true, "checkJs": false, "noEmit": true, "strict": false,
"noImplicitAny": true, "target": "es2022", "module": "nodenext", "types": [] }, "include": ["src"] }
```

PACKAGE.JSON JavaScript

```
{ "type": "module", "scripts": { "typecheck": "tsc -p ." } }
```

SRC/MODELS.TS TypeScript

```
export interface Student { id: number; name: string; marks: number[] }
export type Grade = "A" | "B" | "C";
```

SRC/GRADES.TS TypeScript

```
import type { Grade, Student } from "./models.js";
export function average(s: Student): number { return s.marks.reduce((a, b) => a + b, 0) /
(s.marks.length || 1); }
export function grade(avg: number): Grade { return avg >= 85 ? "A" : avg >= 60 ? "B" : "C"; }
export function parseLegacy(raw: any): Student { return { id: Number(raw.id), name: String(raw.name),
marks: raw.marks ?? [] }; }
```

SRC/REPORT.JS JavaScript

```
import { average, grade } from "./grades.js";
// still JavaScript: types flow in from grades.ts, so calls are checked when this file is converted
export function report(students) {
  return students.map((s) => `${s.name}: ${grade(average(s))}`);
}
```

SRC/APP.JS JavaScript

```
import { report } from "./report.js";
console.log(report([{ id: 1, name: "Asha", marks: [91, 84] }]));
```

A HALF-MIGRATED PROJECT: JAVASCRIPT AND TYPESCRIPT SIDE BY SIDE, WITH PROGRESS METRICS terminal

```
npx tsc -p . && echo "type check passed (the .js files are included but not yet checked)"
echo "files still in JavaScript: $(find src -name '*.js' | wc -l), in TypeScript: $(find src -name
'*.ts' | wc -l)"
echo "explicit any: $(grep -rn ': any' src --include=*.ts | wc -l), suppressions: $(grep -rn '@ts-
expect-error' src | wc -l)"
```

TERMINAL

```
$ npx tsc -p . && echo "type check passed (the .js files are included but not yet checked)"
type check passed (the .js files are included but not yet checked)
$ echo "files still in JavaScript: $(find src -name '*.js' | wc -l), in TypeScript: $(find src -name
'*.ts' | wc -l)"
files still in JavaScript: 2, in TypeScript: 2
$ echo "explicit any: $(grep -rn ': any' src --include=*.ts | wc -l), suppressions: $(grep -rn '@ts-
expect-error' src | wc -l)"
explicit any: 1, suppressions: 0
```

19.1.2 Starting loose and tightening strictness over time

Turning on `strict` in a large JavaScript codebase can produce thousands of errors at once, which stalls the migration. Start with the most valuable and least noisy checks, fix the errors, then enable the next. A commonly used order: `noImplicitAny` (find untyped parameters), `strictNullChecks` (the biggest source of real bugs, and usually the most work), `strictFunctionTypes` and `strictBindCallApply`, `strictPropertyInitialization`, `useUnknownInCatchVariables`, then `strict: true`, and finally extras such as `noUncheckedIndexedAccess`. For very large codebases, tools such as **ts-migrate** (Airbnb) add `any` and `@ts-expect-error` automatically so the code compiles under the new flag, and teams then remove those markers gradually; plugins such as `typescript-strict-plugin` enforce strict mode file by file.

Stage	Settings	Goal
1. coexist	`allowJs`, `strict: false`, `noEmit`	TypeScript compiles alongside JavaScript
2. no silent any	`noImplicitAny: true`	every parameter and variable has a known type
3. null safety	`strictNullChecks: true`	undefined and null are handled explicitly
4. full strict	`strict: true`	all strict checks, including future ones
5. extra safety	`noUncheckedIndexedAccess`, `exactOptionalPropertyTypes`, `noImplicitOverride`	the strictest practical setup
6. clean up	lint rules `no-explicit-any`, `ban-ts-comment`; remove `allowJs`	no escape hatches left

TSCONFIG.JSON JavaScript

```
{ "compilerOptions": { "noEmit": true, "strict": false, "target": "es2022", "types": [] }, "include": ["src"] }
```

SRC/CART.TS TypeScript

```
export function total(items, coupon?) {
  const sum = items.reduce((s, i) => s + i.price * i.qty, 0);
  return coupon ? sum * (1 - coupon.pct / 100) : sum;
}

const names: string[] = ["Asha", "Ravi"];
const found = names.find((n) => n.startsWith("Z"));
console.log(found.length);

class Batch {
  title: string;
  size = 30;
}
export { Batch };
```

COUNTING THE ERRORS EACH STRICTER FLAG WOULD PRODUCE BEFORE ENABLING IT terminal

```
for flag in noImplicitAny strictNullChecks strict; do echo "with --$flag: $(npx tsc --noEmit -p . --$flag 2>&1 | grep -c 'error TS') error(s)"; done
npx tsc --noEmit -p . --strict 2>&1 | head -n 4
```

TERMINAL

```
$ for flag in noImplicitAny strictNullChecks strict; do echo "with --$flag: $(npx tsc --noEmit -p . --$flag 2>&1 | grep -c 'error TS') error(s)"; done
with --noImplicitAny: 4 error(s)
with --strictNullChecks: 1 error(s)
with --strict: 6 error(s)
$ npx tsc --noEmit -p . --strict 2>&1 | head -n 4
src/cart.ts(1,23): error TS7006: Parameter 'items' implicitly has an 'any' type.
src/cart.ts(1,30): error TS7006: Parameter 'coupon' implicitly has an 'any' type.
src/cart.ts(2,29): error TS7006: Parameter 's' implicitly has an 'any' type.
src/cart.ts(2,32): error TS7006: Parameter 'i' implicitly has an 'any' type.
```

19.1.3 Handling third-party libraries without types

During migration you will import libraries that have no types. Work through the options in order (7.2): (1) check whether a newer version of the library ships types, or whether `@types/library` exists (`npm view @types/library`); (2) consider replacing an abandoned, untyped library with a typed modern alternative; (3) write a minimal declaration file in a `types/` folder describing only the functions you use (11.4.3), which documents your actual usage and gives real safety; (4) as a temporary measure, a shorthand `declare module "lib";` makes the import compile with `any`, which you should track and replace. Wrap untyped libraries in a small typed module of your own (an **adapter**), so the untyped surface appears in one file and the rest of the codebase imports typed functions.

TYPES/OLD-SMS.D.TS TypeScript

```
declare module "old-sms";      // temporary: everything is any (tracked in issue LBF-87)
```

SRC/SMS.TS TypeScript

```
// The adapter: the only file that touches the untyped library
import oldSms from "old-sms";

export interface SmsResult { id: string; status: "QUEUED" | "SENT" | "FAILED" }

export function sendSms(to: string, text: string): Promise<SmsResult> {
  if (!/^[6-9]\d{9}$/.test(to)) return Promise.reject(new Error(`invalid mobile number ${to}`));
  return new Promise((resolve, reject) => {
    oldSms.send({ to: "+91" + to, body: text.slice(0, 160) }, (err: unknown, res: unknown) => {
      if (err) return reject(err);
      const r = res as Partial<SmsResult>;
      if (typeof r?.id !== "string") return reject(new Error("unexpected response from old-sms"));
      resolve({ id: r.id, status: r.status === "SENT" || r.status === "FAILED" ? r.status : "QUEUED"
});
    });
  });
}
```

AN ADAPTER MODULE GIVES AN UNTYPED LIBRARY A TYPED SURFACE terminal

```
npx tsc -p . && node dist/app.js
```

```
TERMINAL
$ npx tsc -p . && node dist/app.js
msg-4418 QUEUED
invalid mobile number 12345
```

19.2 Best practices

These guidelines, drawn from the TypeScript team, large open-source projects and books such as *Effective TypeScript*, summarise much of this book. They aim for code where the types are **accurate** (they describe what really happens at run time), **useful** (they catch mistakes and help the editor) and **unobtrusive** (they do not drown the logic in annotations).

19.2.1 Prefer unknown to any

`any` turns the type checker off and spreads silently; `unknown` accepts the same values but forces you to check before use (3.6.2). Use `unknown` for external data, `catch` variables, generic "anything" parameters and deserialised values, and narrow it with guards or schemas. When `any` is truly necessary (complex

library typings, a temporary migration step), keep it as local as possible (inside one function, as `as any` on one expression rather than a parameter type), never let it escape into return types, and enforce the rule with `@typescript-eslint/no-explicit-any` and the `no-unsafe-*` rules.

ANY LETS A BUG THROUGH; UNKNOWN FORCES A CHECK main.ts

```
function parseConfigAny(json: string): any { return JSON.parse(json); }
function parseConfigUnknown(json: string): unknown { return JSON.parse(json); }

const a = parseConfigAny('{"port": "8080"}');
const portA: number = a.port;                    // compiles: any hides that port is a string
console.log(portA + 1);                          // "80801": a bug that reached run time

const u = parseConfigUnknown('{"port": "8080"}');
const portU = typeof u === "object" && u !== null && "port" in u ? Number(u.port) : 3000;   // forced
to check
console.log(portU + 1);
```

TERMINAL

```
$ tsc --strict main.ts
(no errors reported)
$ node main.js
80801
8081
```

19.2.2 Let inference work; annotate boundaries

Annotate where types are **contracts**: function parameters, return types of exported functions and public methods, object literals that must match an interface (preferably with `satisfies`), and values from outside. Leave local variables, callback parameters and obvious return types to inference: redundant annotations add noise, can be **less precise** than inference (annotating a literal as `string` loses its literal type) and must be updated by hand. Explicit return types on exported functions also act as a guard: a mistake inside the function is reported there rather than at a distant caller (4.1.2), and they speed up type checking (15.6.1).

ANNOTATE CONTRACTS, INFER THE REST main.ts

```
interface Course { code: string; fee: number; level: "beginner" | "advanced" }

// Boundary: parameter and return types are the contract
export function discounted(courses: readonly Course[], pct: number): Course[] {
  const factor = 1 - pct / 100;                          // inferred: number
  return courses.map((c) => ({ ...c, fee: Math.round(c.fee * factor) }));   // c inferred from the
array
}

// Over-annotated: noisy, and the annotation is less precise than inference
const levelA: string = "advanced";                       // string: no longer fits Course["level"]
const levelB = "advanced";                               // "advanced": fits
const course: Course = { code: "TS101", fee: 9999, level: levelB };

// satisfies at a boundary: checked against the type, precise type kept (15.1.2)
const FEES = { TS101: 9999, PY201: 14999 } satisfies Record<string, number>;
console.log(discounted([course], 10)[0].fee, levelA, Object.keys(FEES));
```

OUTPUT

```
8999 advanced [ 'TS101', 'PY201' ]
```

19.2.3 Model domain data with unions and literal types

Encode business rules in types so that invalid data cannot be represented (15.2.2): literal unions instead of plain strings for statuses and categories, discriminated unions instead of objects full of optional fields, branded types for identifiers and units (15.2.1), `readonly` for data that must not change, and `Result` types for expected failures. Then exhaustive `switch` statements with `never` make the compiler list every place to update when the domain changes.

DOMAIN TYPES THAT MAKE INVALID PAYMENTS UNREPRESENTABLE main.ts

```
// Before: everything optional, nothing guaranteed
interface PaymentLoose { method: string; cardLast4?: string; upiId?: string; bank?: string; amount:
number; status: string }

// After: each payment method carries exactly its own data, and status is a closed set
type PaymentMethod =
  | { kind: "card"; last4: string; network: "visa" | "rupay" | "mastercard" }
  | { kind: "upi"; vpa: `${string}@${string}` }
  | { kind: "netbanking"; bank: "SBI" | "HDFC" | "ICICI" };
type PaymentStatus = "created" | "authorised" | "captured" | "refunded" | "failed";
interface Payment { readonly id: string; readonly amountPaise: number; method: PaymentMethod; status:
PaymentStatus }

function describe(p: Payment): string {
  const how = (() => {
    switch (p.method.kind) {
      case "card": return `${p.method.network} card ending ${p.method.last4}`;
      case "upi": return `UPI ${p.method.vpa}`;
      case "netbanking": return `${p.method.bank} net banking`;
      default: { const unreachable: never = p.method; return unreachable; }
    }
  })();
  return `${p.id}: Rs ${(p.amountPaise / 100).toFixed(2)} via ${how} [${p.status}]`;
}

const loose: PaymentLoose = { method: "upi", bank: "SBI", amount: -5, status: "done-ish" };   //
nonsense, but compiles
console.log(loose.status);
console.log(describe({ id: "pay_01", amountPaise: 999900, method: { kind: "upi", vpa: "asha@okbank" },
status: "captured" }));
console.log(describe({ id: "pay_02", amountPaise: 1499900, method: { kind: "card", last4: "4242",
network: "rupay" }, status: "authorised" }));
```

OUTPUT

```
done-ish
pay_01: Rs 9999.00 via UPI asha@okbank [captured]
pay_02: Rs 14999.00 via rupay card ending 4242 [authorised]
```

19.2.4 Keep types close to the code that uses them

Define a type next to the code that owns it: a component's props in the component file, a function's options type next to the function, a module's data types in that module, exported only if others need them. A single giant `types.ts` holding every interface in the project becomes a dumping ground: unrelated code changes together, ownership is unclear and unused types accumulate. Shared types belong where they are shared: the domain module that defines the concept, or a `shared` package between front end and back end (18.4). Derive related types (DTOs, props subsets, form state) from the source type with utility types instead of copying fields.

Do	Avoid
`Button.tsx` exports `ButtonProps` next to `Button`	all props interfaces in `src/types/index.ts`
`courses/model.ts` defines `Course`; others import it	the same `Course` interface copied into five files
`type CourseCard = Pick<Course, "code" | "title">`	a hand-written `CourseCard` that drifts from `Course`
`const ROLES = [...] as const; type Role = (typeof ROLES)[number]`	a `Role` union and a separate array that can disagree
types inferred from schemas (`z.infer`) at boundaries	an interface and a validator written separately

19.3 Type challenges

19.3.1 Practising with the type-challenges collection (easy to hard)

type-challenges (github.com/type-challenges/type-challenges) is a popular collection of type-level puzzles graded warm-up, easy, medium, hard and extreme. Each challenge gives a description, a template and test cases written with `Expect<Equal<...>>` (the helper used in Module 14); you solve it in the browser Playground until every test compiles. Start with the easy ones (`Pick`, `Readonly`, `TupleToObject`, `First`, `Length`, `Exclude`, `Awaited`, `If`, `Concat`, `Includes`, `Push`, `Parameters`), which rebuild the utility types and drill `keyof`, mapped types, conditional types and `infer`. The medium ones (`Omit`, `DeepReadonly`, `TupleToUnion`, `Chainable`, `Trim`, `Replace`, `CamelCase`) build fluency with recursion and template literals. Use them to deepen your understanding, not as everyday application code.

SOLUTIONS TO TEN TYPE CHALLENGES, VERIFIED BY THE COMPILER

main.ts

```
type Equal<A, B> = (<T>() => T extends A ? 1 : 2) extends (<T>() => T extends B ? 1 : 2) ? true :
false;
type Expect<T extends true> = T;

// Easy: First of Array
type First<T extends readonly unknown[]> = T extends readonly [infer F, ...unknown[]] ? F : never;
// Easy: Length of Tuple
type Length<T extends readonly unknown[]> = T["length"];
// Easy: Tuple to Object
type TupleToObject<T extends readonly PropertyKey[]> = { [K in T[number]]: K };
// Easy: If
type If<C extends boolean, T, F> = C extends true ? T : F;
// Easy: Includes (exact match)
type Includes<T extends readonly unknown[], U> = T extends [infer H, ...infer R] ? (Equal<H, U>
extends true ? true : Includes<R, U>) : false;
// Medium: Trim
type Space = " " | "\n" | "\t";
type Trim<S extends string> = S extends `${Space}${infer R}` ? Trim<R> : S extends `${infer
L}${Space}` ? Trim<L> : S;
// Medium: CamelCase
type CamelCase<S extends string> = S extends `${infer H}-${infer C}${infer R}` ?
`${H}${Uppercase<C>}${CamelCase<R>}` : S;
// Medium: TupleToUnion and Chainable options
type TupleToUnion<T extends readonly unknown[]> = T[number];

const tuple = ["tesla", "model 3", "model X"] as const;
type cases = [
  Expect<Equal<First<[3, 2, 1]>, 3>>,
  Expect<Equal<First<[]>, never>>,
  Expect<Equal<Length<typeof tuple>, 3>>,
  Expect<Equal<TupleToObject<typeof tuple>, { tesla: "tesla"; "model 3": "model 3"; "model X": "model
X" }>>,
  Expect<Equal<If<false, "a", "b">, "b">>,
  Expect<Equal<Includes<["Kars", "Esidisi", "Wamuu"], "Kars">, true>>,
  Expect<Equal<Includes<[1, 2, 3], 4>, false>>,
  Expect<Equal<Trim<"  hello world \n">, "hello world">>,
  Expect<Equal<CamelCase<"learn-being-forward">, "learnBeingForward">>,
  Expect<Equal<TupleToUnion<[1, "a", true]>, 1 | "a" | true>>,
];
console.log("all 10 type challenges pass");
```

OUTPUT

```
all 10 type challenges pass
```

Level	Challenges to start with	Skills practised
warm-up / easy	Hello World, Pick, Readonly, TupleToObject, First, Length, Exclude, Awaited, If, Concat, Includes, Push, Unshift, Parameters	keyof, mapped types, indexed access, conditional types, infer
medium	Get Return Type, Omit, Readonly 2, Deep Readonly, Tuple to Union, Chainable Options, Last, Pop, Promise.all, Type Lookup, Trim, Capitalize, Replace, Append Argument, Permutation, Flatten, CamelCase	recursion, template literals, variadic tuples, distribution
hard and above	Simple Vue, Currying, Union to Intersection, Get Required, Get Optional, String to Union, Camelize, IsAny, Join, Deep Pick	variance tricks, advanced inference, type-level algorithms

19.4 Capstone projects

A portfolio of finished, well-typed projects is the strongest proof of your TypeScript skills. Choose one or two of the projects below that match the roles you want, and finish them properly: strict mode, no stray `any`, tests, linting, a README with screenshots and setup steps, CI on GitHub Actions, and a live deployment.

Project	Main skills	Best for
Typed expense tracker (front end)	React or vanilla TS, discriminated unions, reducers, localStorage, charts	front-end roles
Typed REST API (Express or NestJS)	Express/NestJS, Zod, Prisma, auth, testing, OpenAPI	back-end and Node.js roles
Full-stack app with shared types	monorepo, shared schemas or tRPC, React + API, end-to-end types	full-stack roles
Published npm utility library	generics, advanced types, declaration files, tsup, type tests, semver	library and platform roles; shows depth

19.4.1 Typed expense tracker (front end)

Features: add, edit and delete expenses with amount, category, date and payment method; filter by month and category; monthly totals and category breakdown charts; budgets per category with warnings; persistence in `localStorage` (typed wrapper, 13.4) with schema versioning; import and export as CSV or JSON with validation. Typing highlights: `Expense` with a branded `Paise` amount (integers avoid floating-point money errors), a `Category` literal union derived from a constant, a `useReducer` with discriminated-union actions (18.1.2), and a validated JSON import. The core logic below runs without any framework, so it can be tested and reused with React, Vue or plain DOM code.

THE TYPED CORE OF AN EXPENSE TRACKER: BRANDED MONEY, LITERAL CATEGORIES AND A REDUCER main.ts

```
declare const brand: unique symbol;
type Paise = number & { readonly [brand]: "Paise" };
const rupees = (r: number) => Math.round(r * 100) as Paise;
const fmt = (p: Paise) => `Rs ${(p / 100).toLocaleString("en-IN", { minimumFractionDigits: 2 })}`;

const CATEGORIES = ["food", "travel", "rent", "books", "fun"] as const;
type Category = (typeof CATEGORIES)[number];
interface Expense { readonly id: number; date: `${number}-${number}-${number}`; amount: Paise;
category: Category; note?: string }

type Action =
  | { type: "add"; expense: Omit<Expense, "id"> }
  | { type: "remove"; id: number }
  | { type: "setBudget"; category: Category; amount: Paise };
interface State { expenses: Expense[]; budgets: Partial<Record<Category, Paise>>; nextId: number }

function reducer(state: State, action: Action): State {
  switch (action.type) {
    case "add": return { ...state, expenses: [...state.expenses, { ...action.expense, id: state.nextId
}], nextId: state.nextId + 1 };
    case "remove": return { ...state, expenses: state.expenses.filter((e) => e.id !== action.id) };
    case "setBudget": return { ...state, budgets: { ...state.budgets, [action.category]: action.amount
} };
  }
}
```

```
function monthlyReport(state: State, month: `${number}-${number}`) {
  const inMonth = state.expenses.filter((e) => e.date.startsWith(month));
  const byCategory = {} as Record<Category, Paise>;
  for (const c of CATEGORIES) byCategory[c] = inMonth.filter((e) => e.category === c).reduce((s, e) =>
s + e.amount, 0) as Paise;
  const warnings = CATEGORIES.filter((c) => (state.budgets[c] ?? Infinity) < byCategory[c]);
  return { total: inMonth.reduce((s, e) => s + e.amount, 0) as Paise, byCategory, warnings };
}

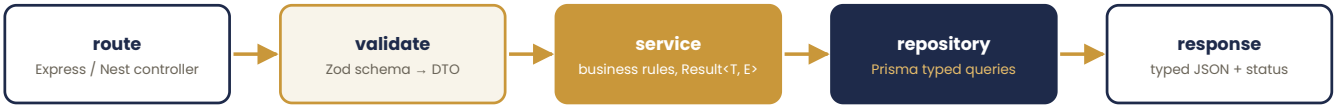
let state: State = { expenses: [], budgets: {}, nextId: 1 };
const actions: Action[] = [
  { type: "setBudget", category: "food", amount: rupees(4000) },
  { type: "add", expense: { date: "2026-10-01", amount: rupees(12000), category: "rent" } },
  { type: "add", expense: { date: "2026-10-03", amount: rupees(2450.5), category: "food", note:
"groceries" } },
  { type: "add", expense: { date: "2026-10-05", amount: rupees(1899), category: "books", note:
"Effective TypeScript" } },
  { type: "add", expense: { date: "2026-10-09", amount: rupees(1820), category: "food" } },
  { type: "add", expense: { date: "2026-09-28", amount: rupees(640), category: "travel" } },
];
state = actions.reduce(reducer, state);

const r = monthlyReport(state, "2026-10");
console.log("October total:", fmt(r.total));
for (const c of CATEGORIES) if (r.byCategory[c] > 0) console.log(`  ${c.padEnd(6)}
${fmt(r.byCategory[c]).padStart(13)} ${"#".repeat(Math.round(r.byCategory[c] / r.total * 30))}`);
console.log("over budget:", r.warnings.join(", ") || "none");
```

```
OUTPUT
October total: Rs 18,169.50
  food     Rs 4,270.50 #######
  rent    Rs 12,000.00 ####################
  books    Rs 1,899.00 ###
over budget: food
```

19.4.2 Typed REST API with Express or NestJS

Build a production-style API for a domain you know, for example course enrolments: resources with full CRUD, pagination and filtering; authentication with hashed passwords and JWT (or sessions); role-based authorisation; validation of every input with Zod (or class-validator in NestJS); Prisma or Drizzle with migrations and seed data; consistent error responses; structured logging; OpenAPI documentation generated from the schemas; integration tests with Vitest and Supertest against a test database; a Dockerfile and deployment. Use the patterns from 18.2 and 18.3: DTOs derived from schemas, typed middleware, Request augmentation for the authenticated user, and a `Result`-returning service layer that the route handlers translate to HTTP status codes.



THE REQUEST PATH THROUGH A TYPED API

19.4.3 Full-stack app with shared types

Combine both sides in a monorepo (17.2) with three packages: `shared` (schemas, inferred types, constants), `api` and `web`. Use either the shared-contract approach of 18.4.1 or tRPC for end-to-end inference. The project demonstrates the main promise of TypeScript in a team: change a field in one schema, run `tsc -b`, and the compiler lists every place in the API and the UI that must change. The

Student Management app outlined in the Module 18 checkpoint is a good starting point; add authentication, search, pagination, optimistic UI updates and end-to-end tests with Playwright.

19.4.4 Publish a small typed npm utility library

Publishing a library shows a deep understanding of types, because your declarations are the product. Choose something small and useful: a typed event emitter (14.4.2), a `Result` type with combinators (15.2.3), a validation helper, a money and GST formatter, an Indian phone and PIN code validator, or a typed `localStorage` wrapper. Write it in strict TypeScript with generics and overloads where they help, generate ESM and CommonJS builds and `.d.ts` files with tsup (17.3.3), test both behaviour (Vitest) and types (`expectTypeOf`, 18.5.3), check the package with `@arethetypeswrong/cli` and `npm pack --dry-run`, document it with TSDoc comments and a README, version it with semantic versioning and publish with `npm publish --access public` (ideally from CI with provenance).

PACKAGE.JSON JavaScript

```
{
  "name": "@lbf/typed-emitter",
  "version": "1.0.0",
  "type": "module",
  "exports": { ".": { "types": "./dist/index.d.ts", "default": "./dist/index.js" } },
  "files": ["dist"],
  "sideEffects": false,
  "scripts": { "build": "tsc", "test": "vitest run", "prepublishOnly": "npm run build" },
  "license": "MIT"
}
```

SRC/INDEX.TS TypeScript

```
/** A map of event names to the payload each event carries. */
export type EventMap = Record<string, unknown>;

/** A strongly typed event emitter. */
export interface Emitter<E extends EventMap> {
  /** Subscribe; returns a function that unsubscribes. */
  on<K extends keyof E>(event: K, handler: (payload: E[K]) => void): () => void;
  /** Emit an event with its payload. Returns the number of handlers called. */
  emit<K extends keyof E>(event: K, payload: E[K]): number;
}

export function createEmitter<E extends EventMap = Record<string, any>>(): Emitter<E> {
  const handlers = new Map<keyof E, Set<(payload: never) => void>>();
  return {
    on(event, handler) {
      if (!handlers.has(event)) handlers.set(event, new Set());
      handlers.get(event)!.add(handler as (payload: never) => void);
      return () => { handlers.get(event)?.delete(handler as (payload: never) => void); };
    },
    emit(event, payload) {
      const set = handlers.get(event);
      set?.forEach((h) => (h as (p: typeof payload) => void)(payload));
      return set?.size ?? 0;
    },
  };
}
```

A TINY TYPED LIBRARY: SOURCE, GENERATED DECLARATIONS AND A QUICK SMOKE TEST

terminal

```
npx tsc -p .
ls dist
cat dist/index.d.ts
node -e "import('./dist/index.js').then(({ createEmitter }) => { const e = createEmitter();
e.on('enrolled', (p) => console.log('enrolled', p.student, 'in', p.course)); e.emit('enrolled', {
student: 'Asha', course: 'TS101' }); })"
```

TERMINAL

```
$ npx tsc -p .
$ ls dist
index.d.ts
index.js
$ cat dist/index.d.ts
/** A map of event names to the payload each event carries. */
export type EventMap = Record<string, unknown>;
/** A strongly typed event emitter. */
export interface Emitter<E extends EventMap> {
    /** Subscribe; returns a function that unsubscribes. */
    on<K extends keyof E>(event: K, handler: (payload: E[K]) => void): () => void;
    /** Emit an event with its payload. Returns the number of handlers called. */
    emit<K extends keyof E>(event: K, payload: E[K]): number;
}
export declare function createEmitter<E extends EventMap = Record<string, any>>(): Emitter<E>;
$ node -e "import('./dist/index.js').then(({ createEmitter }) => { const e = createEmitter();
e.on('enrolled', (p) => console.log('enrolled', p.student, 'in', p.course)); e.emit('enrolled', {
student: 'Asha', course: 'TS101' }); })"
enrolled Asha in TS101
```

19.5 Interview preparation

TypeScript interviews for front-end, back-end and full-stack roles usually combine: conceptual questions on the type system, live coding in TypeScript (often typing an existing JavaScript function, or a small feature in React or Node.js), sometimes a type-level exercise such as implementing a utility type, and a discussion of your projects. Interviewers look for **judgement** as much as knowledge: knowing when a simple union beats a clever generic, and when to validate instead of asserting.

19.5.1 Core topics checklist

Question	What a strong answer covers	Book
any vs unknown?	both accept any value; any disables checking and spreads, unknown requires narrowing before use; use unknown for external data and catch variables	3.6
interface vs type?	both describe object shapes; type also names unions, tuples, primitives, mapped and conditional types; interfaces extend and merge, give clearer errors; pick a convention	5.4
What are **generics** and constraints?	type parameters link input and output types; extends constrains them; keyof and T[K]; defaults; inference; avoid unnecessary parameters	10
How does **narrowing** work?	control-flow analysis; typeof, truthiness, equality, in, instanceof, discriminated unions, type predicates, assertion functions; never for exhaustiveness	8
Name some **utility types**.	Partial, Required, Readonly, Pick, Omit, Record, Exclude, Extract, NonNullable, ReturnType, Parameters, Awaited; and how they are built	12, 14

Question	What a strong answer covers	Book
What is **never**?	the empty (bottom) type: functions that never return, impossible branches, exhaustiveness checks, filtering in mapped and conditional types	3.6.4, 8.2.3
enums or unions?	enums generate runtime objects (numeric reverse mappings, const enums), not erasable; literal unions or as const objects are lighter and fit JSON	6.4
What are **declaration files**?	.d.ts files hold types only; generated with declaration: true, shipped by packages or @types, written for JS libraries; declare, module augmentation	11.4
Structural vs nominal typing?	TypeScript compares shapes; brands or private fields simulate nominal types; excess property checks for fresh literals	5.7, 15.2.1
Types at run time?	types are erased; validate external data with guards or schemas (Zod) and infer types from schemas	1.1.4, 15.3
as const and satisfies?	as const keeps literals and readonly; satisfies checks without widening; together for typed configuration	6.3.3, 15.1
Strict mode?	what strict enables, strictNullChecks in particular; extra flags such as noUncheckedIndexedAccess; migrating loose to strict	2.5.4, 17.1, 19.1

Also prepare quick answers on: `readonly` versus `const`; optional properties versus `| undefined`; function overloads; abstract classes versus interfaces; access modifiers versus `#private`; covariance and contravariance of function parameters; `type` imports and `verbatimModuleSyntax`; how TypeScript is compiled (tsc versus bundlers that only strip types); decorators (standard versus legacy); and how you would type a React component or an Express handler.

19.5.2 Implementing utility types in interviews

A common exercise is "implement `Pick`" (or `Omit`, `Readonly`, `ReturnType`, `DeepPartial`) on a whiteboard or in the Playground. Talk through the building blocks as you write: which operator iterates over keys (mapped types), how to constrain the keys (`K extends keyof T`), how to look up a property type (`T[P]`), when a conditional type with `infer` is needed, and how recursion and the special cases for functions and arrays work. Then test your type with two or three examples, including an edge case (optional properties, unions, `never`).

UTILITY TYPES YOU SHOULD BE ABLE TO WRITE FROM MEMORY, WITH TESTS

main.ts

```
type Equal<A, B> = (<T>() => T extends A ? 1 : 2) extends (<T>() => T extends B ? 1 : 2) ? true :
false;
type Expect<T extends true> = T;

// "Implement Pick": iterate over the chosen keys, look up each property type
type MyPick<T, K extends keyof T> = { [P in K]: T[P] };

// "Implement Omit": keep the keys that are not in K (Exclude distributes over the key union)
type MyOmit<T, K extends keyof any> = { [P in keyof T as P extends K ? never : P]: T[P] };

// "Implement ReturnType": pattern-match the function type with infer
type MyReturnType<F> = F extends (...args: never[]) => infer R ? R : never;

// "Implement Readonly" and "DeepPartial" with the special cases interviewers ask about
type MyReadonly<T> = { readonly [P in keyof T]: T[P] };
type DeepPartial<T> = T extends (...args: never[]) => unknown ? T
  : T extends readonly (infer E)[] ? DeepPartial<E>[]
  : T extends object ? { [P in keyof T]?: DeepPartial<T[P]> }
  : T;

// "Implement Awaited" recursively
type MyAwaited<T> = T extends PromiseLike<infer V> ? MyAwaited<V> : T;

interface Todo { title: string; description?: string; completed: boolean; meta: { tags: string[];
owner: { id: number } } }

type tests = [
  Expect<Equal<MyPick<Todo, "title" | "completed">, { title: string; completed: boolean }>>,
  Expect<Equal<MyOmit<Todo, "meta" | "completed">, { title: string; description?: string }>>,   //
keeps optionality
  Expect<Equal<MyReturnType<() => Promise<Todo[]>>, Promise<Todo[]>>>,
  Expect<Equal<MyReadonly<{ a: number }>, { readonly a: number }>>,
  Expect<Equal<DeepPartial<Todo>["meta"], { tags?: string[]; owner?: { id?: number } } | undefined>>,
  Expect<Equal<MyAwaited<Promise<Promise<string>>>, string>>,
];
console.log("interview utility types verified:", 6, "tests");
```

OUTPUT

```
interview utility types verified: 6 tests
```

19.5.3 Explaining your projects

When asked about a project, explain it with the **STAR** structure (Situation, Task, Action, Result) and make the TypeScript decisions visible: how you modelled the domain (unions for states, branded IDs), where you validated data (schemas at the API boundary), how types are shared between front end and back end, how strict the configuration is and why, a bug the types caught or a refactoring they made safe, and what you would improve. Expect follow-ups such as "Why did you use a type assertion here?", "How do you know the API returns this shape?", "How would you add a new payment method?" (answer: add a union member and let the exhaustive switches show every place to update).

Situation	Task	Action	Result
Our institute tracked enrolments in spreadsheets; data was inconsistent and payments were missed.	Build a full-stack enrolment system with reliable data and online payments.	TypeScript monorepo: Zod schemas shared by Express + Prisma API and React UI, discriminated unions for payment states, strict mode, Vitest + type tests, CI.	Live for 3 batches, zero type-related production bugs; adding UPI payments took one day because the compiler listed every place to change.

On GitHub, pin these projects with clear READMEs: what the project does, a screenshot or demo link, the tech stack, how to run it (`npm install`, environment variables, `npm run dev`), how to test it, and a short "design decisions" section explaining the type-level choices. A clean commit history, CI badges, and a strict `tsconfig.json` visible in the repository all signal professional habits.

Migrate JavaScript incrementally: add TypeScript with allowJs, check JavaScript with checkJs and JSDoc, rename leaf modules first, type the boundaries, then tighten strictness flag by flag (noImplicitAny, strictNullChecks, strict, extras), tracking remaining .js files, any and suppressions; give untyped libraries types via @types, minimal declaration files or a typed adapter module. Best practices: prefer unknown to any, annotate boundaries and let inference work, model the domain with literal and discriminated unions, brands and readonly, and keep types next to their code while deriving related types. Practise type-level skills with type-challenges. Build capstones (typed expense tracker, typed REST API, full-stack app with shared types, a published typed npm library), and prepare for interviews with the core-topics checklist, live implementations of utility types, and STAR explanations of your projects that highlight the type-level decisions.

PRACTICE CHECKPOINT

Migrate one earlier **JavaScript project fully to strict TypeScript**, solve **15 type challenges**, and **publish a capstone project on GitHub**.

Sample solution (migration part). The program below is the result of migrating the JavaScript to-do module from the LBF JavaScript course to strict TypeScript: the data is modelled with literal unions and readonly properties, external data is validated, every function has typed boundaries, and the build passes with `strict`, `noUncheckedIndexedAccess` and `exactOptionalPropertyTypes`. The 15 type challenges can be tracked with the same `Expect<Equal>` technique as 19.3, and the capstone published following 19.4 and 19.5.3.

SRC/TODOS.TS TypeScript

```
// Before (JavaScript): todos were { id, text, done, priority } with any values, and JSON was trusted.
export type Priority = "low" | "normal" | "high";
export interface Todo { readonly id: number; text: string; done: boolean; priority: Priority; due?:
string }
type Filter = "all" | "open" | "done";

const PRIORITIES: readonly Priority[] = ["low", "normal", "high"];

function isTodo(v: unknown): v is Todo {
  if (typeof v !== "object" || v === null) return false;
  const t = v as Record<string, unknown>;
  return typeof t.id === "number" && typeof t.text === "string" && typeof t.done === "boolean" &&
    PRIORITIES.includes(t.priority as Priority) && (t.due === undefined || /^\d{4}-\d{2}-
\d{2}$/.test(String(t.due)));
```

```
}

export function parseTodos(json: string): { todos: Todo[]; rejected: number } {
  const data: unknown = JSON.parse(json);
  if (!Array.isArray(data)) return { todos: [], rejected: 0 };
  const todos = data.filter(isTodo);
  return { todos, rejected: data.length - todos.length };
}

export function addTodo(list: readonly Todo[], text: string, priority: Priority = "normal", due?:
string): Todo[] {
  const id = Math.max(0, ...list.map((t) => t.id)) + 1;
  const todo: Todo = due === undefined ? { id, text: text.trim(), done: false, priority } : { id,
text: text.trim(), done: false, priority, due };
  return [...list, todo];
}

export const toggle = (list: readonly Todo[], id: number): Todo[] => list.map((t) => (t.id === id ? {
...t, done: !t.done } : t));

export function visible(list: readonly Todo[], filter: Filter): Todo[] {
  switch (filter) {
    case "all": return [...list];
    case "open": return list.filter((t) => !t.done);
    case "done": return list.filter((t) => t.done);
  }
}

const rank: Record<Priority, number> = { high: 0, normal: 1, low: 2 };
export const byPriority = (list: readonly Todo[]): Todo[] => [...list].sort((a, b) => rank[a.priority]
- rank[b.priority] || a.id - b.id);

// ---- demo
const { todos, rejected } = parseTodos('[{"id":1,"text":"Finish Module
19","done":false,"priority":"high","due":"2026-10-15"},' +
  '{"id":2,"text":"Revise generics","done":true,"priority":"normal"},
{"id":3,"text":"bad","done":"no","priority":"urgent"}]');
let list = addTodo(todos, "  Publish capstone on GitHub ", "high");
list = toggle(list, 1);
console.log(`loaded ${todos.length} (rejected ${rejected})`);
for (const t of byPriority(visible(list, "all"))) console.log(`${t.done ? "[x]" : "[ ]"} #${t.id}
${t.text} (${t.priority}${t.due ? ", due " + t.due : ""})`);
console.log("open:", visible(list, "open").map((t) => t.id).join(", "));
```

SAMPLE SOLUTION: A JAVASCRIPT MODULE MIGRATED TO STRICT TYPESCRIPT

terminal

```
npx tsc -p . && echo "strict build passed"
node dist/todos.js
```

```
TERMINAL
$ npx tsc -p . && echo "strict build passed"
strict build passed
$ node dist/todos.js
loaded 2 (rejected 1)
[x] #1 Finish Module 19 (high, due 2026-10-15)
[ ] #3 Publish capstone on GitHub (high)
[x] #2 Revise generics (normal)
open: 3
```

Congratulations on completing the TypeScript Programming course. You can now write strictly typed applications, design types that model real domains, work with the compiler and its tooling, and use

TypeScript across the front end, the back end and the database. Keep building, keep the compiler on your side, and keep moving forward.